Temporal Databases

Time

“It's present everywhere, but occupies no space.
We can measure it, but we can't see it, touch it,
get rid of it, or put it in a container.
Everyone knows what it is and uses it every day,
but no one has been able to define it.
We can spend it, save it, waste it, or kill it,
but we can't destroy it or even change it,
and there's never any more or less of it.”

~~Je~~spersen and Fitz-Randolph, From Sundials to Atomic Clocks

Time Structure

- Linear: total order on instants
- Hypothetical (possible futures): tree rooted on now
- Directed Acyclic Graph (DAG): possible futures may merge
- Periodic/cyclic time: weeks, months…
- Assume a linear time structure
- Boundedness
  - Unbounded
  - Time origin exists (bounded from the left)
  - Bounded time (bounds on two ends)
- Nature of bound
  - Unspecified
  - Specified

Time Density

- Discrete
  - Time line is shaped by integers
  - Time line is composed of a sequence of non-decomposable time periods, of some fixed minimal duration, termed chronons
  - Between each pair of chronons is a finite number of other chronons
- Dense
  - Time line is shaped by rational numbers
  - Infinite number of instants between each pair of chronons
- Continuous
  - Time line is shaped by real numbers
  - Infinite number of instants between each pair of chronons
- Distance may optionally be defined

What is temporal DB?

- Temporal databases include all DB applications that require some aspect of time when organizing their information.
- Temporal DB applications have been developed since the early days of database usage. However, in creating these applications, it was mainly left to the application developers to discover, design, program, and implement the temporal concepts.

Why now?

- Decreasing cost of storage
- Data warehouse technology adoption in the majority of the domains
- The need of maintaining and processing historical data (legal, security, etc...)

Why need temporal data?

Two questions should answer:

- Does your organization need to know the situation as it was known at a particular date (e.g. aviation - the fuel price for a certain date)?

- Does your business use information that was effective in the past or will become effective in the future (e.g. the new address of the customer)?

Temporal DBs — Motivation

- Conventional databases represent the state of an enterprise at a single moment of time (snapshot)
- Many applications need information about the past (time-varying data)
  - Banking (account transactions to compute balance)
  - Medical (patient history)
  - Financial (payroll)
- Temporal DBs: a system that manages time varying data

Why time varying data?

Examples of application domains dealing with time varying data:

- Financial Apps (e.g. history of currency exchange data)
- Insurance Apps (e.g. when were the policies in effect)
- Reservation Systems (e.g. when is which room in a hotel booked)
- Medical Information Management Systems (e.g. patient records)
- Decision Support Systems (e.g. planning future contingencies)
- HR applications (e.g. Date tracked positions in hierarchies)

Comparison

Conventional DBs:

— Evolve through transactions from one state to the next

— Changes are viewed as modifications to the state

— No information about the past

— Snapshot of the enterprise

Temporal DBs:

— Maintain historical information

— Changes are viewed as additions to the information stored in the database

— Incorporate notion of time in the system

— Efficient access to past states

Temporal Databases

- Temporal Data Models: extension of relational model by adding temporal attributes to each relation
- Temporal Query Languages: TQUEL, SQL3
- Temporal Indexing Methods and Query Processing

Temporal Databases

- Temporal data types
- Kinds of time
- Temporal statements
- There are three fundamental temporal data types:
  - Instant
  - Interval
  - Period

Instant

- Instant: something happened at an instant of time: eg, now, 30th of September 2025, 19:30, as we speak at the course
- An instant is an anchored location on the timeline. An SQL-92 provides: DATE (a particular day, with a year in the range ad 1–9999), TIME (a particular second within a range of 24 hours), and TIMESTAMP (a particular fraction of a second, defaulting to microsecond, of a particular day).

(Time) Interval

- A length of time (e.g., three months)
- An interval is relative; an instant is absolute. An interval can be added to an instant, yielding another instant.
- A time interval is a directed duration of time, an amount of time with a known length, but without exact starting or ending instants.
- The distance between two instants is an interval. Unlike instants, intervals have direction. An interval can be positive or negative, denoting a shift to the future or to the past.

Interval

- The SQL-92 interval type is complex. SQL differentiates year-month intervals and day-time intervals. The first,an integral number of years or months; the latter, an integral number of days, hours, minutes, seconds, or fractions of a second. This distinction is due to varying month lengths in the Gregorian calendar.
- The individual units (months, hours, seconds) are termed granules, so an interval value is a (signed) integer number of granules.
- Intervals are combinations of the fields year, month, day, hour, minute, and second, though not all combinations are allowed.

Period

- An instant has no duration. Yet facts in the database are true over a duration of time. To express when a fact holds in the enterprise, a period is associated with that fact.
- Period: an anchored duration of time (e.g., the spring semester, 29th of September 2025 through 27th of February, 2026)
- SQL-92 includes support for instants and intervals. Most DBMS products, though, only support instants, with intervals being simulated with integers or floating-point numerics. Periods are always left to the application developer to simulate using supplied data types.
- As SQL-92 does not provide a period data type, there are no period literals in that language. Periods must instead be specified by their constituent datetime and interval literals.

Time Predicates

- For such a diverse set of types (DATE, TIME, TIMESTAMP, TIME WITH TIME ZONE, TIMESTAMP WITH TIME ZONE, and two variants of INTERVAL: year-month and day-time), SQL-92 supports only four classes of temporal predicates: equality, less than, is null, and overlaps.
- There are several variants of the equality predicate; these variants apply to all the data types. When applied to two expressions, '**=**' determines whether the values of these expressions are identical. When applied to a value and a set of values (of the same type), **=ANY** determines if the left-hand value is identical to at least one of the values in the right-hand set. **=SOME** and IN are non orthogonal equivalents. **MATCH** also relies on equality testing.

Taxonomy of time

- *User defined time*: an uninterpreted time value
- *Transaction time* databases:

  — Transaction time is the time when a fact is stored in the database
- *Valid time* databases:

  - Valid time is the time that a fact becomes effective in reality
- *Bi-temporal* databases:

  — Support both notions of time

Temporal databases

- We can use these two dimensions to distinguish between different forms of temporal database
  - A rollback database stores data with respect to transaction time
  - A historical database stores data with respect to valid time
  - A bi-temporal database stores data with respect to both valid time and transaction time.

Temporal Database Features

Temporal databases support managing and accessing temporal data by providing one or more of the following features:

- A time period datatype, including the ability to represent time periods with no end (infinity or forever)
- The ability to define valid and transaction time period attributes and bitemporal relations
- System-maintained transaction time

Temporal Database Features

- Temporal primary keys, including non-overlapping period constraints
- Temporal constraints, including non-overlapping uniqueness and referential integrity
- Update and deletion of temporal records with automatic splitting and coalescing of time periods
- Temporal queries at current time, time points in the past or future, or over durations
- Predicates for querying time periods

Example

- Sales example: data about sales are stored at the end of the day
- Transaction time is different than valid time
- Valid time can refer to the future also!
  - Credit card: O1/2026-O1/2030

Transaction Time DBs

- Time evolves in discrete manner, usually is associated with the transaction number (or SCN):

  T1 -> T2 -> T3 -> … -> Tn

- A record R is extended with an interval [t.start, t.end). When we insert an object at t1 the temporal attributes are updated -> [t1, now)
- Updates can be made only to the current state!
  - Past cannot be changed
  - “Rollback” characteristics

Transaction Time DBs

- Transaction time records the time period during which a database entry is accepted as correct.
- This enables queries that show the state of the database at a given time.
- Transaction time periods can only occur in the past or up to the current time.
- In a transaction time table, records are never deleted. Only new records can be inserted, and existing ones updated by setting their transaction end time to show that they are no longer current.

Transaction Time DBs

- Deletion is logical (never physical deletions)
  — When an object is deleted at t2, its temporal attribute changes from [t1, now) -> [t1, t2)
  — Object is “alive” from insertion to deletion time, ex. t1 to t2. If “now” then the object is still alive

eid	salary	start	end
10	20K	9/93	10/94
20	50K	4/94	*
33	30K	5/94	6/95
10	50K	1/95	*



Transaction Time DBs

- Requirements for index methods:
  - — Store past logical states
  - — Support addition/deletion/modification changes on the objects of the current state
  - — Efficiently access and query any database state

Transaction Time DBs

- Queries:

  - Timestamp (timeslice) queries: ex. “Give me all employees at 09/2025”

  - Range-timeslice: “Find all employees with id between 100 and 200 that worked in the company on 09/2025”

  - Interval (period) queries: “Find all employees with id in [100,200] from 01/2023 to 09/2025”

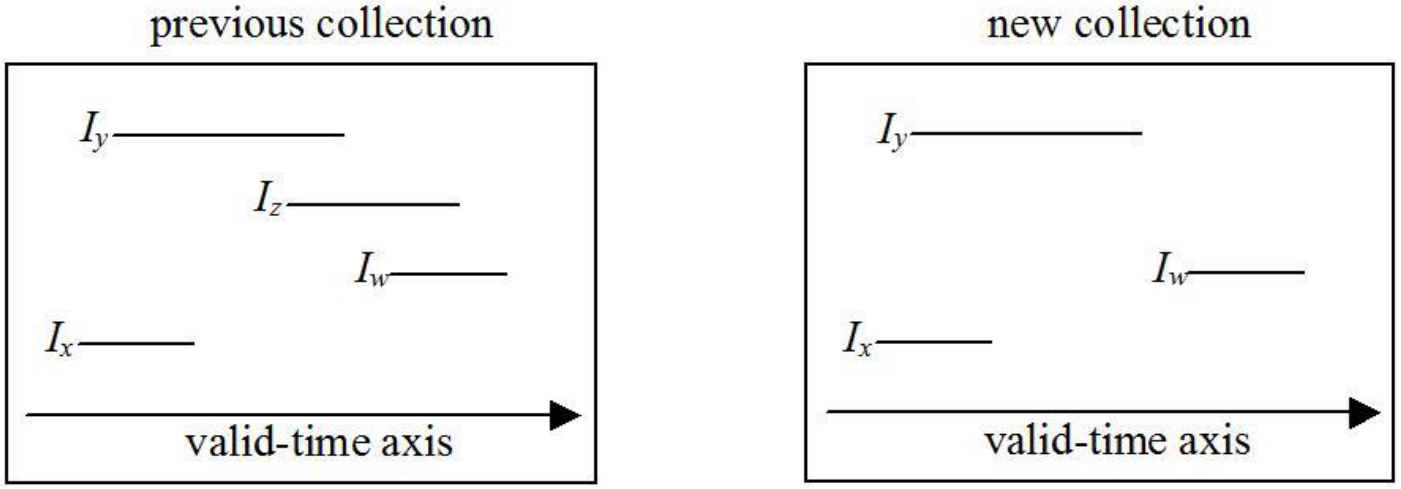
Valid Time DBS

- Valid time is the time for which a fact is true in the real world.
- A valid time period may be in the past, contain the current time, or occur in the future.
- Time evolves continuously
- Each object is a line segment representing its time span (eg. Credit card valid time)
- Physical deletion is possible
- Support full operations on interval data:
  - Deletion at any time
  - Insertion at any time
  - Value change (modification) at any time (no ordering)

Valid Time

- Event table
  - single chronon identifier
- State table
  - interval timestamps

Valid Time DBS



The reality "best know now !"

Valid Time DBS

- The time interval is closed at its lower bound and open at its upper bound
- Requirements for an Index method:

— Store the latest collection of interval-objects

— Support add/delete/modify changes to this collection

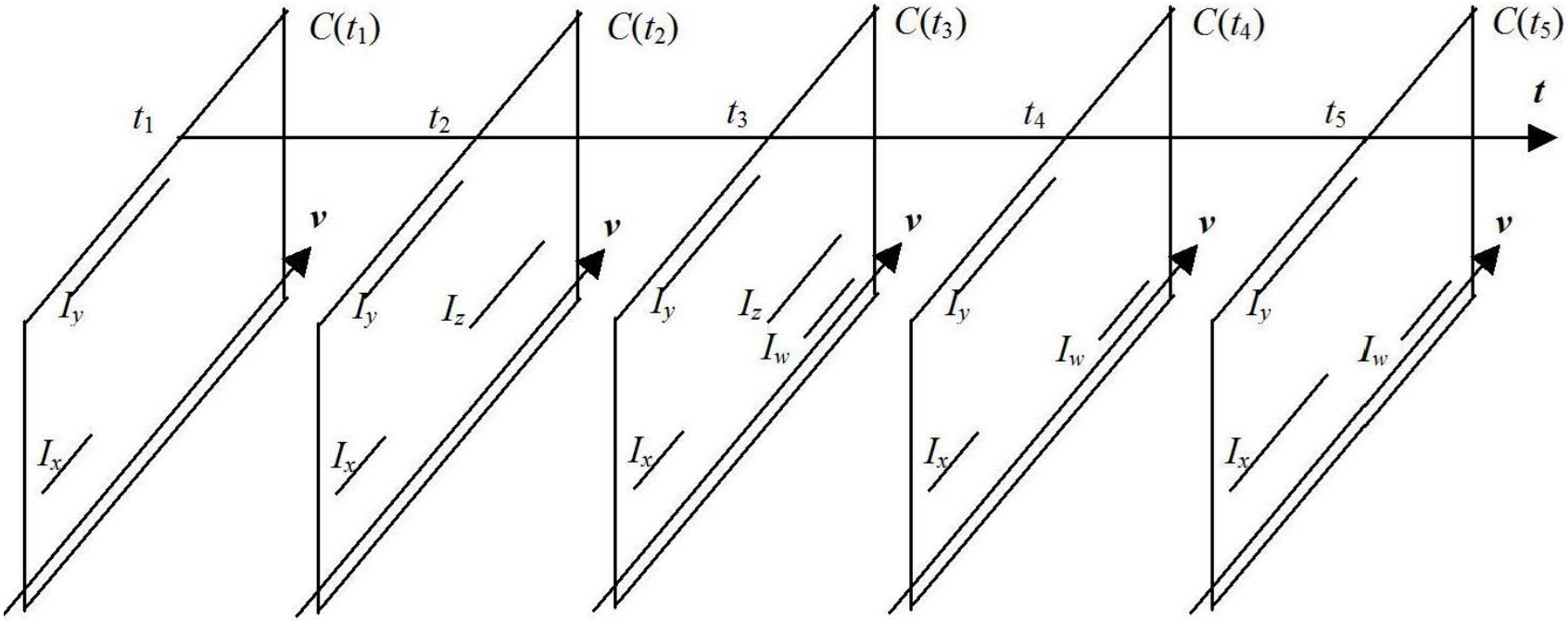
— Efficiently query the intervals in the collection

- Timestamp query
- Interval (period) query

Bi-temporal DBs

- A bi-temporal relation contains both valid and transaction time. This provides both historical and rollback information. Historical information (e.g.: "Where did John live in 1992?") is provided by the valid time. Rollback (e.g.: "In 1992, where did the database believe John lived?") is provided by the transaction time.
- It is a transaction-time database as each record is an interval (plus the other attributes of the record)
- At each timestamp, it is a valid time database

Bi-temporal DBs



Examples

- Model a car hire problem
  - as event table
  - as state table
- Ideas how to store transaction time for transactions that affect several tables?

SQL 2011 Standard

- There is no commonly accepted definition of temporal data. Let us consider temporal data to be any data with one or more associated time periods during which that data is considered to be in effect or valid.
- A (SQL) standard has to deal amongst others with:
  - data type and table definition standardization,
  - table definition modification,
  - (referential) integrity enforcement
  - data manipulation

SQL 2011 Standard

- A time period is a mathematical interval on the time line, bounded by a start time and an end time.
- It was introduced a period data type, defined as an ordered pair of two datetime values, for the purpose of associating time periods with the rows of a table. **SQL-2011 has not adopted this solution.**
- Adding a new data type to the SQL standard (or to an SQL product) is a costly operation because of the new data type has to be supported also in all the software that form the ecosystem surrounding SQL.

SQL 2011 Standard

- Instead of adding a period type SQL 2011 adopts period definitions as **table meta data**.
- The model adopted is a closed-open period model.
- A period definition as table component identifies a pair of columns that store the period start and end time.
- CREATE TABLE and ALTER TABLE statements are enhanced with **syntax for creating or modifying period definitions**.
- The period start and end columns are conventional columns, with separate names.
- The period name occupies the same name space as column names, a period can't have the same name as a column.

SQL 2011 Standard

- *transaction time support* provided by system versioned tables. The name of the system time period is specified by the standard as *SYSTEM_TIME*.

- *valid time support* provided by tables storing application time period. The name of an application time period can be any *user-defined* name.

- can be defined at most one application time period and at most one system time period per table.

- allows existing databases that store periods using a pair of date time columns to take advantage of the SQL-2011 extensions more easily.

- as most DBMSs do not support a period type, the period information using a pair of columns of datetime data type is the solution.

SQL 2011 Standard

- Application (valid) time period: applications that are interested in storing time periods during which the data is believed to be valid in the real world.
- Example: insurance application, keep track of the specific policy details of a given customer that are in effect at any given point in time.
- User be free to assign any time values, either in the past, current or in the future, for the start and end times.
- User be permitted to update the validity periods of the rows as errors are discovered or new information is made available.
- Any table that contains a period definition with a user-defined name is an application-time period table.

SQL 2011 Standard

```
CREATE TABLE Emp(
ENo INTEGER,
EStart DATE,
EEnd DATE,
EDept INTEGER,
PERIOD FOR EPeriod (EStart, EEnd)
)
```

Users can pick any name for the name of the period as well as for the names of columns that act as the start and end of the period.

The data types of the period start and end columns must be either **DATE** or a **timestamp** type, and data types of both columns must be the same.

SQL 2011 Standard

```
INSERT INTO EmpVALUES (22217, DATE ‘2010-01-01’,DATE '2011-11-12', 3)
```

- The conventional UPDATE statement can be used to modify the rows of application-time period tables.
- The conventional DELETE statement can be used to delete rows of application-time period tables.

New feature of SQL-2011 is the ability to specify changes that are effective within a specified period.

Provided by a syntactic extension to both UPDATE and DELETE statements lets users specify the period of interest.

```
UPDATE Emp FOR PORTION OF EPeriod FROM DATE '2011-02-03' TO DATE '2011-09-10'SET EDept = 4 WHERE ENo = 22217
```

SQL 2011 Standard

- To execute this statement, the DBMS locates all rows whose application-time period overlaps the period P from Feb. 3, 2011 to Sept. 10, 2011; (periods follow closed-open semantics in SQL-2011) so P includes Feb. 3, 2011 but excludes Sept. 10, 2011. Any overlapping row whose application-time period is contained in P is simply updated.
- If an overlapping row whose application-time period has a portion either strictly before or strictly after P, then that row gets split into two or three contiguous rows depending on the extent of overlap, and of these, the row whose application-time period is contained in P is updated.

SQL 2011 Standard

- The DELETE statement is similarly enhanced with **FOR PORTION OF** syntax to facilitate deletes that are only effective within a specified period.

```
DELETE Emp FOR PORTION OF EPeriod FROM DATE
'2011-02-03' TO DATE '2011-09-10' WHERE ENo
= 22217
```

- Similar to the UPDATE example, any row whose application-time period is contained in P from Feb. 3,2011 to Sept. 10, 2011 is simply deleted. If an overlapping row whose application-time period has a portion either strictly before or strictly after P, then that row gets split into two or three contiguous rows, and of these, the row whose application-time period is contained in P is deleted.

SQL 2011 Standard

It must be possible to forbid overlapping application-time periods:

```
ALTER TABLE EmpADD PRIMARY KEY (ENo, EPeriod
WITHOUT OVERLAPS)
```

Consider the following definition:

```
CREATE TABLE Dept(
DNo INTEGER,
DStart DATE,
DEnd DATE,
DName VARCHAR(30),
PERIOD FOR DPeriod (DStart, DEnd),
PRIMARY KEY (DNo, DPeriod WITHOUT OVERLAPS))
```

SQL 2011 Standard

- Assume also that we want to make sure that at every point in time, every value in EDept column corresponds to some value of DNo column in Dept table, i.e., every employee at every point in time during her employment belongs to a department that actually exists at that point in time. How should this work?
- It must be possible to forbid a row in a child table whose application-time period is not contained in the application-time period of a matching row in the parent table, which can be specified with this syntax:

```
ALTER TABLE EmpADD FOREIGN KEY (Edept,
PERIOD EPeriod)  REFERENCES Dept (DNo,
PERIOD DPeriod)
```

SQL 2011 Standard

With this referential constraint definition, the sample data is prohibited as a constraint violation. More generally, for a given child row, *it is not necessary that there exists exactly one matching row in the parent table* whose application-time period contains the application-time period of the child row. As long as the application-time period of a row in the child table is contained in the *union of application-time periods* of two or more contiguous matching rows in the parent table, the referential constraint is considered satisfied.

SQL 2011 Standard

In SQL-2011, application-time period tables can be queried using the regular query syntax.

```
SELECT Name, Edept FROM Emp WHERE ENo = 22217 AND EStart <= DATE '2011-01-02' AND EEnd > DATE '2011-01-02'
```

A simpler way to formulate the above query would be to employ one of the period predicates provided in SQL-2011 for expressing conditions involving periods:

`CONTAINS, OVERLAPS, EQUALS, PRECEDES,SUCCEEDS, IMMEDIATELY PRECEDES,` and `IMMEDIATELY SUCCEEDS.`

SQL 2011 Standard

The query could be expressed using the CONTAINS predicate:

```
SELECT Ename, Edept FROM Emp WHERE ENo =
22217 AND EPeriod CONTAINS DATE '2011-01-02'
```

```
SELECT Ename, Edept FROM Emp WHERE ENo =
22217 AND EStart < DATE '2011-01-01' AND
EEnd > DATE '2010-01-01'
```

(the above query uses the closed-open model)

The same query using the OVERLAPS predicate:

```
SELECT Ename, Edept FROM Emp WHERE ENo =
22217 AND EPeriod OVERLAPS PERIOD (DATE
'2010-01-01', DATE '2011-01-01')
```

SQL 2011 Standard

- System-versioned tables are used in applications that maintain a history of data changes either for business reasons, legal reasons, or both. A typical example of such applications is a banking application, where it is necessary to keep previous states of customer account information
- customers can be provided with a detailed history of their accounts.

Any update or delete of a row must automatically record the old state of the row before performing the update or delete.

SQL 2011 Standard

- The **system**, rather than the user, **maintains the start and end times of the periods** of the rows, and that users be unable to modify the content of historical rows or the periods associated with any of the rows.
- Any updates to the periods of rows in a system-versioned table must be performed only by the system as a result of updates to the non-period columns of the table or as a result of row deletions.

- guarantees that the recorded history of data changes cannot be altered, which is critical to meet auditing and compliance regulations.

SQL 2011 Standard

- Any table that contains a period definition with the standard-specified name, SYSTEM_TIME, and includes the keywords WITH SYSTEM VERSIONING in its definition is a system-versioned table.
- Users can pick any name they want for the names of columns that act as the start and end columns of the SYSTEM_TIME period. Though SQL-2011 allows the data types of the period start and end columns to be either DATE or a timestamp type (as long as the data types of both columns are the same), in practice, most implementations will provide the TIMESTAMP type with the highest fractional seconds precision as the data type for the system-time period start and end columns.

SQL 2011 Standard

```
CREATE TABLE Emp(
ENo INTEGER,
Sys_start TIMESTAMP(12) GENERATED ALWAYS AS
ROW START,
Sys_end TIMESTAMP(12) GENERATED ALWAYS AS
ROW END,
EName VARCHAR(30),
PERIOD FOR SYSTEM_TIME (Sys_start, Sys_end)
) WITH SYSTEM VERSIONING
```

Similar to application-time periods, system-time periods use closed-open period mode.

SQL 2011 Standard

2) INSERT into a system-versioned table automatically sets the value of Sys_start column to the transaction timestamp, a special value associated with every transaction and sets the value of Sys_end column to the highest value of the column's data type.

3) UPDATE and DELETE on system-versioned tables only operate on current system rows. Users are not allowed to:

- update or delete historical system rows.
- modify the system-time period start or the end time of both current system rows and historical system rows.

SQL 2011 Standard

At any given point in time, a row in a system-versioned table is regarded as current system row if the system-time period of that row contains the current time. A row that is not a current system row is regarded as a historical system row.

System-versioned tables differ from application-time period tables in the following respects:

1) In contrast to the application-time period tables,users are not allowed to assign or change the values of Sys_start or Sys_end columns; they are assigned (and changed) automatically by the database system. This is the reason why the definitions of Sys_start or Sys_end columns must include the keywords GENERATED ALWAYS.

SQL 2011 Standard

4) UPDATE and DELETE on system-versioned tables result in the automatic insertion of a historical system row for every current system row that is updated or deleted. An UPDATE statement on a system-versioned table first inserts a copy of the old row with its system-time period end time set to the transaction timestamp, indicating that the row ceased to be current as of the transaction timestamp. It then updates the row while changing its system-period start time to the transaction timestamp,indicating that the updated row to be the current system row as of the transaction timestamp

SQL 2011 Standard

A DELETE statement on a system-versioned table does not actually delete the qualifying rows; instead it changes the system-time period end time of those row to the transaction timestamp, indicating that those rows are no longer current as of the transaction timestamp.

SQL 2011 Standard

Primary key and referential constraints on system-versioned tables

The definition and enforcement of constraints on system-versioned tables is considerably simpler than the definition and enforcement of constraints on application-time period tables. This is because constraints on system-versioned tables need only be enforced on the current system rows. Historical system rows in a system-versioned table form immutable snapshots of the past.

SQL 2011 Standard

Any constraints that were in effect when a historical system row was created would have already been checked when that row was a current system row, so there is never any need to enforce constraints on historical system rows. Consequently, there is no need to include the system-period start and end columns or the period name in the definition of primary key and referential constraints on system-versioned tables. For example, the following

```
ALTER TABLE Emp ADD PRIMARY KEY (ENo)
```

SQL 2011 Standard

The above constraint ensures there exists exactly one current system row with a given ENo value. Similarly, the following ALTER TABLE statement specifies a referential constraint between Emp and Dept tables:

```
ALTER TABLE Emp ADD FOREIGN KEY
(Edept)REFERENCES Dept (DNo)
```

The above constraint is again enforced only on the current system rows of Emp and Dept tables.

SQL 2011 Standard

Because system-versioned tables are intended primarily for tracking historical data changes, queries on system-versioned tables often tend to be concerned with retrieving the table content as of a given point in time or between any two given points in time. SQL-2011 provides *three syntactic extensions* for this specific purpose. These are allowed only in queries on system-versioned tables.

The first extension is the **FOR SYSTEM_TIME AS OF** syntax that is useful for querying the table content as of a specified point in time.

SQL 2011 Standard

The other two extensions are
the period specified in the (FROM ... TO ...)
corresponds to a closed-open period model while the
period specified in the (BETWEEN ... AND ...) corresponds
to a closed-closed period model.

SQL 2011 Standard

Bitemporal tables

- combine the capabilities of both system-versioned and application-time period tables.

- the user is in charge of supplying values for the application-time period start and end columns.

- INSERT sets the value of system-time period start column to the transaction timestamp, and the value of system time period end column to the highest value of the column's data type.

- the UPDATE statement as well as UPDATE with FOR PORTION OF application-period can be used to modify the rows of bitemporal tables.

SQL 2011 Standard

- the DELETE statement as well as DELETE with FOR PORTION OF application-period can be used to delete rows from bitemporal tables.

- only current rows in system-time can be updated or deleted and a historical system row is automatically inserted for every current system row that is updated or deleted.

Queries on bitemporal tables can specify predicates on both application-time periods as well as system-time periods to qualify rows that will be returned as the query result.

Elemente de DB Temporale in Produse Comerciale

Introducere

- Consideram urmatoarele: o tabela *Orders* cu definitia

```
Create table Orders(
        order_number int not null primary key,
        product_id int not null,
        qty   int not null,
        order_date date
);
```

Am dori sa aflam perioadele dintre comenzi pentru fiecare produs

- Cum am scrie o astfel de interogare?

Introducere

ORDER_NUMBER	PRODUCT_ID	QTY	ORDER_DATE
1	1000	10	10-MAY-16
2	3000	11	02-JUN-16
3	1000	20	23-JUN-16
4	2000	33	11-SEP-16
5	3000	3	12-NOV-16
6	1000	12	13-APR-17

```
SELECT product_id, order_date,
LAG (order_date,1) OVER (PARTITION BY product_id
ORDER BY order_date) AS prev_order_date
FROM orders order by 1, 2;
```

PRODUCT_ID	ORDER_DATE	PREV_ORDER_DATE
1000	10-MAY-16	(null)
1000	23-JUN-16	10-MAY-16
1000	13-APR-17	23-JUN-16
2000	11-SEP-16	(null)
3000	02-JUN-16	(null)
3000	12-NOV-16	02-JUN-16

Introducere

- Bazele de date relationale moderne, cum ar fi **Oracle**, **Microsoft SQL Server** si **PostgreSQL**, ofera facilitati avansate de gestionare a datelor temporale. Acestea sunt esentiale pentru a pastra si gestiona versiuni multiple ale datelor in timp, permitand interogarea starilor anterioare si urmarirea modificarilor

Introducere

- Valid Time poate fi relevant in scenarii de:
  - **Information Lifecycle Management** (ILM) si alte aplicatii unde este important sa se cunoasca momentul cand anumite date au devenit valide (din perspectiva aplicatiei) si cand a devenit nevalida (daca a devenit)
  - **Data correction** cand trebuie stocate date incorecte si marcata perioada cand au fost considerate valide, perioada cand datele corectate sunt vazute ca fiind valide

Oracle VT si TT

- **Valid Time** în Oracle este gestionat printr-o funcționalitate denumită **Temporal Validity**. Aceasta permite adăugarea și definirea explicită a perioadelor de valabilitate pentru înregistrările din tabele. Utilizatorii pot defini perioade **PERIOD FOR** care indică timpul valid în care datele sunt considerate active sau relevante. Aceasta oferă suport direct pentru valid time, permițând interogarea și gestionarea datelor în funcție de valabilitatea lor într-o anumită perioadă de timp.

Oracle VT si TT

- **Transaction Time** este gestionat implicit de Oracle prin funcționalitatea **Flashback Data Archive** (FDA). Aceasta păstrează toate modificările datelor în tabelele arhivate, inclusiv timpul exact în care tranzacțiile au fost realizate. Prin intermediul funcțiilor de interogare AS OF și VERSIONS BETWEEN, utilizatorii pot reveni la stările anterioare ale datelor și pot accesa istoricul complet al modificărilor, ceea ce reprezintă o gestionare automată a transaction time.

Tehnologia Flashback

- Este folosita cand apare o corupere logica in baza de date Oracle si este necesara o recuperare rapida si usoara. Cand au loc erori umane este greu de identificat ce obiecte au fost afectate de o tranzactie eronata.

Tehnologia Flashback permite sa:

- diagnosticam erorile introduse in baza de date si sa reparam defectele.
- vizualizam tranzactiile ce au contribuit la modificarea unei inregistrari
- vizualizam toate versiunile unei inregistrari dintr-o anumita perioada de timp
- vizualizam datele cum existau la un anumit moment in timp in trecut.

Flashback – Facilitati DBA

- **Flashback Database**: restaureaza intreaga baza de date la un anumit moment in timp, folosind loguri optimizate pentru Oracle flashback, (alternativa la backup si forward recovery).
- **Flashback Table**: restaurarea facila a unei tabele la un anumit moment in timp; utila cand coruperea logica afecteaza o tabela sau o multime restransa de tabele din db.
- **Flashback Drop**: restaureaza o tabela stearsa in mod accidental. Restaureaza atat tabela cat si indexurile, constringerile si triggerii folosind Recycle Bin (un container logic pentru toate obiectele sterse).
- **Flashback Transaction**: realizeaza o anulare (undo) a efectului unei singure tranzactii, si optional, a tuturor tranzactiilor dependente, folosind o singura operatie PL/SQL sau un wizard din Enterprise Manager.

Flashback – Facilitati App Dev

- **Flashback Transaction Query**: vizualizare tuturor modificarilor unei anumite tranzactii; util cand o tranzatie eronata modifica mai multe inregistrari intr-o tabela.
- **Flashback Query**: interogheaza date la un moment specific in trecut. Se poate utiliza pentru a observa si a reconstrui date corupte logic, ce au putut fi modificate sau sterse necorespunzator.
- **Flashback Versions Query**: furnizeaza diferite versiuni ale unei inregistrari in cadrul unei perioade de timp.

Tehnologia Flashback

- Flashback Database foloseste **fisiere log flashback** pentru operatia de flashback. Flashback Drop foloseste **recycle bin**. Restul tehnicilor folosesc **undo data**.
- Nu toate capabilitatile flashback modifica baza de date. Unele sunt doar metode de interogare a versiunilor datelor. Acestea reprezinta unelte de analiza a problemelor si ajuta la recuperarea la erori. Rezultatele acestor interogari ajuta la:
  - Determinarea tipului de operatie (ce modifica db-ul) flashback pentru rezolvarea problemei .
  - Furnizeaza rezultatul interogarii unei instructiuni INSERT, UPDATE, sau DELETE ce permite repararea datelor eronate.

Beneficiile Flashback

- Tehnologia Flashback este un avans revolutionar in recuparea la erori
- Tehnicile traditionale de recuperare sunt lente
  - intreaga baza de date sau un fisier trebuie restaurat, nu doar datele incorecte
  - fiecare schimbare trebuie examinata in fisierele de log

- Flashback este rapid
  - Schimbarile sunt indexate dupa inregistrare si dupa tranzactie
  - Doar datele modificate sunt restaurate

- Comenzile Flashback sunt simple
  - Proceduri ce nu presupun pasi complexi multipli

Tehnologia Flashback

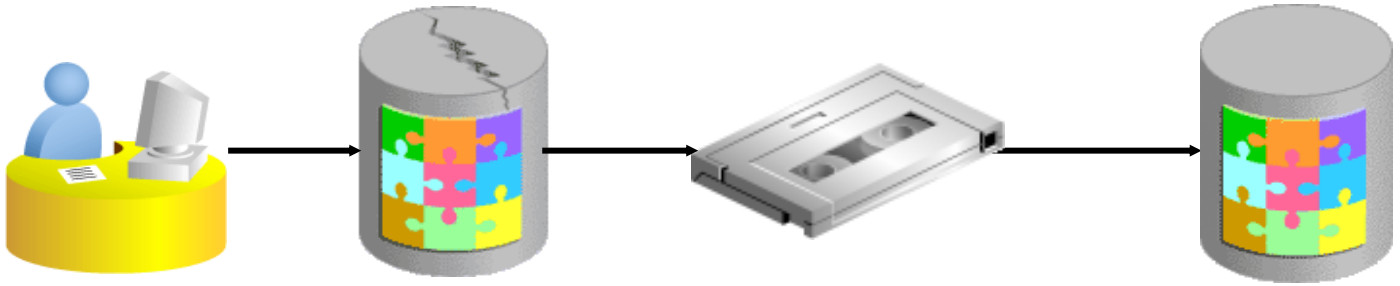
Nivelul Obiect	Scenarii exemplu	Tehnologia Flashback	Utilizare	Date afectate
Database	Truncate table; Modificari nedorite in mai multe tabele	Database	Flashback logs	TRUE
Table	Drop table	Drop	Recycle bin	TRUE
	Update cu clauza `WHERE` gresita	Table	Undo data	TRUE
	Compara date curente cu date din trecut	Query	Undo data	FALSE
	Compara versiuni ale unei inregistrari	Version	Undo data	FALSE
Tx	Investigheaza stari anterioare ale datelor	Transaction	Undo data	FALSE

Flashback Database

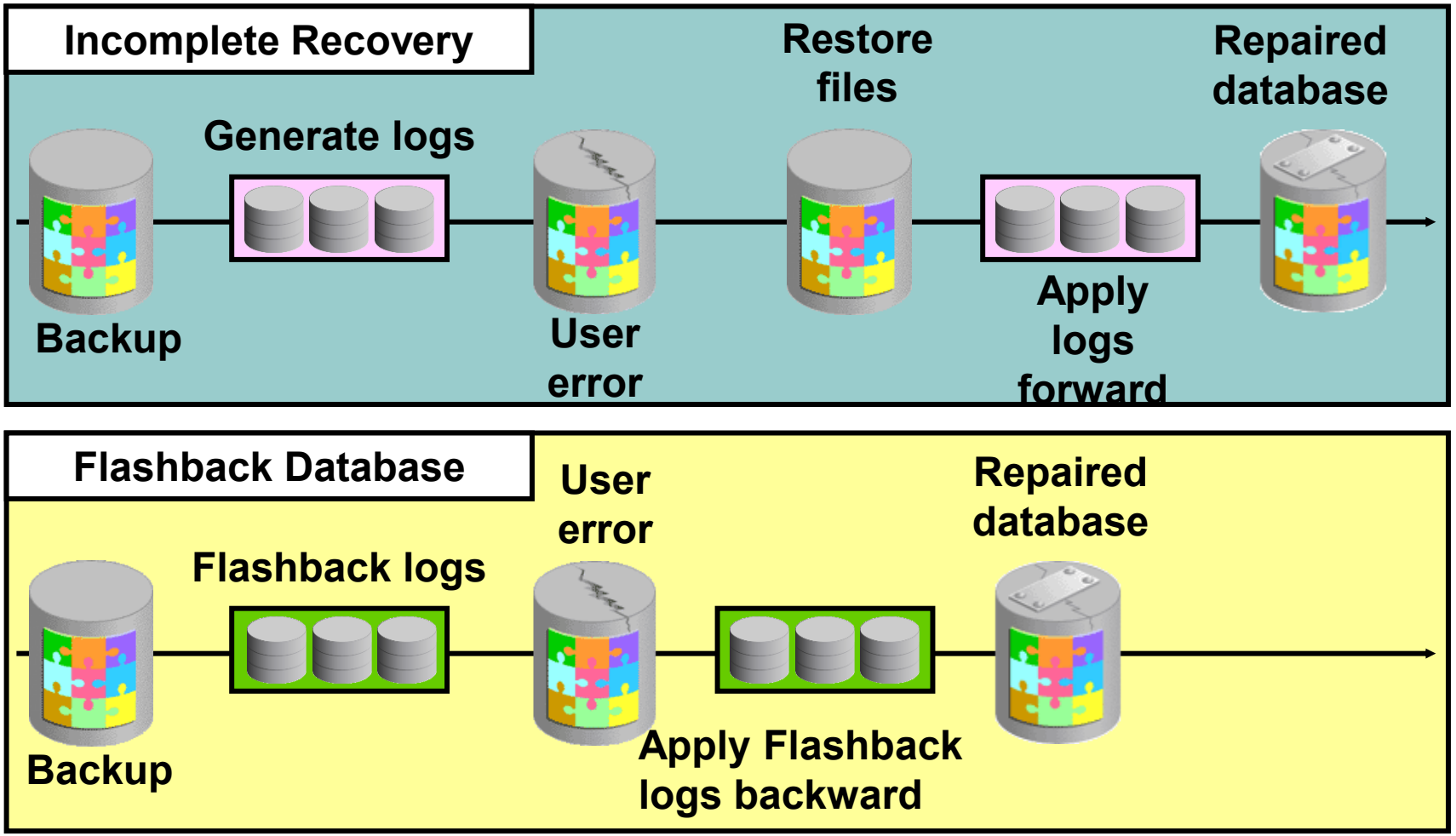
- Restaurarea facila si rapida a bazei de date la un moment anterior in timp prin anularea tuturor modificarilor ce au avut loc de la acel moment. Operatia este rapida pentru ca nu se restaureaza un backup. Se poate utiliza pentru anularea modificarilor ce au dus la coruperea logica a datelor.
- Pentru pierderi ale mediului fizic sau corupere fizica, se pot folosi doar metode traditionale de recuperare

Flashback Database

- Operatia Flashback Database:
  - Functioneaza ca un buton rewind pentru baza de date
  - Se poate folosi in cazuri de corupere logica facuta de utilizator



Flashback Database
Reduce Timpul de Restore



Configurare Flashback Database



1. Configure the flash recovery area
2. Set the retention target
3. Enable Flashback Database

```
SQL>  ALTER SYSTEM SET
            DB_FLASHBACK_RETENTION_TARGET=2880
            SCOPE=BOTH;
```

```
SQL> ALTER DATABASE FLASHBACK ON;
```

Monitorizare Flashback Database

- Ajustare dimensiune flash recovery disk quota:

```
SQL> SELECT estimated_flashback_size,
            flashback_size
     FROM   V$FLASHBACK_DATABASE_LOG;
```

- Determinare fereastra flashback curenta:

```
SQL> SELECT oldest_flashback_scn,
     oldest_flashback_time
     FROM   V$FLASHBACK_DATABASE_LOG;
```

- Monitorizare logging in Flashback Database logs:

```
SQL> SELECT *
    FROM   V$FLASHBACK_DATABASE_STAT;
```

Exemple Flashback Database

```
RMAN> FLASHBACK DATABASE TO TIME =
    > TO_DATE('2019-05-27 16:00:00',
    > 'YYYY-MM-DD HH24:MI:SS');

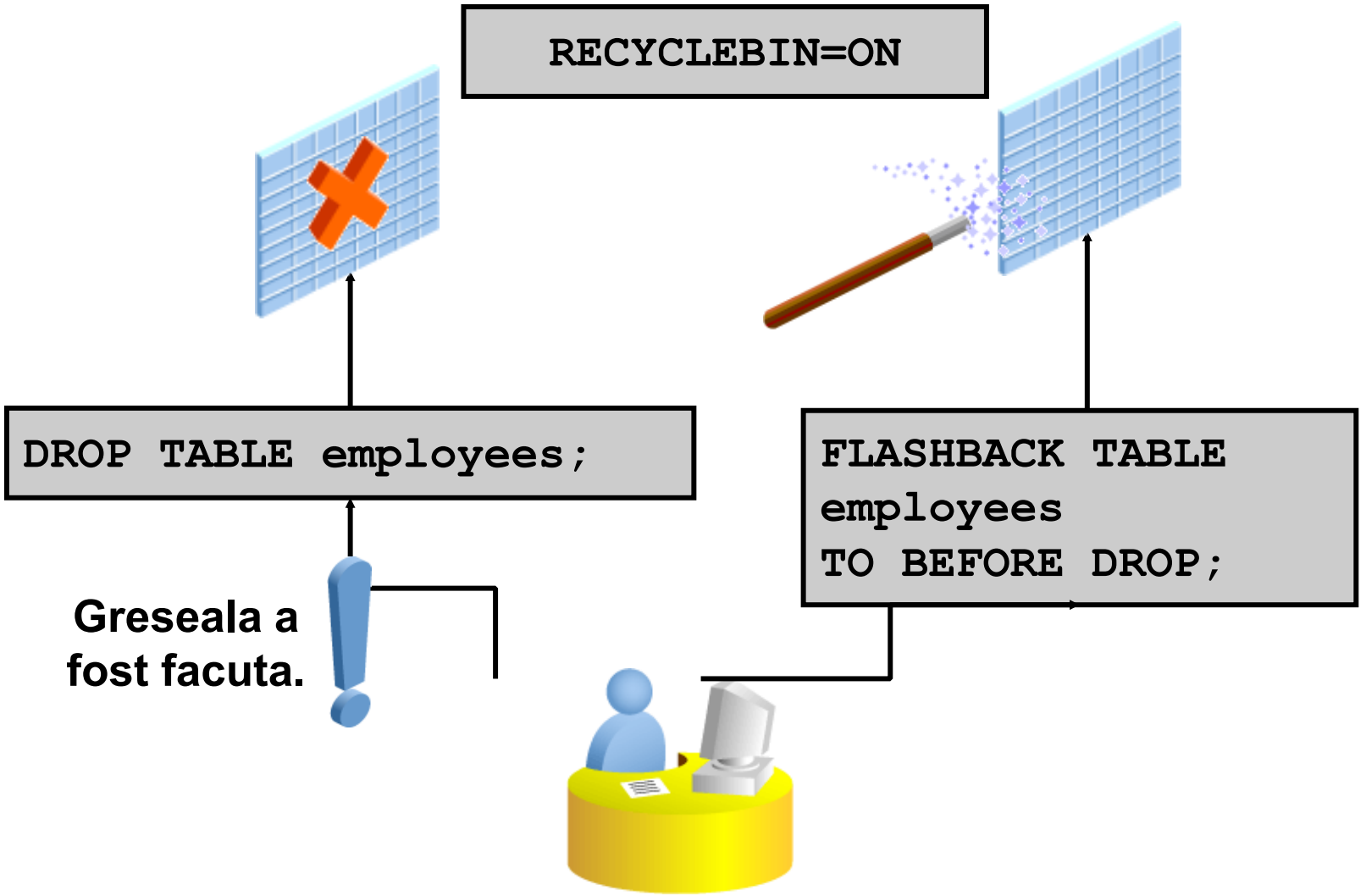
RMAN> FLASHBACK DATABASE TO SCN=23565;

RMAN> FLASHBACK DATABASE
    > TO SEQUENCE=223 THREAD=1;
```

```
SQL> FLASHBACK DATABASE
     TO TIMESTAMP(SYSDATE-1/24);

SQL> FLASHBACK DATABASE TO SCN 53943;
```

Flashback Drop si Rec. Bin



Flashback Drop

- In versiunile mai vechi ale Oracle database, daca se stergea o tabela din greseala, trebuia restaurata baza de date la un moment anterior pentru a recupera tabela. Procedura este de obicei de durata si resulta in pierderea efectelor unor tranzactii.
- In Oracle Database 10g se introduce facilitatea Flashback Drop, ce se poate utiliza pentru a anula efectul unei instructiuni DROP TABLE fara a folosi “point-in-time recovery”.
- Parametrul de initializare RECYCLEBIN este folosit pentru a configura existenta/inexistenta capabilitatii Flashback Drop prin valoarea ON, respectiv OFF. Daca este OFF, tabela stearsa nu va fi plasata in recycle bin. Daca este ON, tabela stearsa va fi plasata in recycle bin si poate fi restaurata. Implicit, parametrul RECYCLEBIN are valoare ON.

Flashback Drop

- Comanda FLASHBACK TABLE ... TO BEFORE DROP este utila pentru a restaura o tabela si toate obiectele dependente din recycle bin. Se poate specifica pentru restaurare numele original sau cel atribuit de sistem avut cand obiectul fost sters.
- Daca se specifica numele original si in recycle bin exista mai multe obiecte cu acelasi nume, obiectul cel mai recent mutat in recycle bin este restaurat primul (LIFO). Pentru o versiune anterioara se poate specifica numele generat de sistem sau se executa instructiuni FLASHBACK TABLE ... TO BEFORE DROP pana cand tabela dorita este restaurata.

Flashback Drop

- Daca in schema destinatie exista deja un obiect cu numele tabelei sterse, la restaurare se va genera eroare daca nu este specificata clauza RENAME TO.
- La operatia de flashback a unei tabele sterse, elementele atasate recuperate, indexuri, triggeri, constraingeri, vor avea numele din recycle bin. Se recomanda ca pentru obtinerea unor nume mai semnificative sa se interogheze recycle bin si DBA_CONSTRAINTS inainte de flashback pentru a redenumi aceste obiecte atasate cu nume sugestive.

Interogare Recycle Bin

```
SELECT owner, original_name, object_name,
       type, ts_name, droptime, related, space
FROM dba_recyclebin
WHERE can_undrop = 'YES';
```

```
SELECT original_name, object_name,
       type, ts_name, droptime, related, space
FROM user_recyclebin
WHERE can_undrop = 'YES';
```

```
SQL> SHOW RECYCLEBIN
```

Flashback Query

- Pentru Oracle Flashback Query se foloseste o instructiune SELECT cu o clauza **AS OF**. Oracle Flashback Query returneaza datele existente la un anumit moment in trecut. Interogarea refera in mod explicit momentul de timp cu ajutorul unui timestamp sau a unui System Change Number (SCN). Va furniza valorile ce au fost salvate (committed) la acel moment de timp.
- Utilizarea Oracle Flashback Query include:
  - Recuperarea datelor sterse sau anularea (undo) schimbarilor incorecte salvate (commited): de exemplu, din greseala s-au actualizat sau sters randuri, apoi s-a salvat, se poate imediat anula acest lucru.
  - Compararea datelor curente cu corespondentul lor din trecut.

Flashback Query

- Exemplu: se poate rula un raport zilnic ce arata toate schimbarile fata de ziua anterioara. Se pot compara inregistrari individuale ale tabelei, sau, se pot determina intersectii, reuniuni de inregistrari.
- Simplifica arhitectura aplicatiei eliminand nevoia de stocare a unor atribute temporale: Oracle Flashback Query permite furnizarea unor date din trecut direct din baza de date.
- Permite un mecanism self-service de corectie a erorilor, permitand utilizatorilor sa anuleze sau sa corecteze erorile.

Flashback Query

```
CREATE TABLE my_emp(
  empno NUMBER,
  last_name VARCHAR2(30),
  start_time TIMESTAMP,
  end_time TIMESTAMP,
PERIOD FOR user_valid_time (start_time,
end_time));

INSERT INTO my_emp VALUES (100, 'Ames', '01-Jan-
10', '30-Jun-11');
INSERT INTO my_emp VALUES (101, 'Burton', '01-
Jan-11', '30-Jun-11');
INSERT INTO my_emp VALUES (102, 'Chen', '01-Jan-
12', null);
```

Flashback Version Query

- Utilizat pentru regasirea versiunilor unei anumite inregistrari ce a existat intr-un interval de timp. O noua versiune a inregistrarii este creata atunci cand este implicata intr-o operatie de COMMIT.
- Oracle Flashback Version Query se specifica prin clauza VERSIONS BETWEEN a unei instructiuni SELECT. Sintaxa este:

```
VERSIONS {BETWEEN {SCN|TIMESTAMP} start AND end}
```

Unde start si end sunt expresii ce reprezinta inceputul si sfarsitul intervalului de interes. Intervalul include (start si end).

- Oracle Flashback Version Query furnizeaza o tabela cu o inregistrare pentru fiecare versiune a randului ce a existat pe durata intervalului specificat. Fiecare inregistrare va contine pseudocoloane (metadata) despre versiunea inregistrarii.

Flashback Version Query

VERSIONS_STARTSCN
VERSIONS_STARTTIME

System Change Number-ul (SCN) de inceput sau TIMESTAMP-ul cand a fost creata versiunea inregistrarii. Aceasta pseudocoloana indica momentul cand inregistrarea a avut valorile reflectate in versiune. Pseudocoloana se poate folosi pentru a determina timpul tinta pentru Oracle Flashback Table sau Query. Daca pseudocoloana este NULL, versiunea a fost creata inainte de timpul de start.

VERSIONS_ENDSCN
VERSIONS_ENDTIME

SCN sau TIMESTAMP-ul cand versiunea inregistrarii a expirat. Daca este NULL, fie versiunea corespunde unui DELETE, fie versiunea era curenta la momentul interogat.

Flashback Version Query

VERSIONS_XID

Identificatorul tranzactiei ce a creat versiunea inregistrarii.

VERSIONS_OPERATION

Operatia efectuata in tranzactie: **I** - insert, **D** - delete, **U** - update.
Versiunea corespunde inregistrarii dupa un INSERT, inainte de DELETE, respectiv UPDATE.
Pentru actualizarea valorii unei chei de index Oracle Flashback Version Query poate trata UPDATE ca fiind 2 operatii, DELETE plus INSERT, fiind prezente 2 versiuni, prima cu **D** uramat de **I** in VERSIONS_OPERATION.

Flashback Version Query

```
SELECT versions_startscn, versions_starttime,
       versions_endscn, versions_endtime,
       versions_xid, versions_operation,
       name, salary
  FROM employees
  VERSIONS BETWEEN TIMESTAMP
      TO_TIMESTAMP('2003-07-18 14:00:00', 'YYYY-
MM-DD HH24:MI:SS')
  AND TO_TIMESTAMP('2003-07-18 17:00:00', 'YYYY-
MM-DD HH24:MI:SS')
  WHERE name = 'JOE';
```

Flashback Transaction Query

- Utilizat pentru regasirea metadatelor si istoricului pentru o anumita tranzactie sau pentru toate tranzactiile dintr-un interval de timp. Oracle Flashback Transaction Query interogheaza view-ul dictionarului de date FLASHBACK_TRANSACTION_QUERY.
- Coloana UNDO_SQL furnizeaza codul SQL opus dpdv logic operatiei DML efectuata de tranzactie. Acest cod se poate utiliza pentru a inversa pasii logici efectuati in tranzactie. Sunt cazuri cand acest cod nu este exact opusul tranzactiei originale. Un SQL_UNDO la un DELETE va coespunde cu un INSERT ce nu neaparat va insera o inregistrare la acelasi ROWID de unde a fost sters.

Flashback Transaction Query

- Instructiunea de mai jos va interoga view-ul FLASHBACK_TRANSACTION_QUERY pentru informatii despre tranzactii, incluzand ID-ul tranzactie, operatia, SCN de start si end, utilizatorul operatiei, codul SQL opus logic operatiei:

```
SELECT xid, start_scn, commit_scn, operation,
logon_user, undo_sql
FROM flashback_transaction_query
WHERE xid = HEXTORAW('000200030000002D');
```

```
000200030000002D  195243  195244  DELETE   HR        insert into "HR"."EMP"
("EMPNO","EMPNAME","SALARY") values ('111','Mike','655');
000200030000002D  195243  195244  INSERT   HR        delete from
"HR"."DEPT" where ROWID = 'AAAKD4AABAAAJ3BAAB';
000200030000002D  195243  195244  UPDATE   HR        update "HR"."EMP" set
"SALARY" = '555' where ROWID = 'AAAKD2AABAAAJ29AAA';
000200030000002D  195243  113565  BEGIN  HR
4 rows selected
```

Tabel cu Suport Valid Time

```
-- Crearea tabelei cu Temporal Validity:
CREATE TABLE angajati (
    id NUMBER PRIMARY KEY,
    nume VARCHAR2(100),
    pozitie VARCHAR2(100),
    salariu NUMBER,
    valid_from DATE,
    valid_to DATE,
    PERIOD FOR valid_time (valid_from, valid_to)
);
```

În acest exemplu PERIOD FOR valid_time este folosit pentru a defini perioada de valabilitate a fiecărui rând. valid_from și valid_to reprezintă începutul și sfârșitul perioadei de valabilitate pentru înregistrarea respectivă.

Tabel cu Suport Valid Time

- Inserarea datelor (Create)

```
INSERT INTO angajati (id, nume, pozitie, salariu,
valid_from, valid_to)
VALUES (1, 'Maria Ionescu', 'Manager', 5000, DATE
'2023-01-01', DATE '2024-01-01’);

INSERT INTO angajati (id, nume, pozitie, salariu,
valid_from, valid_to)
VALUES (2, 'Andrei Popescu', 'Developer', 4000, DATE
'2023-05-01', DATE '2024-05-01');
```

Inserează doi angajați, fiecare cu o perioadă de valabilitate specificată.

Tabel cu Suport Valid Time

- Citirea datelor (Read)

Pentru a interoga datele valide la un anumit moment, folosim **AS OF**:

```
SELECT * FROM angajati
  AS OF PERIOD FOR valid_time DATE '2023-06-01’;
```

Returnează toate înregistrările valabile la data de 1 iunie 2023.

- Actualizarea datelor (Update)

Actualizeaza datele unui angajat și a extinde perioada de valabilitate:

```
UPDATE angajati
SET salariu = 5500, valid_to = DATE '2025-01-01'
WHERE id = 1;
```

Actualizează salariul și extinde perioada de valabilitate pentru angajatul cu ID-ul 1 până la data de 1 ianuarie 2025.

Tabel cu Suport Valid Time

- Ștergerea datelor (Delete)

Pentru a șterge o înregistrare specifică din tabelă:

```
DELETE FROM angajati WHERE id = 2;
```

Elimina înregistrarea pentru angajatul cu ID-ul 2.

- Interogare cu Valid Time pentru interval de timp

Interogăm datele valide într-un interval de timp specificat:

```
SELECT * FROM angajati
  WHERE PERIOD FOR valid_time OVERLAPS (DATE '2023-
01-01', DATE '2023-12-31');
```

Returneaza toate înregistrările valide pentru perioada specificată, adică pentru întregul an 2023.

Alte Exemple de Utilizare

- Se poate crea un view ce furnizeaza stari anterioare folosind clauza AS OF in instructiunea SELECT ce defineste view-ul. Daca se specifica o valoare relativa la o anumita instanta de timp, starea anterioara este recalculata la fiecare accesare a view-ului. De exemplu:

```
CREATE VIEW hour_ago AS
  SELECT * FROM angajati
     AS OF PERIOD FOR valid_time TIMESTAMP
(SYSTIMESTAMP - INTERVAL '60' MINUTE);
```

Alte Exemple de Utilizare

- Se poate folosi clauza AS OF in self-joins sau in operatii cum ar fi INTERSECT sau MINUS, pentru a extrage si/sau compara date la timpi diferiti. Rezultatele se pot stoca/salva prin prefixarea Oracle Flashback Query cu:
  - CREATE TABLE AS SELECT
  - INSERT INTO TABLE SELECT
- Interogarea urmatoare reinsereaza in tabela de angajati inregistrarile ce existau acum 1 ora:

```
INSERT INTO angajati
    (SELECT * FROM angajati
     AS OF PERIOD FOR valid_time TIMESTAMP
(SYSTIMESTAMP - INTERVAL '60' MINUTE)
    )
    MINUS SELECT * FROM angajati);
```

Adaugarea Suport VT

- Pentru a adauga unei tabele existente capabilitati VT fara a adauga in mod explicit coloane pentru perioada de timp, la instructiunea ALTER TABLE se poate specifica clauza ADD PERIOD FOR. Daca, de exemplu, la executia CREATE TABLE nu s-au specificat coloanele START_TIME si END_TIME, se poate apela:

```
ALTER TABLE my_emp ADD PERIOD FOR
user_valid_time;
```

- Instructiunea de mai sus adauga doua coloane ascunse tabelei MY_EMP: USER_VALID_TIME_START si USER_VALID_TIME_END. Se pot insera inregistrari ce specifica valori pentru aceste coloane, dar aceste coloane nu apar in rezultatul unei instructiuni DESCRIBE, iar in instructiunea SELECT vor fi afisate si aceste 2 coloane doar daca lista din select include explicit numele acestora.

Oracle References

- http://www.oracle.com/webfolder/technetwork/tutorials/obe/db/12c/r1/ilm/temporal/temporal.html
- https://docs.oracle.com/cd/B28359_01/appdev.111/b28424/adfns_flashback.htm

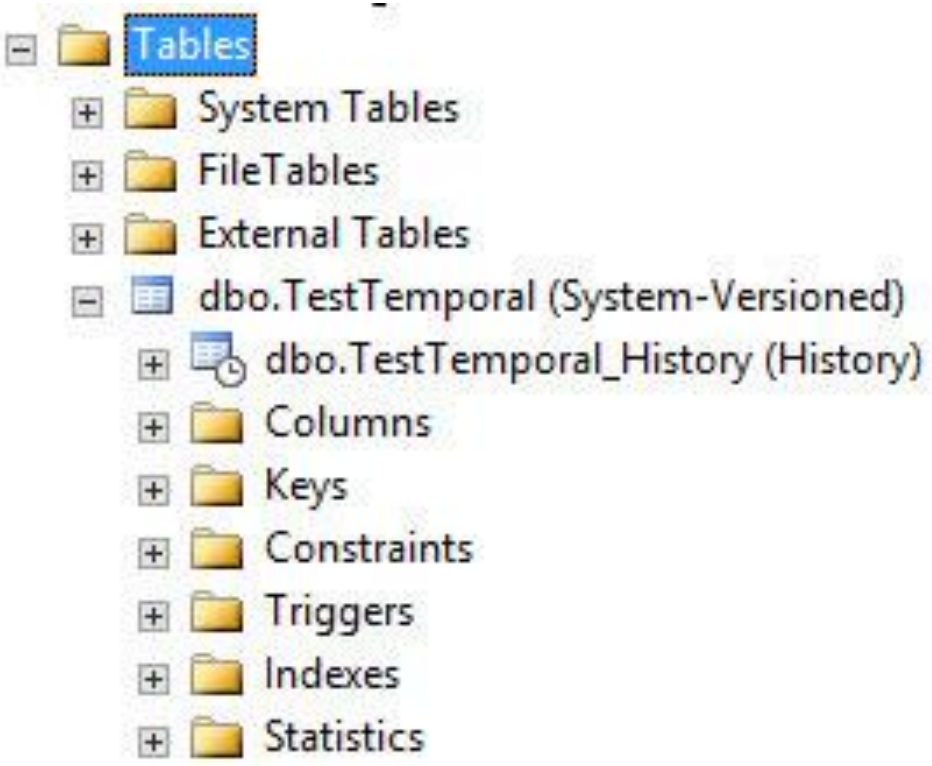
Tabele Temporale in MS Sql Server

Introducere

SQL Server include suport pentru date temporale prin System-Versioned Temporal Tables, introdusă în 2016.

- System-Versioned Temporal Tables: permit urmărirea automată a istoricului modificărilor înregistrărilor. Se folosesc două tabele: una pentru datele curente și alta pentru datele istorice.
- Fiecare înregistrare are două coloane suplimentare, SysStartTime și SysEndTime, care indică perioada în care un rând a fost valabil.
- Interogarea datelor istorice: folosind comanda FOR SYSTEM_TIME, se pot interoga datele într-un anumit interval de timp (AS OF, FROM TO, BETWEEN, etc.).
- SQL Server gestionează automat istoria modificărilor, ceea ce face ca acest mecanism să fie foarte eficient pentru urmărirea datelor pe termen lung.

Introducere



Introducere

- Tabelele temporale (tabele versionate sistem) se mai numesc tabele de asertie, (*assertion* tables), ele capturand durata de viata a unei inregistrari pe baza datelor fizice cand inregistrarile au fost actualizate sau sterse. Pentru moment nu suporta versionare bazata pe date logice.
- Exemplu: se da o tabela ce stocheaza preturi de produse; daca se actualizeaza pretul la 12PM cu o instructiune UPDATE, tabela temporala va inregistra istoricul pretului vechi pana la 12PM in acea zi. Incepand cu 12PM, noul pret este valid. Dar daca s-ar fi dorit o schimbare incepand cu 1PM (schimbare logica)? Ar trebui sincronizat perfect momentul pentru instructiunea de UPDATE la 1PM in loc de 12PM.

Crearea Tabelei Versionate Sistem

- Pentru o tabela temporala nou creata se aplica:
  - Trebuie definita o cheie primara
  - Trebuie definite 2 coloane pentru start si end date cu tipul de data datetime2. Daca este necesar, ele pot fi marcate cu flagul HIDDEN. Aceste coloane vor fi numite coloane pentru perioada SYSTEM_TIME.
  - Nu sunt permisi declansatori (trigger) INSTEAD OF.
  - Nu se poate utiliza In-memory OLTP
- Exista unele limitari:
  - Tabela temporala si cea de istoric nu pot fi FILETABLE
  - Tabela de istoric nu poate avea constrangeri
  - Instructiunile INSERT si UPDATE nu pot referi coloanele perioadei SYSTEM_TIME
  - Datele din tabela de istoric nu pot fi modificate

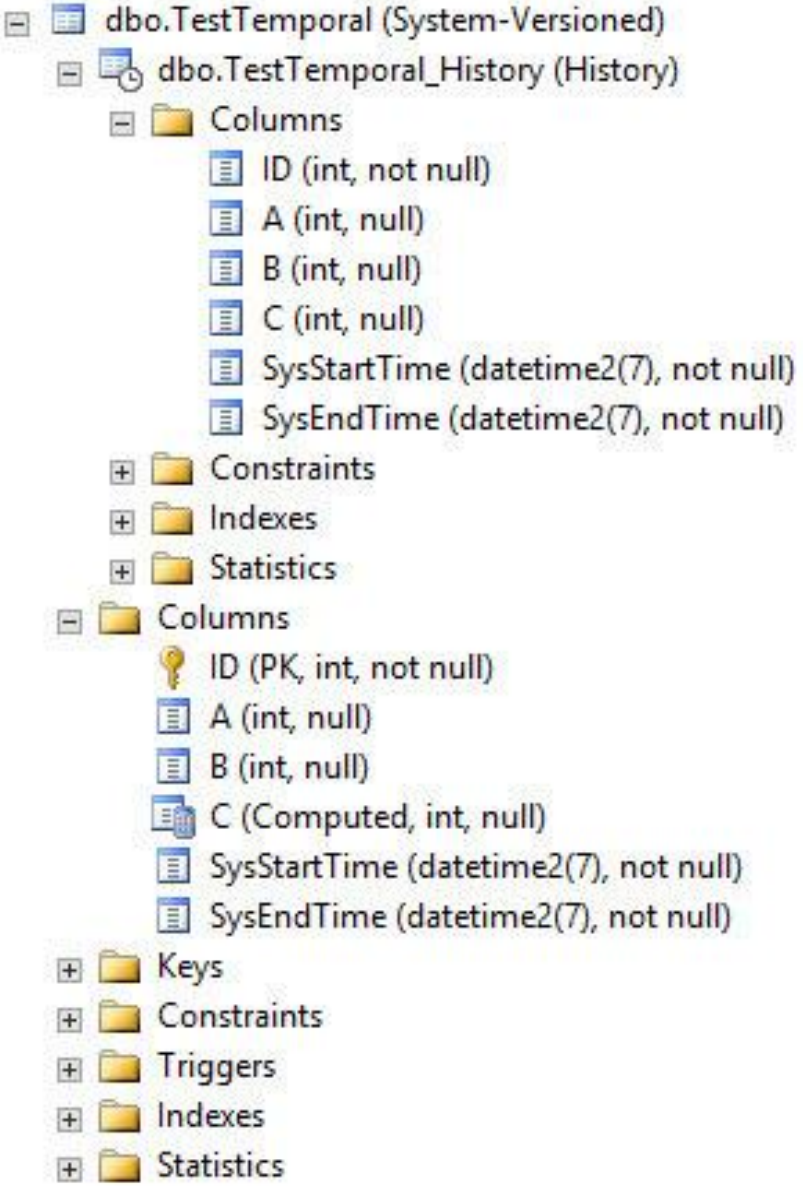
Crearea Tabelei Versionate Sistem

```
CREATE TABLE dbo.TestTemporal
     (ID int primary key
     ,A int
     ,B int
     ,C AS A * B
     ,SysStartTime datetime2 GENERATED ALWAYS AS ROW
START NOT NULL
     ,SysEndTime datetime2 GENERATED ALWAYS AS ROW
END NOT NULL
     ,PERIOD FOR SYSTEM_TIME
(SysStartTime,SysEndTime)) WITH(SYSTEM_VERSIONING =
ON (HISTORY_TABLE=dbo.TestTemporal_History));
```

- Daca nu se specifica un nume pentru tabela de istoric, SQL Server va genera automat unul cu structura: *dbo.MSSQL_TemporalHistoryFor_xxx*, unde *xxx* este id-ul obiect al tabelei principale.

Crearea Tabelei Versionate Sistem

- Tabela de istoric are o multime identica de coloane cu cea de baza, dar fara constrangeri. Are multimea proprie de indexuri si statistici. Crearea unor indexuri clustered poate creste performanta simtitor.



Operatii cu Tabele Versionate Sistem

```
-- Initial Load
INSERT INTO dbo.TestTemporal(ID, A, B, C)
VALUES      (1,2,3,6)
      ,(2,4,5,20)
      ,(3,0,1,0);

SELECT * FROM dbo.TestTemporal;
```

Results | Messages

	ID	A	B	C	SysStartTime	SysEndTime
1	1	2	3	6	2015-07-07 06:20:35.3951761	9999-12-31 23:59:59.9999999
2	2	4	5	20	2015-07-07 06:20:35.3951761	9999-12-31 23:59:59.9999999
3	3	0	1	0	2015-07-07 06:20:35.3951761	9999-12-31 23:59:59.9999999

Operatii cu Tabele Versionate Sistem

```
-- Modify Data
DELETE FROM dbo.TestTemporal
WHERE ID = 2;

UPDATE dbo.TestTemporal
SET A = 5
WHERE ID = 3;
```

```
SELECT * FROM dbo.TestTemporal;
```

100 %

Results | Messages

	ID	A	B	C	SysStartTime	SysEndTime
1	1	2	3	6	2015-07-07 06:20:35.3951761	9999-12-31 23:59:59.9999999
2	3	5	1	5	2015-07-07 06:23:27.7719520	9999-12-31 23:59:59.9999999

```
SELECT * FROM [dbo].[TestTemporal_History];
```

100 %

Results | Messages

	ID	A	B	C	SysStartTime	SysEndTime
1	2	4	5	20	2015-07-07 06:20:35.3951761	2015-07-07 06:23:27.7719520
2	3	0	1	0	2015-07-07 06:20:35.3951761	2015-07-07 06:23:27.7719520

Schimbare Tabela - Versionare Sistem

- SQL Server permite conversia unei tabele existente la o tabela temporala. Pentru aceasta operatie este necesara existenta unei chei primare (daca nu exista se creaza). Se adauga tabelei 2 coloane de tip datetime2 si optiunea GENERATED ALWAYS AS ROW START/END se aplica cu PERIOD FOR SYSTEM_TIME (nameFrom, nameTo).
- Ambele optiuni se executa prin intermediul comenzii ALTER TABLE. A doua comanda ALTER stabileste proprietatea SYSTEM_VERSIONING, furnizand optional un nume pentru proprietatea HISTORY_TABLE.

Schimbare Tabela - Versionare Sistem

```
CREATE TABLE Department_Exist (
      DeptID int NOT NULL PRIMARY KEY CLUSTERED,
      DeptName varchar(50) NOT NULL,
      ManagerID INT  NULL,
      ParentDeptID int NULL )
GO
ALTER TABLE dbo.Department_Exist
ADD SysStartTime datetime2 GENERATED ALWAYS AS ROW START
CONSTRAINT DF_Department_Exist_SysStartTime DEFAULT
SYSUTCDATETIME() NOT NULL,
SysEndTime datetime2 GENERATED ALWAYS AS ROW END
CONSTRAINT DF_Department_Exist_SysEndTime DEFAULT CONVERT
(DATETIME2, '9999-12-31 23:59:59') NOT NULL,
PERIOD FOR SYSTEM_TIME (SysStartTime,SysEndTime)
GO
ALTER TABLE dbo.Department_Exist
      SET (SYSTEM_VERSIONING = ON (HISTORY_TABLE =
dbo.Department_ExistHistory))
GO
```

Resources

https://docs.microsoft.com/en-us/sql/relational-databases/tables/temporal-tables?view=sql-server-ver15
https://www.sqlshack.com/performing-a-sql-server-audit-using-system-versioned-temporal-tables/

PostgreSQL

PostgreSQL nu are suport nativ pentru tabele temporale la nivelul SQL Server sau Oracle, dar oferă posibilitatea implementării acestora folosind extensii și anumite tehnici la nivel de aplicație.

- **Extensia Temporal Tables**: extensie care poate fi adăugată pentru a implementa gestionarea automată a versiunilor istorice ale datelor, similar cu tabelele temporale din SQL Server. Odată activată, permite păstrarea automată a înregistrărilor vechi într-o tabelă de istoric și interogarea acestora folosind funcții specifice.
- Utilizarea manuală a perioadelor de validitate: PostgreSQL permite utilizatorilor să implementeze manual gestionarea datelor temporale, prin adăugarea coloanelor valid_from și valid_to pentru a marca perioadele de validitate ale datelor, împreună cu trigger-uri pentru gestionarea actualizărilor și a ștergerilor.

PostgreSQL

1. Instalarea extensiei temporal_tables

Trebuie instalata și activata extensia pentru a adăuga suport pentru tabele temporale în PostgreSQL.

```
CREATE EXTENSION IF NOT EXISTS temporal_tables;
```

Aceasta instalează extensia necesară pentru gestionarea automată a tabelelor cu versiuni istorice.

PostgreSQL

2. Crearea unei tabele cu suport pentru istoricul datelor

Exemplu de tabela pentru a păstra informații despre angajați:

```
CREATE TABLE angajati (
    id SERIAL PRIMARY KEY,
    nume VARCHAR(100),
    pozitie VARCHAR(100),
    salariu INTEGER
);
```

Activarea gestionarii automate a istoricului pentru această tabelă, folosind extensia temporal_tables.

```
SELECT create_history_table('angajati');
```

Creează automat o tabelă de istoric asociată cu tabelul angajati și păstrează toate modificările făcute asupra datelor.

PostgreSQL

3. Inserarea datelor (Create)

Inserăm câteva date în tabelul angajati:

```
INSERT INTO angajati (nume, pozitie, salariu)
VALUES ('Maria Ionescu', 'Manager', 5000),
       ('Andrei Popescu', 'Developer', 4000);
```

4. Actualizarea datelor (Update)

La actualizarea datele în tabelul principal, modificările anterioare sunt automat salvate în tabelul de istoric.

```
UPDATE angajati
SET salariu = 5500
WHERE nume = 'Maria Ionescu';
```

După actualizare valoarea anterioară a salariului pentru Maria Ionescu va fi stocată automat în tabelul de istoric creat anterior.

PostgreSQL

5. Interogarea datelor din istoric

Pentru a interoga datele istorice se foloseste tabelul de istoric:

```
SELECT * FROM angajati_history;
```

Returnează toate modificările istorice ale tabelei angajati, inclusiv versiunile anterioare ale rândurilor.

6. Ștergerea datelor (Delete)

La fel ca în cazul operațiilor de actualizare, ștergerile sunt păstrate în istoricul tabelei:

```
DELETE FROM angajati WHERE nume = 'Andrei Popescu';
```

După ștergerea lui Andrei Popescu, versiunea anterioară a acestuia va fi păstrată în tabelul de istoric angajati_history.

PostgreSQL

7. Vizualizarea unui anumit moment din istoric

Pentru vizualizarea datelor la un anumit moment, se poate folosi o interogare pentru a extrage versiunea din tabelul de istoric:

```
SELECT * FROM angajati_history
WHERE start_time <= '2023-01-01 00:00:00' AND
end_time > '2023-01-01 00:00:00';
```

Permite accesarea stărilor anterioare ale tabelei la un moment specific în timp.

PostgreSQL

https://wiki.postgresql.org/wiki/Temporal_Extensions

https://github.com/arkhipov/temporal_tables

https://github.com/xocolatl/periods

Cloud Databases



Course 2-2 – Cloud Intro

What Is the Traditional Approach?

- For individuals and small companies: develop a monolithic app, contact a hosting company, rent a server, install and configure relevant tools, copy the app with FTP, run the app.

- For large corporations: Discuss the budget, approve the budget, build a data center, go over the budget, hire people to manage the data center, deploy apps, upgrade data center.

Cloud Intro

What does Cloud mean?

In simple terms, cloud computing means on-demand availability of various computing resources. Typical benefits of using the cloud include:

- Flexibility
- Cost (savings)
- Speed
- Security
- Performance
- Reliability
- ...

History

First public offering came from a large company (Amazon) as a result of over-provisioning (to handle holiday sales) in 2006.

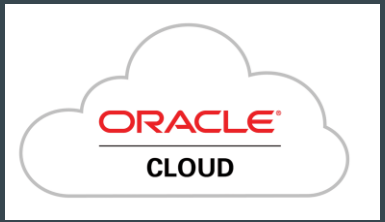
In 2008, Google launched App Engine, one of the first public PaaS solutions.

Main Providers







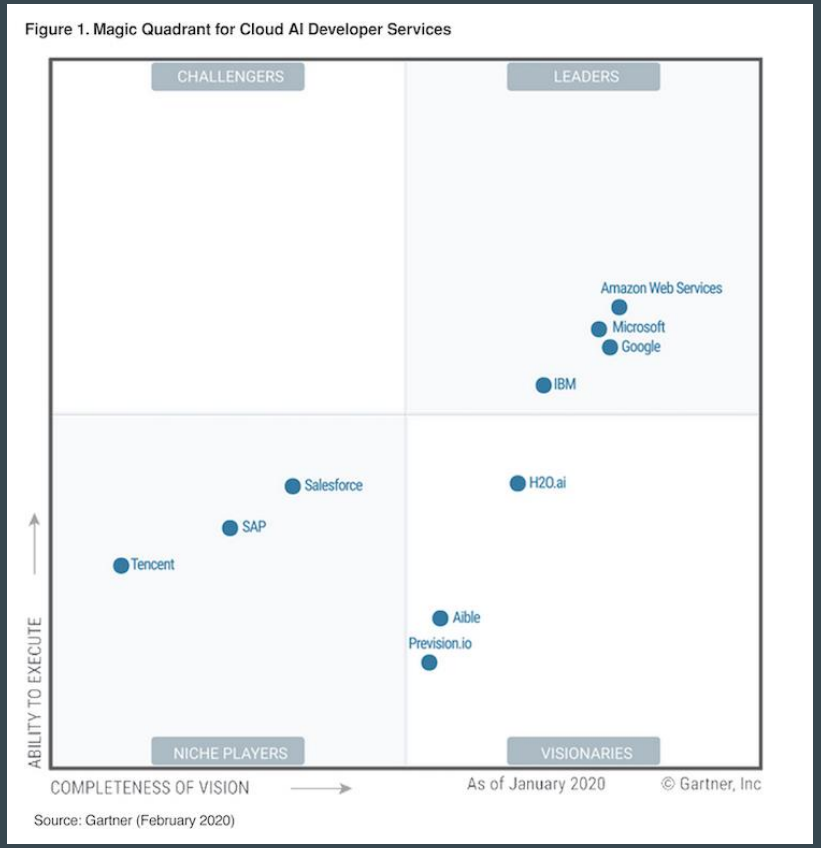






Gartner Magic Quadrant





Cloud Classification

Deployment Models:

- **Public**
- Private
- Community
- Hybrid

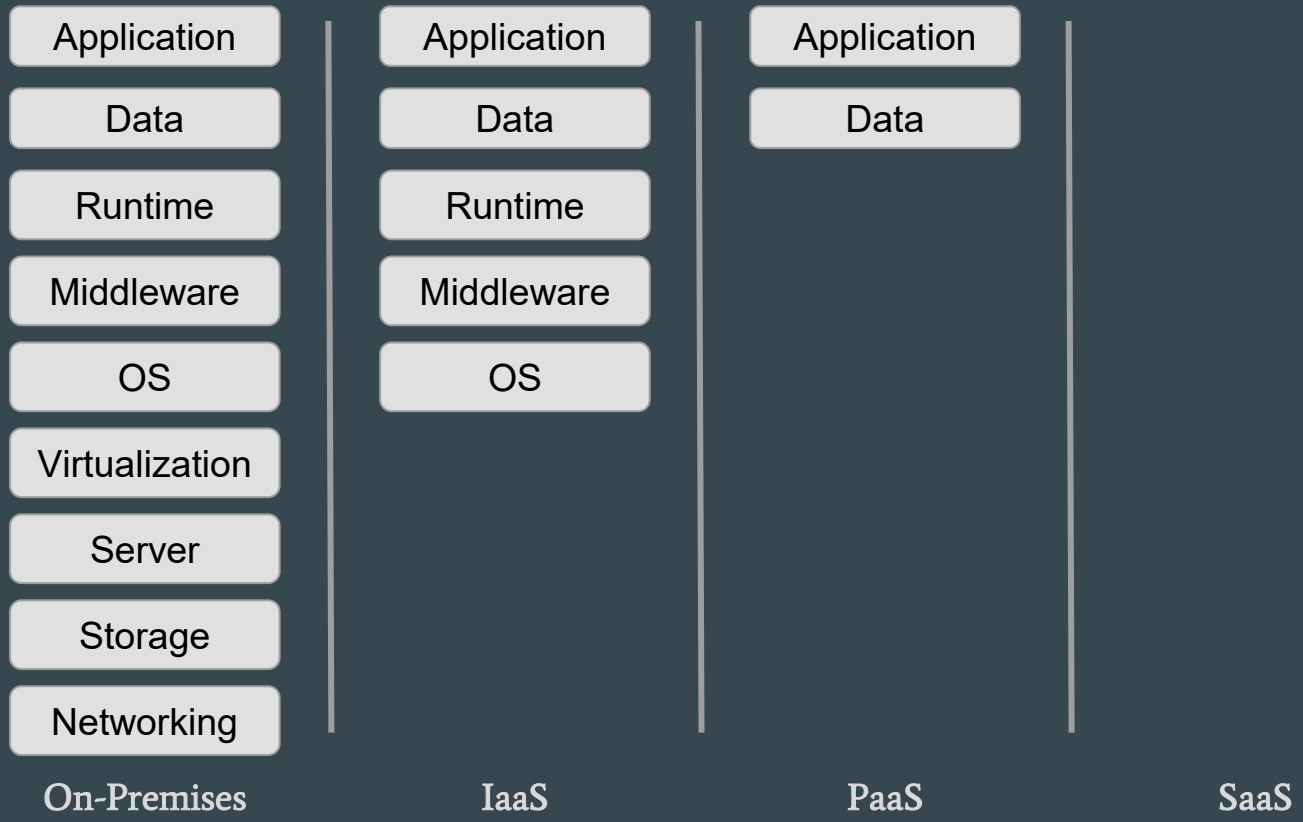
Service Models:

- **IaaS (infrastructure** as a service**)**
- **PaaS (platform** as a service**)**
- **SaaS (software** as a service**)**

Many *aaS nowadays: BaaS, DBaaS, FaaS (most of them are PaaS)

The NIST Model

IaaS, PaaS, SaaS - What We Manage



Examples for Each Service Model

IaaS

Usually virtual machine services. Other service models are built on top of them.

E.g. Amazon EC2, Google's GCE etc.

PaaS

Services that let you develop your solution without worrying about how the system runs.

E.g. Cloudfoundry, Google App Engine, Azure App Service, managed databases (RDS, CloudSQL etc.)

SaaS

No implementation, you just use the provided functionality. Interact with them through UI or API.

E.g. Google Suite, Office365, Stripe, Dropbox, Mailgun

Working with the Cloud

1. Create an account
2. Setup billing
    a. Most providers offer decent free tiers
3. Configure and launch resources
    a. From the web console
    b. Using the CLI
    c. Infrastructure **a**s **C**ode
4. *Stop/terminate them when not used
    a. Usually billing stops when stopping the resources

Standards/Compliance

Assure customers best practices are followed

- CSA (Cloud Security Alliance) - Best practices for cloud security
- ISO 9001 - Global Quality Standard
- ISO 27001 - Information Security Management
- ISO 27017 - 27001 in the context of cloud
- ISO 27018 - Personal data protection in cloud
- PCI DSS - Card payments security
- HIPAA - Health information protection

"Running on complaint infrastructure makes our products also compliant”

Pricing Models

Most services are billed based on the actual usage:

- **Time based**: pay per second/hour etc of the service being online
  - Usually applies to IaaS and PaaS offerings
- **Usage based**: pay per processing unit (request, transaction, read, write etc)
  - Mostly for PaaS/SaaS

Prices can be **fixed**, **tiered** and/or **dynamic**.

There is also the **subscription**-based model: agree to pay each month a certain price and have access to the agreed services. Usually applies to SaaS offerings and some providers - e.g. SAP offers both

Pricing Models - Time vs Usage

Time Based

Usually applies to virtual machines, (relational) databases and general PaaS services (e.g. Cloudfoundry).

You pay the same for any amount of traffic.

Easy to control costs.

Usage Based

Usually applies to fully managed services (FaaS, DBaaS - usually NoSQL) and storage services.

You pay based on the traffic.

You cannot throttle the usage without stopping the service.

Some services offer both - e.g. Azure API Management

Pricing Models - Fixed, Tiered, Dynamic

Fixed

You pay the same no matter what. Usually applies to on-demand virtual machines

Instance	vCPU(s)	RAM	Temporary storage	Pay as you go
B1S	1	1 GiB	4 GiB	€0.0135/hour
B1MS	1	2 GiB	4 GiB	€0.0234/hour
B2S	2	4 GiB	8 GiB	€0.0473/hour

Azure Windows Virtual Machines Pricing

Tiered

You pay less per unit the more you use it. Usually applies to storage services and bandwidth cost.

	Storage pricing
S3 Standard - General purpose storage for any type of data, typically used for frequently accessed data	
First 50 TB / Month	$0.023 per GB
Next 450 TB / Month	$0.022 per GB
Over 500 TB / Month	$0.021 per GB

Dynamic

Usually based on supply and demand. More spare capacity, smaller prices.

	Linux/UNIX Usage	Windows Usage
General Purpose - Current Generation		
a1.medium	$0.0084 per Hour	N/A*
a1.large	$0.0234 per Hour	N/A*
a1.xlarge	$0.0336 per Hour	N/A*
a1.2xlarge	$0.0672 per Hour	N/A*
a1.4xlarge	$0.1343 per Hour	N/A*
a1.metal	$0.13	
t2.micro	$0.00	
t2.small	$0.00	

	Linux/UNIX Usage	Windows Usage
General Purpose - Current Generation		
a1.medium	$0.005 per Hour	N/A*
a1.large	$0.0098 per Hour	N/A*
a1.xlarge	$0.0197 per Hour	N/A*
a1.2xlarge	$0.0394 per Hour	N/A*
a1.4xlarge	$0.0788 per Hour	N/A*
a1.metal	$0.0788 per Hour	N/A*
t2.micro	$0.0035 per Hour	$0.0081 per Hour
t2.small	$0.0069 per Hour	$0.0159 per Hour

Pricing Models - Discounts

Credits

Most providers offer **sign-up** credits, **educational** credits and **start-up** credits. You can also earn them from various contests or partnerships.

They also provide always free service tiers.

Reservation

If you know you will be using a certain service for a long time, you can commit to it and get a considerable price cut.

E.g. AWS Reserved Instances and Savings Plans can reduce the cost by 30 to 70%

Sustained Use

Some providers will automatically apply discounts to your bill if you use certain resources continuously.

E.g. Google Cloud sustained use discounts for GCE and App Engine (30% if the instance is running the entire month)

Guidelines & Principles

Architecture Frameworks

AWS Well-Architected

Google Architecture Framework

Azure Well-Architected

Operational Excellence

- Automation, Monitoring, Testing

Security

- IAM, Defence in depth, Traceability

Reliability

- Monitoring, Scaling, Testing

Performance

- Scaling, Choosing right, Design

Cost Optimization

- Choosing right, scaling, Understanding the services and offers

The 12-Factor App

Link

Codebase

Dependencies

Config

Backing Services

Build, release, run

Processes

Port Binding

Concurrency

Disposability

Dev/Prod Parity

Logs

Admin Processes

Pragmatic video about it

Relevant Links

1. List with engineering blogs from top companies (most relevant: Netflix, Uber, Atlassian)
2. AWS re:Invent 2019 Playlist
3. Google Cloud Next `19 Playlist
4. Microsoft Developer Youtube Channel
5. Last Week in AWS

Summary

1. Why you would care
2. Involvement is important
3. The cloud and how it compares to on-premises
4. Providers
5. Cloud classification
6. Standards
7. Pricing models
8. Principles



Course 3

Cloud Databases

What is a Scalable System?

Match the Required Load



How do we handle traffic spikes? (e.g. Black Friday)



How do we handle gradual growth of the product?



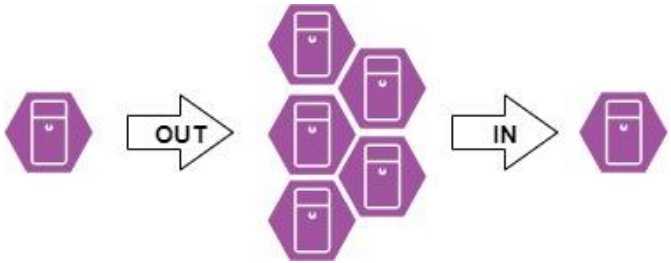
How can we minimize cost if our system is used only during certain timeframes?

Types of Scalability

Vertical
(Up/Down)



Horizontal
(In/Out)



Comparison

Vertical

- Usually easier/straightforward to implement (update the VM).
- Compatible with any app architecture (e.g. stateful apps).
- Doesn’t require additional components.
- Usually requires restarting the service.
- Might be the only approach for certain components (e.g. RDBMS)

Horizontal

- Infinite scale.
- Requires stateless apps (promotes better architectures).
- Requires load balancers (not always).
- Does not touch already existing VMs (i.e. no downtime) (unless when scaling in).

Context

We will focus on IaaS mostly (i.e. working with VMs).

While vertical scaling can definitely help, we will focus on horizontal scaling.

- Possibly unlimited scale
- Doesn’t require restart
- More interesting to talk about

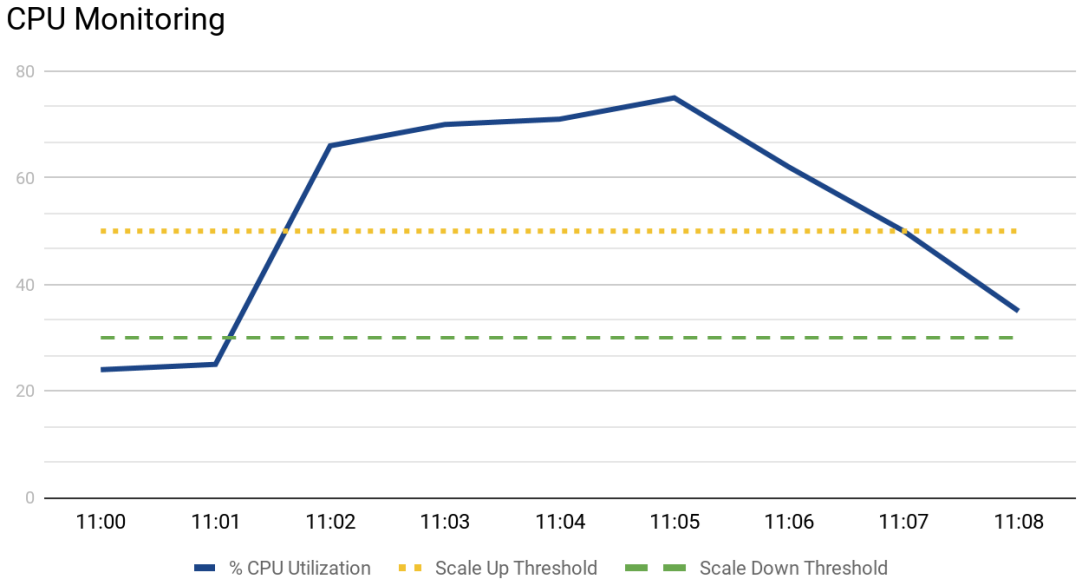
Scaling Strategies - Dynamic

Scale based on various **metrics** provided by the monitoring software:

- CPU, # of requests, memory, etc.

Choose the right metric for your system.

E.g. for a chat app, # of connections might be more relevant than CPU utilization.



Scaling Strategies - Dynamic

Aim to keep the load under 50% to have a buffer until additional VMs are ready

- Monitoring delay
- Scaling takes place only after sustained load.
  - E.g. scale up only if the load was above the threshold for 2 minutes
- Resource provisioning/initialization/setup
- The load balancer might take some time acknowledge the new VM
  - E.g. waits for the health check to pass at least 3 times

Scaling Strategies - Scheduled

Useful for predictable load patterns such as:

- Low usage at night
- High usage from Wednesday until Friday
- Events

Should be used when possible since it avoids scaling delays

- A.k.a. Resources are pre-warmed
- Providers can also pre-warm certain managed resources
  - E.g. managed load balancers

Traffic Routing

How do we route traffic when working with horizontal scaling?

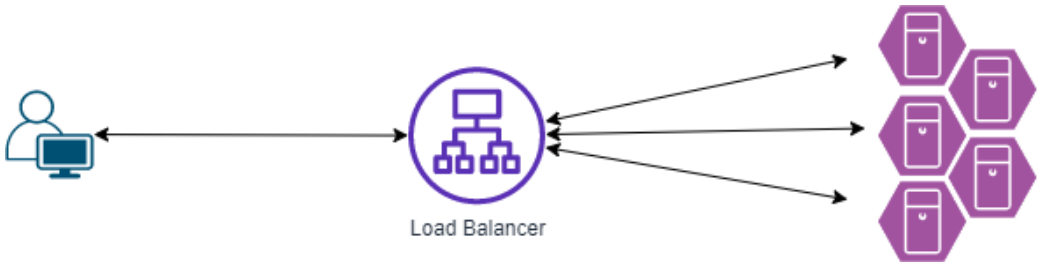
Multiple approaches:

- Have a load balancer in front of the VMs
  - I.e. a single point of entry
- DNS load balancing
  - I.e. rely on the DNS service to redirect traffic to VMs
- Make the app aware of scaling
  - Rare, usually used for scaling databases

Load Balancers

Serve multiple purposes:

- Balance traffic across multiple targets
- Monitor the health of target resources
- Serve as a single point of entry
- (Some) Caching
- Others such as SSL offloading, authentication, A/B testing, etc.



(Some) Routing Strategies

Round-Robin

- Simplest, route traffic in a loop. Good enough for simple use cases

Weighted Round-Robin

- Prefer certain targets more than others (useful when target vary in sizes)

Least Connections

- Useful when requests vary in load

Application Layer Scalability

Stateless Applications

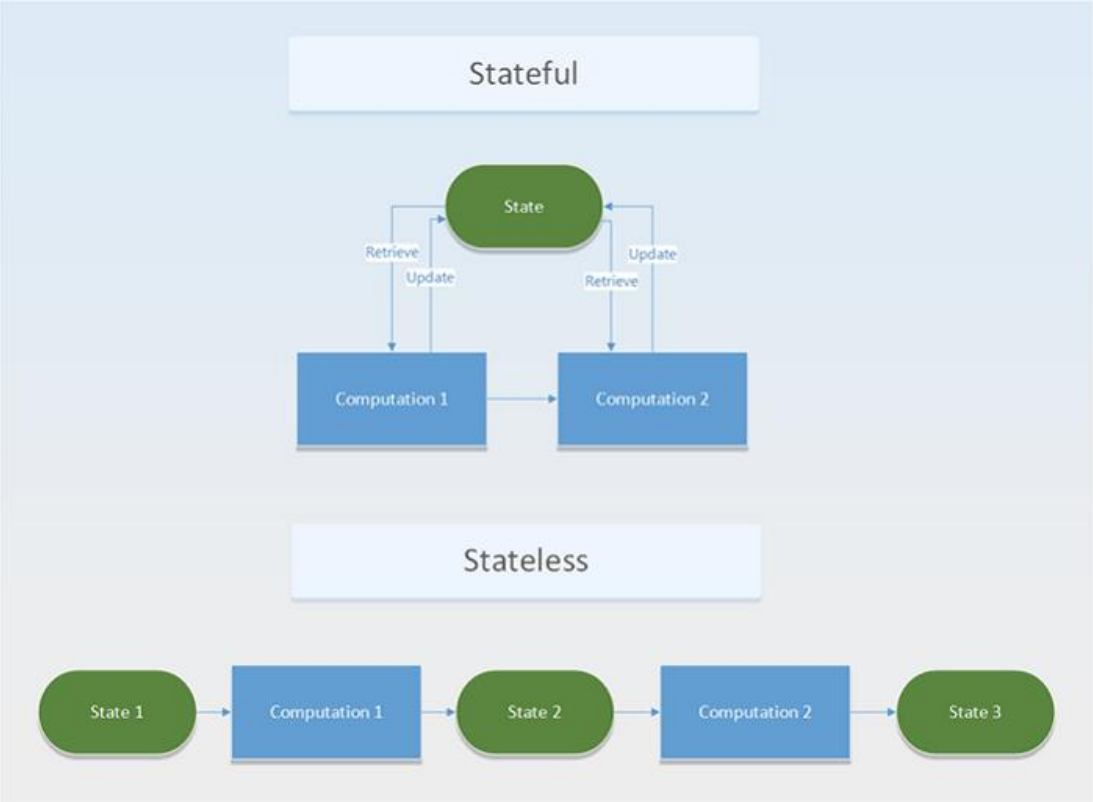
Applications were traditionally stateful (e.g. authentication sessions/shopping carts handled by the application layer).



Stateless applications handle requests independently.

- They don’t rely on previous requests
- The entire context is deduced based on the current request).

Stateless Applications



source

Pets vs Cattle



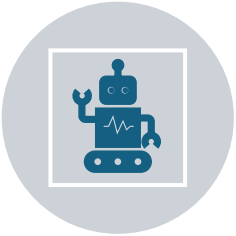
Rather cruel, but good analogy.



Pets are hand-built servers that we cannot just get rid of.



Cattle, on the other hand, can easily be replaced.



Automation is the main point.

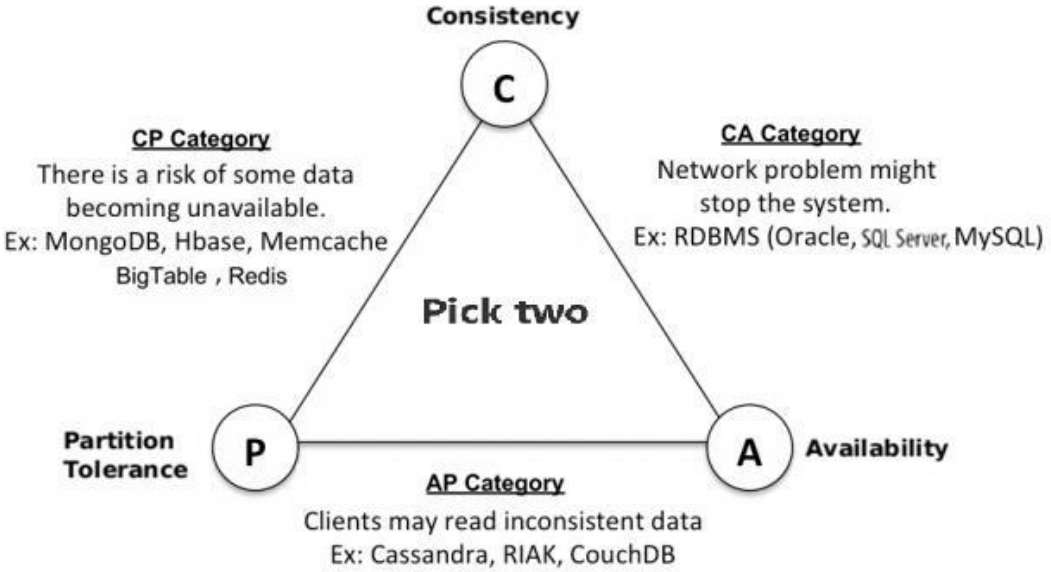
Database Layer Scalability

Context

NoSQL scale much easier than SQL databases (their main selling point).

We will cover different DB types later.

The CAP Theorem



Consistency Types

Strong consistency

- When data is changed, every client sees the new data immediately.

Eventual consistency

- Some clients might see the new data with some delay
- ACID vs BASE

Certain DBs can be tuned to favor different consistency types.

Check out the Cosmos DB consistency levels

Scaling for Reads



Nodes dedicated just for reading.



Still a single node for writing.



Data replication is usually asynchronous (i.e. leading to eventual consistency).



Clients must be aware of it - i.e. read from nodes dedicated for reading.

Partitioning



Split the data into multiple subsets/groups/**partitions.**



Each item in a given partition has the same partition key.



Doesn’t imply multiple databases/nodes.



Still **improves performance as queries are usually executed within one partition**.



Queries should contain the partition key otherwise the DB engine has to look through all partitions.

Sharding

Similar to partitioning, but actually involves distributing the data across multiple nodes.

Based on sharding key.

Not well supported/implemented in RDBMS.

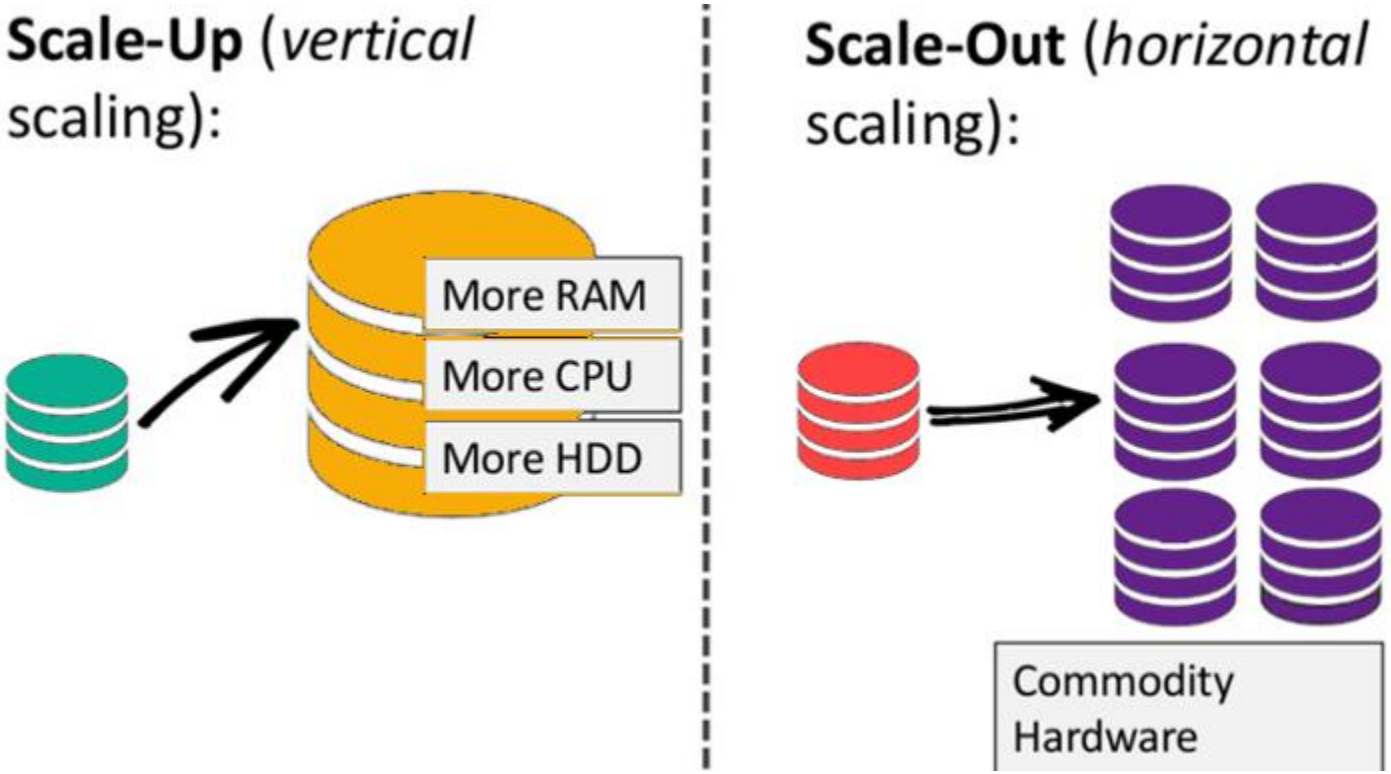
RDBMS vs NoSQL

Course 4

(Traditional) Workloads

- OLTP:
  - The most used (at least at school)
  - Based on transactions
  - Known access patterns with predefined schema
  - Write-heavy
- OLAP:
  - Used for data warehouses
  - Access patterns not known entirely (Ad-hoc)
  - Read heavy
  - Large data volumes
  - Possibly long operation (minutes, hours)

Scaling Strategies



RDBMS vs NoSQL

- typical story cycle in IT: every new technology destroys and replaces an older one; PCs replaced mini-computers. Smartphones unseated cameras and flip phones (they are coming back :-D). Online streaming wipes out video rental and music CDs.
- Are big data technologies wiping out relational database management systems?

NoSQL

- Stands for Not Only SQL
- Non-relational data storage systems
- Usually do not require a fixed table schema
- The concept of joins is eliminated
- All NoSQL offerings relax one or more of the ACID properties
- Meta S. Brown: <<if you need to get the data to deliver precise answers, then you've got to use a relational database. If you need an approximate answer in a big hurry, then a NoSQL database is the way to go. >>

NoSQL – The Beginning

- Three major papers were the seeds of the NoSQL movement
  - BigTable on top of GFS(Google)
  - Dynamo (Amazon)
    - Gossip protocol (discovery and error detection)
    - Distributed key-value data store
    - Eventual consistency
  - CAP Theorem

GFS

Google has reexamined traditional choices /assumptions and explored radically different points in the design space:

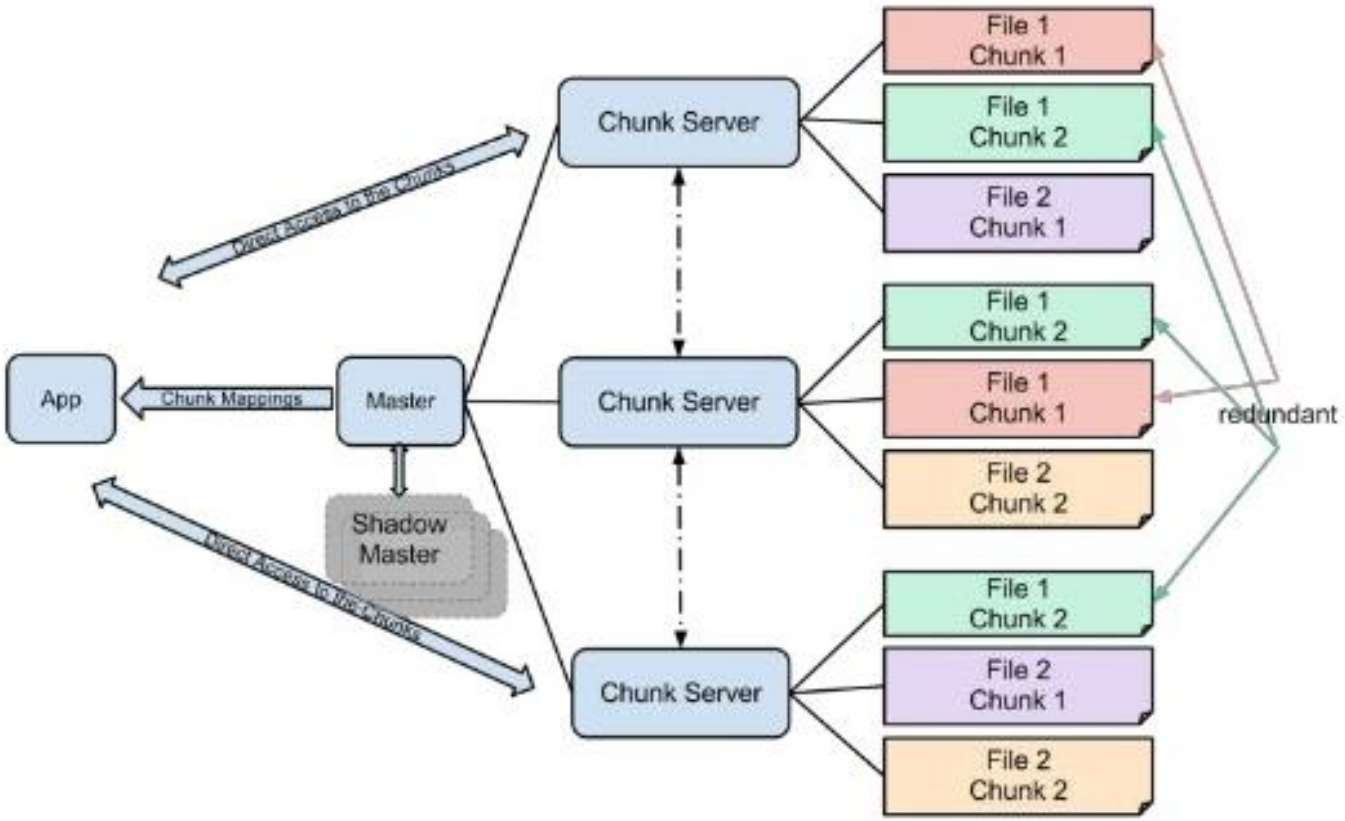
1. First, **component failures** are the norm rather than the exception
   - The system is built from many inexpensive commodity components that often fail. It must constantly monitor itself and detect, tolerate, and recover promptly from component failures on a routine basis
2. Second, **files are huge** by traditional standards. Multi-GB files are common
3. Third, most files are **mutated by appending new data** rather than overwriting existing data
4. Fourth, **co-designing the applications and the file system API** benefits the overall system by increasing flexibility

GFS Architecture

- GFS is a cluster of computers. A cluster is simply a network of computers. Each cluster might contain hundreds or even thousands of machines. In each GFS cluster there are three main entities:

1. Clients
2. Master servers
3. Chunk servers.

GFS Architecture



Source: wikipedia

GFS Architecture

- Master Server is the coordinator for the cluster. Its task include:

1. Maintaining an operation log, that keeps track of the activities of the cluster. The operation log helps keep service interruptions to a minimum if the master server crashes, a replacement server that has monitored the operation log can take its place.
2. The master server also keeps track of metadata, which is the information that describes chunks. The metadata tells the master server to which files the chunks belong and where they fit within the overall file.

GFS Architecture

- Client can be other computers or computer applications that make a file request. Requests can range from retrieving and manipulating existing files to creating new files on the system.
- Chunk Servers store 64-MB file chunks. The chunk servers don't send chunks to the master server. Instead, they send requested chunks directly to the client. The GFS copies every chunk multiple times and stores it on different chunk server. Each copy is called a replica. By default, the GFS makes three replicas per chunk, but users can change the setting and make more or fewer replicas if desired.

Big Table

- Is a large scale, fault tolerant, self managing system that includes terabytes of memory and petabytes of storage. It can handle millions of reads/writes per second.
- Is a distributed hash mechanism built on top of GFS. It is not a relational database. It doesn't support joins or SQL type queries.
- Provides lookup mechanism to access structured data by key.
- Machines can be added and deleted while the system is running and the whole system just works.
- Each data item is stored in a cell which can be accessed using a row key, column key, or timestamp (**wide column store**).
- Has three different types of servers: (Master, Tablet ,Lock Servers)

Big Table

- Is a sparsely populated table that can scale to billions of rows and thousands of columns, allowing you to store terabytes or even petabytes of data.
- A single value in each row is indexed; this value is known as the row key.
- Is ideal for storing very large amounts of single-keyed data with very low latency.
- It supports high read and write throughput at low latency

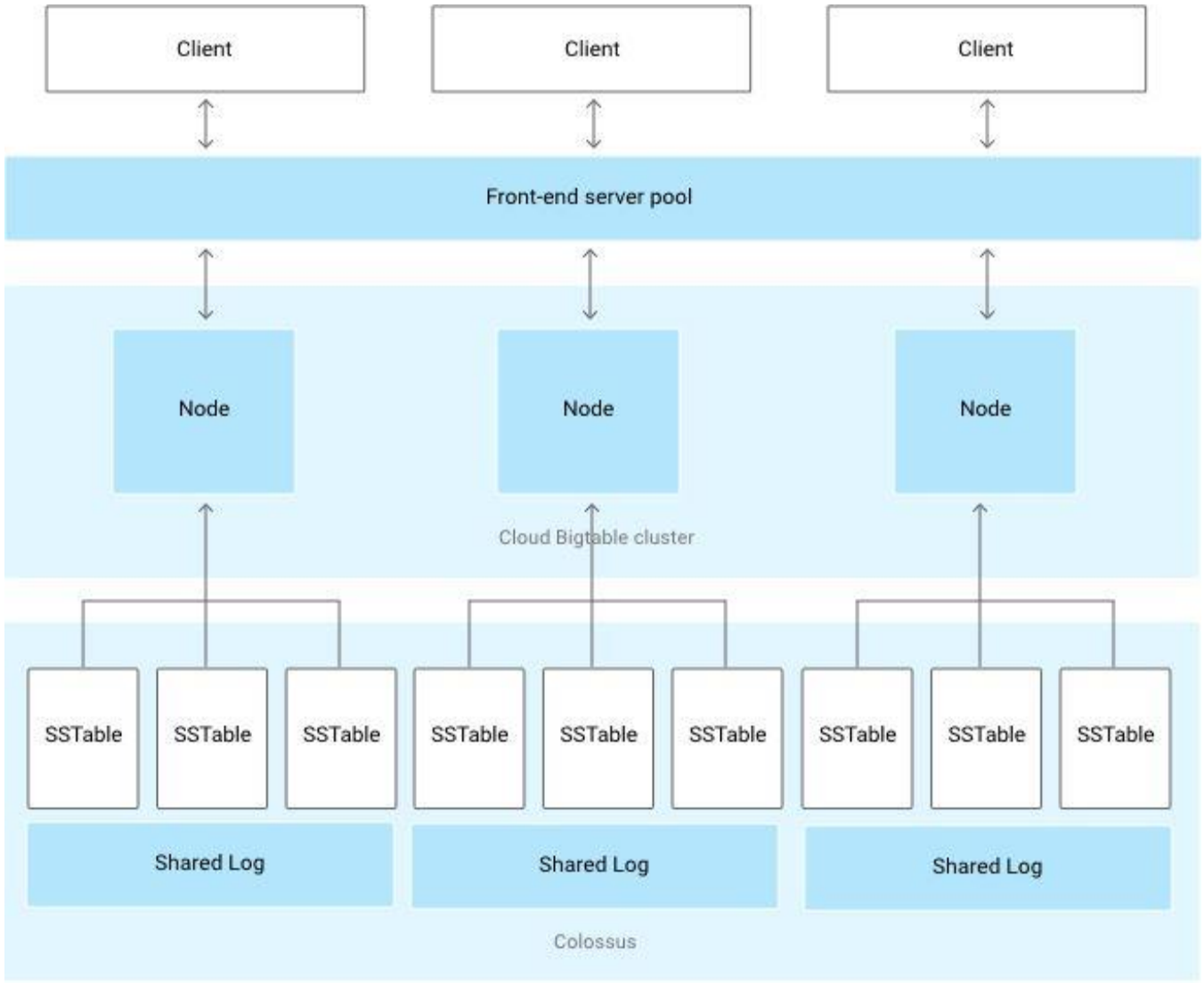
Big Table

Row Key	Column Family: profile	Column Family: activity
user001	name: John email: john@example.com	last_login: 2025-10-19 pages_viewed: 12
user002	name: Alice email: alice@example.com	last_login: 2025-10-18 pages_viewed: 8
user003	name: Bob email: bob@example.com	last_login: 2025-10-20 pages_viewed: 15
user004	name: Carol email: carol@example.com	last_login: 2025-10-17 pages_viewed: 5
user005	name: Dave email: dave@example.com	last_login: 2025-10-20 pages_viewed: 20

Big Table

- Stores data in massively scalable tables, each of which is a sorted key/value map. The table is composed of rows, each of which typically describes a single entity, and columns, which contain individual values for each row.
- Each row is indexed by a single row key, and columns that are related to one another are typically grouped together into a column family. Each column is identified by a combination of the column family and a column qualifier, which is a unique name within the column family.
- Each row/column intersection can contain multiple cells at different timestamps, providing a record of how the stored data has been altered over time. Bigtable tables are sparse; if a cell does not contain any data, it does not take up any space.

Big Table



Tablet servers

Tablets (blocks of rows)

Filesystem

Source: cloud.google.com/bigtable/

CAP Theorem

- Three properties of a system: consistency, availability and partitions
- You can have at most two of these three properties for any shared-data system
- To scale out, you have to partition. That leaves either consistency or availability to choose from
  - In almost all cases, probably is best to choose availability over consistency

Availability

- Traditionally, thought of as the server/process available five 9's (99.999 %).
- However, for large node system, at almost any point in time there's a good chance that a node is either down or there is a network disruption among the nodes.
  - Want a system that is resilient in the face of network disruption

Consistency

- A consistency model determines rules for visibility and apparent order of updates.
- For example:
  - Row R is replicated on nodes N1 and N2
  - Client C1 writes row R to node N2
  - Some period of time t elapses
  - Client C2 reads row R from node N1
  - Does client C2 see the write from client C1?
  - Consistency is a continuum with tradeoffs
  - For NoSQL, the answer would be: **maybe**
  - CAP Theorem states: Strict Consistency can't be achieved at the same time as availability and partition-tolerance.

ACID vs BASE

- While RDBMS uses ACID (Atomicity, Consistency, Isolation, Durability) as a mechanism for ensuring the consistency of data, non-relational DBMS use BASE.
- BASE stands for Basically Available, Soft state, and Eventual Consistency. Of these, eventual consistency is most important because it is responsible for conflict resolution when data is in motion between nodes in a distributed implementation.
- The data state is maintained by the software and the access model relies on basic availability.

Eventual Consistency

- When no updates occur for a long period of time, eventually all updates will propagate through the system and all the nodes will be consistent
- For a given accepted update and a given node, eventually either the update reaches the node or the node is removed from service

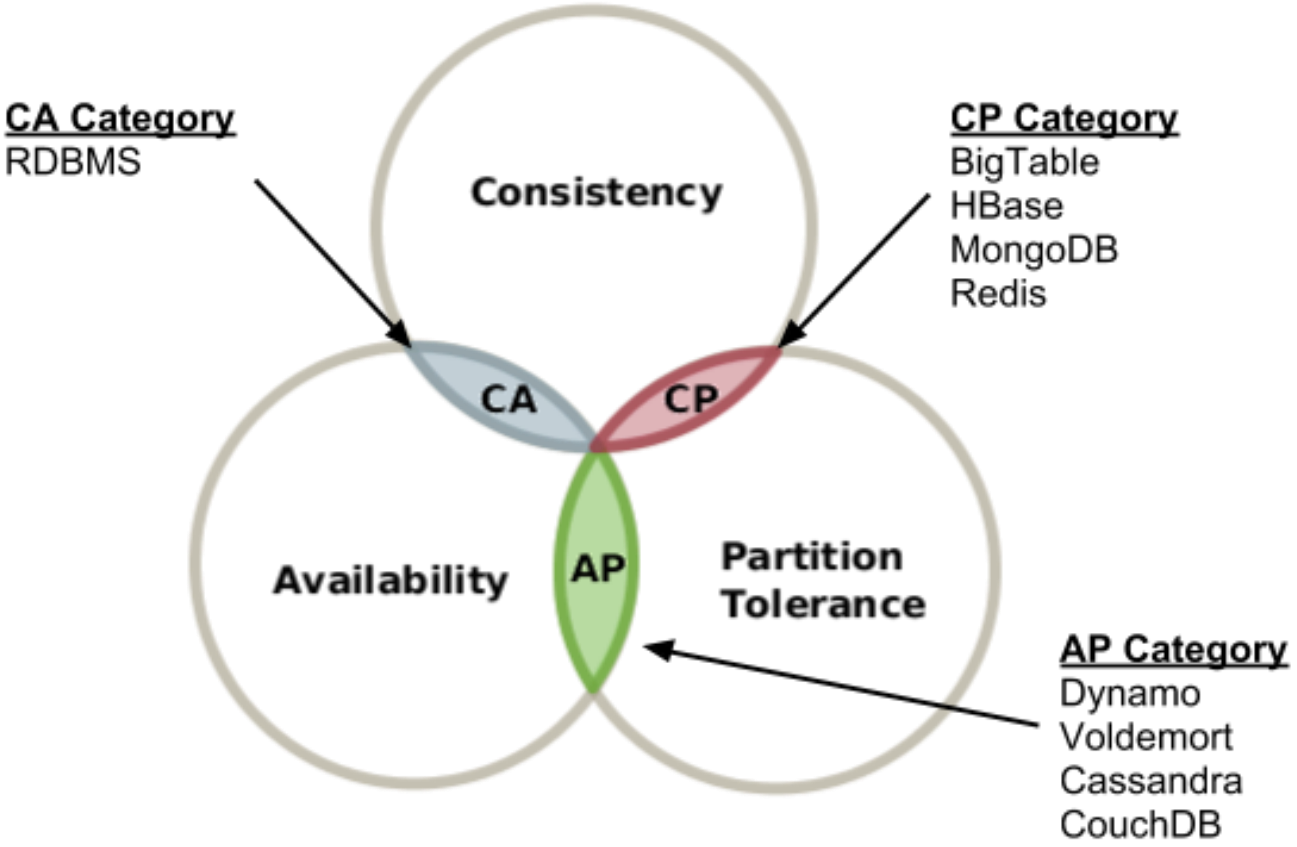
ACID vs BASE

- ACID:
  - Strong consistency.
  - Less availability.
  - Pessimistic concurrency.
  - Complex.
- BASE:
  - Availability is the most important thing (CAP).
  - Weaker consistency (Eventual).
  - Best effort.
  - Simple and fast.
  - Optimistic.

CAP

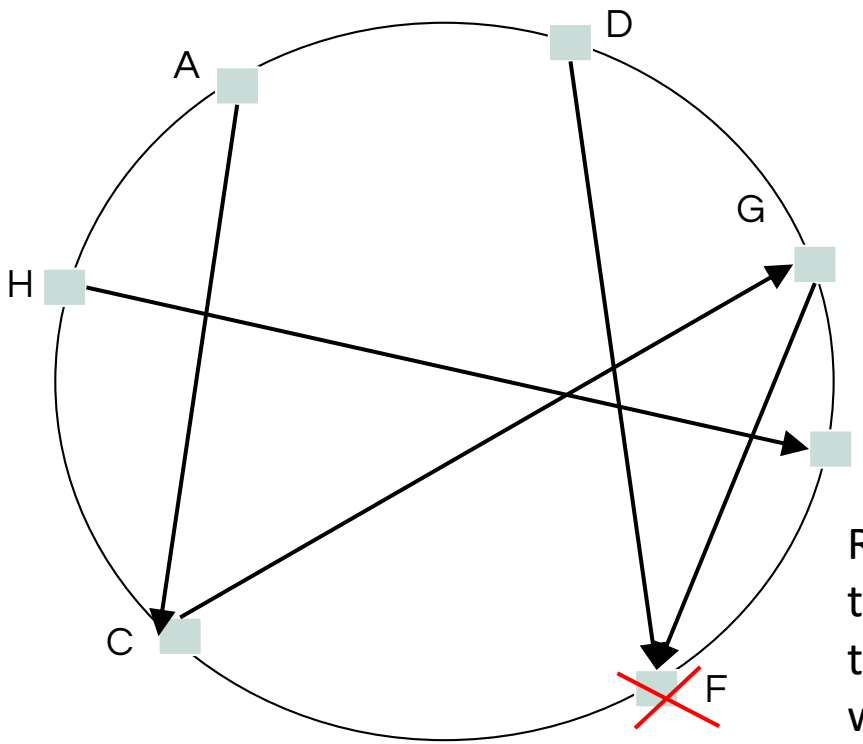
- **C**onsistency: Clients should read the same data. There are many levels of consistency.
  - Strict Consistency – RDBMS.
  - Tunable Consistency – Cassandra.
  - Eventual Consistency – Amazon Dynamo.
- **A**vailability: Data to be available.
- **P**artition Tolerance: Data to be partitioned across network segments due to network failures.

CAP


CA Category
RDBMS
Consistency
CP Category
BigTable
HBase
MongoDB
Redis
CA
CP
Availability
AP
Partition
Tolerance
AP Category
Dynamo
Voldemort
Cassandra
CouchDB

Gossip Protocol

One of the preferred communication protocols in a distributed environment is Gossip Protocol.

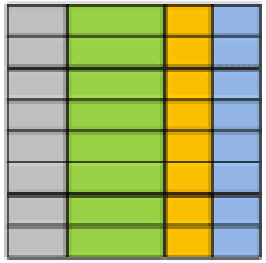


- All the nodes talk to each other peer wise.
- There is no global state.
- No single point of coordination.
- If one node goes down and there is a Quorum, load for that node is shared among others.
- Self managing system.
- If a new node joins, load is also distributed.

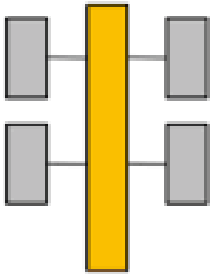
Requests coming to F will be handled by the nodes who take the load of F, let's say C with the **hint** that it took the requests which were for F, when F becomes available, F will get this Information from C. Self healing property.

Storage Models

Relational



Analytical (OLAP)



Column-Family



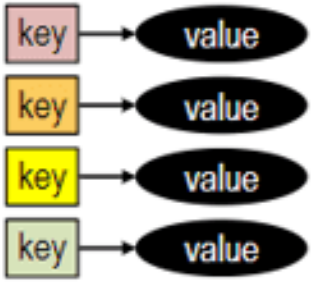
Graph



Document



Key-Value



Storage Models

- Key-value stores:
  - Processing a constant stream of small reads and writes.
- Document databases:
  - Natural data modeling. Programmer friendly. Rapid development. Web friendly, CRUD.
- *RDMBS*:
  - OLTP. SQL. Transactions. Relations.
- Columnar:
  - Handles size well. Massive write loads.
  - High availability. Multiple-data centers.
  - MapReduce
- Graph:
  - Graph algorithms and relations.

Key-Value

- The database stores data as a collection of key/value pairs. This is a simple method of storing data, and it is known to scale well.
- The key-value pair is a well established concept in many programming languages, as an associative array or data structure. A **key-value is** also commonly referred to as a **dictionary or hash**.
- The key in a key-value pair must be unique. This is the unique identifier that allows you to access the value associated with that key.

Key-Value

- The value in a key-value store can be anything, from text (long or short), a number, markup code such as HTML, programming code, an image, etc.
- The value could also be a list, or even another key-value pair encapsulated in an object.
- Some key-store DBMSs allow you to specify a data type for the value. For example, you could specify that the value should be an integer. Other DBMSs don't provide this functionality so, the value could be of any type.
- They provide **simple operations: get, put and delete**. The data querying has to be handled at the application level.

Key-Value: example

- DynamoDB:
  - stores data as items in tables, where each item is uniquely identified by a primary key. This key can be:
    - Partition Key (Hash Key): A single attribute that uniquely identifies each item.
    - Composite Key (Partition Key + Sort Key): Allows multiple items with the same partition key but different sort keys, enabling range queries.
  - Each item is essentially a key-value pair, where:
    - The key is the primary key.
    - The value is a flexible set of attributes (like a JSON document).

Partition Key (`UserID`)	Attributes (Value)
`user123`	`{ "theme": "dark", "language": "en" }`
`user456`	`{ "theme": "light", "language": "fr" }`
`user789`	`{ "theme": "dark", "language": "es" }`

Key-Value

Key Features That Enhance Key-Value Behavior:

- **Flexible schema**: Each item can have different attributes. No need to define columns ahead of time.
- **Fast lookups**: Retrieve items directly using the key with single-digit millisecond latency.
- **Secondary indexes**: You can define alternate keys (Global or Local Secondary Indexes) to query by other attributes.
- **Conditional writes**: Ensures atomic updates and prevents race conditions.
- **TTL** (Time to Live): Automatically delete items after a set time, useful for caching.

Document

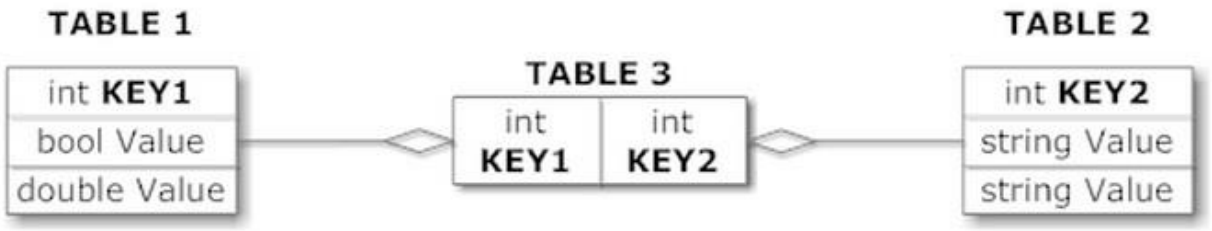
- A document-oriented database is a designed for storing, retrieving, and managing document-oriented, or **semi structured data**. The concept of a document-oriented database is the notion of a Document. While each document-oriented database implementation differs on the details of this definition, in general, they all assume documents encapsulate and encode data (or information) in some standard format or encoding. Encodings used include XML, YAML, JSON and BSON, as well as binary forms like PDF and Microsoft Office documents

Document

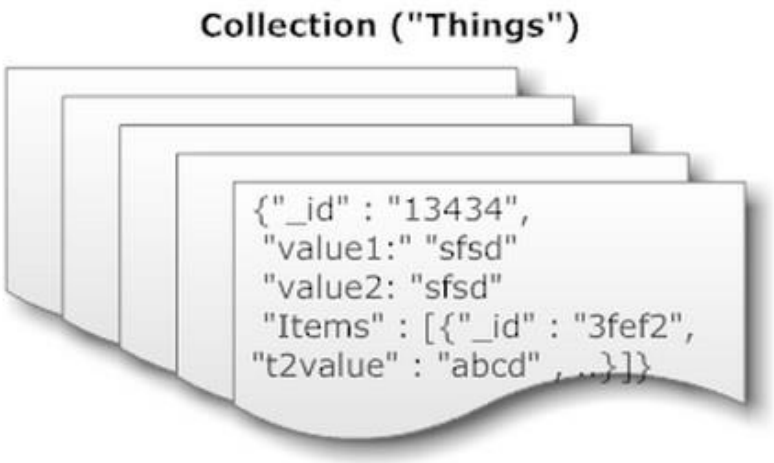
- Document dbs are a type of key-value db: each document has a unique identifier (the key) and the document itself serves as the value.
- The difference between these two models is that:
  - in a key-value database, the data (value) is treated as opaque and the database doesn't know about the data held within it; it's up to the application to understand what data is stored
  - in a document store each document contains some kind of metadata (a degree of structure to the data). They often come with an API or query language that allows users to retrieve documents based on the metadata they contain. They also allow for complex data structures, as you can nest documents within other documents.

Document

Relational Model

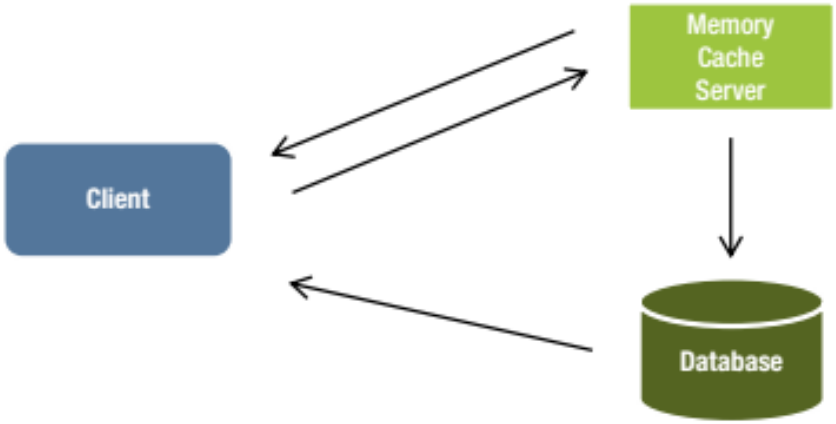


Document Model



source: www.dzone.com

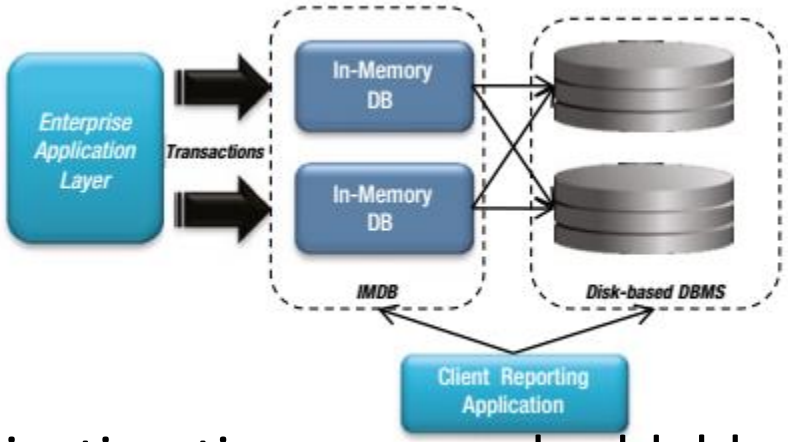
Document - In Memory Caching



- In-Memory Caching
  - Data is deployed between the application and the database to reduce database load
  - Ideal for caching data to memory that is repeatedly accessed.
  - Data is not replicated or persisted across servers.
  - It harnesses the aggregate memory of many distributed machines by using a hashing algorithm.

Facebook uses 800 servers for over 28 TBs of memory EHCache.

Document - In Memory Database



- In Memory Database
  - Data is deployed in the application tier as an embeddable database—for example, Derby
  - Reading and writing data is as fast as accessing RAM. For example, with a 1.8-GHz processor, a read transaction can take less than 5 microseconds, with a write transaction taking less than 15 microseconds.
  - The database fits entirely in physical memory.
  - The data is managed in memory with optimized access algorithms.
  - Transaction logs and database checkpoint files stored to disk.

Document: examples

MongoDB:

- composed of databases containing “collections”. A collection is composed of “documents” and each document is composed of fields.
- index a collection to increase the performance of data lookup. MongoDB returns something called a “cursor” which serves as a pointer to the data.
- offers the option of counting or classifying the data without extracting it. MongoDB supports BSON, the binary implementation of JSON.

Document

CouchDB:

- databases composed of documents consisting of fields and attachments as well as a “description” of the document in the form of metadata that is automatically maintained by the system.
- The underlying technology features all ACID capabilities from the RDBMS world.
- The advantage in CouchDB over relational is that the data is packaged and ready for manipulation or storage rather than scattered across rows and tables.

Columnar

- Relational databases are row oriented, the data in each row of a table is stored together. In a columnar, or column-oriented database, the data is stored across rows. It is very easy to add columns, and they may be added row by row, offering great flexibility, performance, and scalability. When you have volume and variety of data, you might want to use a columnar database. It is very adaptable; you simply continue to add columns.
- They use traditional database languages like SQL to load data and perform queries. The database can more precisely access the data it needs for a query rather than scanning and discarding unwanted data in rows. Query performance is often increased, particularly in very large data sets.

Columnar

There are 2 (somehow similar) types:

- Column stores

Stores data tables by column rather than by row
Serializes all of the values of a column together, similar to an index organization for row oriented dbs
Reduces the amount of data read from disk by compressing the similar columnar data and by reading only the necessary data
Examples

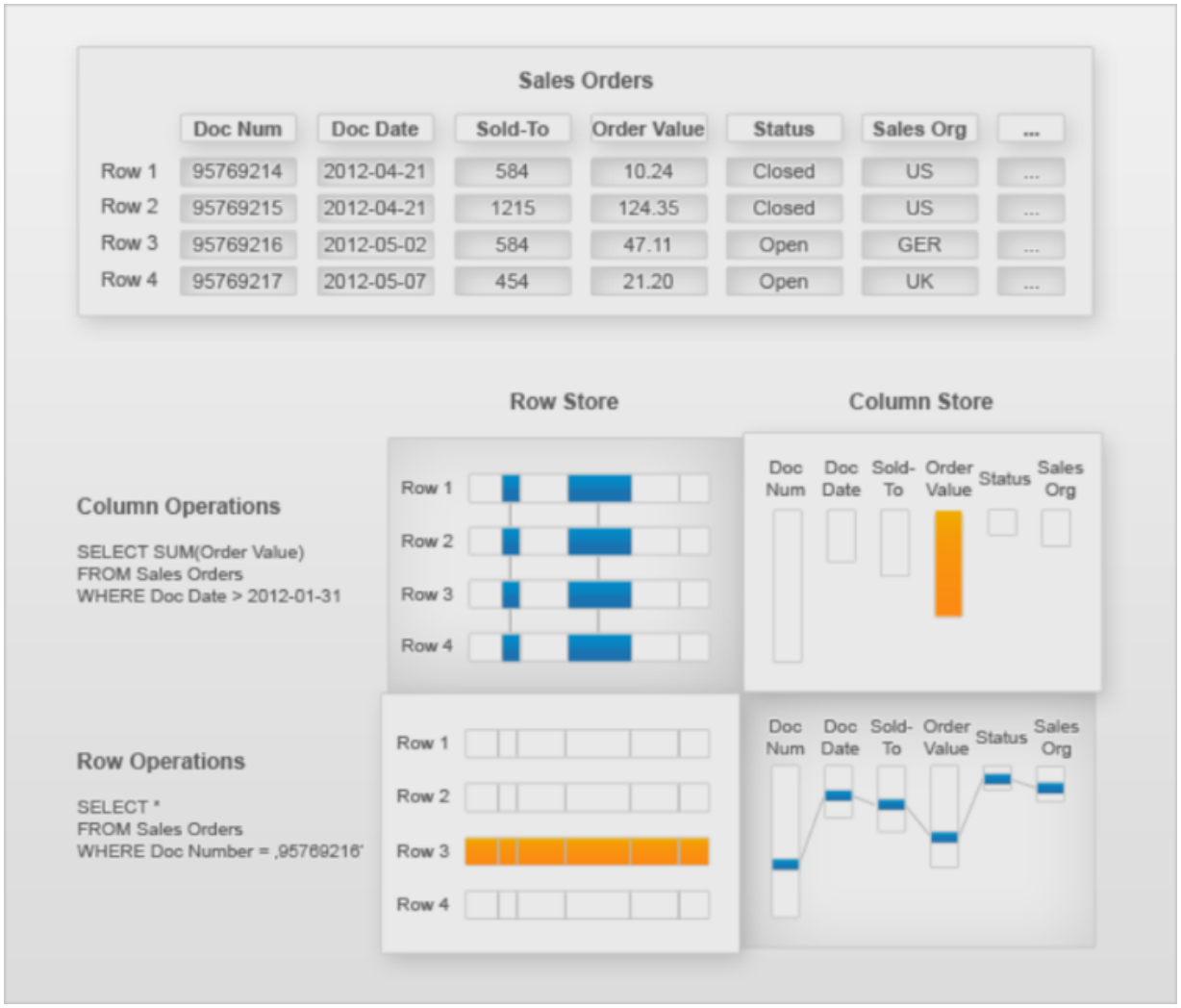
- Apache Parquet, Amazon Redshift
- MariaDB ColumnStore

Columnar



source: http://www.data-

Columnar



source: http://www.data-design.org

Columnar

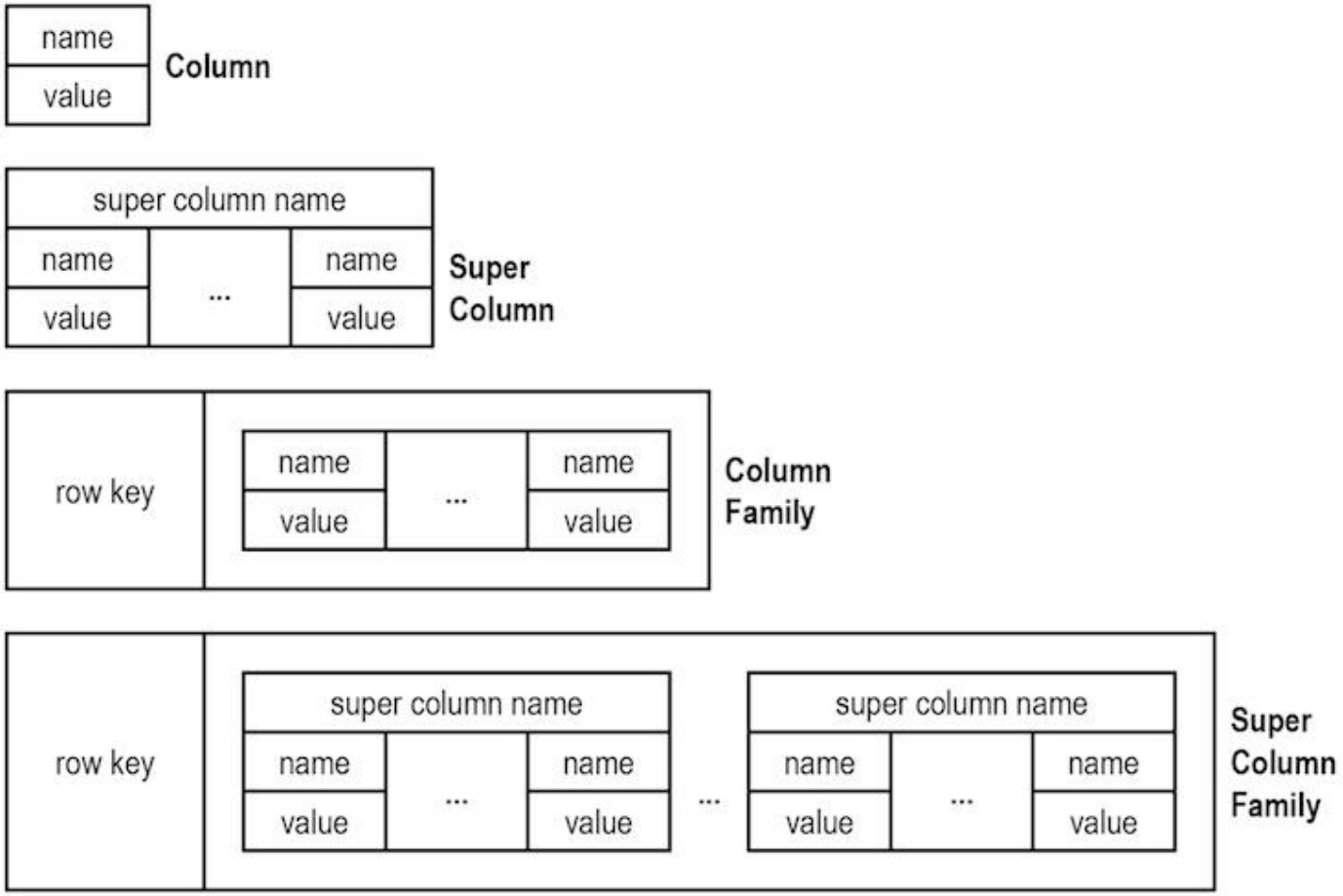
- Wide column databases

Has an architecture that uses persistent, sparse matrix, multi-dimensional mapping (row-value, column-value, and timestamp) in a tabular format meant for massive scalability (over and above the petabyte scale)

Examples:

- BigTable, Cassandra, HBase

Columnar


name
value
Column
super column name
name
value
...
name
value
Super Column
row key
name
value
...
name
value
Column Family
row key
super column name
name
value
...
name
value
...
super column name
name
value
...
name
value
Super Column Family

Columnar: example

- Hbase: the design of HBase is modeled on Google's BigTable. Implementations of HBase are highly scalable, sparse, distributed, persistent multidimensional sorted maps. The map is indexed by a row key, column key, and a timestamp; each value in the map is an uninterpreted array of bytes. When big data implementation requires random, real-time read/write data access, HBase is a very good solution. It is often used to store results for later analytical processing.

Columnar

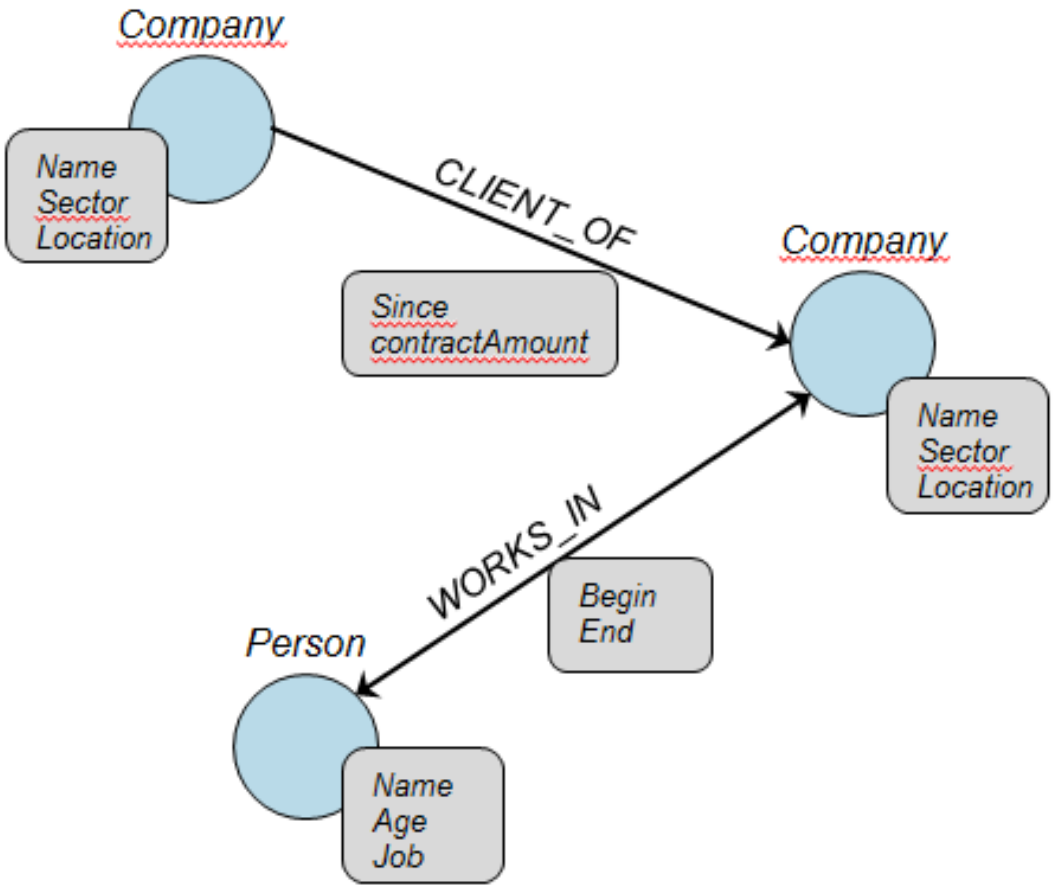
Similarities:

- Column-oriented data access: Both store and access data by columns rather than rows, which can improve performance for certain queries.
- Efficient for sparse data: They handle sparse datasets well, storing only non-null values.
- Good for analytics: Column-based access is ideal for aggregations and filtering on specific fields.
- Compression-friendly: Columnar formats allow better compression since similar data types are stored together.

Graph

- The fundamental structure for graph databases is called "**node-relationship**". This structure is most useful when you must deal with highly interconnected data. Nodes and relationships support properties, a key-value pair where the data is stored. These databases are navigated by following the relationships. This kind of storage and navigation is not possible in RDBMSs due to the rigid table structures and the inability to follow connections between the data wherever they might lead us. A graph database might be used to manage geographic data for oil exploration or to model and optimize a telecommunications provider's networks.

Graph



source: blog.octo.com

Graph: example

- Neo4J: is an ACID transaction database offering high availability through clustering. It is a trustworthy and scalable database that is easy to model because of the node-relationship properties' fundamental structure and how naturally it maps to our own human relationships. It does not require a schema, nor does it require data typing, so it is inherently very flexible.

Graph Connected Data

Property Graphs and RDF (Resource Description Framework)

Think of this as the NoSQL of graphs — pragmatic, developer-friendly, and built for speed.
Nodes: Entities (e.g., Person, Product)
Edges: Relationships (e.g., KNOWS, BOUGHT)
Properties: Key-value pairs on both nodes and edges

the semantic web's native language — formal, standardized, and built for interoperability.
Everything is a triple: **Subject** → **Predicate** → **Object** (e.g., :Alice :knows :Bob)
No properties on edges — instead, you reify relationships (i.e., turn them into nodes)

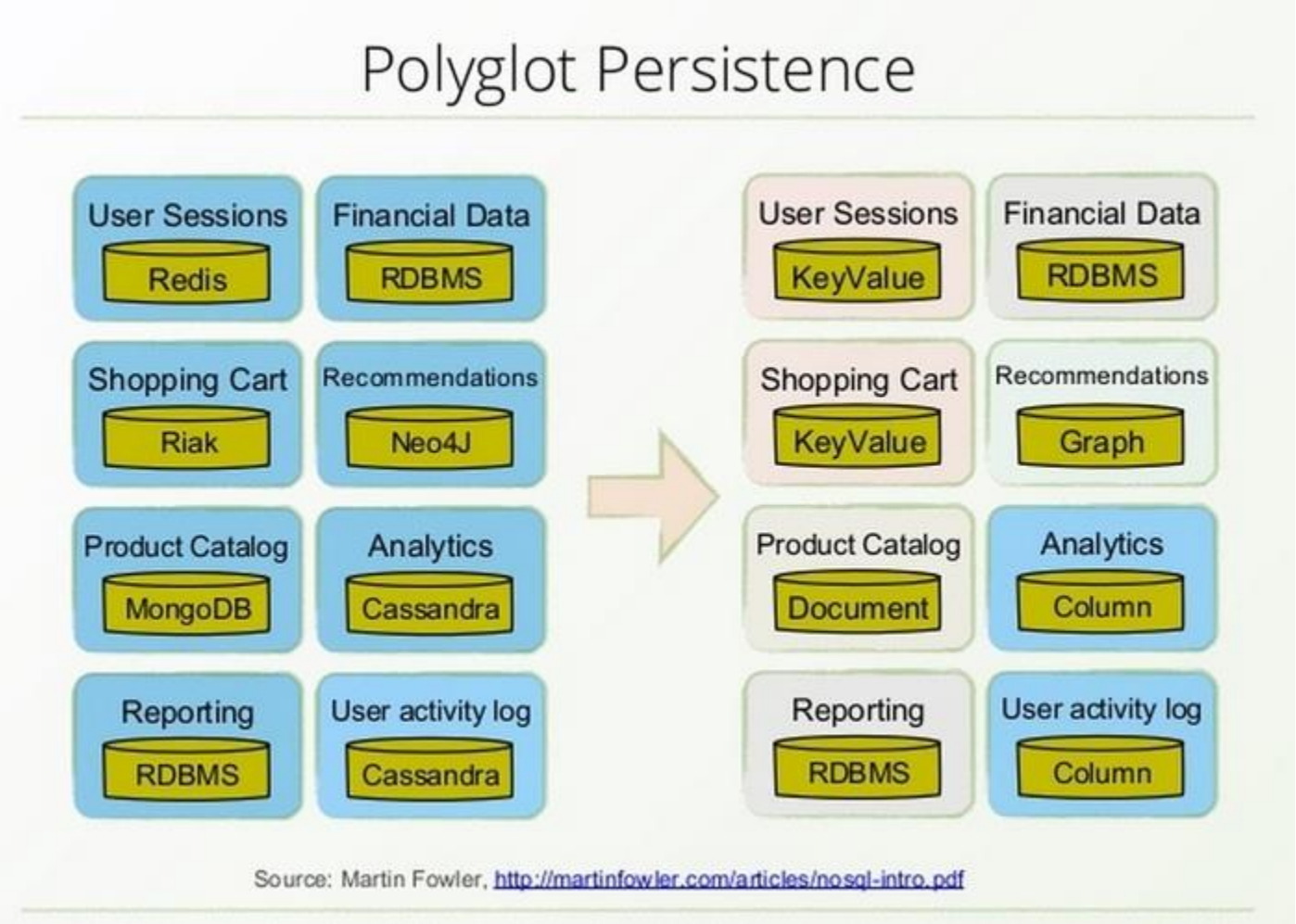
Graph Connected Data

Feature	Property Graph	RDF
Schema	Optional (schema-less or enforced)	Schema via RDFS
Flexibility	High — great for agile modeling	High, but more formal and verbose
Semantics	Implicit (developer-defined)	Explicit (machine-readable ontologies)
Query language	Cypher, Gremlin, PGQL MATCH (a:Person)-[:KNOWS]->(b:Person) RETURN b	SPARQL SELECT ?b WHERE { :Alice :knows ?b }
Use case	Social networks, fraud detection, recommendations	Knowledge graphs, data integration, linked data, ontologies

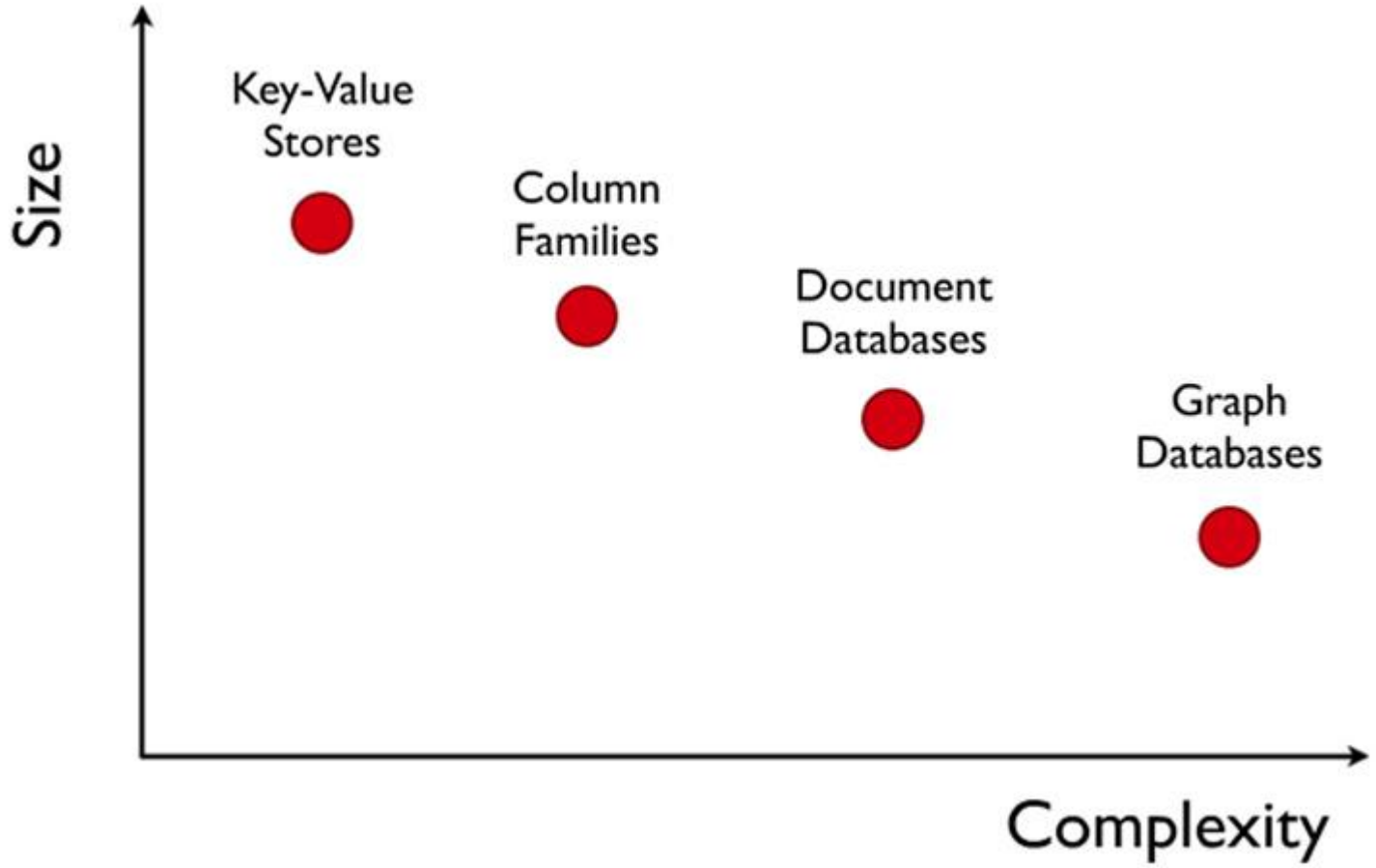
Polyglot

- The official definition of polyglot is “someone who speaks or writes several languages”. The term is redefined as a set of applications that use several core database technologies, the most likely outcome of a big data implementation planning. It is difficult to choose one persistence style no matter how narrow the approach to big data might be. A polyglot persistence database is used when it is necessary to solve a complex problem by breaking that problem into segments and applying different database models. It is then necessary to aggregate the results into a hybrid data storage and analysis solution.

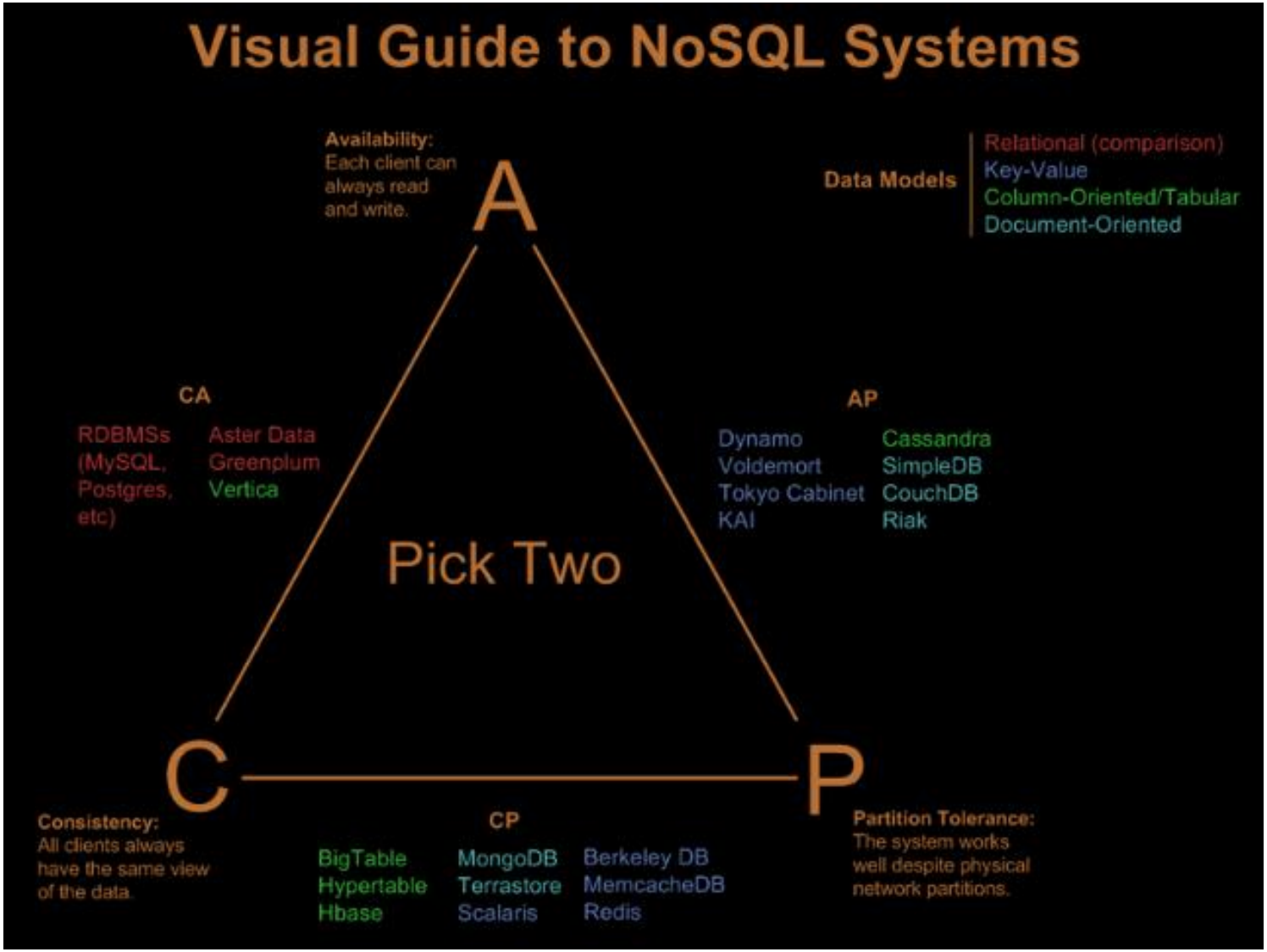
Polyglot


Polyglot Persistence
User Sessions
Redis
Financial Data
RDBMS
Shopping Cart
Riak
Recommendations
Neo4J
Product Catalog
MongoDB
Analytics
Cassandra
Reporting
RDBMS
User activity log
Cassandra
User Sessions
KeyValue
Financial Data
RDBMS
Shopping Cart
KeyValue
Recommendations
Graph
Product Catalog
Document
Analytics
Column
Reporting
RDBMS
User activity log
Column
Source: Martin Fowler, http://martinfowler.com/articles/nosql-intro.pdf

Size vs Complexity



CAP Compliant



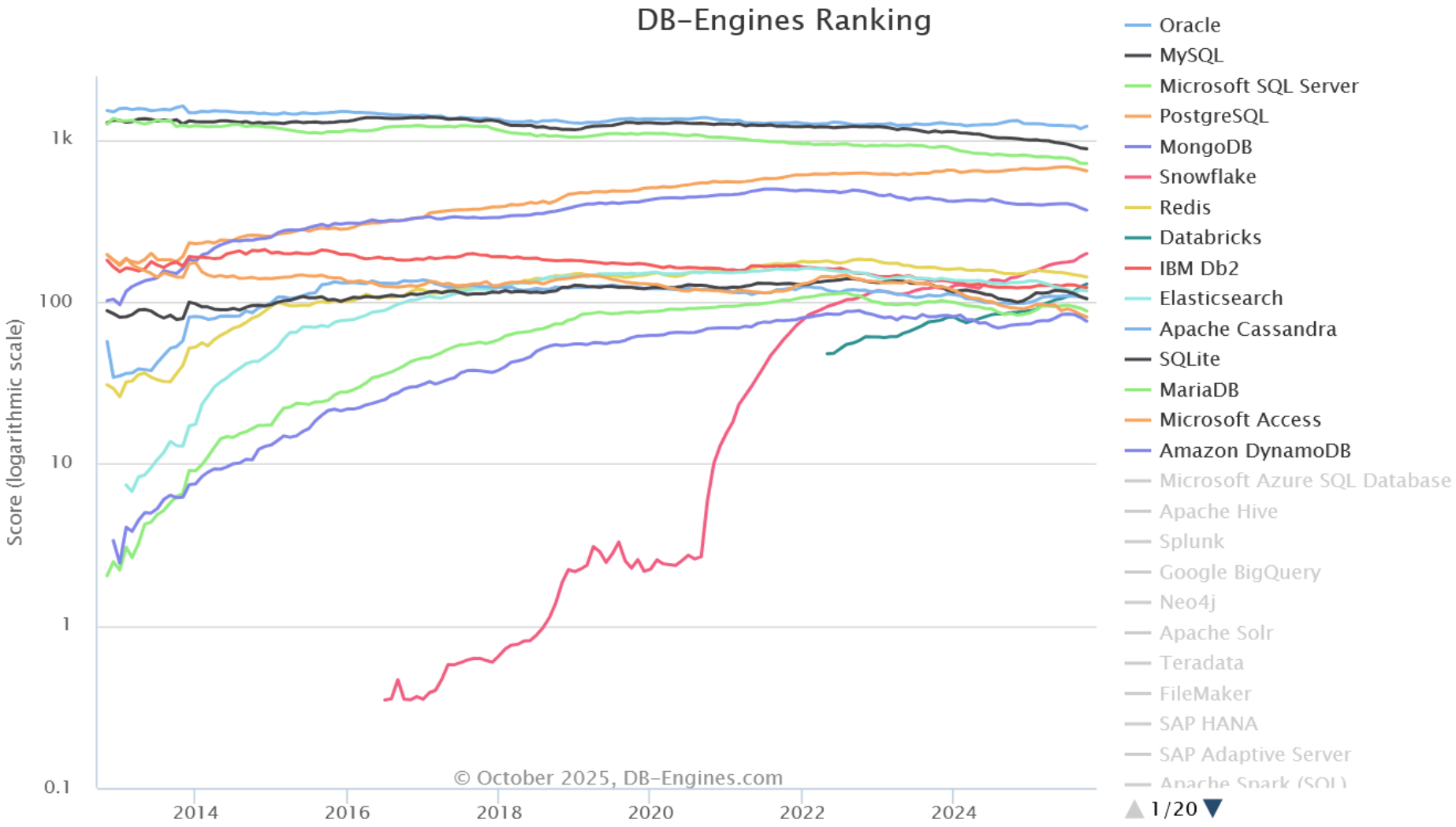
Databases Ranking

425 systems in ranking, October 2025

Rank			DBMS	Database Model	Score		
Oct 2025	Sep 2025	Oct 2024			Oct 2025	Sep 2025	Oct 2024
1.	1.	1.	Oracle	Relational, Multi-model	1212.77	+42.15	-96.67
2.	2.	2.	MySQL	Relational, Multi-model	879.66	-12.11	-143.09
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model	715.05	-2.27	-87.04
4.	4.	4.	PostgreSQL	Relational, Multi-model	643.20	-13.97	-8.96
5.	5.	5.	MongoDB	Document, Multi-model	368.01	-12.49	-37.20
6.	6.	↑ 7.	Snowflake	Relational	198.65	+8.46	+58.05
7.	7.	↓ 6.	Redis	Key-value, Multi-model	142.33	-2.84	-7.30
8.	↑ 9.	↑ 14.	Databricks	Multi-model	128.80	+4.74	+43.21
9.	↓ 8.	9.	IBM Db2	Relational, Multi-model	122.37	-1.81	-0.40
10.	10.	↓ 8.	Elasticsearch	Multi-model	116.67	-1.59	-15.18
11.	↑ 12.	11.	Apache Cassandra	Wide column, Multi-model	105.16	-1.82	+7.56
12.	↓ 11.	↓ 10.	SQLite	Relational	104.56	-3.32	+2.64
13.	13.	↑ 15.	MariaDB	Relational, Multi-model	87.77	-3.69	+2.88
14.	14.	↓ 12.	Microsoft Access	Relational	80.79	-2.82	-11.36
15.	15.	↑ 17.	Amazon DynamoDB	Multi-model	75.91	-4.37	+4.06
16.	16.	16.	Microsoft Azure SQL Database	Relational, Multi-model	75.49	-3.70	+0.95
17.	17.	↑ 18.	Apache Hive	Relational	75.22	-0.88	+22.65
18.	18.	↓ 13.	Splunk	Search engine	73.87	-1.89	-17.40
19.	19.	19.	Google BigQuery	Relational	63.20	-2.80	+12.02
20.	20.	↑ 21.	Neo4j	Graph	52.51	-1.27	+10.01

Source: http://db-engines.com/en/ranking

Databases Ranking



Source: http://db-engines.com/en/ranking

Key-Value Stores

Course 5

Key-Value Stores: The Simplest Model

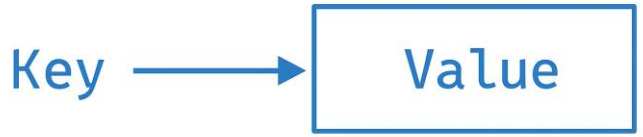
The foundational data model of NoSQL.

Essentially a massive, distributed **hash map**.

Optimized strictly for two operations: **GET(key)** and **PUT(key, value)**.

Structure

- **Key:** Unique identifier (string, hash). Used to locate the data.
- **Value:** An opaque blob of data. Can be JSON, an image, a string, etc.
- **Data Opacity:** The database does not inspect or care about the value's schema or structure.




```
{
  "user:123": { "name": "Alice", "age": 30 },
  "user:124": { "name": "Bob", "age": 25 },
  "session:456": "active"
}
```

Key

Is a unique identifier that acts like an address for retrieving the associated value.

Properties:

- Must be unique across the dataset.
- Typically immutable (changing a key means inserting a new pair).
- Often a string, but can be binary data depending on the database.
- Indexed efficiently (usually through hashing or sorted trees).

Example:

"user:123" or "session:abcdef123456"

Value

Is the actual data stored and retrieved via the key.

Properties:

- Can be any arbitrary blob of data — string, JSON object, image, serialized object, etc.
- The database doesn’t interpret or enforce structure (see data opacity on next slide).
- Often stored as a byte array internally.

Example:

Simple: "active"

Complex: { "name": "Alice", "email": "alice@example.com" }

Binary: a JPEG image file stored as bytes

Data Opacity

Means it treats values as opaque blobs - the database doesn't know or care what's inside them:

- The database cannot query inside the value (no "find users where age > 30").

- It only stores, retrieves, or deletes data based on the key.

- The application layer is responsible for:
    - Interpreting the value's structure
    - Managing serialization/deserialization (e.g., JSON, protobuf)

- Performing logical operations on the contents

Example:

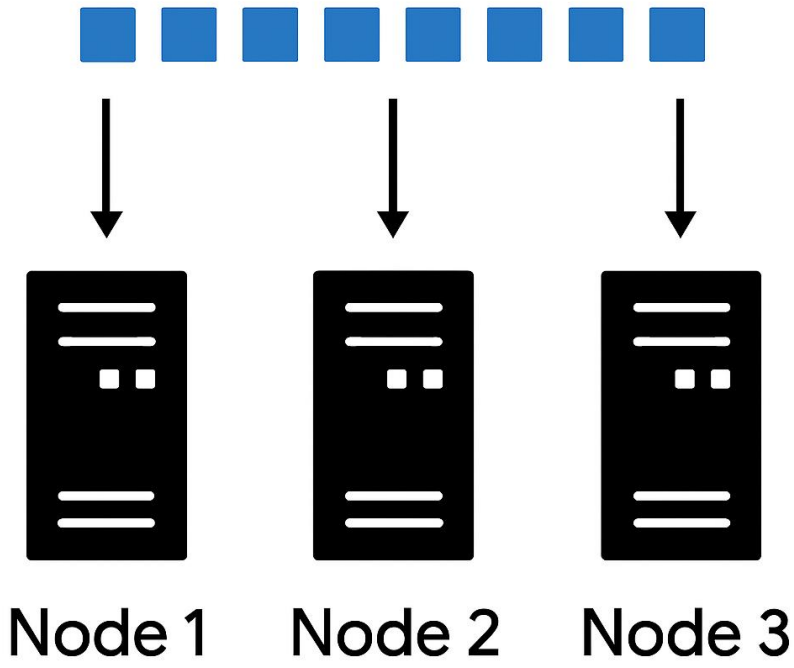
If the value is a JSON object:

{ "name": "Alice", "age": 30 }

The key–value store doesn't know about "name" or "age"; it just stores the entire JSON blob under "user:123".

Performance: O(1) Access Time

- **Speed:** Lookups are typically **O(1)** (constant time) because the key directly maps to the storage location.
- **CAP Model:** Generally **AP** (Available and Partition Tolerant), prioritizing availability.
- **Consistency:** Implements the **BASE** consistency model.
- **Scaling:** Linear and trivial **Horizontal Scaling** (add more nodes)



Internal Implementation

Hash Tables:

- Most in-memory KV stores (Redis) use hash tables for O(1) average access.

Persistent Stores:

- Disk-backed KV stores (LevelDB, RocksDB) use:
  - LSM Trees (Log-Structured Merge Trees): optimized for write-heavy workloads.
  - B-Trees / B+ Trees: used in older or more general-purpose databases.

Hash Table

A hash table maps keys to values using a hash function, which converts the key into an index (or "bucket") in an array.

Used as core data structure by in-memory key–value stores like Redis and Memcached.

Each key is hashed to an integer index:

hash("user:1") -> 3

hash("user:2") -> 7

hash("user:3") -> 3

The index decides where in the table to store the pair.

If two keys hash to the same index (a collision), uses a linked list or open addressing to handle it.

LSM Tree



An LSM Tree (Log-Structured Merge Tree) is optimized for high write throughput on disk.



Used by key–value stores like LevelDB, RocksDB, Cassandra, and DynamoDB.



Instead of updating records in place (which is slow on disks), it:

Writes new data sequentially to a log and an in-memory table (MemTable).

When the MemTable fills up, it is flushed to disk as an immutable sorted file (SSTable).

Over time, SSTables are merged (compacted) to keep lookups efficient.

1: Write to WAL (Write-Ahead Log)
    Appends to log for durability: WAL -> [("A",1), ("C",3), ("B",2)]
2: Insert into MemTable (in-memory sorted tree)
        MemTable -> { A:1, B:2, C:3 }
3: When MemTable is full -> Flush to SSTable
        SSTable_1 (on disk):
            A → 1
            B → 2
            C → 3
4: New writes go into a new MemTable, repeating the process.

When performing GET("B"):
    Search MemTable first (in memory)
    If not found, check immutable MemTables (read-only)
    If still not found, check SSTables on disk (using a bloom filter + binary search)

Common Use Cases

- **Caching Layer:** Stores frequently accessed query results (e.g., Redis).
- **Session Management:** Stores temporary user session data (login status, cart).
- **Rate Limiting:** Tracks API calls associated with a user or IP address.
- **Feature Toggles:** Stores global configuration settings for instant read access.

Data Distribution

Replication:

- Multiple copies of data on different nodes.
- Ensures fault tolerance, high availability, and read scalability.

Replication Models:

- Master–Slave: One node handles writes; others replicate.
- Multi-Master: All nodes can handle writes (eventual consistency).

Sharding (Partitioning):

- Splitting data across nodes based on key ranges or hashes.
- Common method: Consistent Hashing — evenly distributes keys, minimizes data movement on node changes.

Safe Parallel Access

Concurrency Control: Prevents race conditions during concurrent reads/writes.

- Optimistic concurrency: Allow concurrent ops, detect conflicts (e.g., version check).
- Pessimistic concurrency: Lock data during update.

Atomicity:

- Guarantees that an operation is all-or-nothing.
- Example: INCR key in Redis -> atomic counter update.

Isolation:

- Ensures consistent reads (e.g., Redis is single-threaded per instance -> implicit isolation).

Persistence and Recovery

Persistence:

- Ensures data survives crashes or restarts.
- Achieved using:
  - Write-Ahead Log (WAL): Append-only log of updates before applying them.
  - Snapshots: Periodic dumps of in-memory state to disk (e.g., Redis RDB).

Recovery:

- On restart, replay WAL + load latest snapshot to restore state.

Conflict Resolution

Why conflicts happen:

- Concurrent writes on different replicas -> inconsistent state.

Dynamo's Approach:

- Vector Clocks: Track version history of each object.
- Eventual Consistency: Allow temporary divergence, resolve later.
- Conflict Resolution Strategies:
  - Last Write Wins (LWW): Based on timestamp.
  - Read Repair: Resolve conflicts during read.
  - Client-side merge: Application decides how to reconcile.

AWS DynamoDB

A fully managed, multi-region NoSQL database service from AWS.

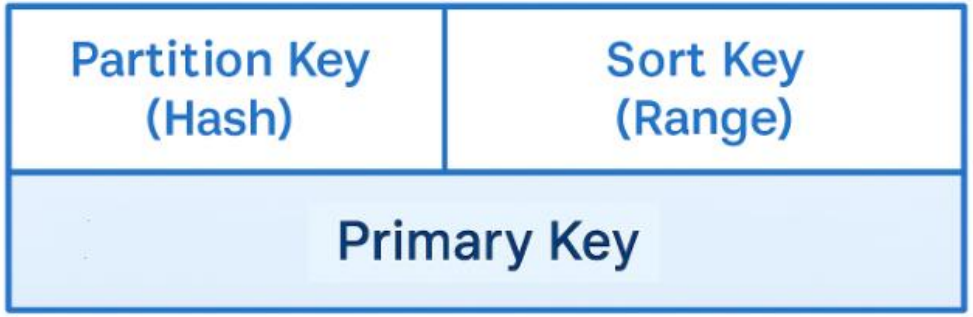
Supports both **Key-Value** and **Document** models.

Engineered for **single-digit millisecond performance** at any scale.

Offers **ACID Transactions** (unlike many traditional K/V systems).

Core Data Model

- DynamoDB's Primary Key is composed of up to two attributes:
  - **Partition Key (Hash Key):** Determines the data's physical location (shard).
  - **Sort Key (Range Key - Optional):** Defines the hierarchical order *within* that partition.



Partition Key: Sharding and Distribution



The Partition Key's hash value assigns data to a specific **physical partition (shard)**.



Must be designed for **high fan-out** to distribute read/write load evenly across partitions.

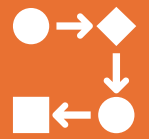


All queries **must** specify the Partition Key.



A poor PK choice leads to **"Hot Partitions"** (performance bottlenecks).

Sort Key: Hierarchy and Range Queries



Organizes items **sequentially** within a single partition (sorted by the Sort Key value).



Enables efficient **range queries** (e.g., "Get all customer orders between June and August").



Allows for data modeling patterns like **Adjacency Lists** within a single table.

Indexes

The Primary Key model is rigid (only fast lookups on PK/SK).

Indexes are separate data structures that allow querying on **non-key attributes**.

Allows the same data to be viewed and queried in multiple ways.

LSI: Strong Consistency within a Partition



Shares the **same Partition Key** as the base table.



Uses a **different Sort Key**.

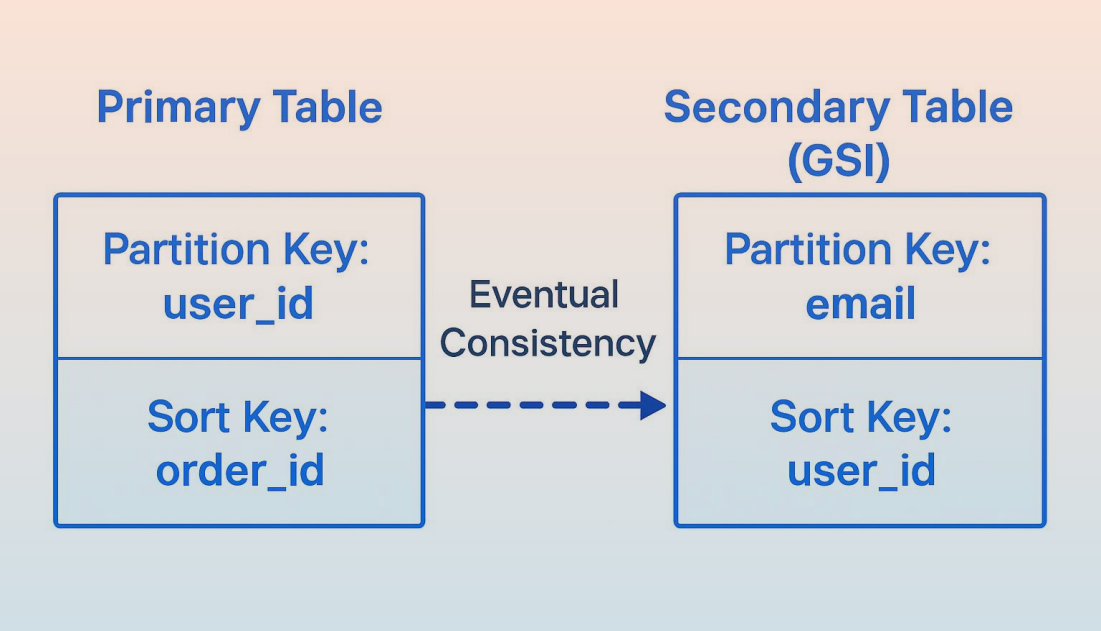


Reads are always **Strongly Consistent** (guaranteed latest data).



Confined to the limits of the original partition.

GSI: Global Flexibility and Scaling



Uses a **different Partition Key** (and optionally a different Sort Key) than the base table.

Ideal for high-fan-out queries (e.g., querying across all users by 'email').

Reads are always **Eventually Consistent** (most data, highest scale).

Can scale and be provisioned **independently** of the main table.

Capacity Modes: Provisioned vs. On-Demand

Provisioned:

- Pay for specified Read/Write Capacity Units (RCUs/WCUs) upfront.
- **Ideal for:** Predictable, steady workloads (cost certainty).
- **Risk:** Can lead to throttling if traffic exceeds provisioned limits.

On-Demand:

- Pay per million actual reads and writes executed.
- **Ideal for:** Spiky, unpredictable traffic (zero capacity planning).
- **Consistency:** Simplest operational model, handles peaks instantly.

Read Consistency: Performance vs. Freshness

Eventually Consistent Reads (Default): Fastest and most performant. A read might not reflect a very recent write.

Strongly Consistent Reads (Optional): Guarantees the latest, most up-to-date value. Consumes more capacity and has slightly higher latency.

ACID Guarantees with Transactions

DynamoDB supports **ACID** (Atomicity, Consistency, Isolation, Durability) transactions.

Allows multi-step updates across multiple items (records) within the same or different tables.

Use Case: Essential for financial or inventory workloads where strict correctness is mandatory.

Data Modeling: Access Patterns First



Anti-Pattern: Normalizing data and expecting joins.



Best Practice: Design the table structure (PK, SK, GSIs) to satisfy all application queries with **single, efficient lookups**.



Key Goal: Identify all access patterns and ensure a primary key or index can serve them.

Open Source KVs

- Redis – In-memory data store, often used for caching, session storage, and real-time analytics.
- Memcached – Simple, in-memory caching system for speeding up dynamic web applications.
- Riak KV – Distributed key-value database with fault tolerance and high availability.
- LevelDB – Embedded key-value store developed by Google; often used in local applications.
- RocksDB – High-performance embedded key-value store developed by Facebook, built on LevelDB.
- BadgerDB – Go-native key-value database inspired by RocksDB.

Commercial/Cloud KVs

- Amazon DynamoDB – Fully managed NoSQL key-value and document database by AWS.
- Azure Cosmos DB (Key-Value API) – Microsoft's globally distributed, multi-model database service.
- Google Cloud Datastore / Firestore (Key-Value mode) – Scalable, managed NoSQL database service.
- Couchbase – Distributed database that supports key-value and document data models.
- Aerospike – High-performance, low-latency distributed key-value store used in real-time systems.

Mastering DynamoDB



K/V: The fastest model, but requires keys for access.



DynamoDB: Uses the **Composite Key** (PK + SK) for core lookups.



GSIs: Crucial for flexibility and satisfying secondary access patterns.



Modeling: Denormalize data to avoid joins, design around your queries.

DynamoDB Core Components

- Tables – Similar to other database systems, DynamoDB stores data in tables. A table is a collection of data.

  Example: table called People can be used to store personal contact information about friends, family, or anyone else of interest. Also can have a Cars table to store information about vehicles that people drive.

- Items – Each table contains zero or more items. An item is a group of attributes that is uniquely identifiable among all of the other items.

  In a People table, each item represents a person. For a Cars table, each item represents one vehicle.

  Items in DynamoDB are similar in many ways to rows, records, or tuples in other database systems. In DynamoDB, there is no limit to the number of items you can store in a table.

- Attributes – Each item is composed of one or more attributes. An attribute is a fundamental data element, something that does not need to be broken down any further.

  An item in a People table contains attributes called PersonID, LastName, FirstName, and other. For a Department table, an item might have attributes such as DepartmentID, Name, Manager, and so on. Attributes in DynamoDB are similar in many ways to fields or columns in other database systems.

Primary Key

DynamoDB supports two different kinds of primary keys:

- **Partition key** – A simple primary key, composed of one attribute known as the partition key.

  Uses the partition key's value as input to an internal hash function. The output from the hash function determines the partition (physical storage internal to DynamoDB) in which the item will be stored.

  In a table that has only a partition key, no two items can have the same partition key value.

  A People table is an example of a table with a simple primary key (PersonID). Access to any item in the People table can be done directly by providing the PersonId value for that item.

People

```
{
    "PersonID": 101,
    "LastName": "Smith",
    "FirstName": "Fred",
    "Phone": "555-4321"
}

{
    "PersonID": 102,
    "LastName": "Jones",
    "FirstName": "Mary",
    "Address": {
                "Street": "123 Main",
                "City": "Anytown",
                "State": "OH",
                "ZIPCode": 12345
    }
}

{
    "PersonID": 103,
    "LastName": "Stephens",
    "FirstName": "Howard",
    "Address": {
                "Street": "123 Main",
                "City": "London",
                "PostalCode": "ER3 5K8"
    },
    "FavoriteColor": "Blue"
}
```

Primary Key

DynamoDB supports two different kinds of primary keys:

- **Partition key and sort key** – Referred to as a composite primary key, this type of key is composed of two attributes. The first attribute is the partition key, and the second attribute is the sort key.

  Uses the partition key value as input to an internal hash function. The output from the hash function determines the partition (physical storage internal to DynamoDB) in which the item will be stored. All items with the same partition key value are stored together, in sorted order by sort key value.

  In a table that has a partition key and a sort key, it's possible for multiple items to have the same partition key value. However, those items must have different sort key values.

  A Music table can have a composite primary key (Artist and SongTitle). Access to any item can be done directly in the Music table by providing the Artist and SongTitle values for that item.

Music


```
{
    "Artist": "No One You Know",
    "SongTitle": "My Dog Spot",
    "AlbumTitle": "Hey Now",
    "Price": 1.98,
    "Genre": "Country",
    "CriticRating": 8.4
}

{
    "Artist": "No One You Know",
    "SongTitle": "Somewhere Down The Road",
    "AlbumTitle": "Somewhat Famous",
    "Genre": "Country",
    "CriticRating": 8.4,
    "Year": 1984
}

{
    "Artist": "The Acme Band",
    "SongTitle": "Still in Love",
    "AlbumTitle": "The Buck Starts Here",
    "Price": 2.47,
    "Genre": "Rock",
    "PromotionInfo": {
        "RadioStationsPlaying": [
            "KHCR",
            "KQBX",
            "WTNR",
            "WJJH"
        ],
        "TourDates": {
            "Seattle": "20150622",
            "Cleveland": "20150630"
        },
        "Rotation": "Heavy"
    }
}
```

Primary Key

A composite primary key gives additional flexibility when querying data. By providing only the value for Artist, DynamoDB retrieves all of the songs by that artist. To retrieve only a subset of songs by a particular artist, can be used a value for Artist along with a range of values for SongTitle.

Secondary indexes

One or more secondary indexes can be created on a table. A secondary index allows querying the data in the table using an alternate key, in addition to queries against the primary key. DynamoDB doesn't require indexes, but they give applications more flexibility when querying data. After creating a secondary index on a table, data can be read from the index in the same way as reading from the table.

DynamoDB supports two kinds of indexes:

- Global secondary index – An index with a partition key and sort key that can be different from those on the table. The primary key values in global secondary indexes don't need to be unique.
- Local secondary index – An index that has the same partition key as the table, but a different sort key.

Secondary indexes

Global secondary indexes (GSIs) are indexes that span the entire table, allowing queries across all partition keys.

- Local secondary indexes (LSIs) are indexes that have the same partition key as the base table but a different sort key.
- Each table in DynamoDB has a quota of 20 global secondary indexes (default quota) and 5 local secondary indexes.

In the example Music table shown previously, you can query data items by Artist (partition key) or by Artist and SongTitle (partition key and sort key). What if you also wanted to query the data by Genre and AlbumTitle? To do this, you could create an index on Genre and AlbumTitle, and then query the index in much the same way as you'd query the Music table.

Consistency

- Local Secondary Index (LSI)

The LSI lives in the same partition as the base item.

When data is read from an LSI, DynamoDB can read from the same storage node as the base table.

Therefore, it supports both:

  - Strongly consistent reads (always reflects the latest write)
  - Eventually consistent reads (faster, cheaper)

- Global Secondary Index (GSI)

The GSI is stored separately, potentially on different physical partitions.

Updates to a GSI happen asynchronously — the system eventually propagates changes from the base table.

Thus:

  - Only eventually consistent reads are possible.
  - A read might momentarily see stale data (e.g., missing or outdated items).

Streams

Optional feature that captures data modification events in DynamoDB tables. The data about these events appears in the stream in near-real time, and in the order that the events occurred.

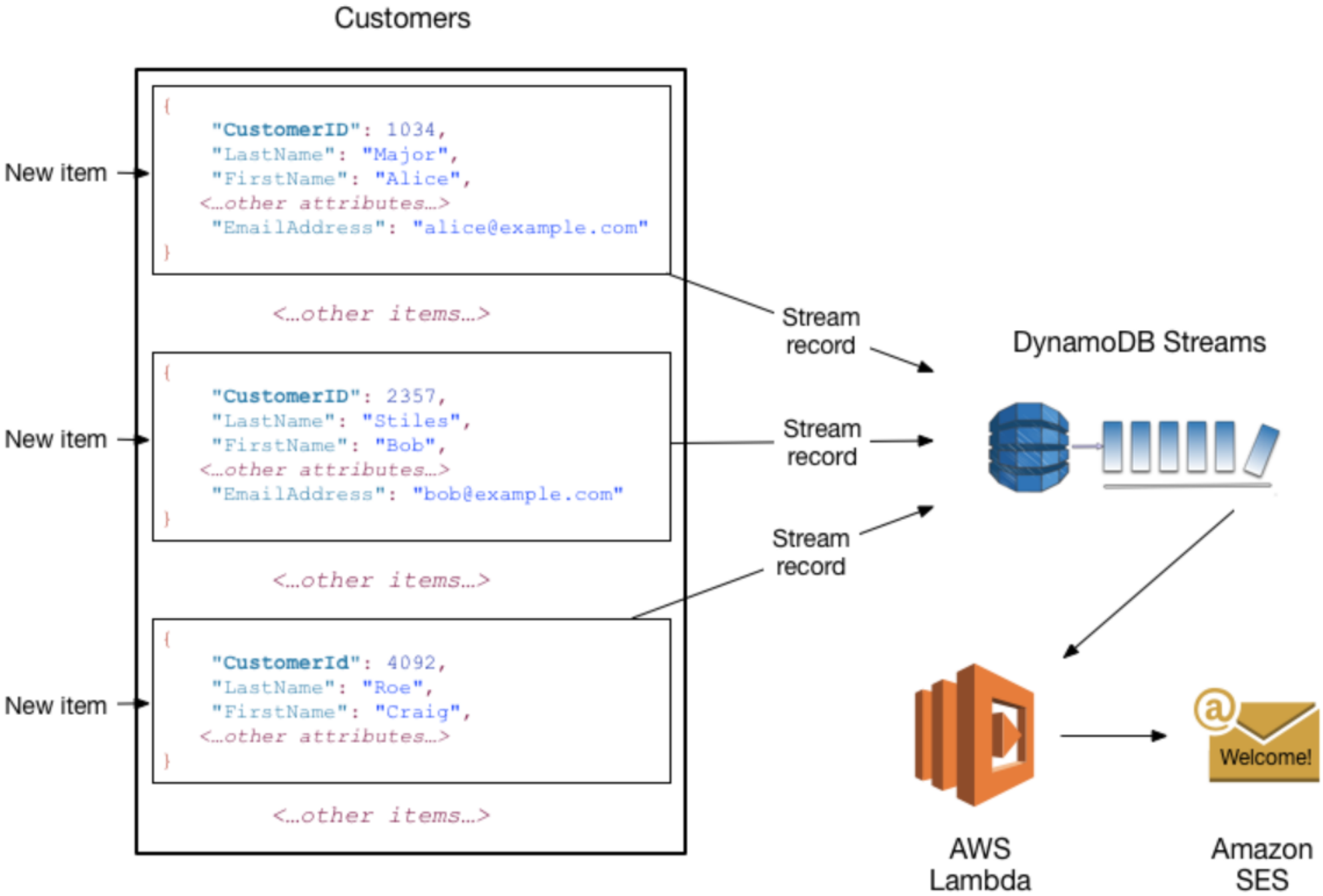
Each event is represented by a stream record. If stream is enabled on a table, Streams writes a stream record whenever one of the following events occurs:

- A new item is added to the table: The stream captures an image of the entire item, including all of its attributes.
- An item is updated: The stream captures the "before" and "after" image of any attributes that were modified in the item.
- An item is deleted from the table: The stream captures an image of the entire item before it was deleted.

Streams

Each stream record also contains the name of the table, the event timestamp, and other metadata. Stream records have a lifetime of 24 hours; after that, they are automatically removed from the stream.

Can be used together with AWS Lambda to create a trigger—code that runs automatically whenever an event of interest appears in a stream. For example, consider a Customers table that contains customer information for a company. When sending a "welcome" email to each new customer and stream is enabled on that table, the stream can be associated with a Lambda function that would run whenever a new stream record appears, but only process new items added to the Customers table. For any item that has an EmailAddress attribute, the Lambda function would invoke Amazon Simple Email Service (Amazon SES) to send an email to that address.


Customers
New item
{
"CustomerID": 1034,
"LastName": "Major",
"FirstName": "Alice",
<…other attributes…>
"EmailAddress": "alice@example.com"
}
<…other items…>
New item
{
"CustomerID": 2357,
"LastName": "Stiles",
"FirstName": "Bob",
<…other attributes…>
"EmailAddress": "bob@example.com"
}
<…other items…>
New item
{
"CustomerId": 4092,
"LastName": "Roe",
"FirstName": "Craig",
<…other attributes…>
}
<…other items…>
Stream record
Stream record
Stream record
DynamoDB Streams
AWS Lambda
Welcome!
Amazon SES

API

Working with Amazon DynamoDB from an application can be done using a few simple API operations; they are organized in categories:

- Control plane
- Data plane
- DynamoDB Streams
- Transactions

Control plane

Control plane operations are used to create and manage DynamoDB tables, work with indexes, streams, and other objects that are dependent on tables.

CreateTable – Creates a new table. Optionally, can be used to create one or more secondary indexes, and enable DynamoDB Streams for the table.

DescribeTable– Returns information about a table, such as its primary key schema, throughput settings, and index information.

ListTables – Returns the names of all of your tables in a list.

UpdateTable – Modifies the settings of a table or its indexes, creates or removes new indexes on a table, or modifies DynamoDB Streams settings for a table.

DeleteTable – Removes a table and all of its dependent objects from DynamoDB.

Data plane

Allows create, read, update, and delete (also called CRUD) actions on data in a table. Some of the data plane operations allow reading data from a secondary index.

PartiQL - a SQL-compatible query language for Amazon DynamoDB, allows to perform these CRUD operations; they can also be done via DynamoDB's classic CRUD APIs that separates each operation into a distinct API call.

ExecuteStatement – Reads multiple items from a table. Can be used to write or update a single item from a table. When writing or updating a single item, the primary key attributes must be specified.

BatchExecuteStatement – Writes, updates or reads multiple items from a table. This is more efficient than ExecuteStatement because application only needs a single network round trip to write or read the items.

Data Plane

Classic API

- Creating data
  - PutItem – Writes a single item to a table; the primary key attributes must be specified, other attributes are not mandatory.
  - BatchWriteItem – Writes up to 25 items to a table. This is more efficient than calling PutItem multiple times because the application only needs a single network round trip to write the items.
- Reading data
  - GetItem – Retrieves a single item from a table; specify the primary key for the item to find. Can retrieve the entire item, or just a subset of its attributes.
  - BatchGetItem – Retrieves up to 100 items from one or more tables. This is more efficient than calling GetItem multiple times because the application only needs a single network round trip to read the items.
  - Query – Retrieves all items that have a specific partition key; must specify the partition key value. Can retrieve entire items, or just a subset of their attributes. Optionally, apply a condition to the sort key values to only retrieve a subset of the data that has the same partition key. Can be used on a table, provided that the table has both a partition key and a sort key. Can be uses on an index, provided that the index has both a partition key and a sort key.
  - Scan – Retrieves all items in the specified table or index. Can retrieve entire items, or just a subset of their attributes. Optionally, can apply a filtering condition to return only the values that are of interest and discard the rest.

Data Plane

Classic API

- Updating data
  - UpdateItem – Modifies one or more attributes in an item; must specify the primary key for the item to modify. Can add new attributes and modify or remove existing attributes. Can also perform conditional updates, so that the update is only successful when a user-defined condition is met. Optionally, can implement an atomic counter, which increments or decrements a numeric attribute without interfering with other write requests.
- Deleting data
  - DeleteItem – Deletes a single item from a table; must specify the primary key for the item that is to be deleted.
  - BatchWriteItem – Deletes up to 25 items from one or more tables. This is more efficient than calling DeleteItem multiple times because the application only needs a single network round trip to delete the items.

Streams

DynamoDB Streams operations are used to disable a stream on a table, and allow access to the data modification records contained in a stream.

ListStreams – Returns a list of all the streams, or just the stream for a specific table.

DescribeStream – Returns information about a stream, such as its Amazon Resource Name (ARN) and where the application can begin reading the first few stream records.

GetShardIterator – Returns a shard iterator, which is a data structure that the application uses to retrieve the records from the stream.

GetRecords – Retrieves one or more stream records, using a given shard iterator.

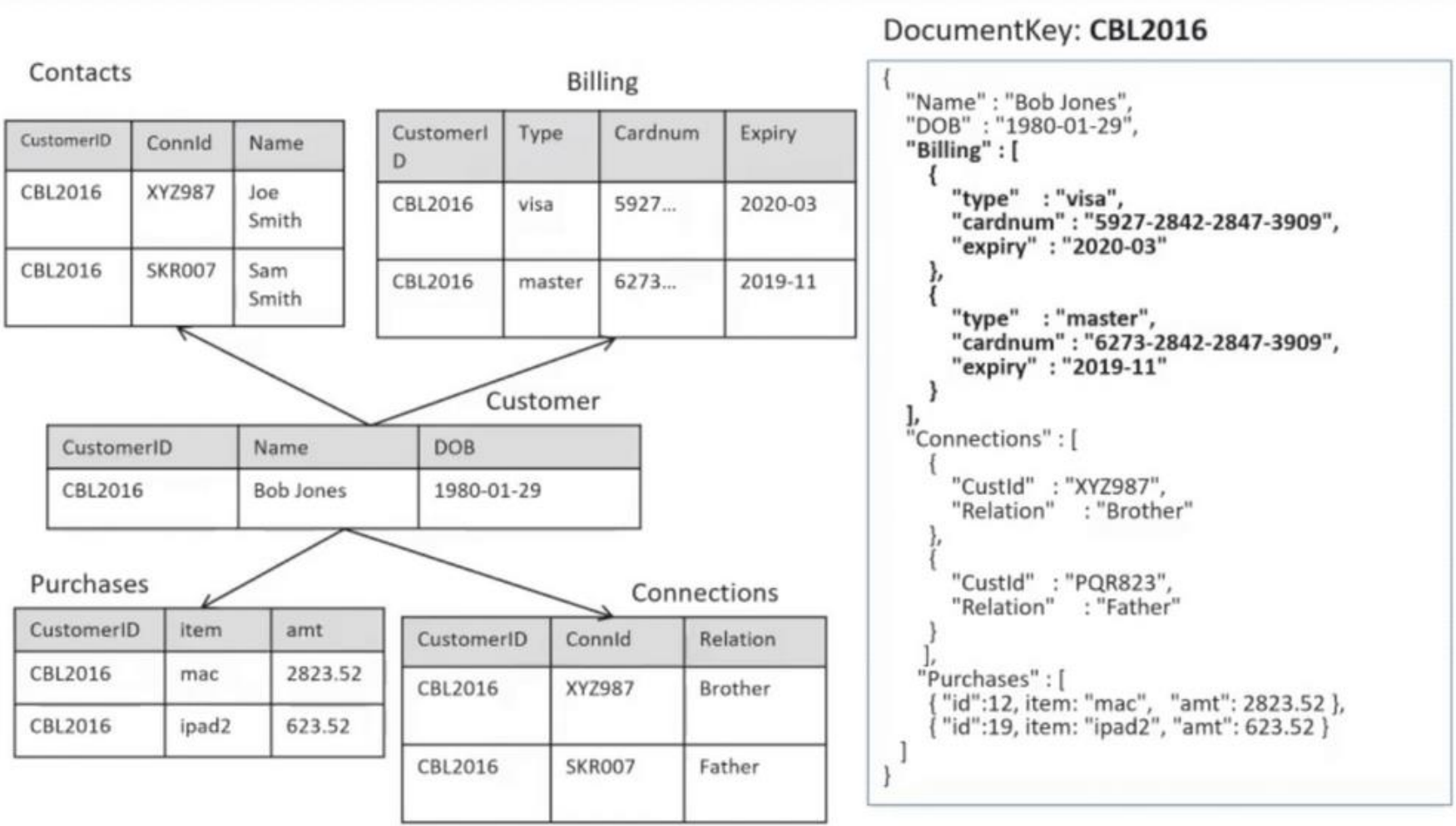
NoSQL – Document Databases

Course 6

Document Databases

Are known for their flexibility, and this flexibility primarily stems from their ability to store data in a schemaless or semi-structured format:

- **Schemaless Design:** a rigid, predefined schema is not enforced. They allow data to be stored in a flexible, document-oriented format, typically using formats such as JSON or BSON (Binary JSON).
- **Dynamic Schema Evolution:** as application requirements evolve, the model can easily adapt without the need for complex migrations. New fields can be added to documents, and existing fields can be modified or eliminated as needed.
- **Nested and Complex Data Structures:** they support nested and complex data structures within documents. As example, a document can contain arrays, sub-documents, or other documents, allowing the representation of complex, nested data.



Source: Couchbase

Document Databases

- **Polymorphic Data**: documents within a collection can represent different types of entities or objects with different attributes. This allows for storing different data types in the same collection.
- **Improved Read and Write Performance**: because associated data is often stored together within a single document, this can provide improved read and write performance for certain types of queries. Selecting a complete document is often more efficient than performing joins.
- **Query Flexibility**: they support flexible and expressive query languages that allow querying based on the content of the documents.
- **Aggregation Frameworks**: they offer aggregation frameworks that enable complex data processing and transformation within the database. This allows for the execution of advanced analysis and data manipulation operations directly on the stored documents.
- **Horizontal Scalability**: they are designed to scale horizontally by distributing data across multiple nodes or servers.

Document Databases

The flexibility offered lies in their ability to adapt to dynamic data models, handle diverse and nested data structures, and provide efficient querying capabilities. This flexibility makes document-oriented databases a suitable choice for applications with changing requirements and complex data structures.

Key-Value Pairs vs. JSON/BSON Documents

- Key-Value Pairs:

Structure: In a key-value pair data structure, each piece of data is associated with a unique identifier known as the key. The key is used to retrieve the corresponding value.

Use Cases: Key-value collections are often simple and efficient, making them suitable for scenarios where fast and direct data access is crucial. These databases are commonly used for caching and metadata storage.

Document Databases

- JSON/BSON Documents

  Structure:

  JSON (JavaScript Object Notation): A data exchange format that uses an easily understandable and readable text format. It supports data types such as objects, arrays, strings, numbers, boolean values, and null.

  BSON (Binary JSON): A binary-encoded serialization of JSON-like documents. BSON extends JSON to include additional data types, such as binary data and the date type, making it more efficient for storage and data exchange.

  Use Cases:

  JSON/BSON documents are the primary data structure in document-oriented databases. Each document is a self-contained unit that represents a set of key-value pairs, where the keys are field names and the values can be simple data types, arrays, nested documents, or a combination thereof.

JSON vs BSON

Feature	JSON (JavaScript Object Notation)	BSON (Binary JSON)
Format	**Text-based**	**Binary-encoded**
Readability	**High**. Easily readable and editable by humans with any text editor.	**Low**. Not human-readable without a specific decoder/tool.
Parsing Speed	**Slower**. The text needs to be parsed into data types (e.g., a date string must be converted to a `Date` object).	**Faster**. Designed for machine processing, allowing quicker serialization and deserialization by the database engine.
Interchange	**Excellent** for APIs and data exchange between different systems due to its ubiquity.	Primarily used **internally** by document databases (like MongoDB) for storage and retrieval.

JSON vs BSON

Data Type	JSON Support	BSON Support (Example Types)
Numeric	Single generic **number** type (float/double).	Separate types for **int32**, **int64**, **double**, and **Decimal128**.
Date/Time	Stored as a **string** (e.g., "2025-11-02T...Z").	Stores a native **Date** type (a 64-bit integer representing milliseconds).
Binary Data	No native type; must be encoded as a **base64 string**.	Has a native **Binary** type.
Identifiers	No native type.	Supports **ObjectId** (used by MongoDB for its default _id field).

JSON vs BSON

Aspect	JSON	BSON
Storage Size	Typically **smaller** for simple data because it lacks the BSON metadata overhead.	**Variable**. Can be **larger** for small documents due to metadata, but often **more efficient** for large, complex, or binary data.
Traversal	Slower, as it lacks a fixed length, meaning the parser must read the entire document to find a nested field.	**Faster**. It encodes **type and length information** for each field. This allows the database to "skip" over parts of the document it doesn't need to read when performing a query.
Indexing	Limited to indexing the string representation of data.	**Powerful**. The database can index and query based on the native BSON types (like a native `Date` object).

BSON Architecture

structured as a series of ordered field-value pairs encoded in a binary format. This structure is designed for fast traversal, scanning, and indexing.
The basic architecture of a BSON document is:

1. Total Document Size: the document begins with a 32-bit integer that specifies the total size of the document in bytes. This allows a parser to skip the entire document quickly.
2. Element List (e_list): a sequence of elements, each representing a field-value pair. The field-value pairs are stored with a defined order.
    - Element Structure: Each element is composed of:
        - Type: A single byte that explicitly indicates the BSON data type (e.g., 0x02 for String, 0x12 for UTC Datetime).
        - Field Name: A C-string (UTF-8 encoded string terminated by a null byte) that serves as the key.
        - Value: The actual data, encoded according to the specified type. For compound types like nested documents or arrays, the value includes its own size prefix.
3. Terminating Null Byte: the document ends with a single null byte (0x00) to mark the end of the element list.

JSON vs BSON

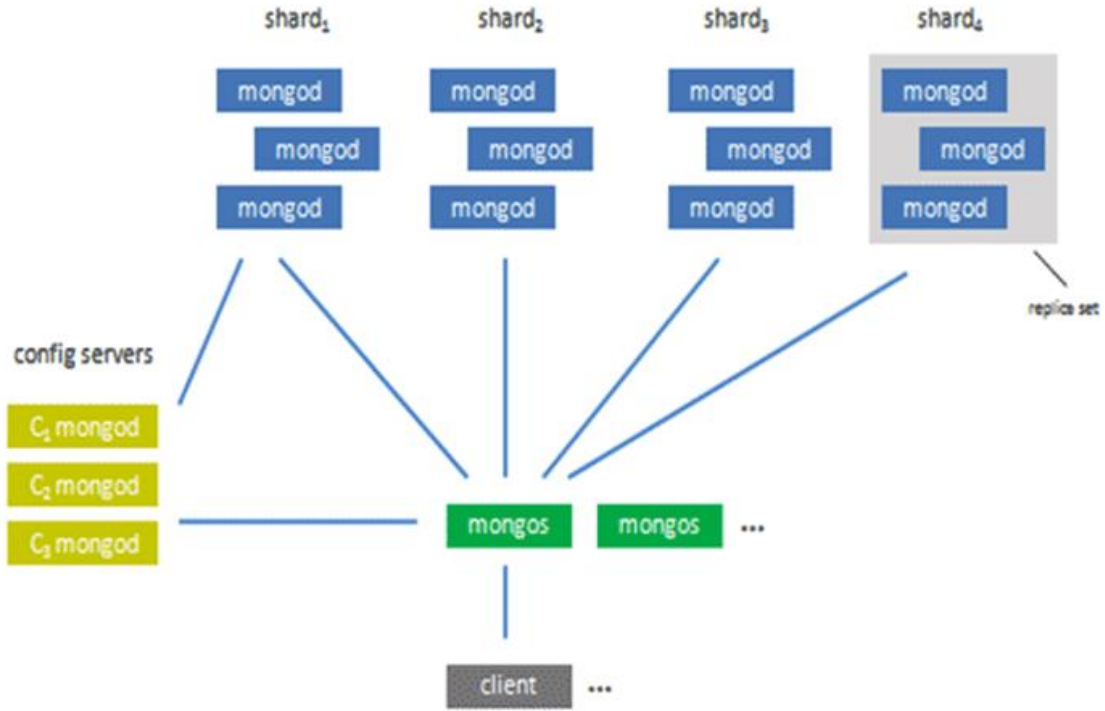
```
{
  "name": "Alice",
  "age": 30,
  "active": true
}
```

Field/Component	BSON Code (Hex) / Value	Description	Purpose/JSON Equivalent
Total Size	1D 00 00 00 (29 in decimal)	4-byte **integer** specifying the **total length** of the document (including this field itself and the terminator).	Allows fast skipping/parsing.
Elem 1: "name" (String)			
Type	2	**type byte** - the next value is a **String**.	name is a string.
Field Name	6E 61 6D 65 00 ("name\0")	The key, followed by a **null terminator**.	"name"
String Size	06 00 00 00 (6 in decimal)	A 4-byte integer specifying the length of the string value **plus its null terminator**.	The string value is 5 characters.
Value	41 6C 69 63 65 00 ("Alice\0")	The UTF-8 string value, also **null-terminated**.	"Alice"
Elem 2: "age" (Integer)			
Type	10	**type byte** - the next value is a **32-bit Integer**.	age is a number.
Field Name	61 67 65 00 ("age\0")	The key, followed by a **null terminator**.	"age"
Value	1E 00 00 00 (30 in little-endian)	The 4-byte binary integer value.	30
Elem 3: "active" (Boolean)			
Type	8	**type byte** - the next value is a **Boolean**.	active is a boolean.
Field Name	69 73 5F 61 63 74 69 76 65 00 ("active\0")	the key, followed by a **null terminator**.	"active"
Value	1	single byte: 01 - true, 00 - false.	TRUE
Document Terminator	0	final single **null byte**.	end of JSON object.

MongoDB

- Is a cross-platform, document-oriented database that offers high performance, high availability, and easy scalability.
- It uses the concept of a collection and a document. The database is a physical container for collections. Each database has its own set of files on the file system. A single MongoDB server typically has multiple databases.
- A **collection** is a group of MongoDB documents (the equivalent of a table). A collection can belong to only one database. No schema is enforced.
- **Documents** within a collection can have different fields. A document is a set of key-value pairs.

MongoDB Architecture



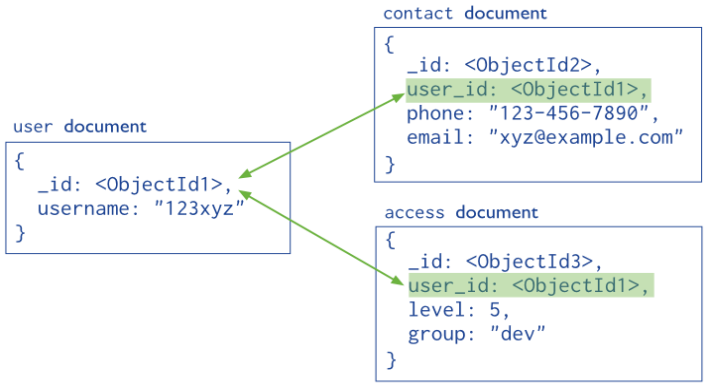
source: mongodb.com

Documents

An important decision in designing data models is the structure of the documents and how the application represents relationships between the data. There are two ways to represent these relationships:

- **References**: Store relationships between data by including links or references from one document to another.
- **Nested** Documents (or **Embedded** Documents): Store related data within a single document structure. MongoDB documents make it possible to embed document structures within a field or an array within a document.

Documents





source: mongodb.com

Relationships

1 to 1 with nesting

```
{
    patron_id: "joe",
    street: "123 Fake Street",
    city: "Faketon",
    state: "MA",
    zip: "12345"
}
```

```
{
    _id: "joe",
    name: "Joe Bookreader",
    address: {
                street: "123 Fake Street",
                city: "Faketon",
                state: "MA",
                zip: "12345"
             }
}
```

Relationships

1 to N with nesting

```
{
    _id: "joe",
    name: "Joe Bookreader"
}

{
    patron_id: "joe",
    street: "123 Fake Street",
    city: "Faketon",
    state: "MA",
    zip: "12345"
}

{
    patron_id: "joe",
    street: "1 Some Other Street",
    city: "Boston",
    state: "MA",
    zip: "12345"
}
```



```
{
    _id: "joe",
    name: "Joe Bookreader",
    addresses: [
                {
                    street: "123 Fake Street",
                    city: "Faketon",
                    state: "MA",
                    zip: "12345"
                },
                {
                    street: "1 Some Other Street",
                    city: "Boston",
                    state: "MA",
                    zip: "12345"
                }
            ]
}
```

Relationships

1 to N
with references

```
{
   title: "MongoDB: The Definitive Guide",
   author: [ "Kristina Chodorow", "Mike Dirolf" ],
   published_date: ISODate("2010-09-24"),
   pages: 216,
   language: "English",
   publisher: {
              name: "O'Reilly Media",
              founded: 1980,
              location: "CA"
            }
}

{
   title: "50 Tips and Tricks for MongoDB Developer",
   author: "Kristina Chodorow",
   published_date: ISODate("2011-05-06"),
   pages: 68,
   language: "English",
   publisher: {
              name: "O'Reilly Media",
              founded: 1980,
              location: "CA"
            }
}
```



```
{
   name: "O'Reilly Media",
   founded: 1980,
   location: "CA",
   books: [12346789, 234567890, ...]
}

{
    _id: 123456789,
    title: "MongoDB: The Definitive Guide",
    author: [ "Kristina Chodorow", "Mike Dirolf" ],
    published_date: ISODate("2010-09-24"),
    pages: 216,
    language: "English"
}

{
   _id: 234567890,
   title: "50 Tips and Tricks for MongoDB Developer",
   author: "Kristina Chodorow",
   published_date: ISODate("2011-05-06"),
   pages: 68,
   language: "English"
}
```

Relationships

1 to N with references

```
{
   title: "MongoDB: The Definitive Guide",
   author: [ "Kristina Chodorow", "Mike Dirolf" ],
   published_date: ISODate("2010-09-24"),
   pages: 216,
   language: "English",
   publisher: {
              name: "O'Reilly Media",
              founded: 1980,
              location: "CA"
            }
}

{
   title: "50 Tips and Tricks for MongoDB Developer",
   author: "Kristina Chodorow",
   published_date: ISODate("2011-05-06"),
   pages: 68,
   language: "English",
   publisher: {
              name: "O'Reilly Media",
              founded: 1980,
              location: "CA"
            }
}
```



```
{
   _id: "oreilly",
   name: "O'Reilly Media",
   founded: 1980,
   location: "CA"
}

{
   _id: 123456789,
   title: "MongoDB: The Definitive Guide",
   author: [ "Kristina Chodorow", "Mike Dirolf" ],
   published_date: ISODate("2010-09-24"),
   pages: 216,
   language: "English",
   publisher_id: "oreilly"
}

{
   _id: 234567890,
   title: "50 Tips and Tricks for MongoDB Developer",
   author: "Kristina Chodorow",
   published_date: ISODate("2011-05-06"),
   pages: 68,
   language: "English",
   publisher_id: "oreilly"
}
```

Documents

Data is stored in documents: JSON-style data structures composed of key and value pairs. On disk, the documents are stored in the BSON serialization format. BSON is a binary representation of JSON documents, although it contains more data types than JSON.

```
var mydoc = {
               _id: ObjectId("5099803df3f4948bd2f98391"),
               name: { first: "Alan", last: "Turing" },
               birth: new Date('Jun 23, 1912'),
               death: new Date('Jun 07, 1954'),
               contribs: [ "Turing machine", "Turing test", "Turingery" ],
               views : NumberLong(1250000)
             }
```

_id

is the primary key for every document in a MongoDB collection. Its role is analogous to a primary key column in a relational database table, but it comes with some unique characteristics essential to how MongoDB operates.
Characteristics:

- Uniqueness: Every document within a collection must have a unique _id field.
- Immutability: The value of the _id field cannot be changed once the document is created.
- Indexing: MongoDB automatically creates a unique index on the _id field for every collection. This ensures fast lookups and efficient enforcement of uniqueness.

_id

Default _id Value: The ObjectId
When insert a document without explicitly providing an _id field, MongoDB automatically generates a value using the ObjectId data type.
Structure of an ObjectId
Is a 12-byte BSON value that is typically represented as a 24-character hexadecimal string. It's designed to be unique, lightweight, and ordered. It is composed of four sections:

4-byte Timestamp: Represents the seconds since the Unix epoch (the document's creation time). This is why ObjectIds are naturally ordered by time.

5-byte Random Value: A machine identifier and process ID combination.

3-byte Incrementing Counter: A counter that starts at a random value.

Because the timestamp is the first 4 bytes, sorting documents by their _id provides a rough chronological ordering, which is often very useful for time-series data or activity feeds.

_id

While the ObjectId is the default and often the best choice, it is not required to use. It can be explicitly set the _id to a value of any valid BSON data type other than an array:

Data Type	Example Use Case
String	Usernames ("john.doe"), email addresses, or custom sequential IDs (e.g., product SKUs).
Integer	Sequential numbers from an external system.
BSON Date	Useful when the date itself is the primary unique identifier for a document.

```
{
  "_id": "user@example.com",
  "name": "User Example",
  " status": "Active“
}
```

MongoDB Index Types

MongoDB supports several index types:

1. Single-Field and Compound Indexes

   Single-Field: An index on a single field in your document (e.g., { email: 1 }).

   Compound: An index on multiple fields (e.g., { country: 1, city: -1 }). The order of the fields is crucial, following the ESR Rule (Equality, Sort, Range) for optimal query efficiency.

2. Multikey Indexes (Arrays)

   If a field in your document is an array and you create an index on it, MongoDB automatically creates a Multikey Index.

   It creates a separate index entry for every element in the array.

   Use Case: Searching documents based on items within a list, such as a document with a tags array.

MongoDB Index Types

3. Geospatial Indexes (2dsphere & 2d)
   - 2dsphere: Used to index and query geospatial data defined on an Earth-like sphere. It supports queries like find documents within a polygon or find the nearest locations.
   - 2d: Used for planar (flat) geometry or simple Cartesian coordinates.
4. Text Indexes
   - Supports full-text search queries on string content in your collections.
   - The index stores all the significant words from the indexed fields and enables linguistic-aware searches.
   - Use Case: Searching a blog post or product catalog for keywords.
5. Hashed Indexes
   - These indexes store a hash of the value of the indexed field.
   - They are primarily used to support sharding based on a hashed shard key.
   - Limitation: Only support equality matches, not range queries.

MongoDB Index Types

Index Type	Example Document	MongoDB Shell Command
Single-Field	{ username: "user1" }	db.users.createIndex({ username: 1 })
Compound	{ country: "USA", city: "NYC" }	db.stores.createIndex({ country: 1, city: -1 })
Unique	{ email: "a@b.com" }	db.users.createIndex({ email: 1 }, { unique: true })
Multikey (Array)	{ tags: ["a", "b"] }	db.products.createIndex({ tags: 1 })
Geospatial	{ loc: [40, 74] }	db.restaurants.createIndex({ loc: "2dsphere" })
Text	{ description: "..." }	db.articles.createIndex({ description: "text" })

MongoDB Index Types

Option	Description	Use Case
unique: true	Prevents the collection from having documents with duplicate values for the indexed key.	Enforcing uniqueness for fields like email or SSN.
sparse: true	Only indexes documents that contain the indexed field. Documents missing the field will **not** be indexed.	When an optional field only needs to be indexed for a subset of documents.
expireAfterSeconds	Allows MongoDB to automatically remove documents after a certain time period. These are called **TTL (Time To Live) Indexes**.	Automatic cleanup of session data, logs, or cache documents.
background: true	Builds the index in the background, minimizing the impact on database operations. (Default is often foreground for faster creation).	Large production collections where downtime is unacceptable.

Queries - Read

Queries in MongoDB are performed using the MongoDB Query Language (MQL) and involve specifying conditions to retrieve documents from collections based on certain criteria. MongoDB provides a rich set of query operators and methods for filtering, sorting, and projecting data.
Read operations include all operations that return a cursor in response to queries, and additionally include a number of aggregation operations that do not return a cursor but have properties similar to queries. These commands include `aggregate`, `count` and `distinct`.
In the Mongo shell, the `find()` and `findOne()` methods are used for read operations.

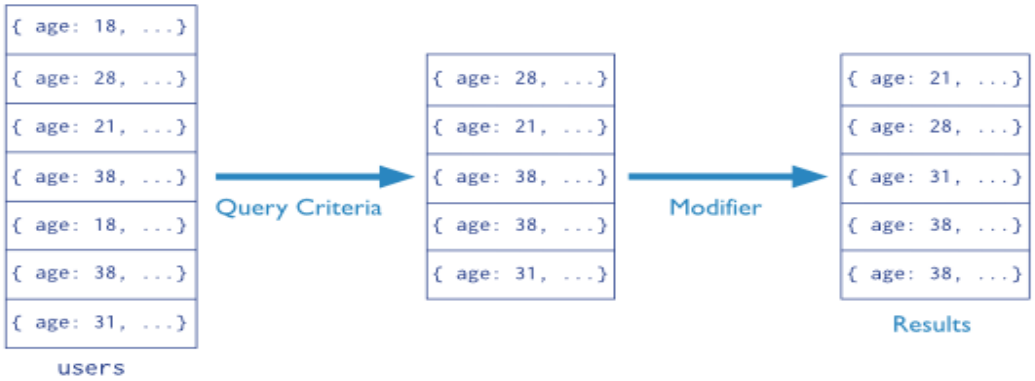
Queries - Read

```
db.users.find(
    { age: { $gt: 18 } },
    { name: 1, address: 1 }
).limit(5)
```

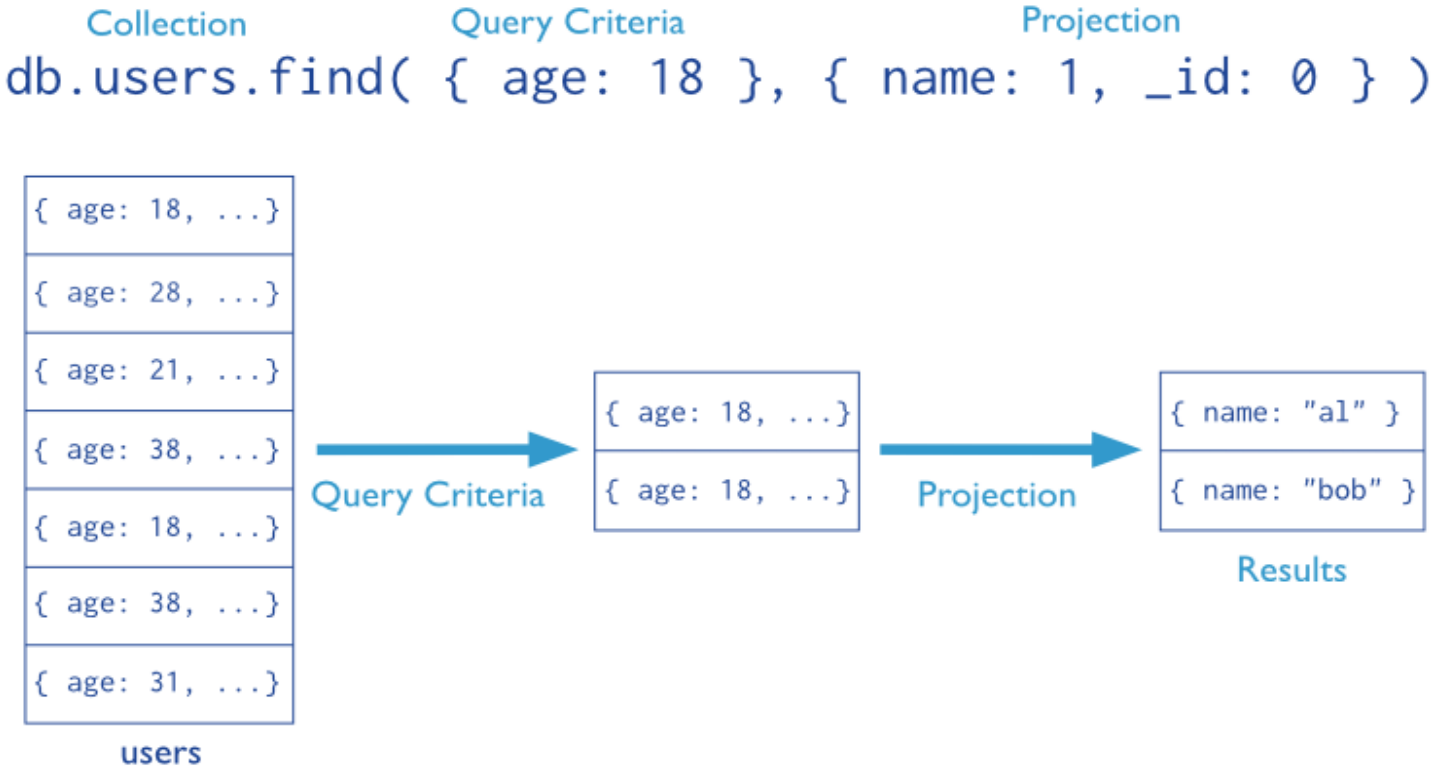
- collection
- query criteria
- projection
- cursor modifier

Collection | Query Criteria | Modifier

```
db.users.find( { age: { $gt: 18 } } ).sort( {age: 1 } )
```



Queries - Read



Queries - Write

Write operations are atomic (at the document level);
A write operation can atomically affect only a single document and a single collection.
Operations that modify more than one document in a collection act on only one document at a time.
A denormalized data model with embedded data facilitates atomic write operations, since a single write operation can insert or update the data for an entity.
In a normalized model, the data would be split across multiple collections and would require multiple write operations that are not collectively atomic.

Queries - Write

- Insert: adding new data into collection

```
db.users.insert (          <- collection
   {
     name: "sue",          <- field: value
     age: 26,              <- field: value
     status: "A"           <- field: value
   }                       } document
)
```

- Update: modify data (criteria can be used)

```
db.users.update(                    <- collection
   { age: { $gt: 18 } },            <- update criteria
   { $set: { status: "A" } },       <- update action
   { multi: true }                  <- update option
)
```

- Remove: delete data (criteria can be used)

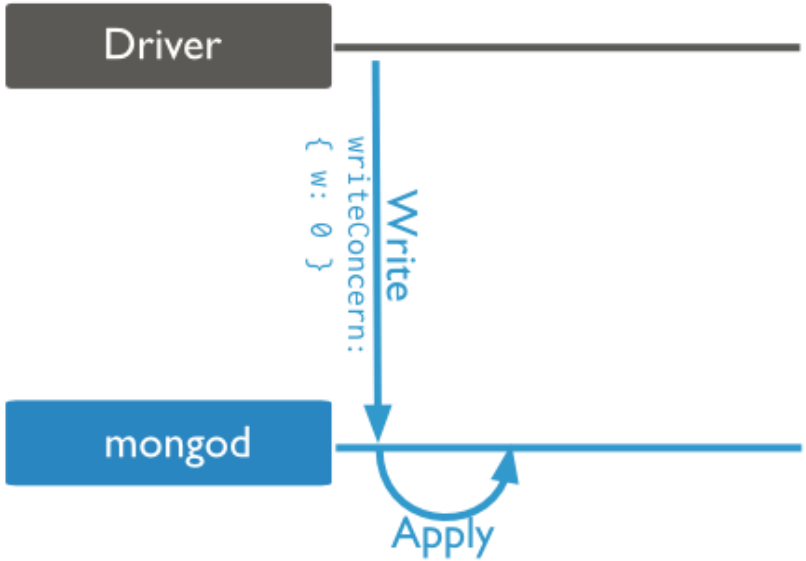
```
db.users.remove(          <- collection
   { status: "D" }        <- remove criteria
)
```

Write Concern

The guarantee provided when reporting the successful execution of a write operation.
The "write concern" configuration determines the level of this guarantee (similar to the isolation level).
Write operations with a weak write concern execute quickly. In some failure cases, write operations initiated with a weak write concern may not be persistent.
With a stronger write concern, the client application must wait longer for the confirmation of write operations.
MongoDB offers different levels of write concern to suit various needs. The write concern can be adjusted to balance performance and persistence requirements.
By default, it is set to "Acknowledged."

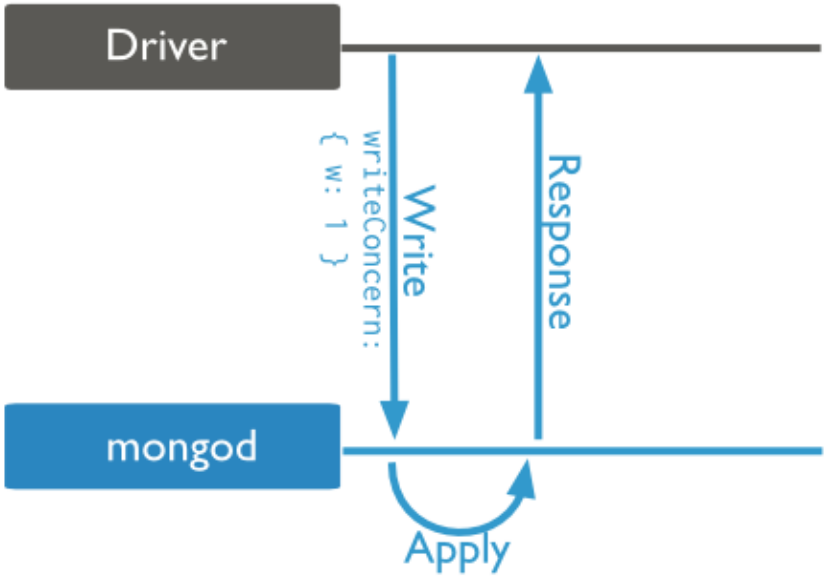
Write Concern - Unacknowledged

Using this mode, MongoDB does not send a confirmation for the write operation.



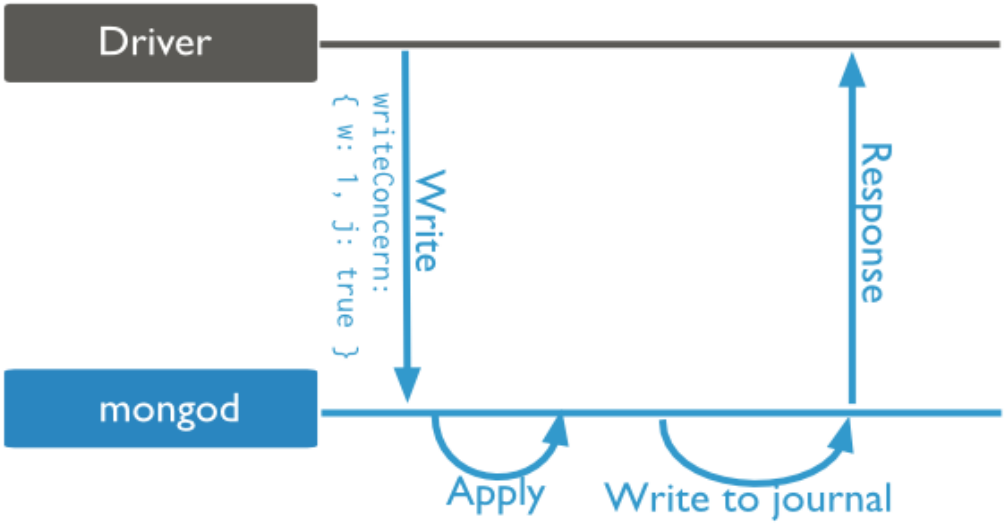
Write Concern - Acknowledged

Using this mode, MongoDB sends a confirmation for the receipt of the write operation request and data saving in memory.



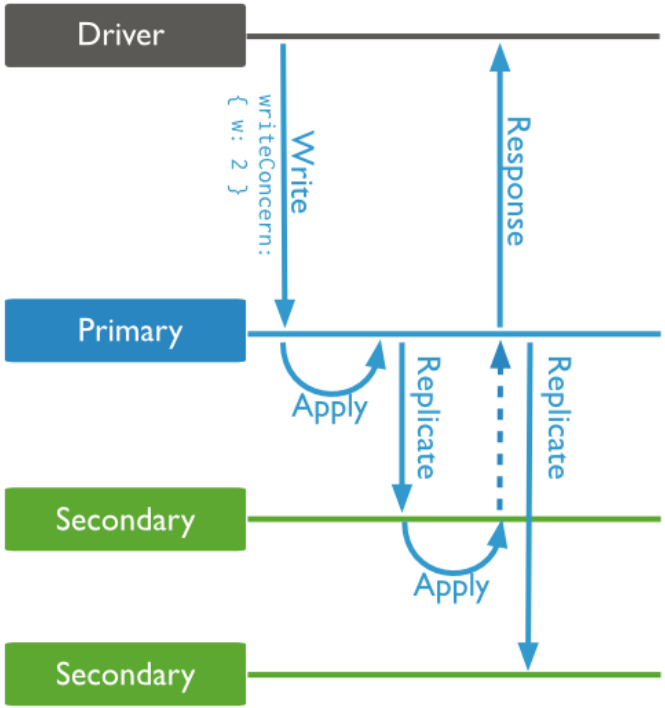
Write Concern - Journaled

In this mode, MongoDB acknowledges the receipt of the write request only after the write operation has been saved to the log file. This ensures that the state can be restored after a shutdown or power failure.



Write Concern – Replica Acknowledged

In this mode, MongoDB guarantees that the write operation is propagated to all components of a replica set.



Aggregations

Aggregation operations process multiple documents and return computed results. They are useful for:

- Grouping values from multiple documents.
- Performing operations on grouped data to return a single result.
- Analyzing data changes over time.

For aggregation operations, you can use:

- Aggregation pipelines (the recommended method).
- Single-purpose aggregation methods (they are simple but lack the capabilities of a pipeline).

Aggregations – Pipes

An aggregation pipeline consists of one or more stages for processing documents:

- Each stage performs an operation on the input documents, such as filtering, grouping, or calculating values.
- The documents generated in one stage are passed to the next stage.
- An aggregation pipeline can return results for groups of documents, such as totals, averages, maximums, or minimums.
- Operations can be performed on collections stored in shards.
- There are limitations on the result size, the number of stages, and the amount of memory used.

Aggregations – Pipes

```
db.orders.aggregate( [

   // Stage 1: Filter pizza order documents by pizza size
   {
      $match: { size: "medium" }
   },

   // Stage 2: Group remaining documents by pizza name and calculate total quantit
   {
      $group: { _id: "$name", totalQuantity: { $sum: "$quantity" } }
   }

] )
```

Aggregations – Uniscope Functions

Single-purpose aggregation methods (they are simple but lack the capabilities of a pipeline):

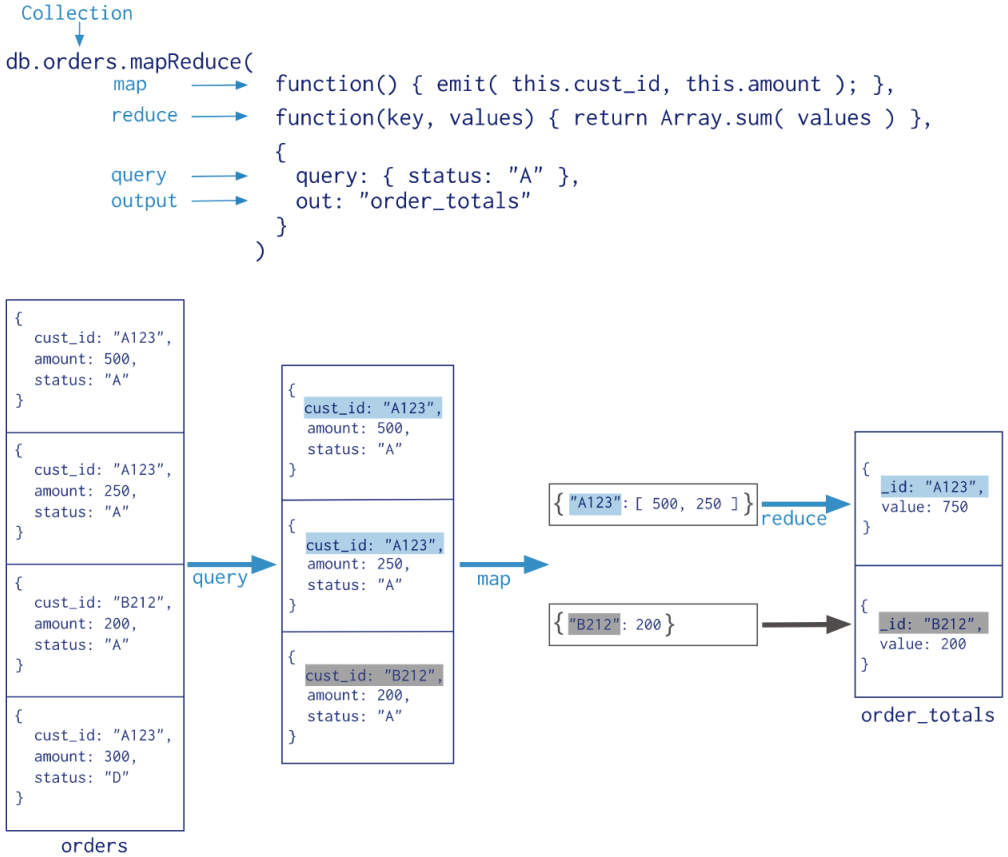
- `db.collection.estimateDocumentCount()`: Provides an approximate total number of documents in a collection.
- `db.collection.count()`: Provides the number of documents in a collection.
- `db.collection.distinct()`: Returns an array of documents that have distinct values for the specified field.

Map - Reduce

It is a paradigm for processing large volumes of data and generating aggregations.

The `map()` function is applied to each input document and emits key-value pairs.

In the `reduce()` stage, multiple values for the same key are aggregated.



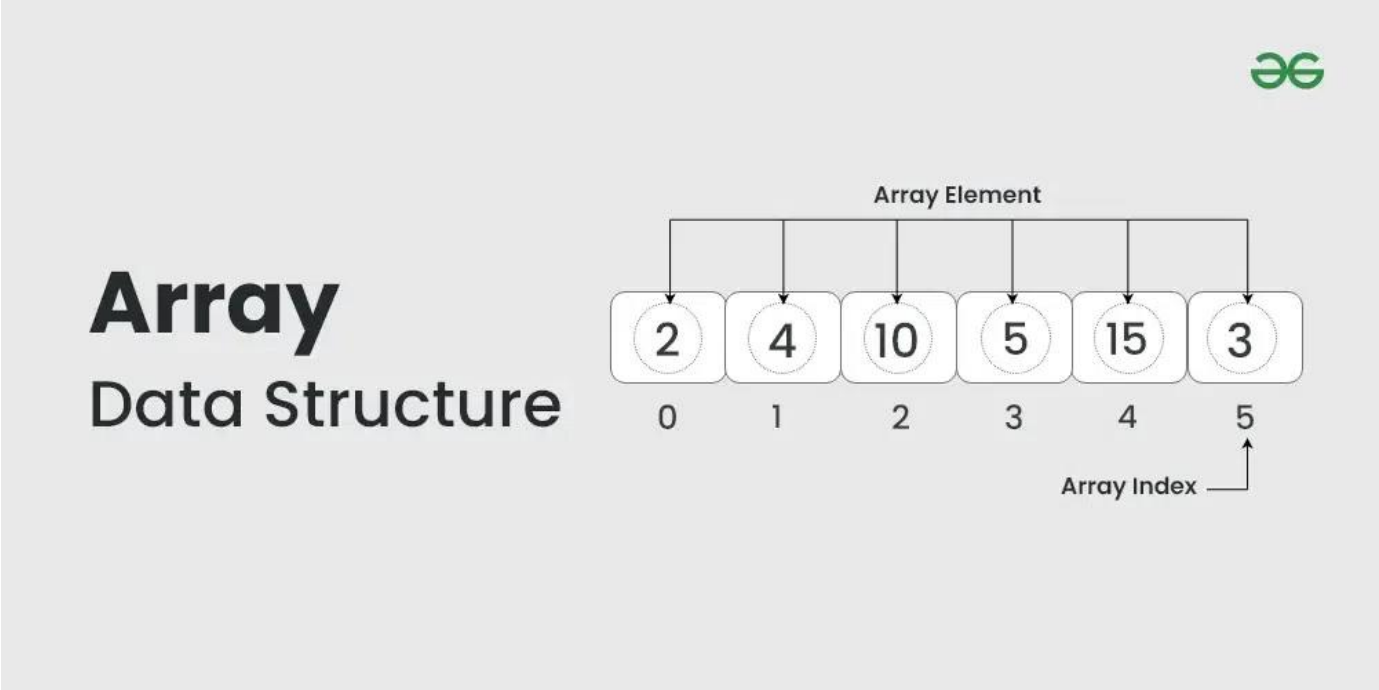
Vector Databases

What, Why & How?

Agenda

- What is a vector
- Embeddings
- What is a vector DB
- How does it work
- Why do we need a vector DB
- Relational DBs vs Vector DBs
- Types of vector DBs

What is a vector?



What is a vector?

Abstract representation of “something”

What is a vector?

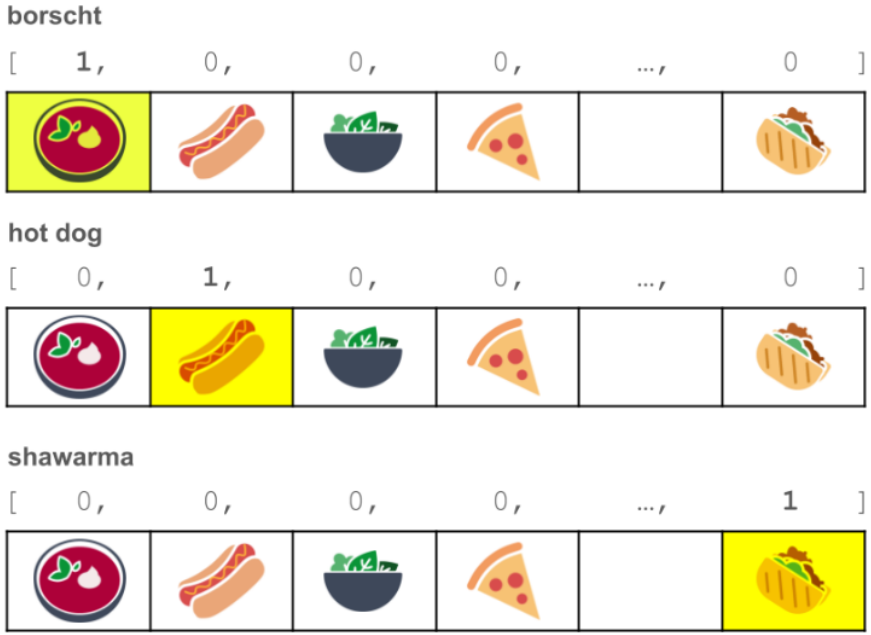
Abstract representation of “something”

Weekly gym schedule

Mo	Tu	We	Th	Fr	Sa	Su
1	0	1	0	0	1	0

What is a vector?

Abstract representation of “something”

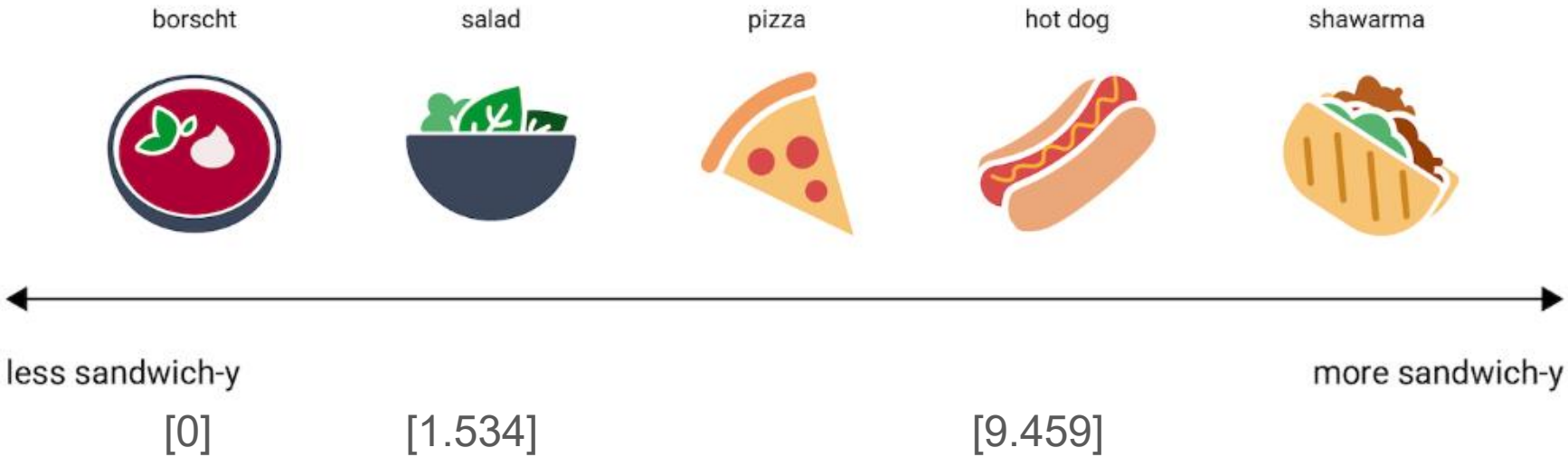


One-hot encoding

What is a vector?

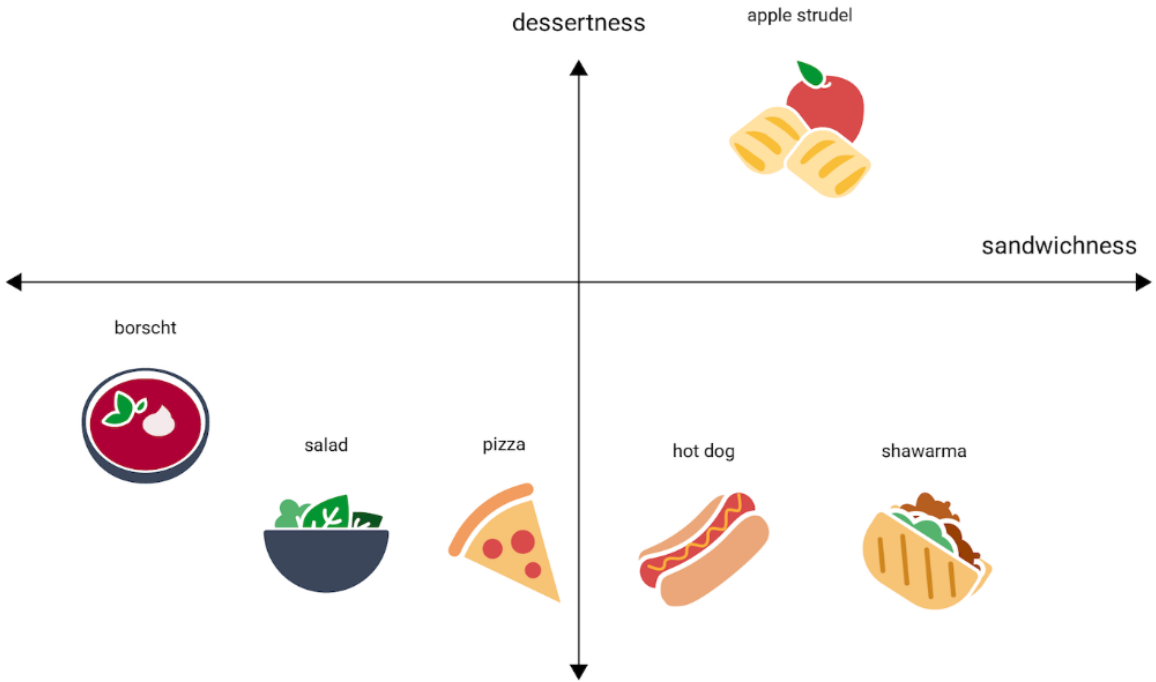
Abstract representation of “something”

One dimensional



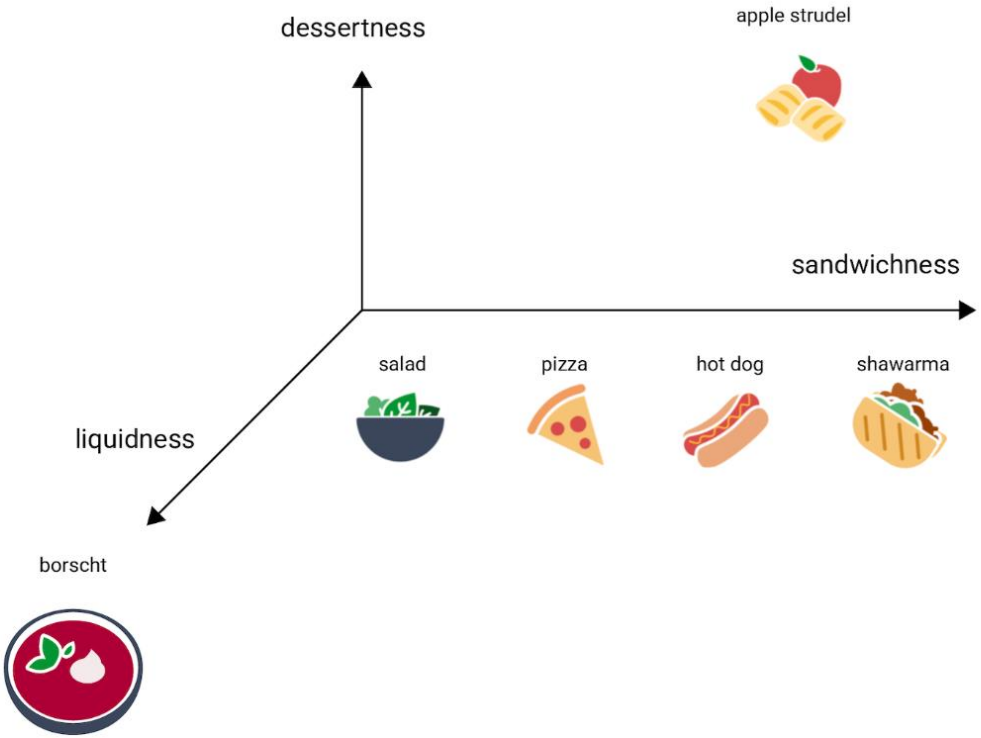
What is a vector?

Abstract representation of “something”



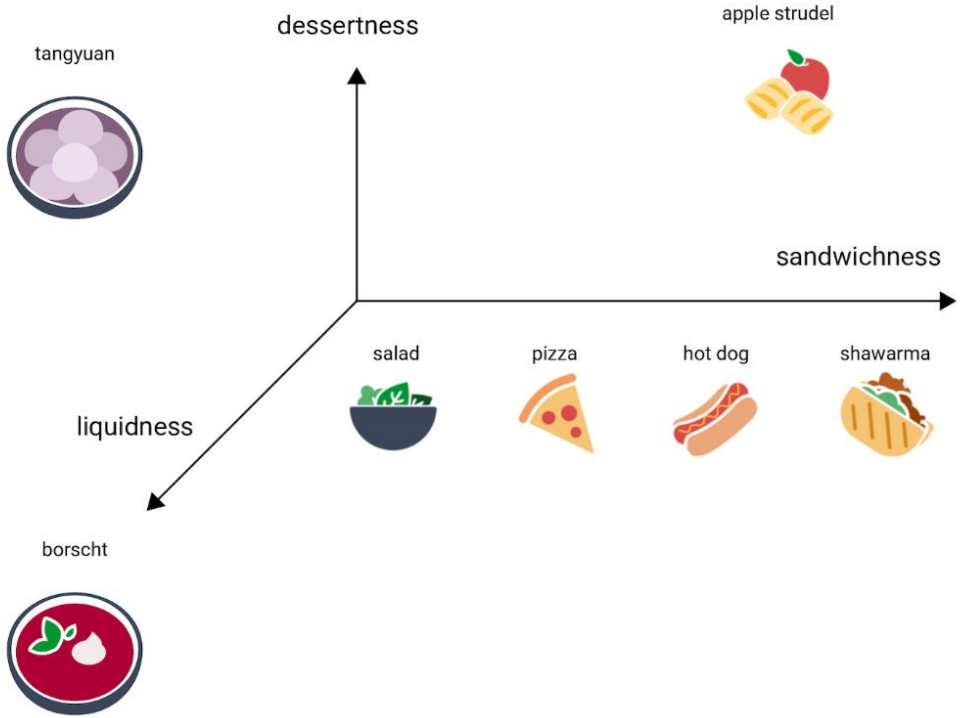
What is a vector?

Abstract representation of “something”



What is a vector?

Abstract representation of “something”



What is a vector?

Abstract representation of “something”

Weekly gym schedule

Mo	Tu	We	Th	Fr	Sa	Su
1	0	1	0	0	1	0

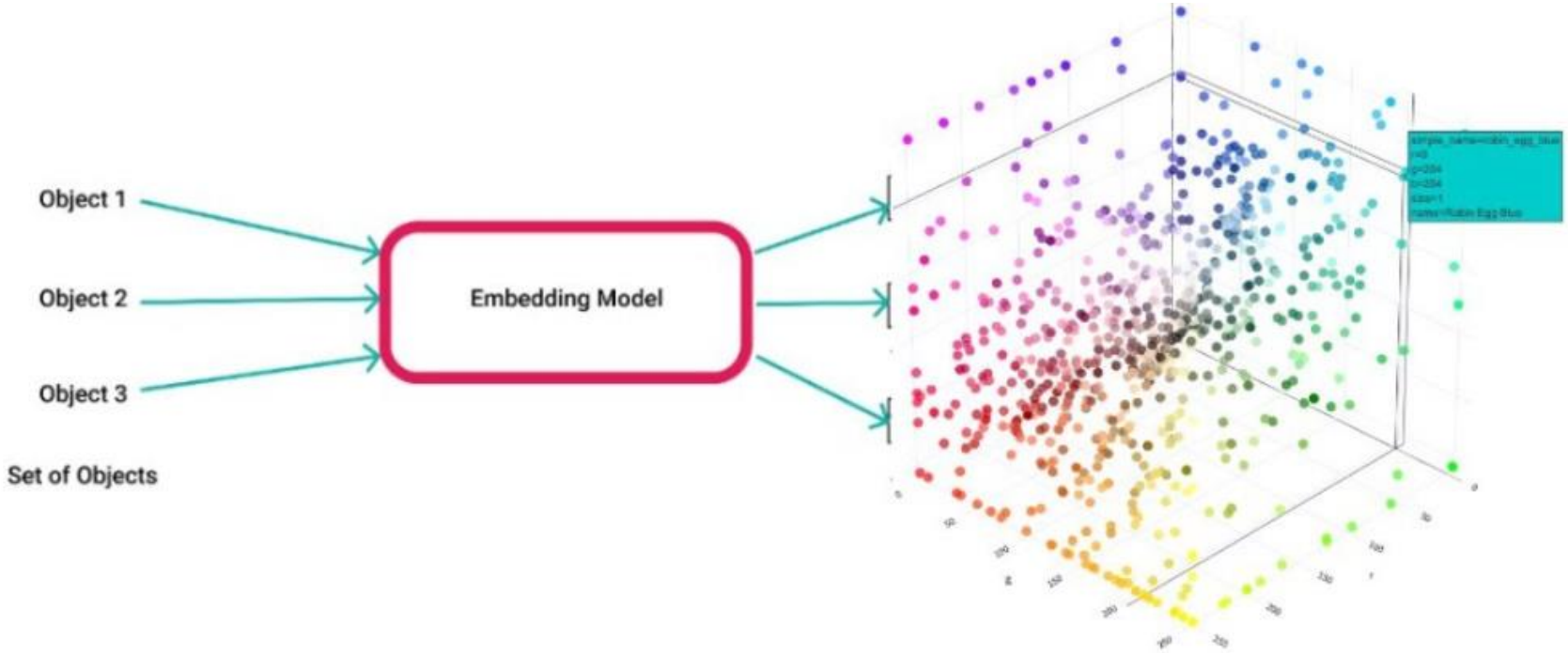
Embeddings

Vectorized representation of “something”

1	0	1	0	0	1	0
1	0	1	0	1	1	1
0	1	0	0	1	0	0
0	0	0	0	0	0	0

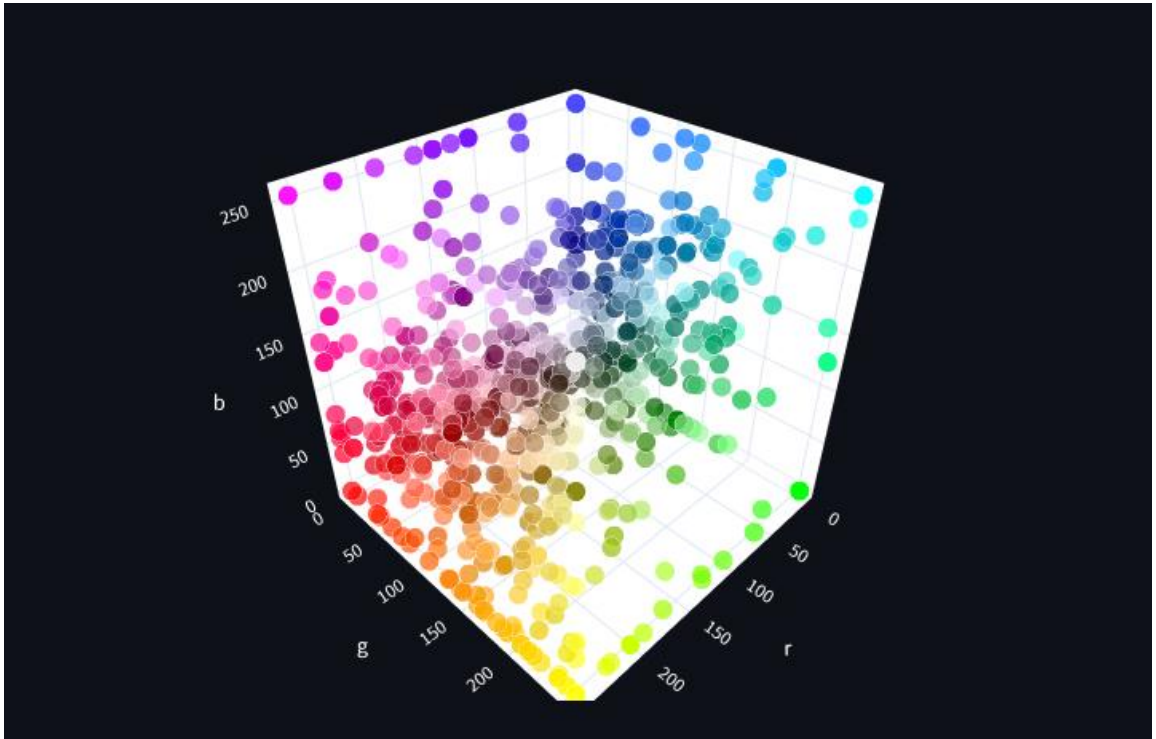
Embedding Models

An algorithm that generates vector representations of “things”



Embedding Models

Data represented in the Embedding Space

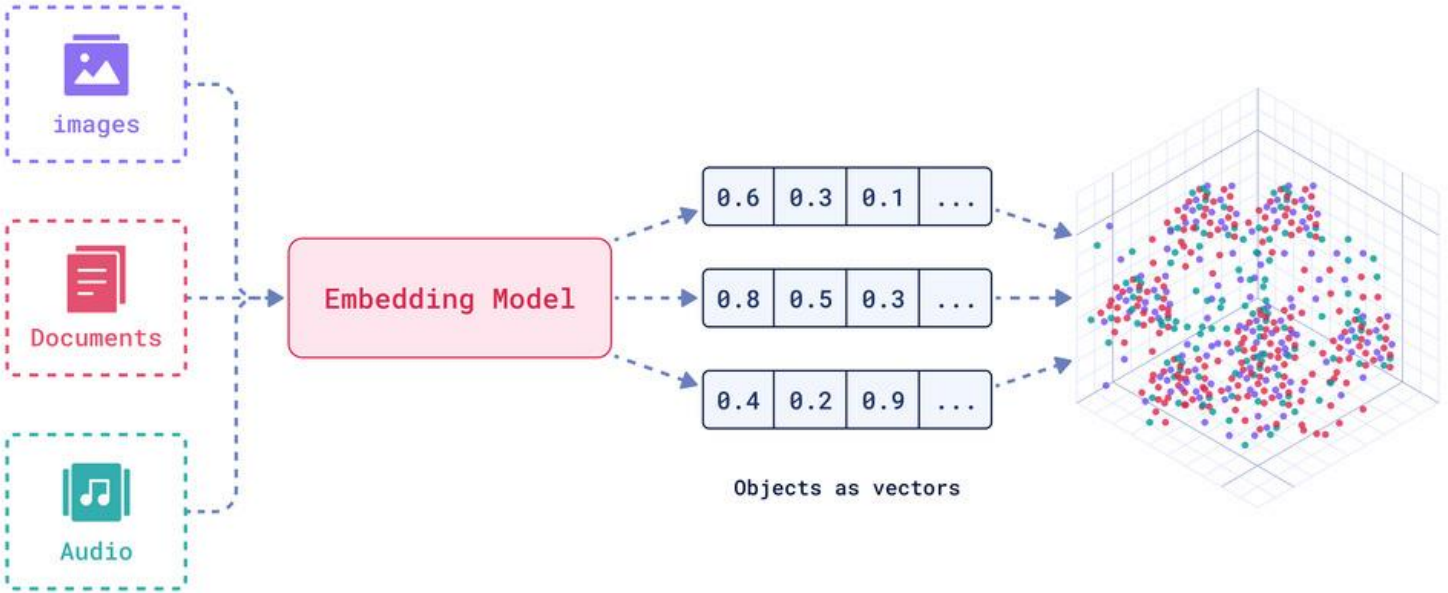


Embedding Models

What things can be embedded?

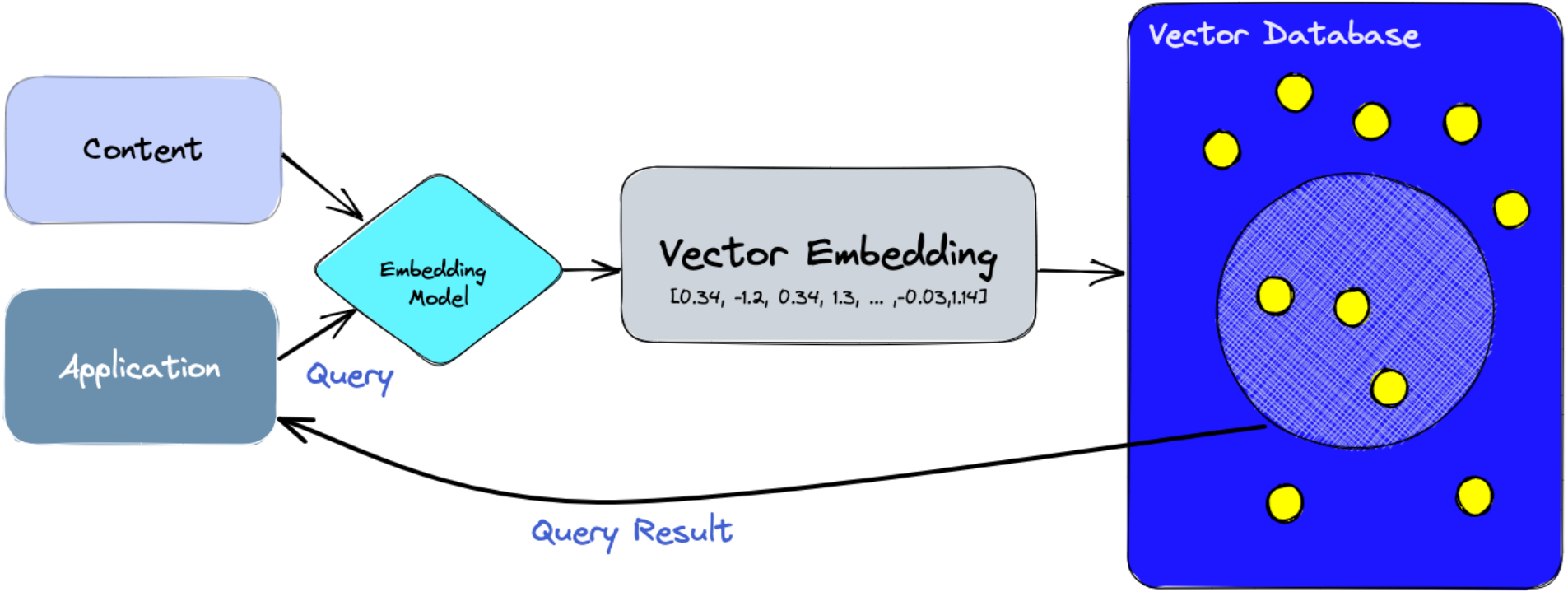
Embedding Models

What things can be embedded? EVERYTHING!



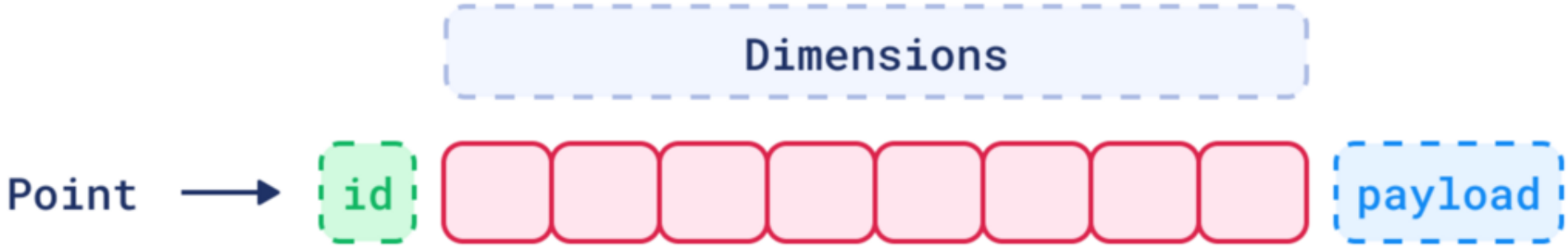
Vector Databases

A database specifically designed to work with vectors(embeddings)



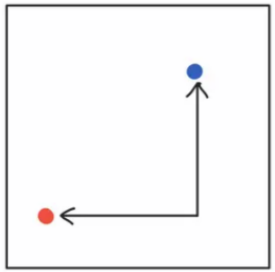
Data representation

Point = Id + Data itself + vector representation + metadata

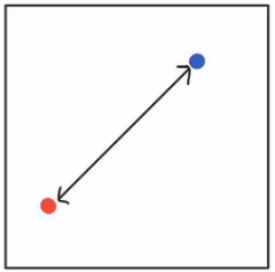


Distance metrics

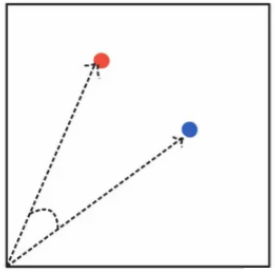
Relations between points in a space



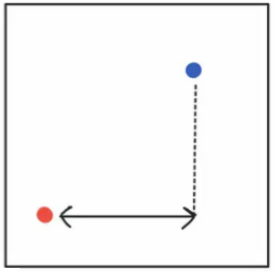
Manhattan



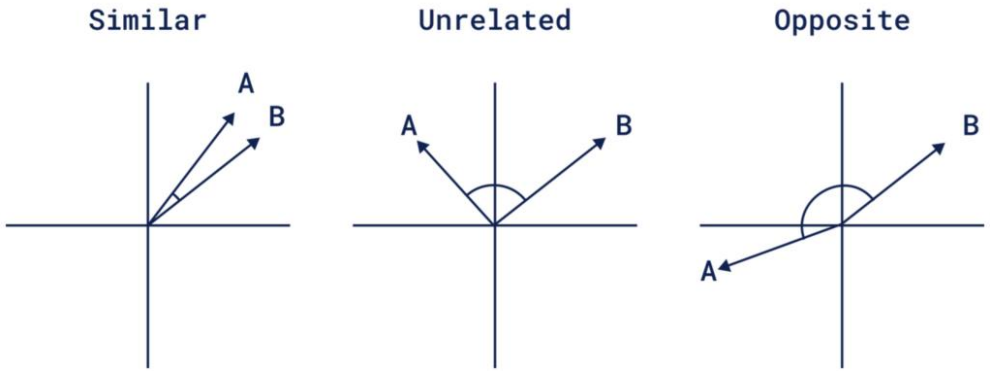
Euclidean

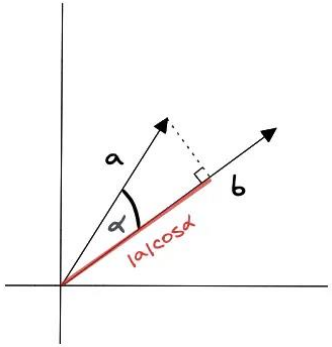


Cosine



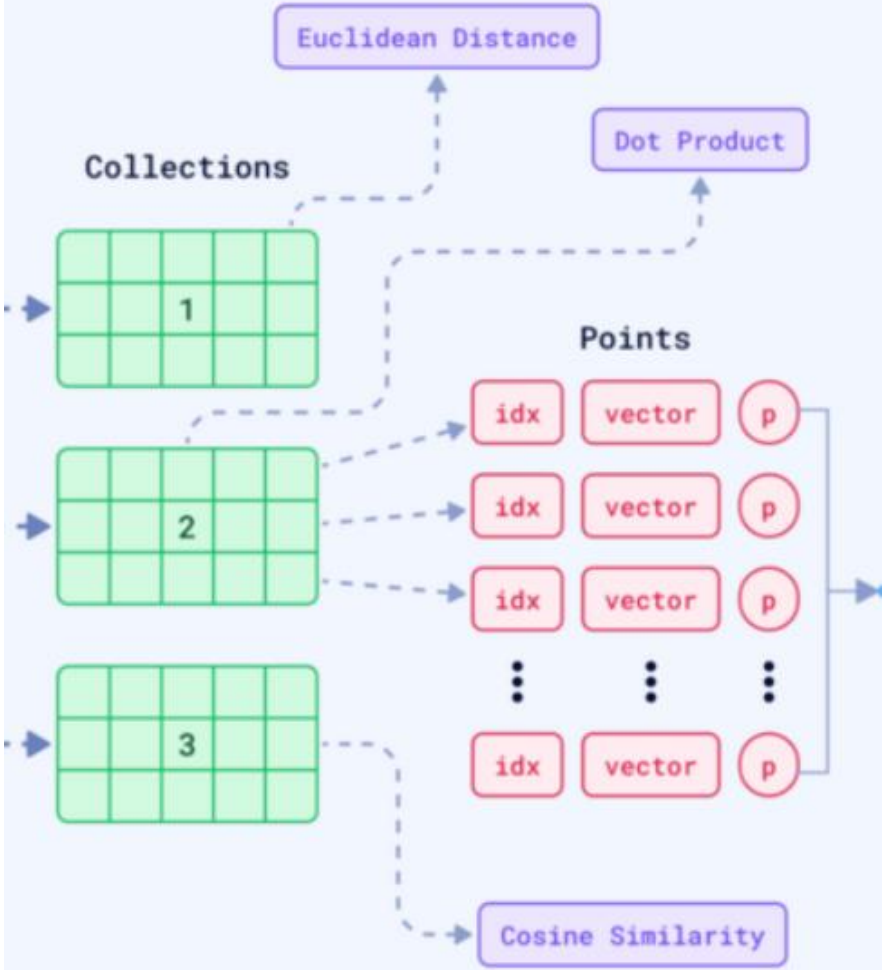
Chebyshev





Data storage

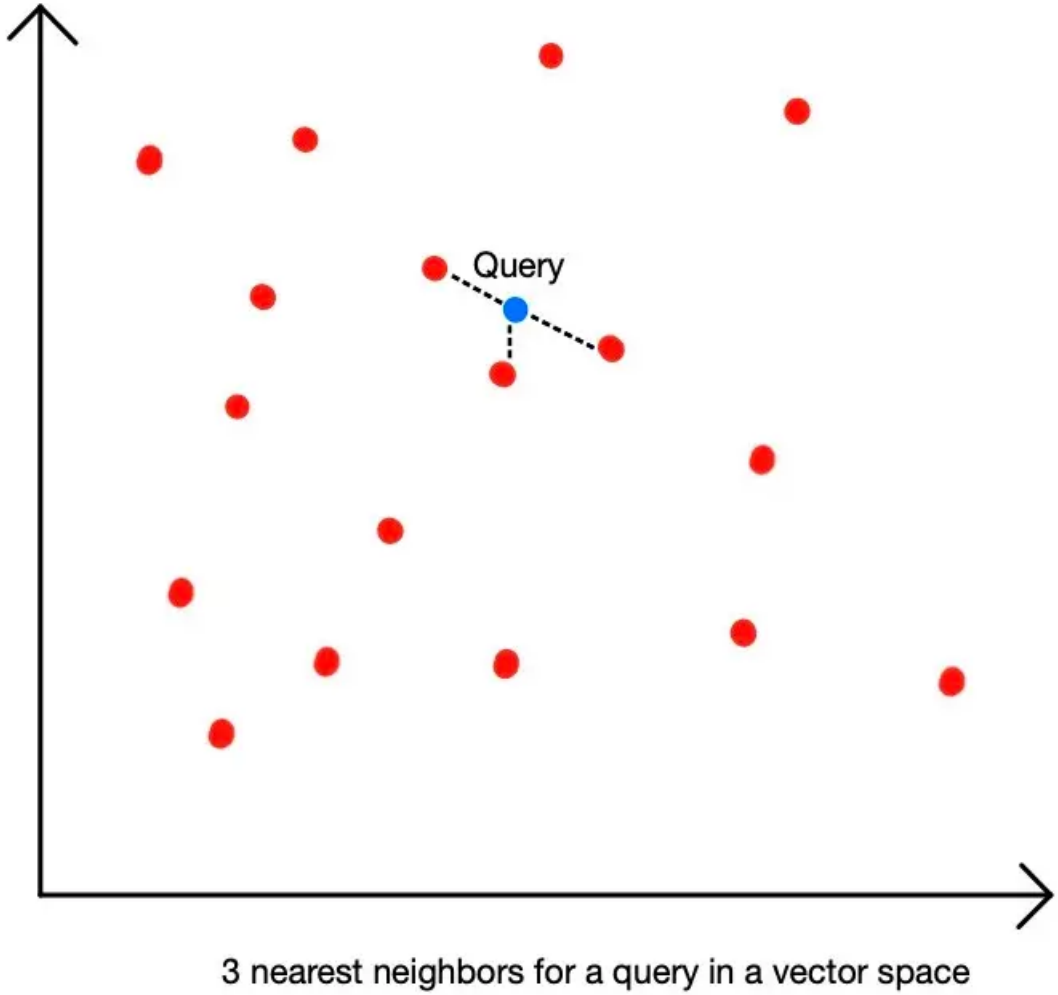
Collections = vector indexes



Indexing algorithms

Enabling fast querying with fast traversable data structures

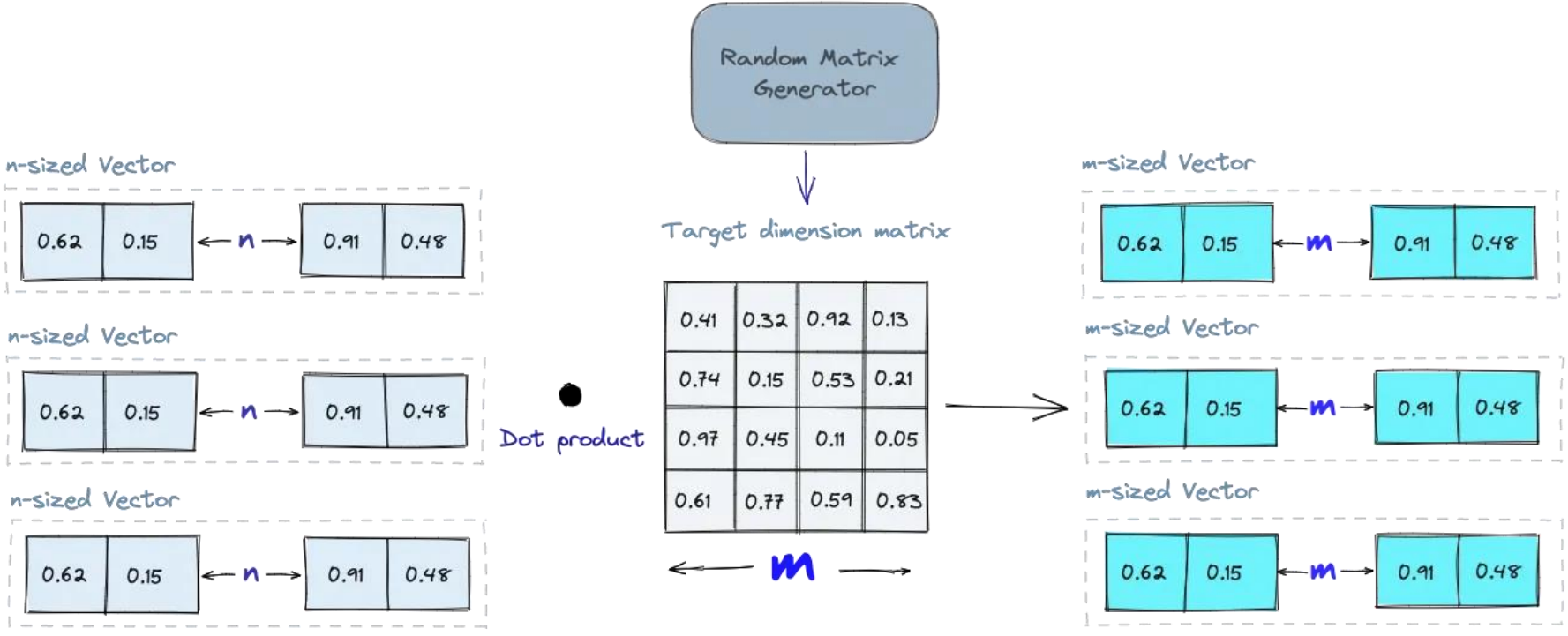
Approximate Nearest Neighbour(ANN)



3 nearest neighbors for a query in a vector space

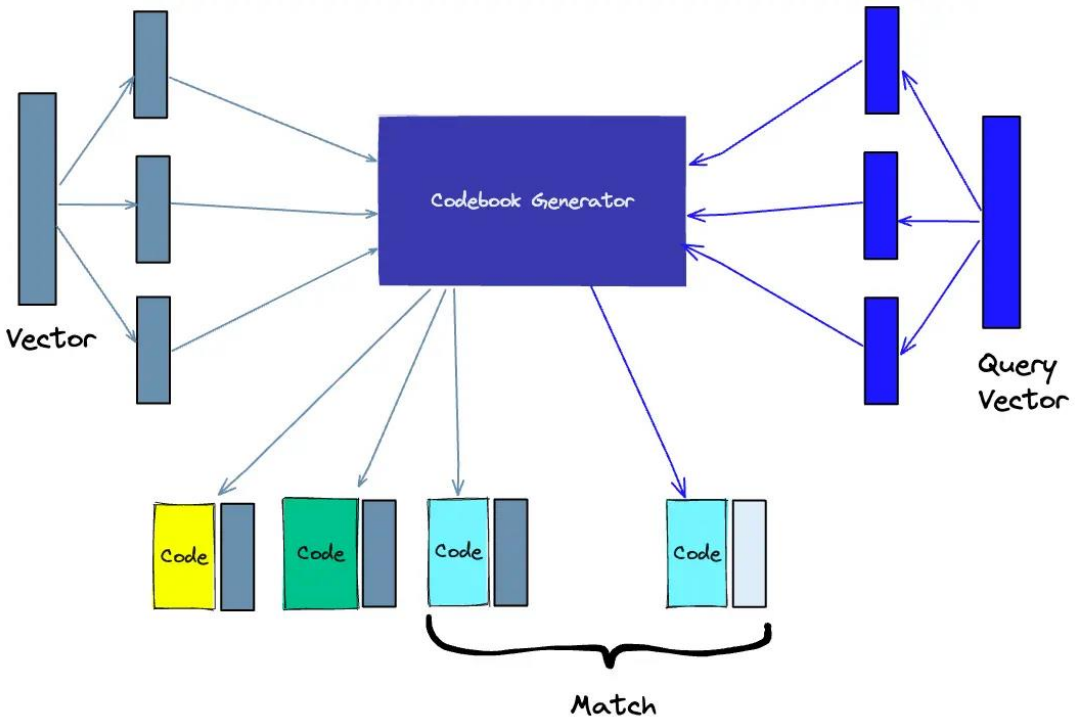
Indexing algorithms

Random Projection - dimensionality reduction



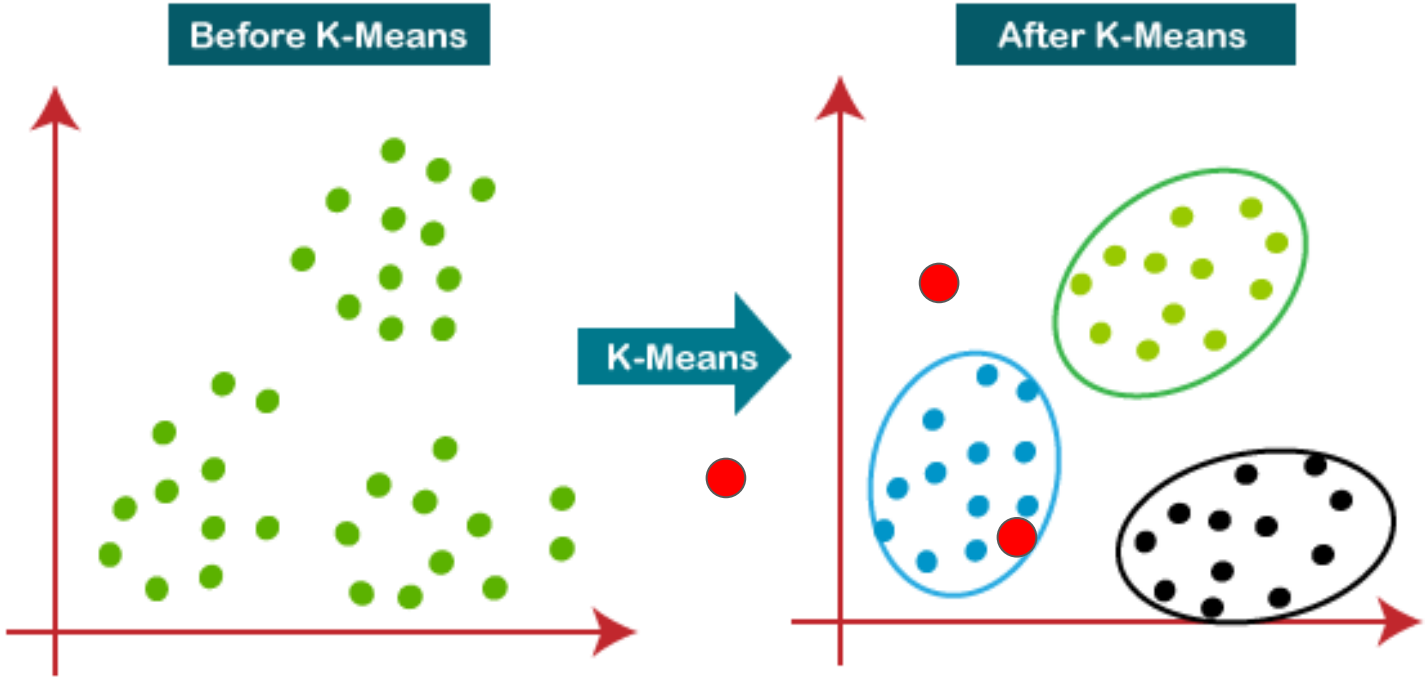
Indexing algorithms

Product Quantization - fractional hashing with k-means clustering for hashes = dimensionality reduction



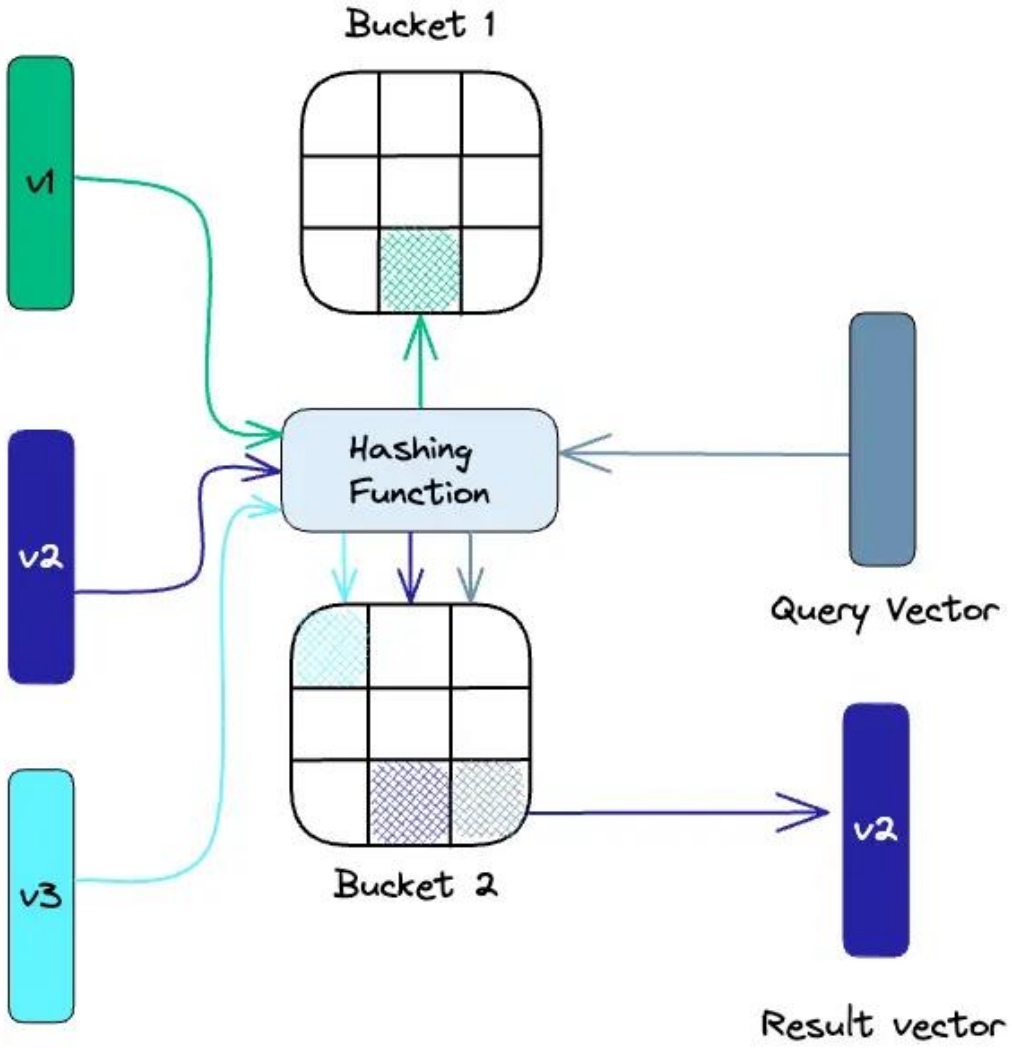
Indexing algorithms

Product Quantization - fractional hashing with k-means clustering for hashes = dimensionality reduction



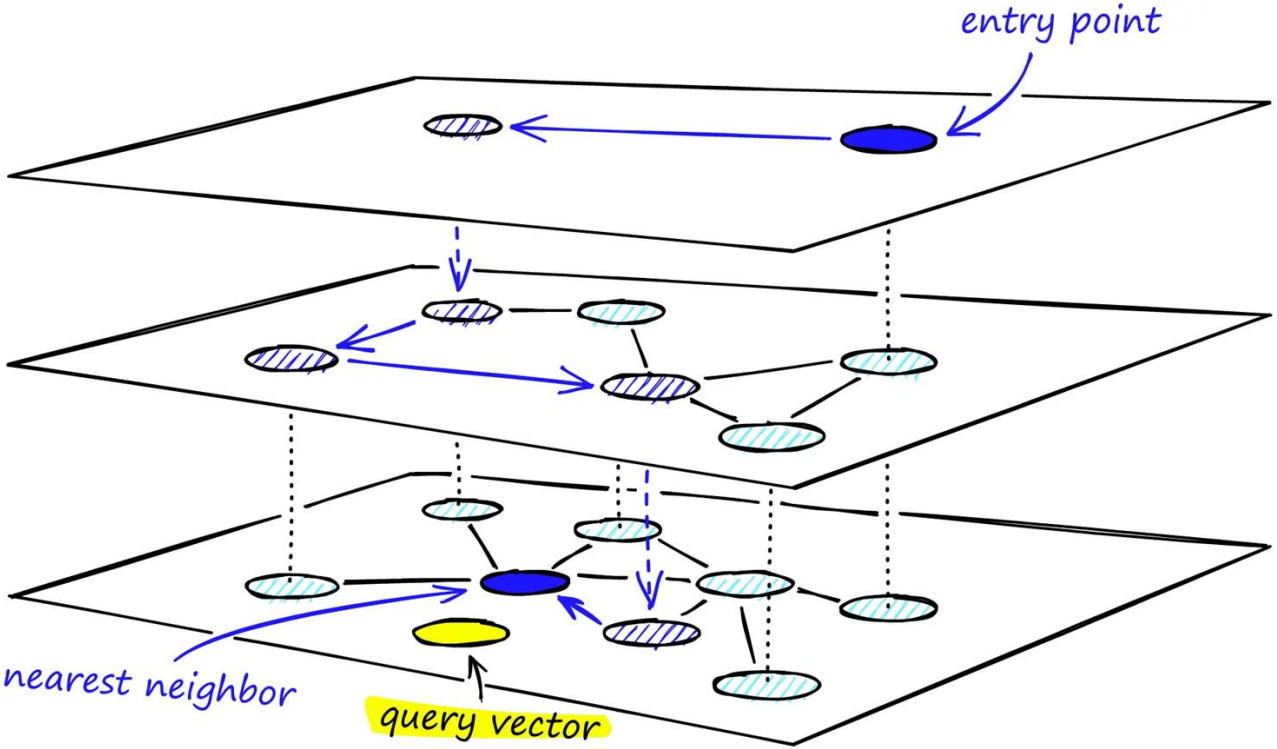
Indexing algorithms

Locality-sensitive hashing - reducing the search space



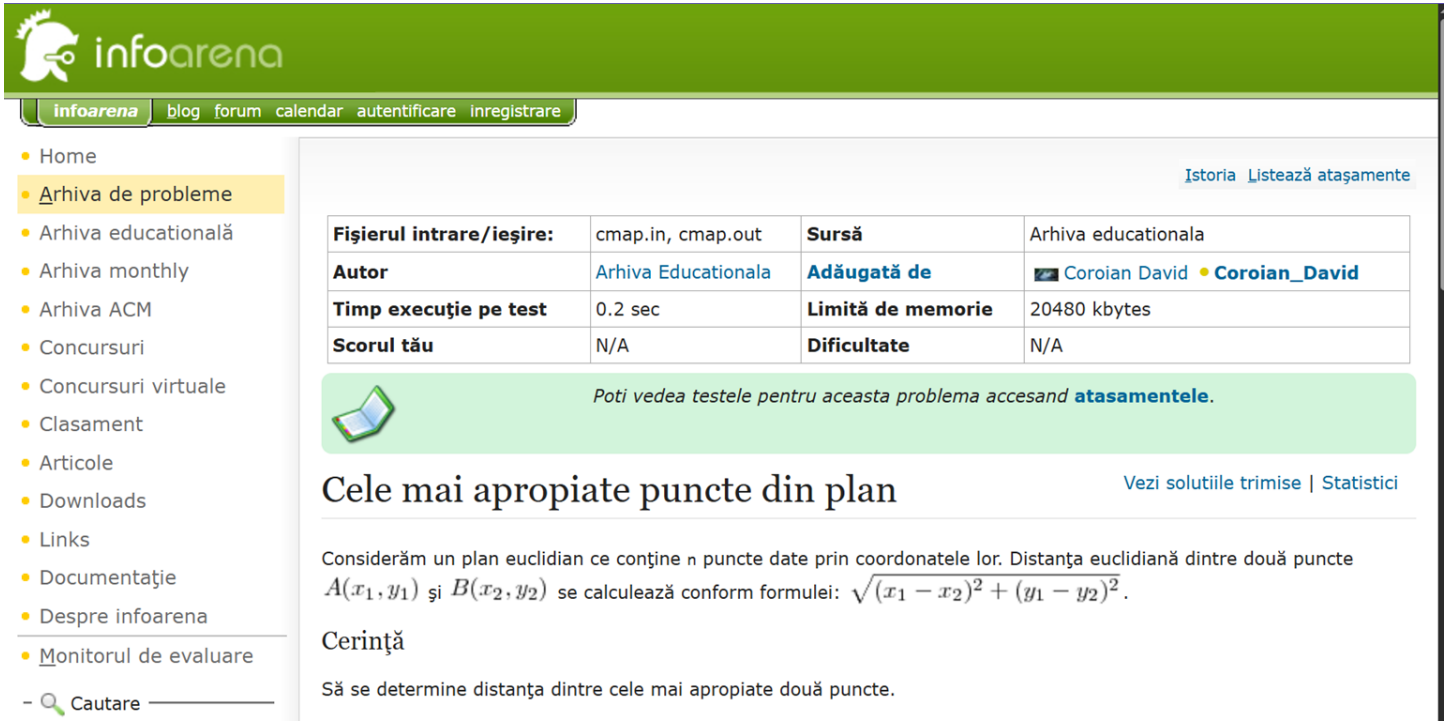
Indexing algorithms

Hierarchical Navigable Small World (HNSW) - state of the art



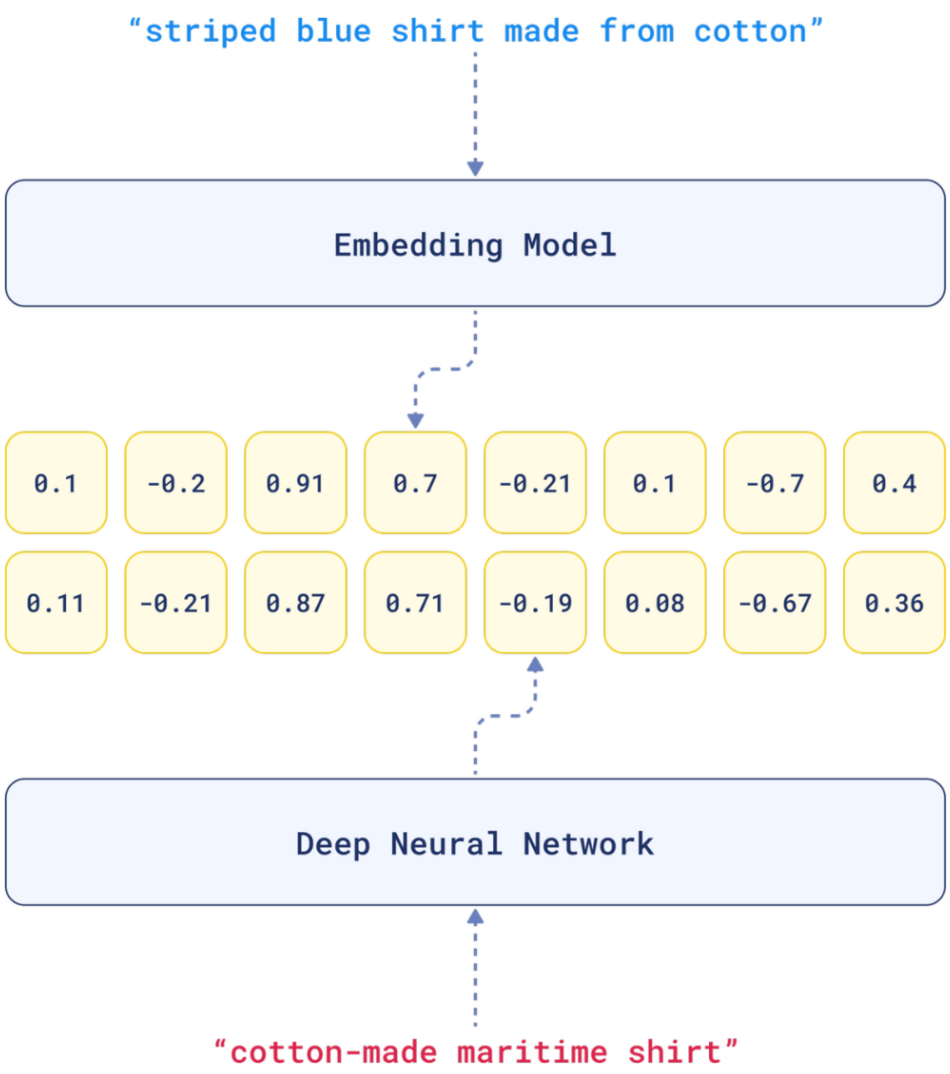
Indexing algorithms

Searching for the closest points in a plane: https://www.infoarena.ro/problema/cmap



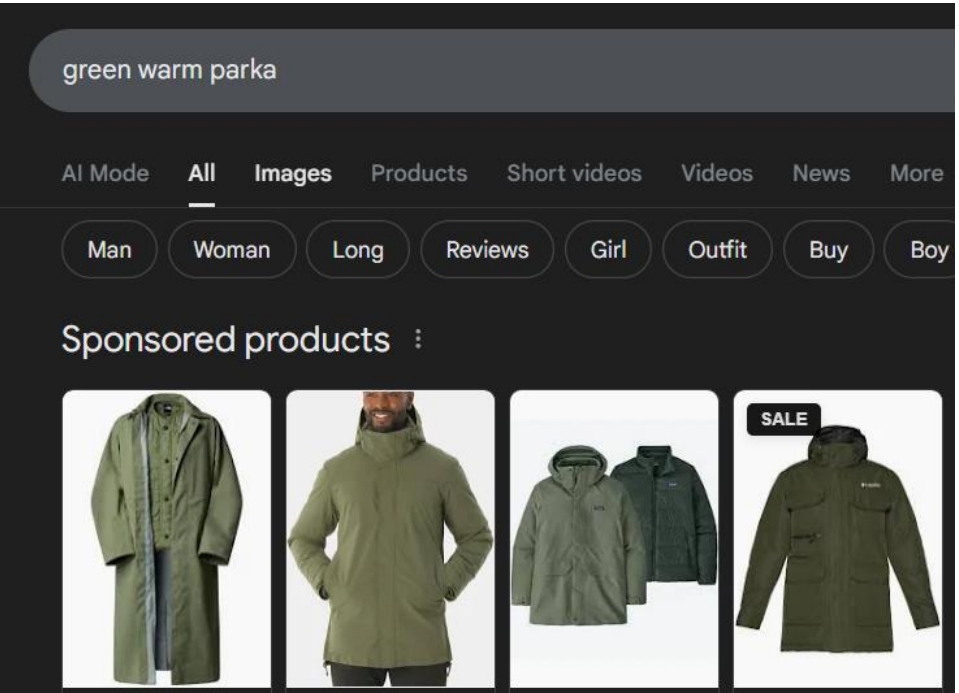
Features of a Vector DB

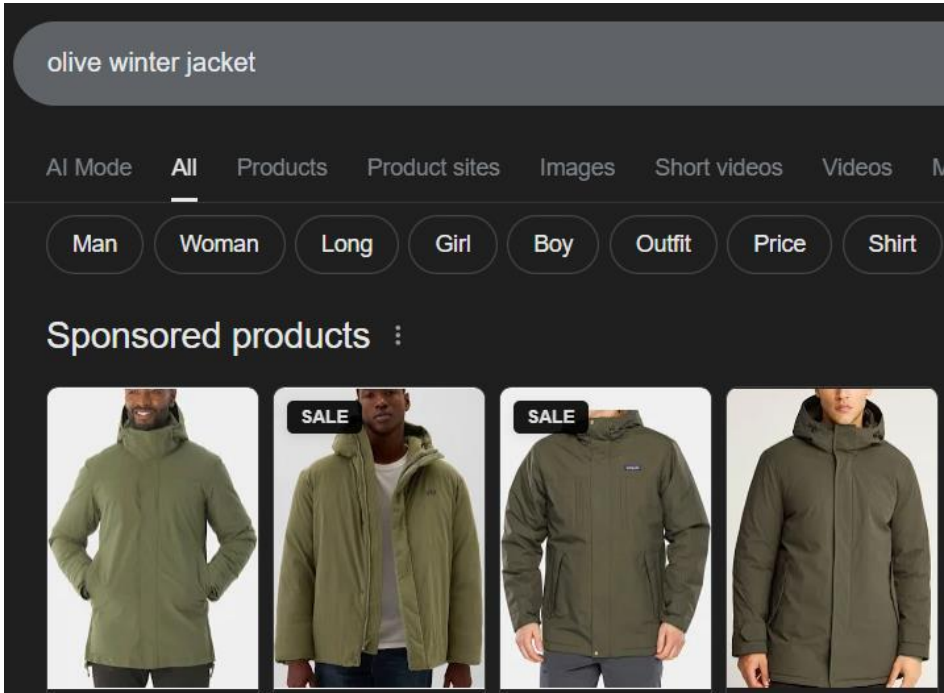
Similarity & Semantic Search



Features of a Vector DB

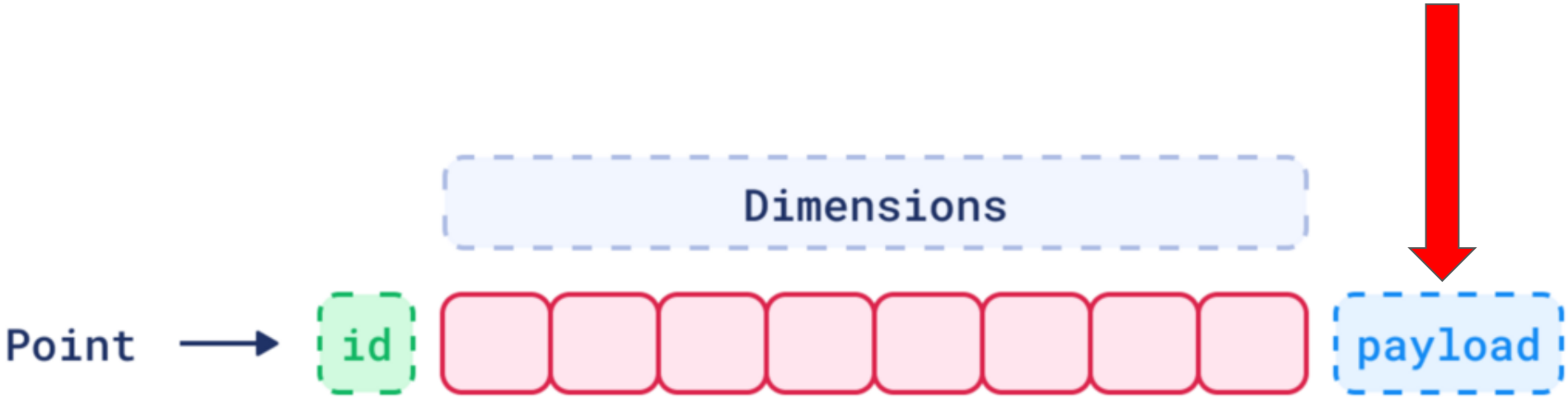
Similarity & Semantic Search





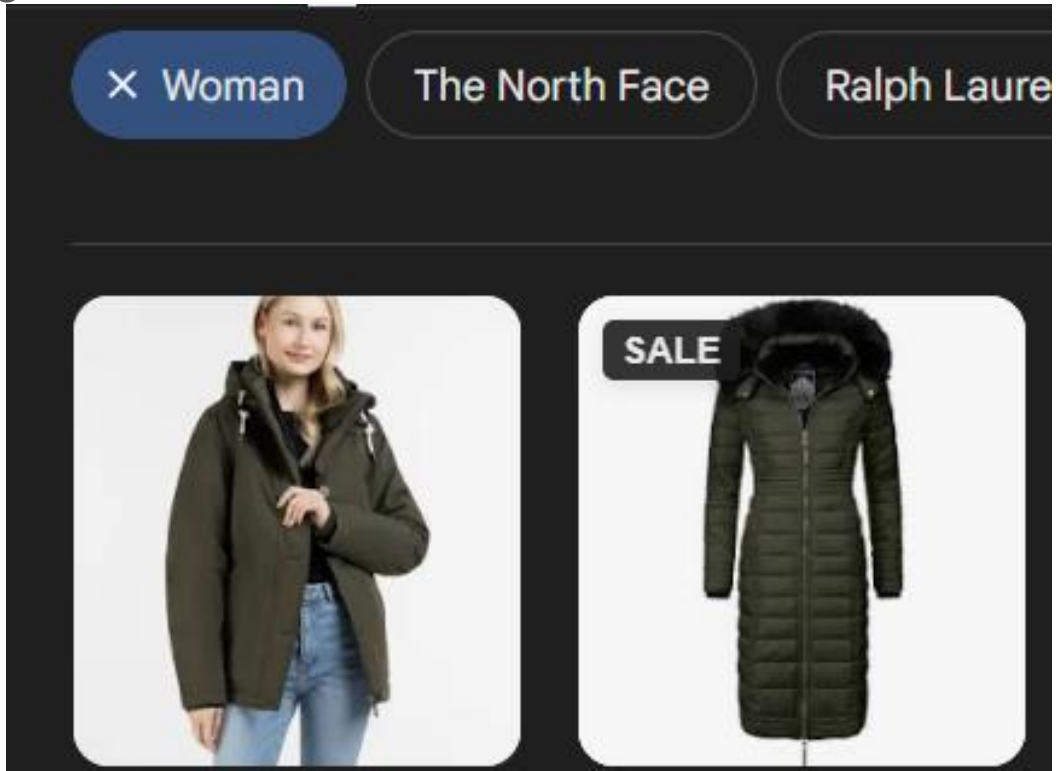
Data representation

Point = Id + Data itself + vector representation + **METADATA**



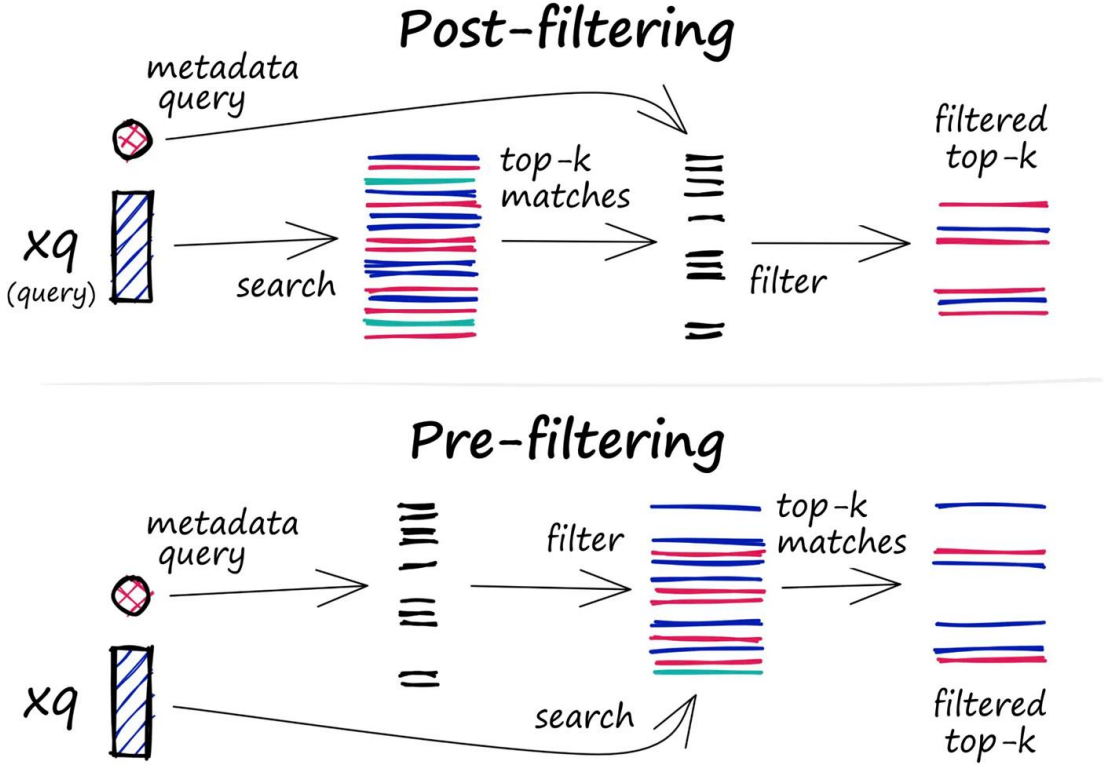
Features of a Vector DB

Metadata filtering



Features of a Vector DB

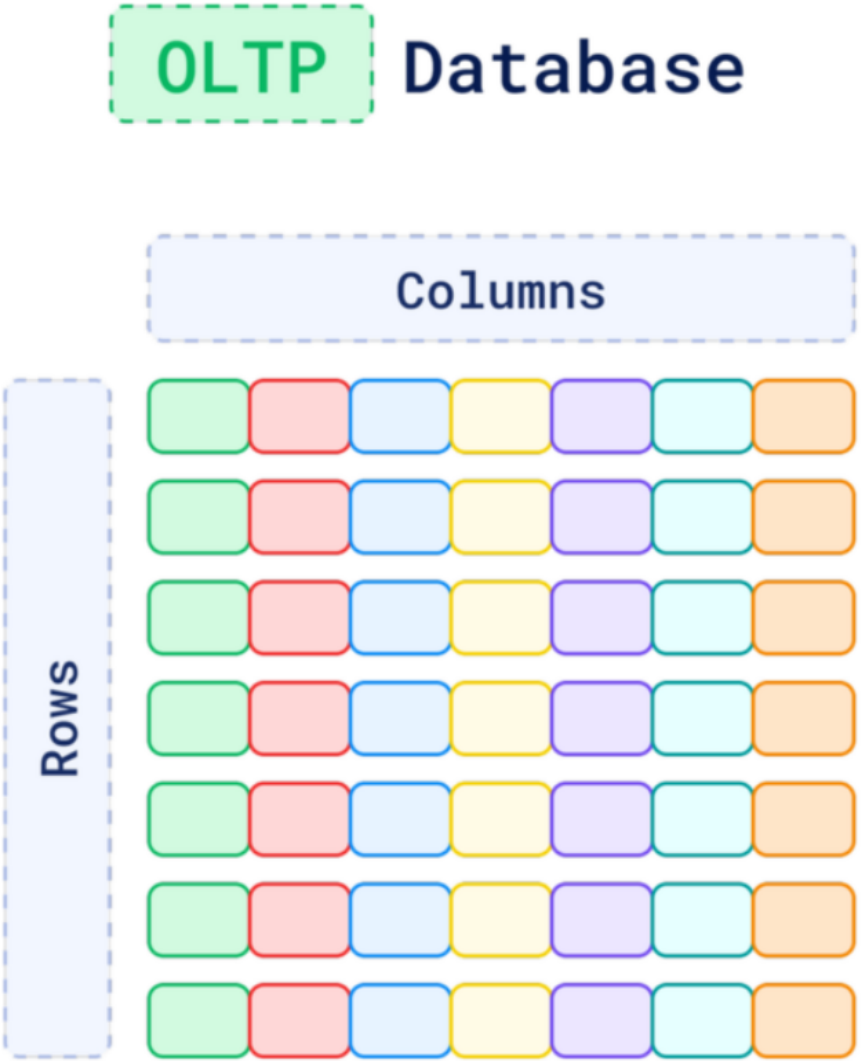
Metadata filtering



Relational DBs vs Vector DBs

Relational DBs:

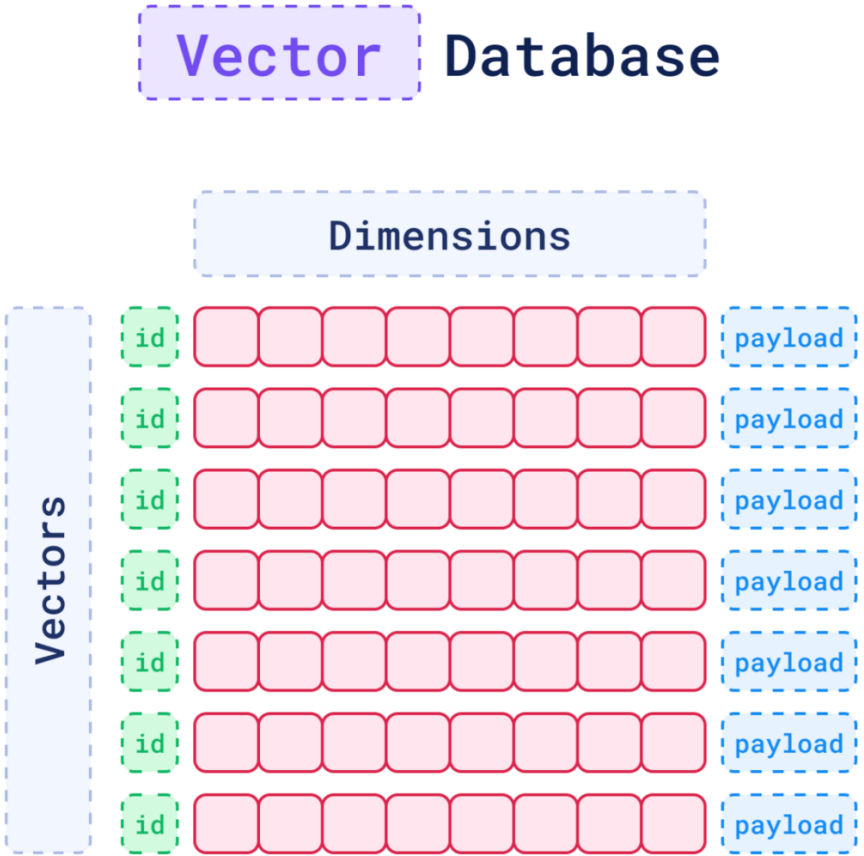
- ACID
- Storage and persistence of data
- Transactions Transaction Transactions
- Table joins, data aggregation
- Precise searches, exact matches
- ~~Horizontal scaling~~ Vertical scaling
- Standard data types



Relational DBs vs Vector DBs

Vector DBs:

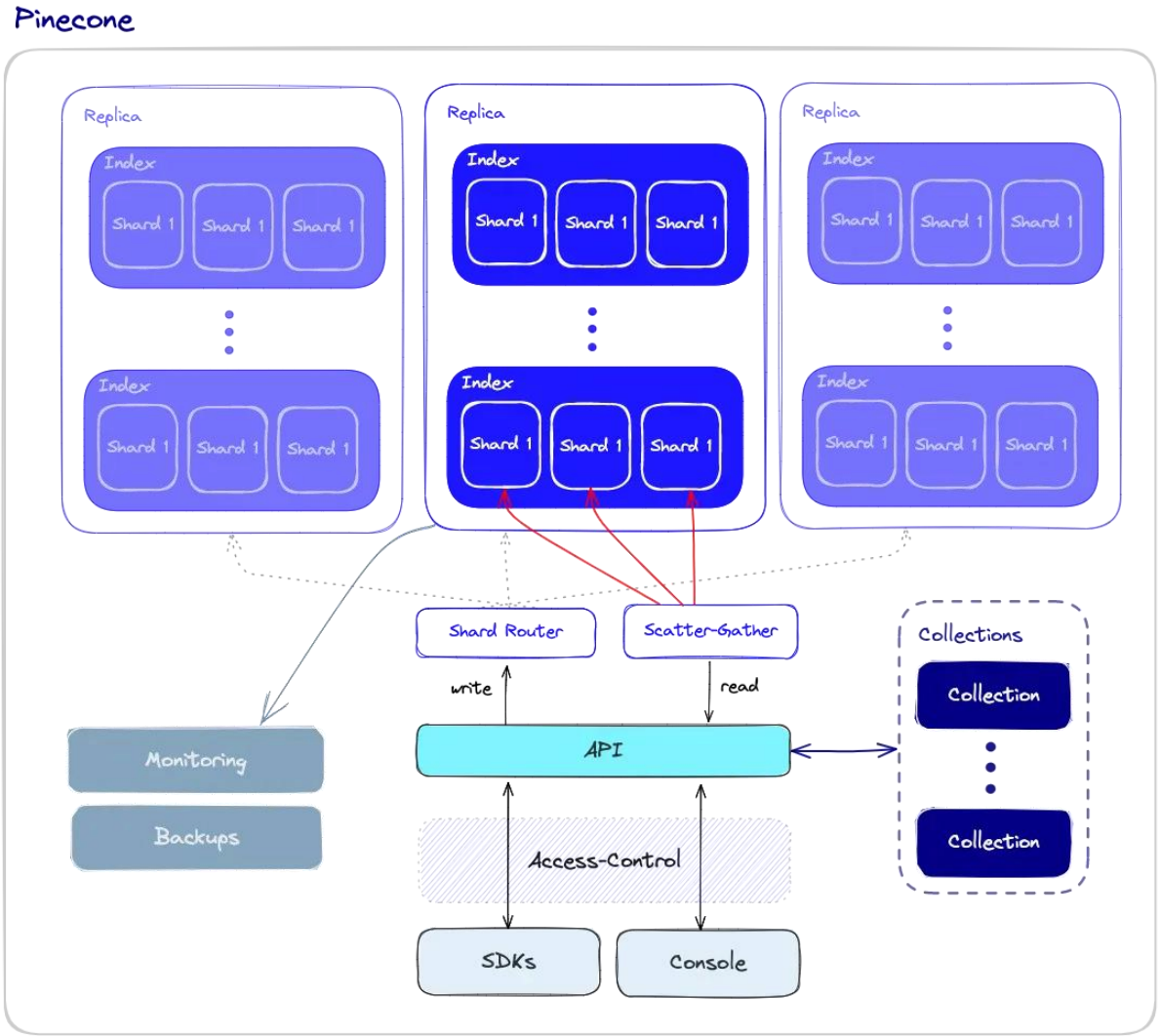
- Search Focused
- Optimized for approximate results
- If it can be vectorized, it can be searched
- Horizontal scaling + Vertical scaling



Types of Vector DBs

Fully fledged:

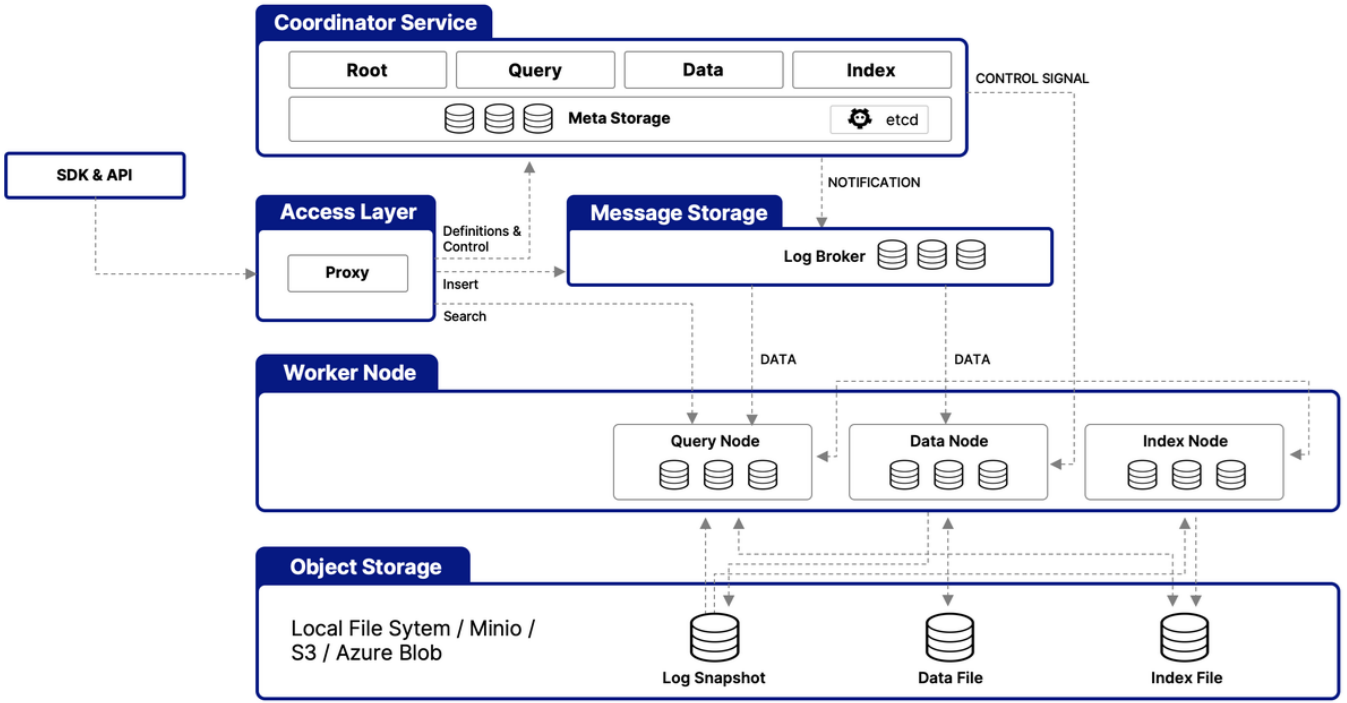
- Pinecone DB



Types of Vector DBs

Fully fledged:

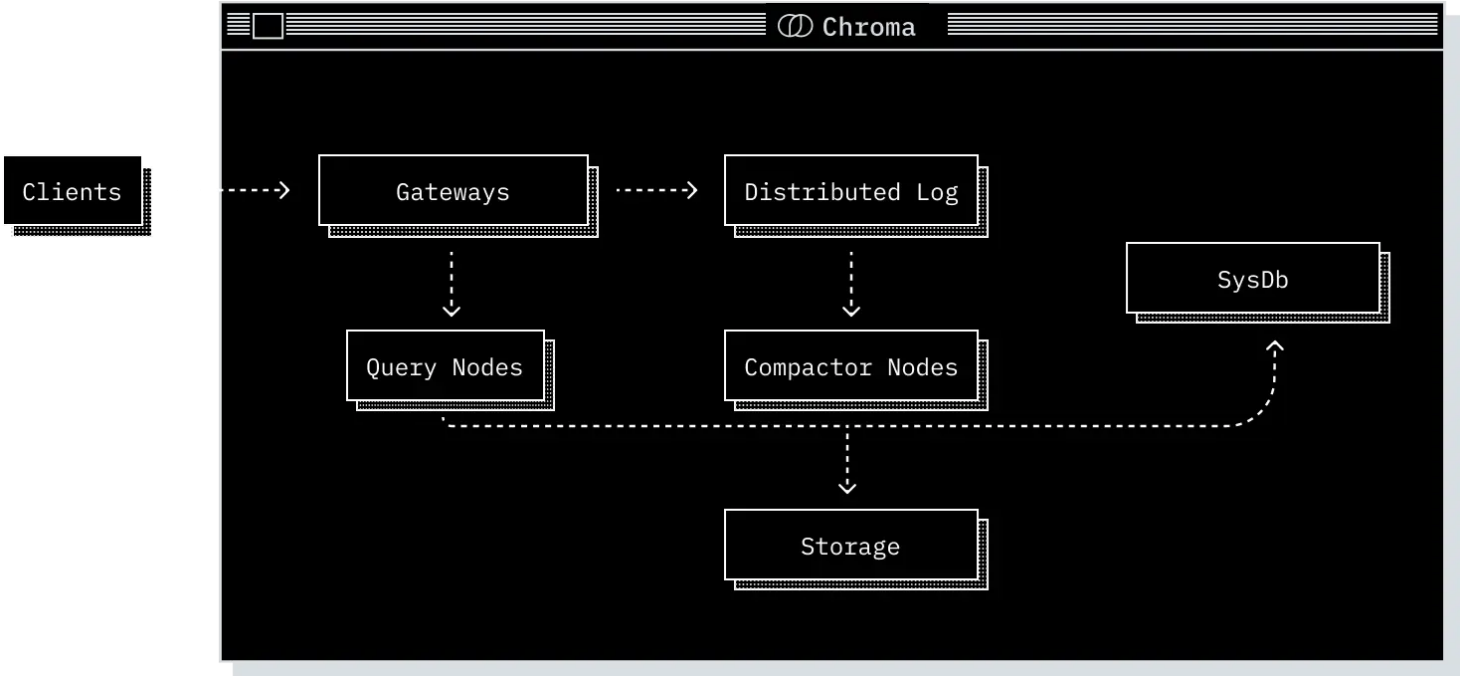
- Pinecone DB
- Milvus



Types of Vector DBs

Fully fledged:

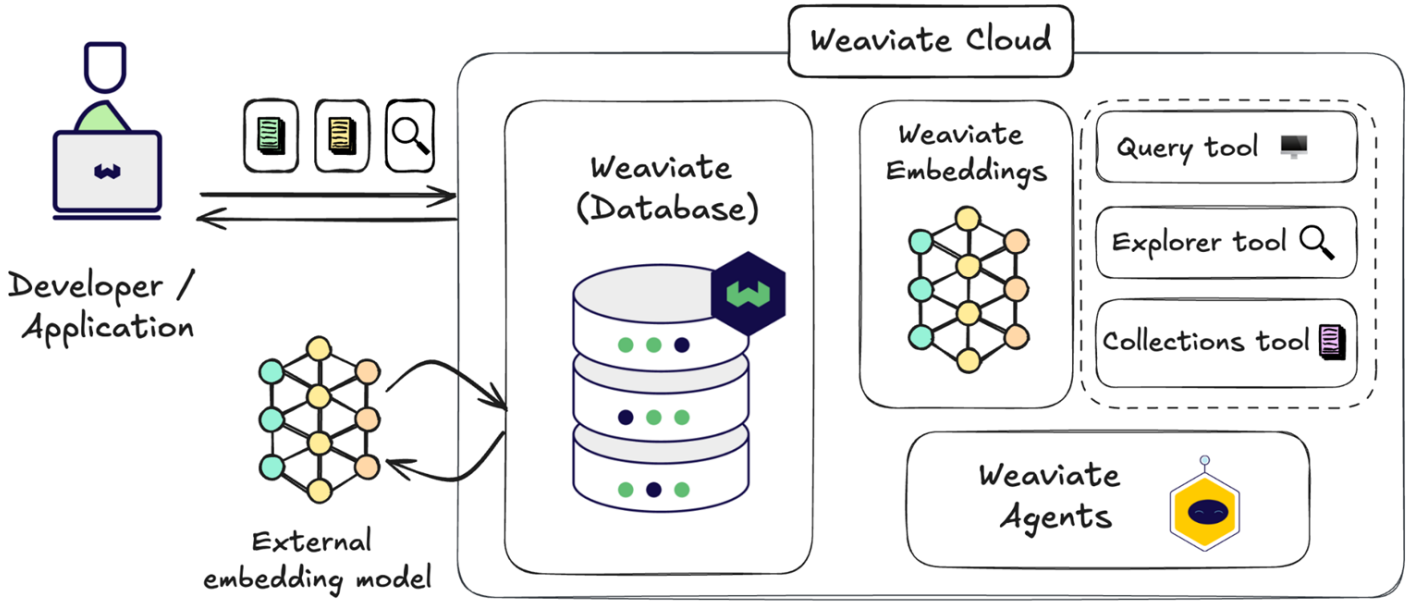
- Pinecone DB
- Milvus
- Chroma DB



Types of Vector DBs

Fully fledged:

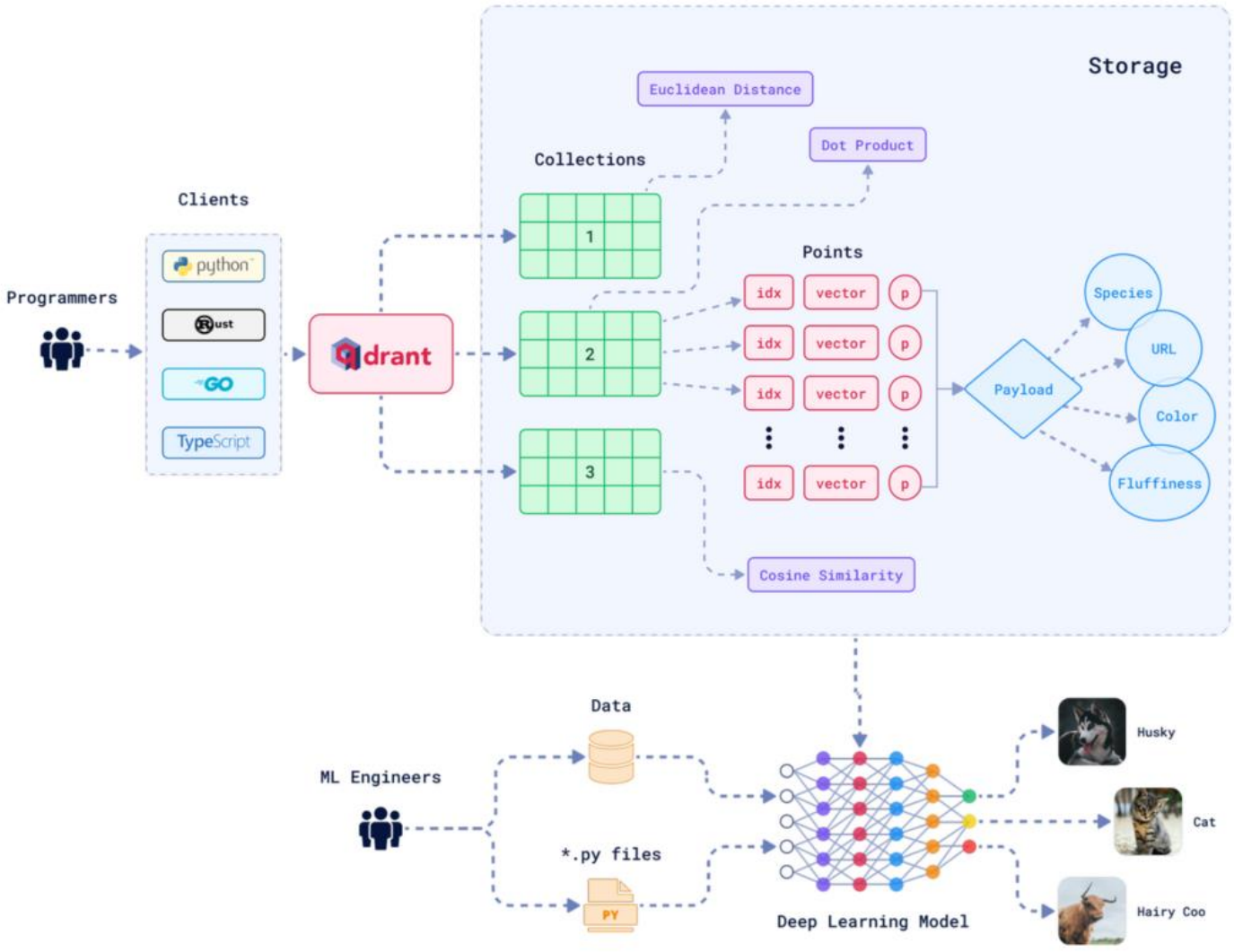
- Pinecone DB
- Milvus
- Chroma DB
- Weaviate



Types of Vecto

Fully fledged:

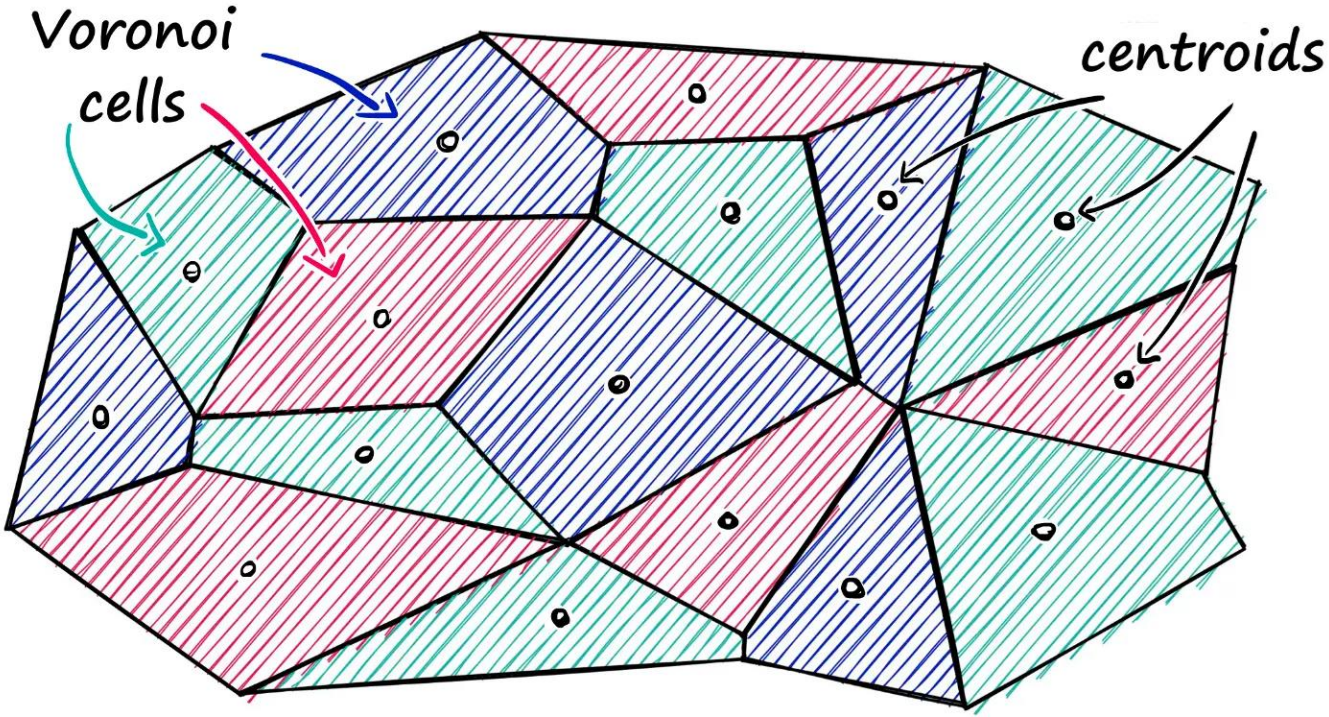
- Pinecone DB
- Milvus
- Chroma DB
- Weaviate
- Qdrant



Types of Vector DBs

Vector search library:

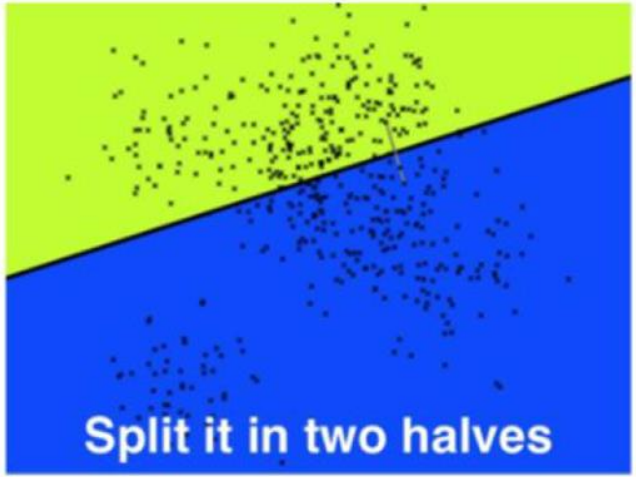
- FAISS

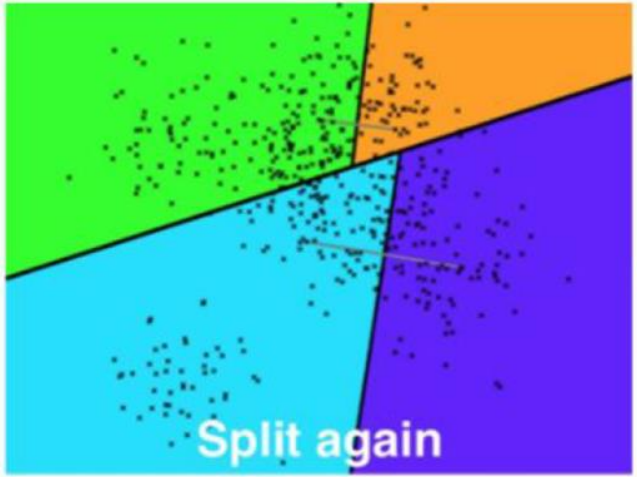


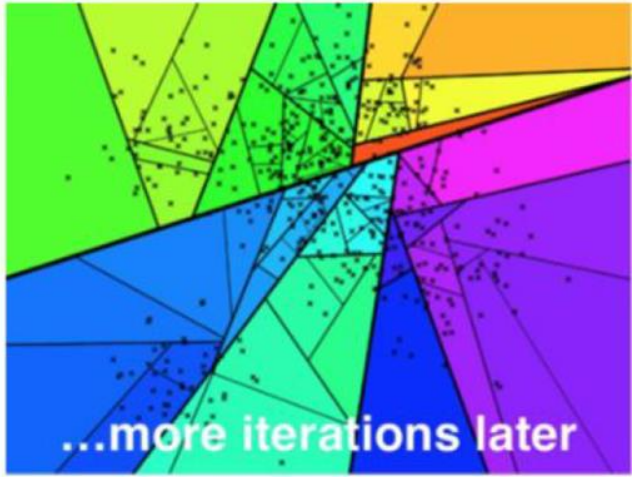
Types of Vector DBs

Vector search library:

- FAISS
- Annoy



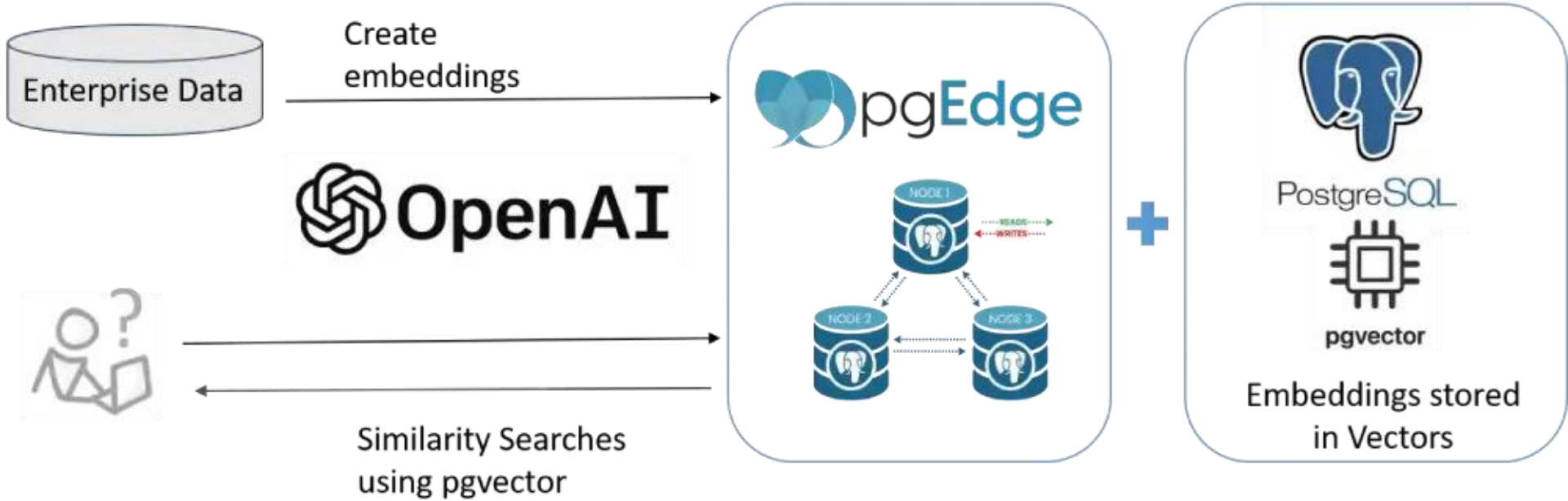




Types of Vector DBs

RDB Plugins:

- pgVector



References

https://zilliz.com/blog/top-5-open-source-vector-search-engines

https://zilliz.com/learn/what-is-vector-database

https://www.pinecone.io/learn/vector-database/

https://www.pinecone.io/learn/what-is-similarity-search/

https://www.pinecone.io/learn/series/faiss/hnsw/

https://qdrant.tech/articles/what-is-a-vector-database/#dense-vs-sparse-vectors

https://cloud.google.com/discover/what-is-a-vector-database

https://milvus.io/intro

https://docs.trychroma.com/docs/overview/architecture

https://www.pinecone.io/learn/series/faiss/faiss-tutorial/

https://engineering.atspotify.com/2023/10/introducing-voyager-spotifys-new-nearest-neighbor-search-library

https://github.com/spotify/annoy

https://docs.weaviate.io/weaviate/concepts/data

https://developers.google.com/machine-learning/crash-course/embeddings



ArtSoft consult

The Art of Software Development

ArtSoft Consult is a Romanian private leading digital engineering company with 140+ team members in the Central and Eastern Europe serving clients across Europe and US.

ArtSoft in a nutshell

- 100% Romanian private capital
- People centric culture
- Tier 1 private clients only
- High customer retention
- Same management team since inception

140+
Professionals

60+
Satisfied clients

80%
Revenue from Repeat Clients

300+
Completed projects

25
Years in business

History & timeline

1999–2002

Founding ArtNet

→ Established in Cluj-Napoca
→ Clients from Germany
→ Working on data mining and data qualifying projects
→ Technology stack: Java, XML, JSP, Oracle
→ Final clients: Bertelsmann, Jewish Museum from Berlin
→ Part of the R&D dept. of Caatoosee AG.
→ Managed 400 developers in Bali
→ Became Oracle Partner

2003

Rebranded to ArtSoft Consult

→ New entity ArtSoft Consult
→ Management team joined
→ New industries: automotive, telecom

2005–2006

Reached 50+ employees

→ Moved into a larger office
→ New industry: Travel
→ Technology stack: mobile apps, data and application migrations
→ Developed and implemented our own agile software development process
→ Became Microsoft partner

2008–2010

Key clients acquired

→ New key clients from Media, Digital marketing, Travel and Medical industries
→ Starting working with Scrum, Scrumban, Kanban
→ Medical software developed with artificial neural networks

2012–2013

Own office building

→ Reached 100 FTE
→ New key clients from Telecom, Education and Manufacturing industry
→ Swiss Quality Award and Forum Managed Care prizes for developed product
→Founding member of Cluj IT Cluster

2015–2016

Technology stack: Cloud and microservices

→ Reached 250 successful projects
→ Became a strategic technology partner for our clients in the Travel, Education, Medicine, Telecom and Digital marketing fields

2020–2022

150 employees

→ Reached 300 successful projects
→ Technology stack: Cloud-native, Serverless architecture, Big data
→ No specialists on the bench

2023–2024

Key clients acquired

→ Consolidation of strategic partnerships
→ ISO 27001:2022 certification
→ New key clients from Manufacturing and Commerce & Marketing industries

The same management team

Our teams are part of the R&D departments of our clients

Strategic technology partner of our clients

What we do



- Software engineering
- UX/UI and graphic design
- QA and testing
- Integration
- Security
- Roll out software products

- Data and application migrations
- Infrastructure operations
- Architecture design/redesign

- Business analysis
- Benchmarking
- Due diligence analysis
- Software architecture & design
- Optimizations
- Project management
- Security consulting

- iOS
- Android
- Cross-platform

- Digital marketing
- Customer support
- Data analysis and data management
- Business development

What we do – Domains of expertise



Enterprise applications



Portals/web applications



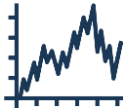
Booking engines and Loyalty systems



Telecom provisioning



Data security & Cloud



Time series



Data warehouse and business intelligence



Artificial neural networks



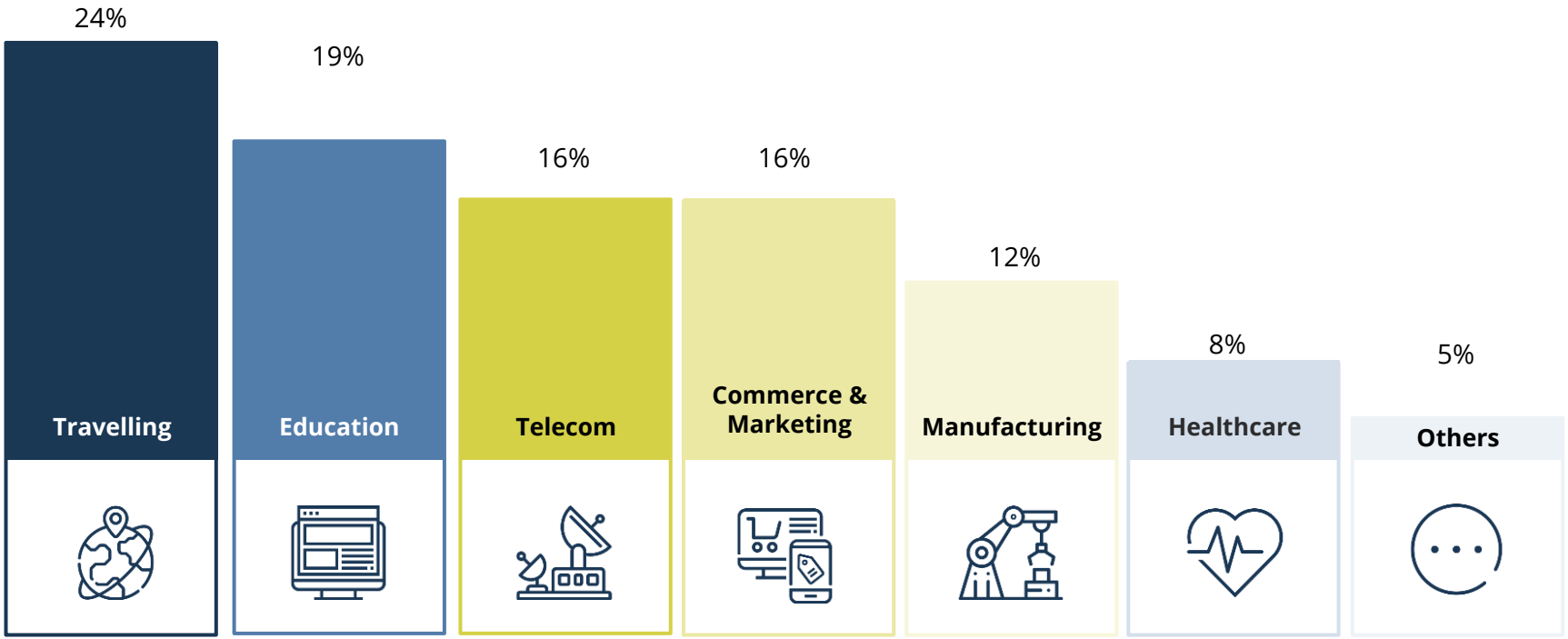
Internet of things

ArtSoft consult

Technologies (selection)



Clients by industry



Applicable to period 2020 – 2024 revenue

ETL Strategies in BI Projects

Extract

Transform

Load

ETL vs ELT

- Full Load
- Incremental (Delta) Load
- Batch ETL
- Real-Time/Streaming

Talend, Informatica, SSIS

Apache NiFi, Airbyte, dbt with timestamps

Pentaho, SSIS, AWS Glue

Apache Kafka, Spark Streaming, Fivetran (CDC)

Change Data Capture (CDC)

Essential for near real-time updates

Log based

Trigger based

Timestamp based

Method	Accuracy	Performance	Complexity	Detects Deletes
Log	High	High	Higher	Yes
Trigger	High	Medium	Medium	Yes
Timestamp	Medium	High	Low	No (only soft)

Log Based CDC

Captures changes directly from the database transaction logs.

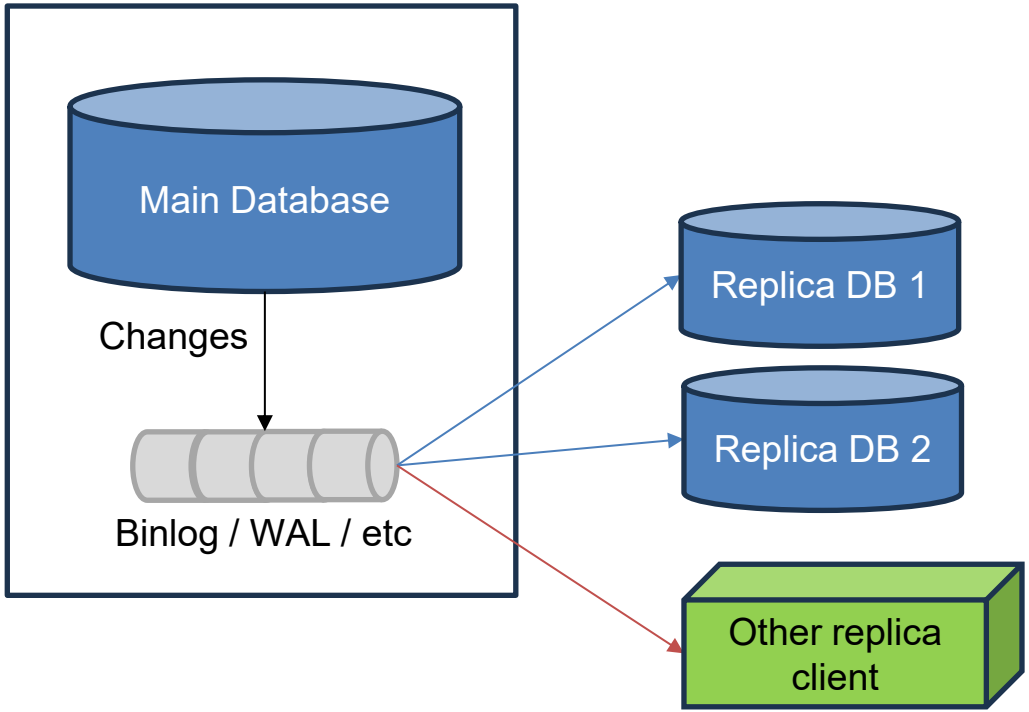
Highly efficient: it avoids querying the entire dataset.

Works well with databases supporting redo logs (MySQL binary logs, PostgreSQL WAL, SQL Server CDC, Oracle LogMiner).

Pros: Minimal performance impact, reliable, scalable.

Cons: Requires log access permissions, compatibility varies by database.

Log Based CDC



MySQL specific:

```
29  SHOW BINARY LOGS;
```

Results 1

SHOW BINARY LOGS | Enter a SQL expression to filter results (use Ctrl+Space)

	Log_name	File_size	Encrypted
1	mysql-bin.856508	111,241,998	No
2	mysql-bin.856509	110,221,102	No
3	mysql-bin.856510	104,873,480	No
4	mysql-bin.856511	105,493,156	No

```
29  SHOW BINARY LOGS;
30  show binlog events in 'mysql-bin.856508' limit 100;
```

Results 1

show binlog events in 'mysql-bin.856508' limit 100 | Enter a SQL expression to filter results (use Ctrl+Space)

		Pos	Event_type			Info
1	mysc	4	Format_desc	,294,	126	Server ver: 8.0.40-commercial, Binlog ver: 4
2	mysc	126	Previous_gtids	,294,	157	
3	mysc	157	Anonymous_Gtid	,294,	237	SET @@SESSION.GTID_NEXT= 'ANONYMOUS'
4	mysc	237	Query	,294,	333	BEGIN
5	mysc	333	Table_map	,294,	469	table_id: 276 (availability_engine.ae_activity)
6	mysc	469	Write_rows	,294,	8,616	table_id: 276

ArtSoft consult

Debezium: CDC for Real-Time Data Streaming



Open-source distributed CDC platform.

Captures changes in databases without modifying application logic.

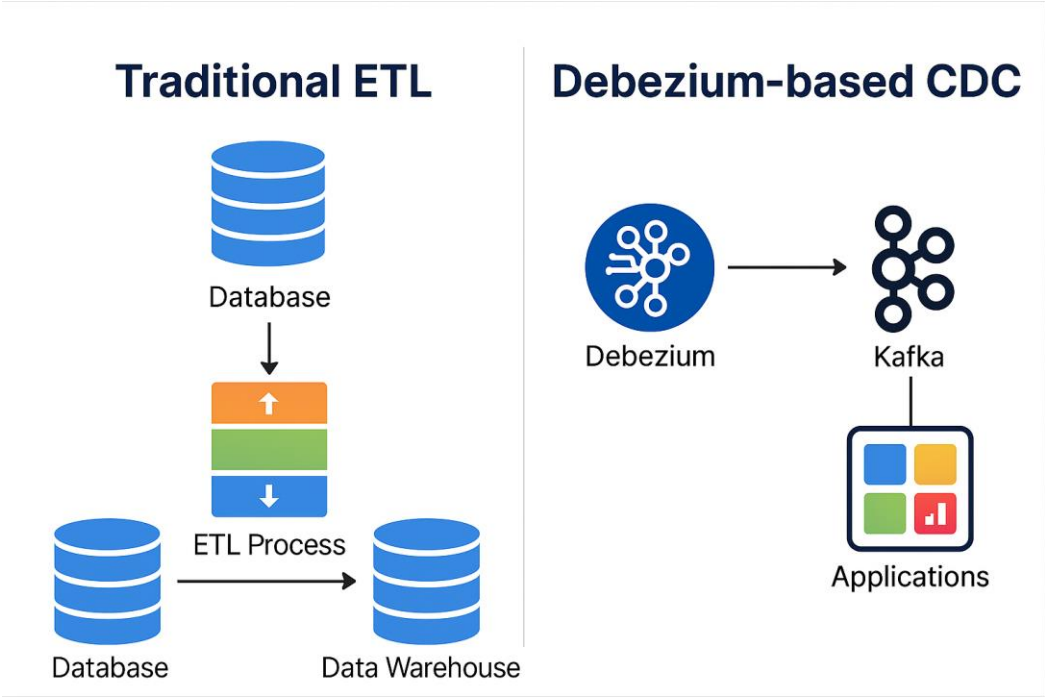
Ideal for event-driven architectures.

Enables real-time ETL and analytics.

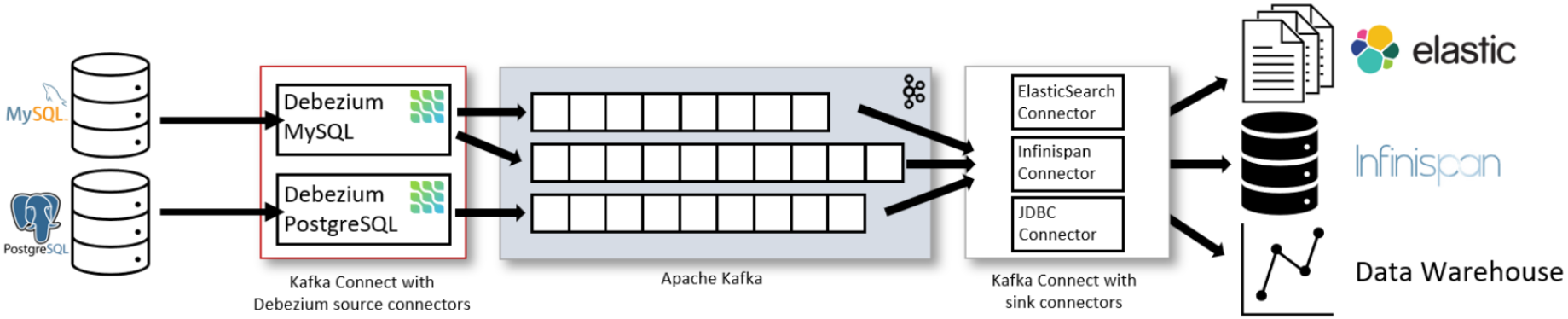
Reduces database polling inefficiencies.

Debezium Core Components

- Source Connectors (database-specific capture agents)
- Kafka Topics (CDC event streams)
- Consumer Applications (BI tools & analytics platforms)



Debezium Architecture



Debezium: Incremental Snapshots

By default gets a full snapshot of the tables. When having huge tables, this snapshot process can take a lot of time and in the meantime the data change events are suppressed. A more flexible approach is to use incremental snapshots. For this to work, there are several prerequisites:

- **primary key** on source table
- **snapshot.mode: no_data** - prevent the initial snapshot
- **signal.data.collection: <signal_table_name>** - the “signal table”

table.include.list: ……,<signal_table_name>,…. - add the “signal table” to the list of tables that will be monitored for changes;

- **incremental.snapshot.watermarking.strategy** - **insert_delete**

ArtSoft consult

Debezium Advantages

1. Transactional Event Ordering: events are streamed in the order of db transactions.
2. Schema Evolution Tracking: detects changes in database schemas.
3. Exactly-Once Message Semantics: avoid duplicate messages.
4. Embedded Mode: can run inside an application instead of using Kafka Connect.
5. Filtering & Data Masking: fine-grained filtering to capture only relevant changes.
6. Automatic Recovery & Reconciliation: detects inconsistencies and automatically resynchronizes missing or failed events.
7. Snapshotting Mechanism: incremental snapshots to backfill historical records.
8. Low-Latency Streaming : works with Kafka Streams, Flink, and Apache Beam for low-latency analytics.

ArtSoft consult

Str. Eugen Ionesco, nr. 1A
Cluj-Napoca, Romania, 400357

www.artsoft-consult.ro

office@artsoft-consult.ro

+40 0264 403 662

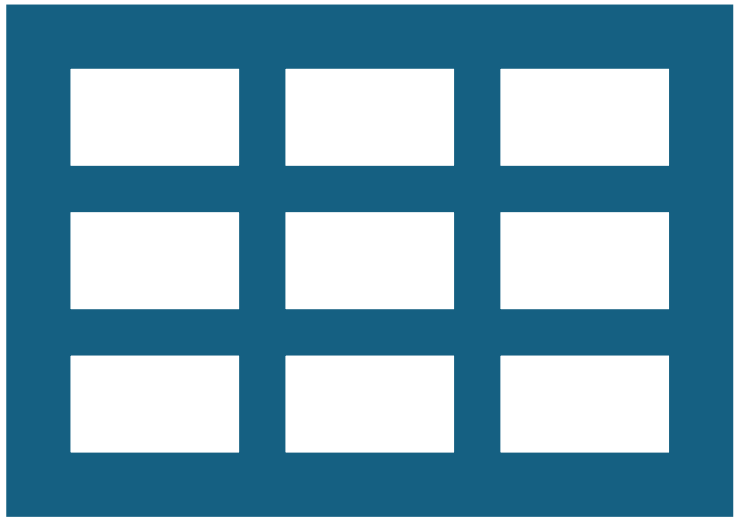
+40 0264 403 489

Artsoft.Consult

@artsoftconsult

artsoft-consult





Course 10

NoSQL – Wide Column

Wide Column Store

Definition: A NoSQL database that organizes data into columns with dynamic rows.

The Hybrid Nature: Sits between Key-Value stores and Relational Databases.

Key Concept: A "two-dimensional key-value store."

Structure:

- Row Key (Primary Index)
- Column Family (Group of related columns)
- Column Qualifier (Dynamic key within the row)
- Timestamp (Versioning)

Row Oriented vs Wide Column

Row-Oriented (RDBMS):	Data stored sequentially by row (e.g., CSV style). Good for OLTP (fetching one user's full profile). Example: •[Block 1: id, name, email, city, theme_style] values: [1,'Alice','alice@email.com','New York','light'] •[Block 2: id, name, email, city, theme_style] values: [2,'Mary','mary@xyz.abc','Frankfurt',NULL]
Wide Column:	Data stored by column families; sparse. Rows can have different columns. Example: •Row 1 -> {name: "Alice", email: "alice@email.com", city: "New York", theme_style: "light"} •Row 2 -> {name: "Mary", email: "mary@xyz.abc", city: "Frankfurt"}

id	name	email	city	theme_style
1	Alice	alice@email.com	New York	light
2	Bob	bob@comp.net	London	dark
3	Mary	mary@xyz.abc	Frankfurt	



Row 1

name	email	city	theme_style
Alice	alice@email.com	New York	light

Row 2

name	email	city	theme_style
Bob	bob@comp.net	London	dark

Row 3

name	email	city
Mary	mary@xyz.abc	Frankfurt

Recap: Big Table

Google published the "Bigtable" paper in 2006.

Storing petabytes of data across thousands of commodity servers (Web Indexing, Google Earth).

A sparse, distributed, persistent multi-dimensional sorted map.

Inspired Apache HBase (Hadoop) and Apache Cassandra (Facebook).

Characteristics



Schema-Flexibility: Columns can be added on the fly without locking the table.



High Availability: Designed to tolerate node failures without downtime.



Scalability: Horizontal scaling (scale-out) using commodity hardware.



Distributed: Data is partitioned across multiple nodes (Sharding).

Distributed Architectures

- Master-Slave (HBase, Bigtable):
  - Master: Handles metadata and coordination.
  - Slaves (RegionServers): Handle data IO.
  - Pro: Strong consistency.
  - Con: Master is a single point of failure (conceptually).

- Peer-to-Peer / Masterless (Cassandra, ScyllaDB):
  - All nodes are equal.
  - Any node can handle a request (Coordinator Node).
  - Pro: Max availability.
  - Con: Eventual consistency complexity.

CAP/PACELC

CAP: Consistency, Availability, Partition Tolerance. Pick two.

Wide Column Positioning:

- AP (Availability + Partition Tolerance): Cassandra, ScyllaDB.
- CP (Consistency + Partition Tolerance): HBase, Bigtable.

Tunable Consistency: The ability to trade latency for consistency per-query (e.g., QUORUM vs. ONE).

Data Distribution

Consistent Hashing

How to distribute data evenly without rehashing everything when a node is added?

The *Token Ring*:

- Data is hashed (e.g., Murmur3) to a token.
- Nodes own ranges of tokens on a circular ring.
- Virtual Nodes (VNodes): Physical nodes own multiple small ranges to improve load balancing.

Inter Node Communication



Gossip Protocol



A peer-to-peer communication protocol where nodes periodically exchange state information.



Failure detection (Phi Accrual Failure Detector).



Schema propagation.



Topology changes.



Conceptualy: "I tell two friends, they tell two friends..." - The cluster state converges quickly.

Read/Write Paths

Write

Log-Structured Merge Tree: Optimized for high write throughput.

Step 1: Commit Log: Append-only file on disk (Durability).

Step 2: MemTable: In-memory sorted data structure.

Step 3: Flush: When MemTable is full, it is flushed to disk as an SSTable (Sorted String Table).

Step 4: SSTables are immutable.

Read

Challenge: Data might exist in the MemTable or across multiple SSTables.

Step 1: Check MemTable.

Step 2: Check Row Cache (if enabled).

Step 3: Check Bloom Filters (Probabilistic structure: "Is the data definitely not here?").

Step 4: Scan Key Cache / Partition Summary.

Step 5: Fetch from SSTables and merge results (Last Write Wins).

Compaction

Write path creates many small SSTables and "Tombstones" (deletion markers).

Compaction: Background process that merges SSTables.

- Purges deleted data (Tombstones).
- Combines fragments into larger files for faster reads.
- Strategies: Size-Tiered (Write heavy), Leveled (Read heavy).

Conceptual Data Model

Keyspace

Container for tables (roughly maps to Database in SQL).

- Column Family / Table: Container for rows.
- Row: Identified by a unique Primary Key.
- Column: Name, Value, Timestamp.

Primary Key

Partition vs. Clustering

The most critical concept in wide column modeling.

Partition Key:

- Determines which node stores the data.
- Responsible for data distribution.

Clustering Key:

- Determines how data is sorted on the disk within the partition.
- Responsible for range queries.

Design

Query-Driven

RDBMS Approach: Normalize data to remove redundancy -> Join on read.

Wide Column Approach: Denormalize data based on access patterns -> Duplicate on write.

Rule: Design your tables to satisfy your queries. Do not model entities first.

Motto: "Space is cheap, reads are expensive."

Use Cases



- High Write Throughput: IoT sensor data, Logging, Clickstreams.
- Time Series Data: Financial ticks, metrics, history.
- Geographically Distributed Apps: Multi-region replication.
- Personalization Engines: User profiles and activity streams.



- ACID Transactions: If you need banking-level atomic transfers between tables.
- Ad-Hoc Queries: "Find users where Age > 25 AND City = 'NY'" (Requires full table scans = slow).
- Joins: Wide column stores generally do not support server-side joins.
- Small datasets: The overhead of a distributed system isn't worth it for < 50GB.

Products

Apache Cassandra: masterless, linear scalability, multi-datacenter replication

Apache Hbase: open source of BigTable, runs on HDFS (Hadoop), strong consistency

ScyllaDB: rewrite of Cassandra in C++, shared-nothing, 10x performance of Cassandra

Google Bigtable: decouples compute from storage, scale to petabytes, Hbase API compatible

Rank			DBMS	Database Model	Score		
Nov 2025	Oct 2025	Nov 2024			Nov 2025	Oct 2025	Nov 2024
1.	1.	1.	Apache Cassandra	Wide column, Multi-model	102.71	-2.45	+5.00
2.	2.	↑ 3.	Microsoft Azure Cosmos DB	Multi-model	22.55	-0.16	-1.40
3.	3.	↓ 2.	Apache HBase	Wide column	19.32	-0.85	-5.93
4.	4.	↑ 5.	ScyllaDB	Wide column, Multi-model	4.13	+0.28	+0.25
5.	5.	↓ 4.	Datastax Enterprise	Wide column, Multi-model	3.34	-0.07	-1.57
6.	6.	6.	Microsoft Azure Table Storage	Wide column	2.83	+0.03	-0.27
7.	7.	↑ 8.	Google Cloud Bigtable	Multi-model	2.65	0.00	-0.28
8.	8.	↓ 7.	Apache Accumulo	Wide column	2.35	+0.00	-0.73
9.	9.	9.	Amazon Keyspaces	Wide column	1.11	-0.04	+0.12
10.	10.	10.	HPE Ezmeral Data Fabric	Multi-model	0.87	+0.05	+0.04

Cassandra

open-source, distributed NoSQL database designed to handle massive amounts of data across many commodity servers while providing high availability and linear scalability with no single point of failure.

It is highly popular for applications that require constant uptime and must handle high-volume, continuous writes, such as IoT platforms, social media, and real-time analytics.

Most widely adopted wide column store.

Excellent detailed documentation.

Standard CQL (Cassandra Query Language) is SQL-like, making it accessible.

Cassandra

Design objectives:

- Full multi-primary database replication
- Global availability at low latency
- Scaling out on commodity hardware
- Linear throughput increase with each additional processor
- Online load balancing and cluster growth
- Partitioned key-oriented queries
- Flexible schema

organizes data within a cluster using:

Keyspace: Specifies the replication strategy for a dataset across different datacenters. Keyspaces serve as containers for tables.

Table: Tables are composed of rows and columns. Columns define the typed schema for a single datum in a table. Tables are partitioned based on the columns provided in the partition key. Can flexibly add new columns to tables with zero downtime.

Partition: the mandatory part of the primary key, used to identify the node in a cluster where the row is stored. All performant queries supply the partition key in the query.

Row: Contains a collection of columns identified by a unique primary key made up of the partition key and optionally additional clustering keys.

Column: A single datum with a type which belongs to a row.

Cassandra

CQL supports features over a partitioned dataset:

- Collection types including sets, maps, and lists
- User-defined types, tuples, functions and aggregates
- Storage-attached indexing (SAI) for secondary indexes
- Local secondary indexes (2i)
- User-defined types, functions and aggregates
- Single-partition lightweight transactions with atomic compare and set semantics
- (Experimental) materialized views

Cassandra explicitly chooses not to implement operations that require cross-partition coordination as they are typically slow and hard to provide highly available global semantics. Does not support:

- Cross-partition transactions
- Distributed joins
- Foreign keys or referential integrity

Dataset Partitioning

Consistent Hashing

Cassandra scales horizontally by hashing data into partitions and replicating them across nodes and failure domains. Since any replica can update any key, each key is versioned. Instead of Dynamo's vector clocks, Cassandra uses a simpler last-write-wins approach: every mutation, including deletes, is timestamped, and the newest value prevails. Effectively, each CQL row is an LWW-Element-Set CRDT for resolving conflicting replica updates.

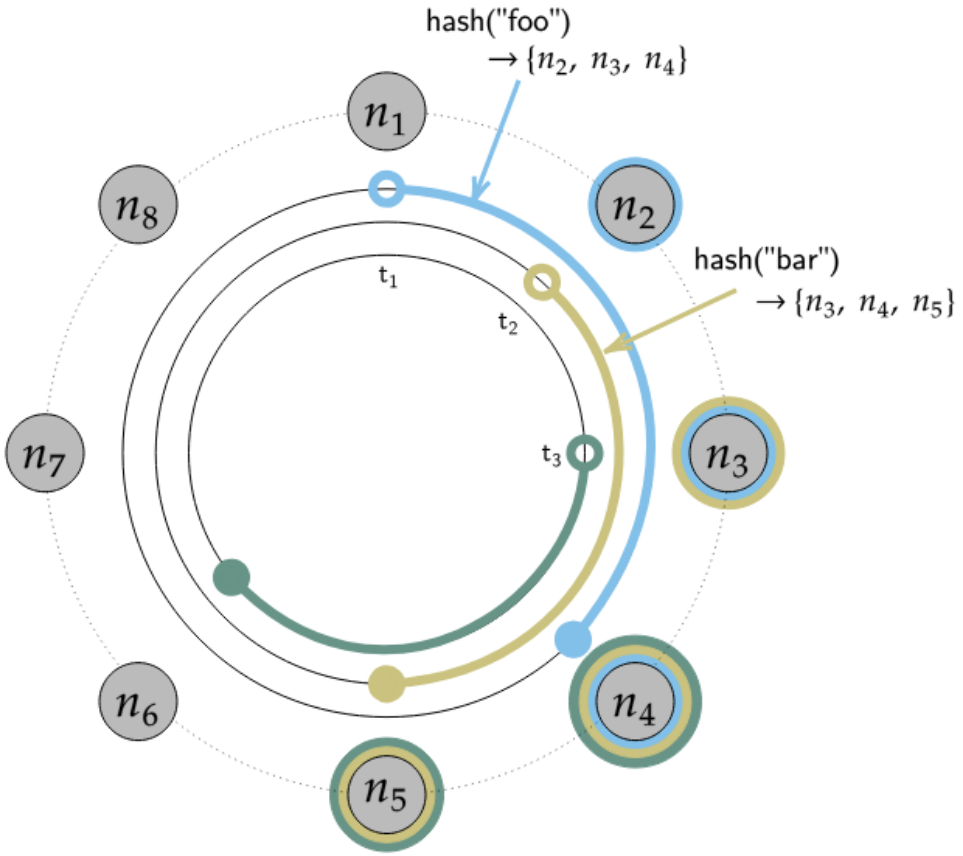
Token Ring

Cassandra distributes data using consistent hashing rather than simple modulo hashing. In naive hashing, adding a node forces most key-to-bucket mappings to change. Instead, Cassandra places nodes at token positions on a continuous hash ring and assigns each key to the next node on the ring, as in Chord. This approach ensures that when nodes are added or removed, only a small portion of keys must be moved.

Dataset Partitioning

$\text{token}(n_1) = t_1$
$\text{token}(n_2) = t_2$
$\text{token}(n_3) = t_3$
...

$\text{range}(t_1, t_2] \rightarrow \{n_2, n_3, n_4\}$
$\text{range}(t_2, t_3] \rightarrow \{n_3, n_4, n_5\}$
$\text{range}(t_3, t_4] \rightarrow \{n_4, n_5, n_6\}$
...



Tokens to Endpoints

Concepts

- *Token*: A single position on the Dynamo-style hash ring.
- *Endpoint*: A single physical IP and port on the network.
- *Host ID*: A unique identifier for a single "physical" node, usually present at one gEndpoint and containing one or more gTokens.
- *Virtual Node* (or vnode): A gToken on the hash ring owned by the same physical node, one with the same gHost ID.

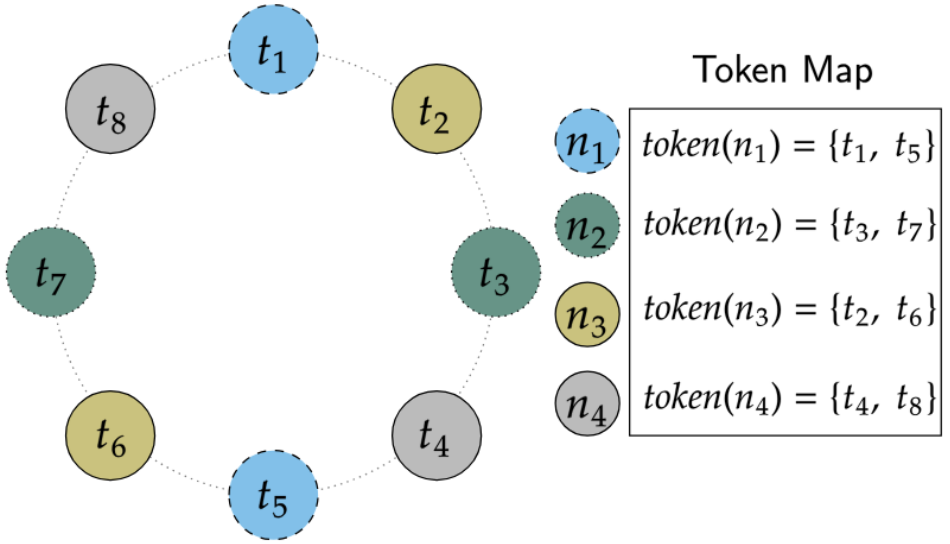
Cassandra's token map tracks which tokens on the hash ring belong to which physical nodes. By giving each node multiple tokens, a cluster can balance data more evenly - for example, four nodes can appear as eight logical positions.

Advantages:

- New nodes take on data evenly, improving cluster balance.
- Decommissioned nodes shed data evenly to others.
- If a node fails, token-aware queries spread load across many nodes.

Disadvantages:

- More tokens create more ring neighbors, increasing failure combination that can reduce availability.
- Cluster-wide tasks, like repairs, slow down as token counts grow.
- Operations spanning many token ranges may see degraded performance.



MultiMaster Replication

- **NetworkTopologyStrategy**:

set a separate replication factor for each datacenter and is preferred even for single-DC clusters, since it simplifies adding new datacenters later.

It also places replicas in different racks, based on the Snitch. If racks ≥ the DC's replication factor, all replicas come from distinct racks; otherwise, each rack gets at least one. This rack awareness can create uneven load—for example, a smaller rack may get more data, or a new single-node rack may receive replicas for the whole ring. To avoid this, many operators map all nodes in one availability zone to a single "rack."

- **SimpleStrategy**:

uses a single integer replication_factor to specify how many nodes store each row (e.g., 3 means three copies).

It ignores datacenters and racks, treating all nodes the same. To select replicas, Cassandra walks the ring from the target token forward, adding each unique node it encounters until it reaches the required number of replicas.

Data Versioning

- **Versioning**:

ensures eventual consistency using timestamp-based versioning. Each mutation gets a timestamp from the client or, if missing, the coordinator's clock, and conflicts resolve with last-write-wins—so accurate clock sync (e.g., NTP) is essential.

Every column in a row receives its own timestamp, and each resolves conflicts independently. Different primary keys in the same partition can therefore update without conflict. CQL collections like maps and sets use the same LWW approach, ensuring concurrent updates also reconcile cleanly.

- **Tunable Consistency**:

ONE: Only a single replica must respond.

TWO: Two replicas must respond.

THREE: Three replicas must respond.

QUORUM: A majority (n/2 + 1) of the replicas must respond.

ALL: All of the replicas must respond.

LOCAL_QUORUM: A majority of the replicas in the local datacenter (whichever datacenter the coordinator is in) must respond.

EACH_QUORUM: A majority of the replicas in each datacenter must respond.

LOCAL_ONE: Only a single replica must respond. In a multi-datacenter cluster, this also guarantees that read requests are not sent to replicas in a remote datacenter.

ANY: A single replica may respond, or the coordinator may store a hint. If a hint is stored, the coordinator will later attempt to replay the hint and deliver the mutation to the replicas. This consistency level is only accepted for write operations.

Gossip

Cassandra's gossip system spreads basic cluster state—membership, protocol versions, token info, and schema versions. Nodes exchange state about themselves and others, using (generation, version) clocks so outdated info can be ignored.

Each node gossips every second by:

- Updating its heartbeat (version) and local view of cluster state
- Contacting a random node
- Occasionally trying unreachable nodes
- Contacting a seed node if not already contacted

Seed nodes are special only during bootstrap: non-seed nodes must reach one to join the cluster, so clusters usually define several seeds per rack or datacenter. After bootstrap, seeds simply get more gossip traffic.

Gossip also drives control-plane actions: schema syncs when versions differ, and token metadata updates that teach nodes who owns which data.

Connecting

```
from cassandra.cluster import Cluster

# Connect to the cluster
cluster = Cluster(['192.168.0.1', '192.168.0.2'])
session = cluster.connect()

# Create Keyspace
session.execute("""
    CREATE KEYSPACE IF NOT EXISTS test_db
    WITH replication = {'class': 'SimpleStrategy', 'replication_factor':
3}
""")
session.set_keyspace('test_db')
```

Create Table

Scenario: Storing student grades. We want to query by Student ID, and order grades by Year.
Code Snippet:

```
CREATE TABLE student_grades (
    student_id UUID,
    year int,
    course_name text,
    grade float,
    PRIMARY KEY ((student_id), year, course_name)
) WITH CLUSTERING ORDER BY (year DESC);
```

Insert Data

Similar to SQL, but no referential integrity checks (Foreign Keys).
Code Snippet:

```
INSERT INTO student_grades (student_id, year, course_name, grade)
VALUES (123e4567-..., 2025, ‘CABD-CLOUDDB', 9.5);
```

Read Data

Queries must be restricted to partition key.
Code Snippet:

```
-- Efficient Query (Hits one node)
SELECT * FROM student_grades WHERE student_id = 123e4567-...;

-- Inefficient Query (Scans ALL nodes - Anti-pattern)
SELECT * FROM student_grades WHERE grade > 9; -- WILL FAIL or TIMEOUT
```

Update and Delete

Update: An update is just an Insert. It writes a new timestamped version.

Delete: Cassandra cannot immediately delete data from disk (SSTables are immutable).

Tombstone: A marker written to indicate data is deleted.

Issues: When deleting too much, reads slow down (skipping over tombstones).

Lightweight Transactions

Useful for "Check-and-Set" operations (e.g., creating a username that must be unique).
Uses Paxos consensus algorithm (Performance penalty: ~4x slower).

Code Snippet:

```
INSERT INTO users (username, password)
VALUES ('admin', '1234')
IF NOT EXISTS;
```

Collections

Cassandra supports Sets, Lists, and Maps within a single column.

Code Snippet:

```
CREATE TABLE users (
    id UUID PRIMARY KEY,
    emails set<text>,
    todo_list list<text>
);

UPDATE users SET emails = emails + {'work@test.com'} WHERE id = ...;
```

Tunable Consistency

Write Consistency:

- *ANY*: Written to hint handoff (fastest, risky).
- *QUORUM*: Written to 51% of replicas (balanced).

Read Consistency:

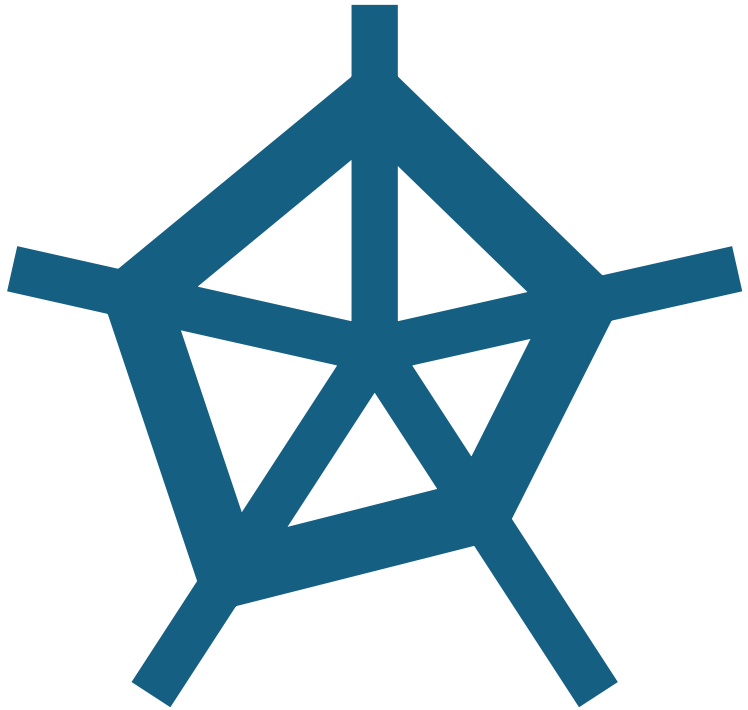
- *ONE*: Returns first response (fastest).
- *QUORUM*: Checks 51% of replicas and repairs data if mismatch (Read Repair).

Materialized Views

Automatic way to denormalize data for different query patterns.
Concept: Base table queries by ID. View queries by Email. Cassandra handles the sync.
Warning: Asynchronous. View update might lag behind Base table update.

Resources

What is Cassandra
Cassandra Architecture explained



Course 11

NoSQL – Graph

Why

Traditional relational models struggle with interconnected datasets.

Graph DBs provide native mechanisms to express, store, and query complex relationships with high performance and semantic clarity.

A graph database is a data management system that stores information using nodes, relationships, and properties.

It models data as a connected network rather than rigid tables.

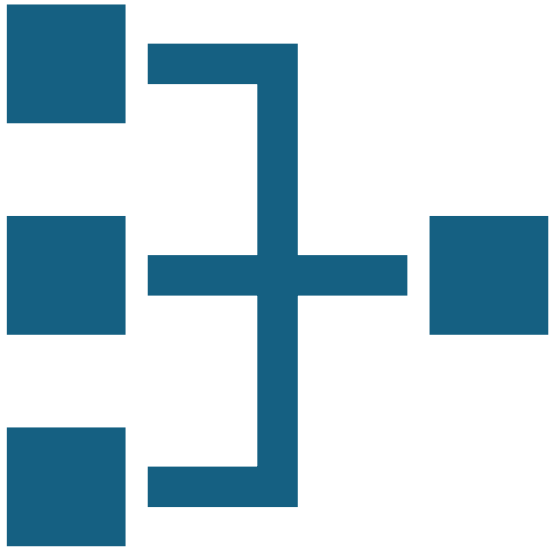
Foundation

Graph databases derive from graph theory, where data is represented as:

V = set of vertices (nodes)

E = set of edges (relationships)

This foundation enables algorithmic operations such as shortest paths, centrality, and traversal-based queries.



Components

- **Nodes** represent entities or objects.
  Examples: Person, Product, City, Device.
  Nodes can contain arbitrary key–value properties.

- **Relationships** represent directed or undirected connections between nodes.
  They can also store properties such as weights, timestamps, or categories.

- **Properties** define attributes for both nodes and relationships, providing flexibility in schema design and allowing highly expressive queries.



LPG

The property graph is the most widely used model in contemporary graph database systems such as Neo4j, TigerGraph, or JanusGraph.

It is characterized by:

- Nodes and edges as first-class entities, each capable of carrying an arbitrary set of key–value properties.
- Labeled nodes, enabling semantic grouping (e.g., Person, Paper, City).
- Typed or labeled edges, representing named relationships (e.g., FRIENDS_WITH, AUTHORED, LOCATED_IN).
- Multi-relational capabilities, meaning multiple edges of various types can exist between the same pair of nodes.

Modeling in an LPG system typically emphasizes descriptive richness and flexibility. As a result, it is well suited for domains requiring heterogeneous relationships, ad-hoc schema evolution, or expressive graph traversals.



RDF

The RDF model forms the foundation of the semantic web and is adopted by graph database systems such as GraphDB, Apache Jena, and Blazegraph.

Its structure is based on:

- Triples of the form (subject, predicate, object), which constitute a directed, labeled graph.
- Uniform representation of nodes and edges, where predicates (edge labels) and nodes are all URIs or literals.
- Schema and semantic constraints expressed through standards such as RDFS and OWL, enabling reasoning and inference.

RDF modeling emphasizes semantic interoperability, machine-understandable vocabularies, and logical inference. It is particularly effective in contexts where shared ontologies or semantic consistency across distributed datasets are required.



Hypergraph

While less common in mainstream graph database engines, hypergraph-based approaches represent relationships involving more than two entities.

In a hypergraph:

- A hyperedge can connect any number of nodes, enabling direct representation of higher-arity relationships.
- Hypernode models sometimes encapsulate sub-graphs as nodes, supporting hierarchical and modular structures.

These approaches are conceptually attractive for domains such as biological networks or knowledge representation, where natural relationships are not strictly binary.



Relation Centric

Some modeling approaches treat relationships as rich entities with their own ontological roles. While LPGs allow edge properties, relation-centric models go further by:

- Allowing edges to participate in additional relationships.
- Providing explicit identity for edges independent of node pairs.
- Supporting reification, where relationships are modeled as nodes to encode meta-relationships.

This approach is valuable for capturing temporal, provenance, or contextual information about relationships themselves; e.g., “Alice knows Bob because of Project X” or “Authored (relationship) occurred at time t”.

Schema (less)



Schema-First (Schema-Driven) Modeling

- Defines node labels, relationship types, and property constraints before data ingestion.
- Encourages consistency, formal validation, and predictable querying patterns.
- Favored in enterprise or regulated environments.

Schema-Optional (Schema-On-Read) Modeling

- Allows data structures to evolve organically.
- Defers strict schema definitions until query or analysis time.
- Suited for exploratory analytics and rapidly changing domains.

Most **LPG** databases support schema-optional modeling, whereas **RDF** environments tend to encourage schema-first modeling through formal ontologies.

Approach



Some contemporary graph systems adopt a hybrid approach, integrating graph structures with relational or document-oriented storage. In such cases:

- Graph models coexist with tabular schemas.
- Certain entities are best stored as documents or relational tables, while relationships and connectivity are represented in graph form.
- The modeling approach becomes an exercise in partitioning data across representational paradigms.

This approach is particularly common in large-scale analytics platforms and multi-model databases such as ArangoDB or Cosmos DB.

Graph vs Relational

Unlike traditional relational databases, which depend heavily on **JOIN** operations to reconstruct relationships across multiple tables, graph databases operate by directly **traversing** explicitly **stored connections** between entities. In a relational system, the process of linking data often becomes increasingly expensive as the number of tables grows or as the depth of the relationships expands. Graph databases, by contrast, maintain pre-materialized edges that encode these relationships natively in the data model.

As a result, queries involving complex networks such as multi-hop paths, densely interconnected records, or variable-length relationship patterns can be executed with substantially higher performance. The traversal-based execution model avoids the computational overhead of repeated JOIN computations and scales more naturally with the structural complexity of the dataset. This makes graph databases particularly effective for domains where connectedness is central to the analytical workload.

Use Cases

- Social Networks

Graph databases naturally express relationships like friendships, followers, communities, and interests, enabling complex analytics.

- Fraud Detection

Financial institutions analyze relationships among accounts, transactions, and devices to detect suspicious patterns using graph traversals.

- Knowledge Graphs

Knowledge graphs integrate vast heterogeneous datasets.

Search engines, recommendation systems, and semantic services rely heavily on graph structures.

- Supply Chain and Logistics

Graph models capture supplier-manufacturer-distributor relationships, enabling route optimization and dependency analysis.

Query Languages

In the ecosystem of graph data management, two query languages have emerged as the most influential and widely adopted:

- **Cypher**, originating from the Neo4j platform,
- **Gremlin**, developed as part of the Apache TinkerPop framework.

Although both languages allow users to explore and manipulate graph-structured data, they embody distinct philosophical and operational approaches.
Cypher is a **declarative, pattern-matching language** that enables users to specify the desired structural configuration of a query such as paths, motifs, or node–relationship patterns without needing to articulate the exact steps for retrieving them.

Gremlin, in contrast, adopts a **procedural, traversal-oriented paradigm**, in which the user describes an explicit sequence of navigation operations that move from vertex to vertex through the graph.

Neo4J

Neo4j is the most widely adopted native graph database.
It offers ACID transactions, a native graph engine, and the Cypher language.

Neo4J Overview

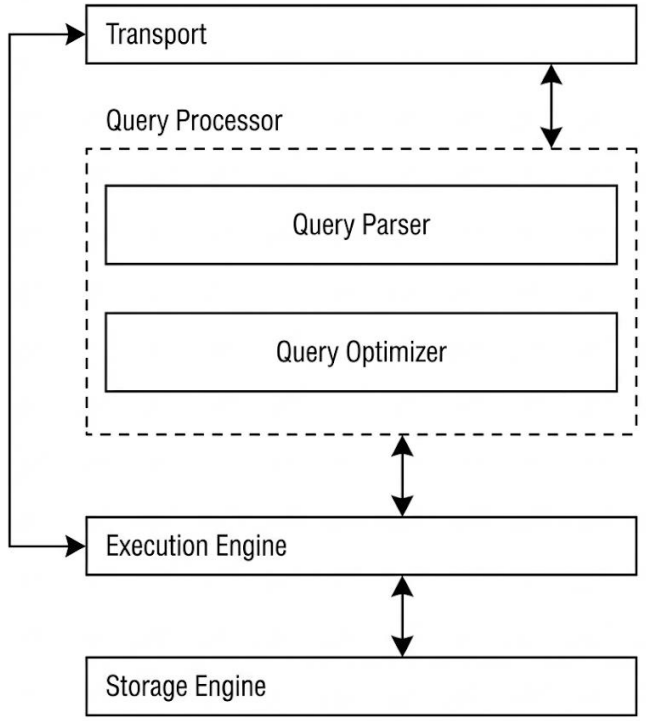
Neo4j’s architecture is built around a **native graph storage engine** and a **graph-optimized processing layer** that store and traverse nodes and relationships without translation overhead. It uses an **ACID-compliant transactional core** supported by a **clustered, highly available deployment model** for scalability and fault tolerance. Together, these components enable efficient graph traversal performance while preserving strong consistency guarantees.

Neo4J Architecture

Transport: at the beginning of any interaction with the database system, a client-issued query enters through the transport subsystem, which subsequently forwards it to the query processor for interpretation and execution.

Query Processor: the query processor parses and validates the incoming statement before the query optimizer determines an efficient execution strategy. In Neo4j, this role is fulfilled by the Cypher Query Planner, which relies on internal statistics, indexes, constraints, and data distribution to guide its optimizations.

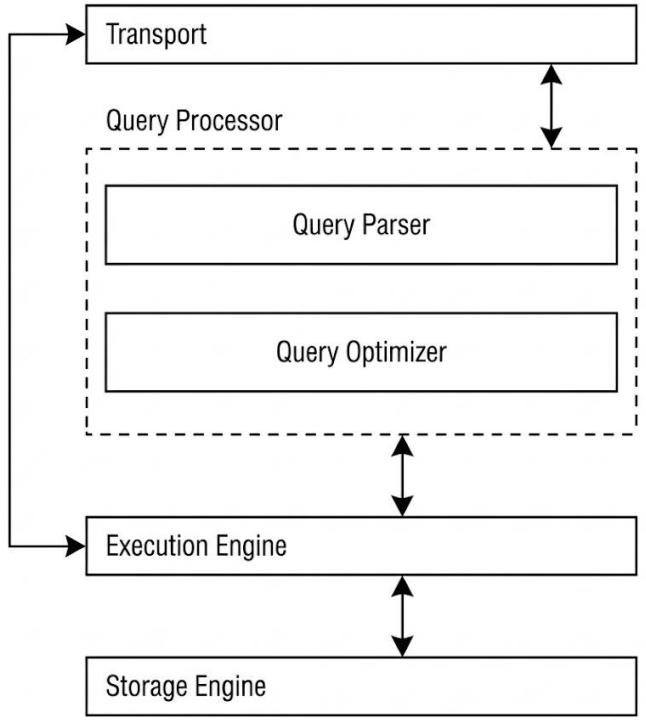
Execution Engine: the query processor produces an execution plan, a structured sequence of operations, which is then carried out by the execution engine to generate and aggregate results. This plan is composed of discrete operators arranged in a tree-like structure, where leaf operators resolve records and propagate their outputs upward, while non-leaf operators consume the intermediate results produced by their branches.



Neo4J Architecture

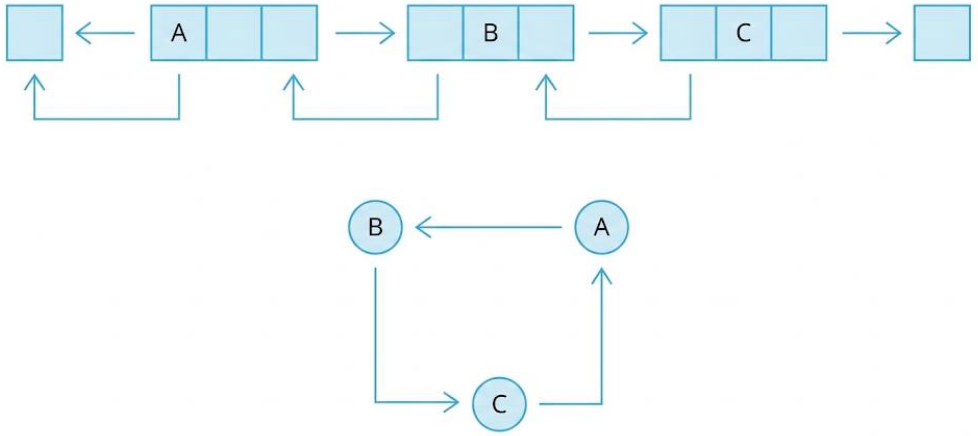
Storage: Data persisted on disk follows the index-free adjacency principle and is organized into store files, each dedicated to a specific record type and fixed-size record format. Although Neo4j employs a disk-based storage model, it depends extensively on in-memory caching to accelerate access, achieving its best performance when frequently accessed data resides in memory and thus minimizes disk I/O overhead.

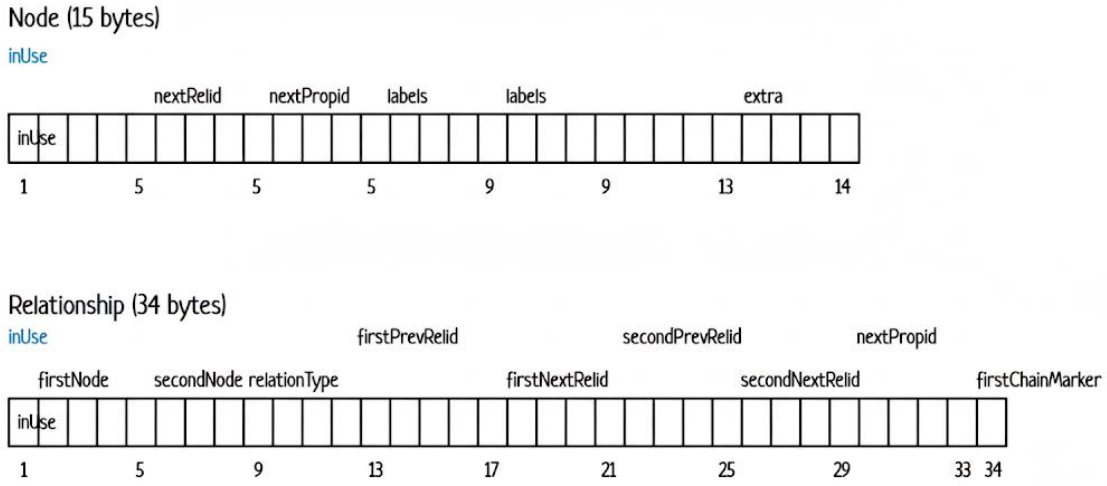
Store File	Record size	Contents
neostore.nodestore.db	15 B	Nodes
neostore.relationshipstore.db	34 B	Relationships
neostore.propertystore.db	41 B	Properties for nodes and relationships
neostore.propertystore.db.strings	128 B	Values of string properties
neostore.propertystore.db.arrays	128 B	Values of array properties
Indexed Property	1/3 * AVG(X)	Each index entry is approximately 1/3 of the average property value size



Neo4J Architecture

Indexing: is essential in traditional DBMSs for accelerating data retrieval, but large indexes can degrade write performance and consume substantial storage. Neo4j avoids these costs by relying on ***index-free adjacency***: each node directly references its neighbors through singly linked lists, while edges form doubly linked lists that link both forward and backward. This native adjacency model enables near constant-time traversal performance, as query cost depends only on the portion of the graph explored rather than on global data size.





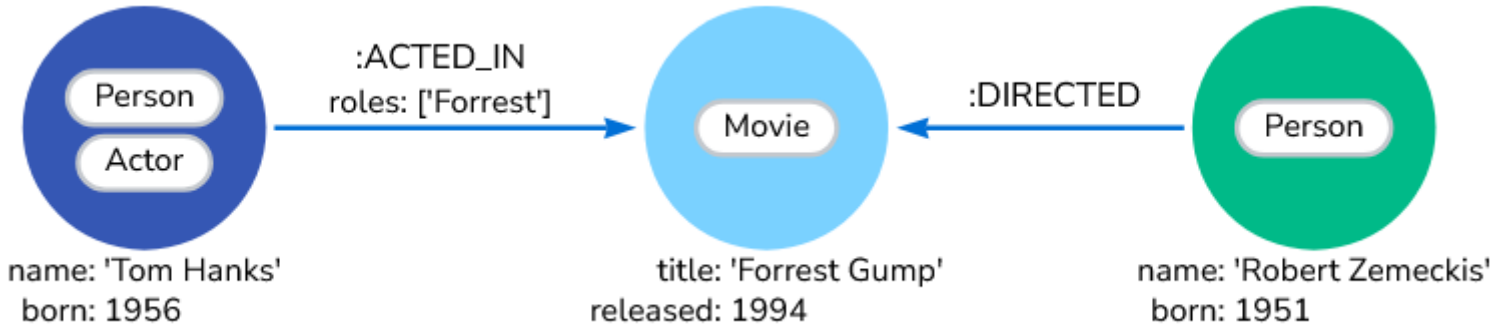
Neo4J Architecture

Neo4j's indexing relies on three global index types: b-tree, token lookup, and full-text, each optimized for different retrieval needs.

- **B-tree** indexes map property values to nodes or relationships for efficient exact-match lookups and ordered access.
- **Token lookup** indexes, created automatically for all labels and relationship types, allow rapid label-based matching and support the efficient construction of other indexes.
- **Full-text** indexes, built on Apache Lucene, serve search-oriented workloads that require flexible text matching.

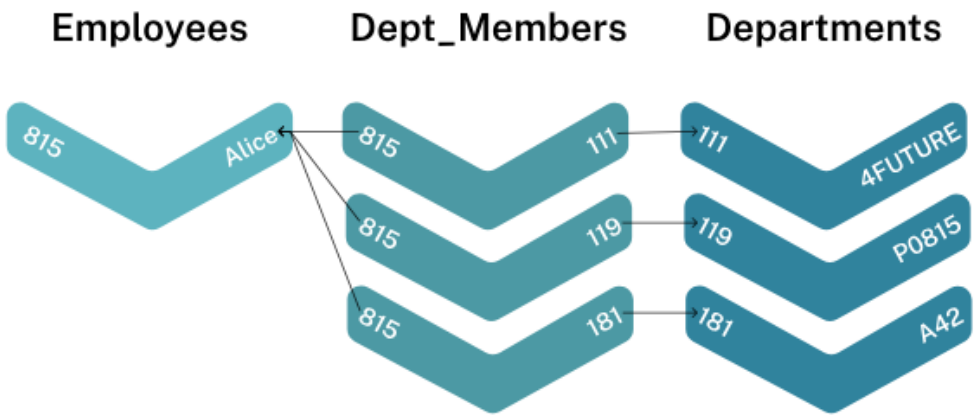
Neo4J Example Graph

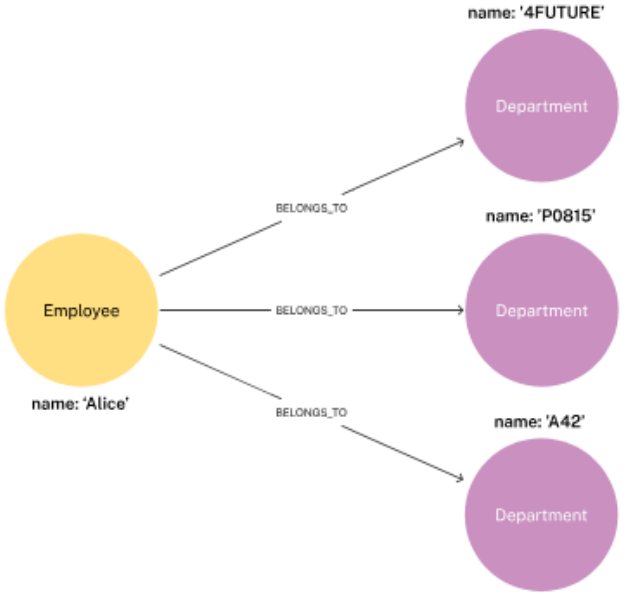
Nodes are used to represent *entities* (discrete objects) of a domain.
Node labels classify nodes into semantic groups, enabling operations to be restricted to specific sets—such as retrieving all nodes labeled *User*. Because labels can be added or removed at runtime, they also serve as markers for transient states, for example indicating suspended accounts or seasonal items. Each node may carry any number of labels, including none.
Relationships express a directed connection between a source and a target node, and may even link a node to itself. Each relationship has a single type that classifies its role and may include properties that further describe its meaning. Through these typed, property-bearing links, nodes form larger structures, ranging from lists and trees to maps and composite entities, supporting complex and densely interconnected graph topologies.
A relationship must have exactly one **relationship type**.
Properties are key–value pairs used to store descriptive information on both nodes and relationships.



Relational vs Graph

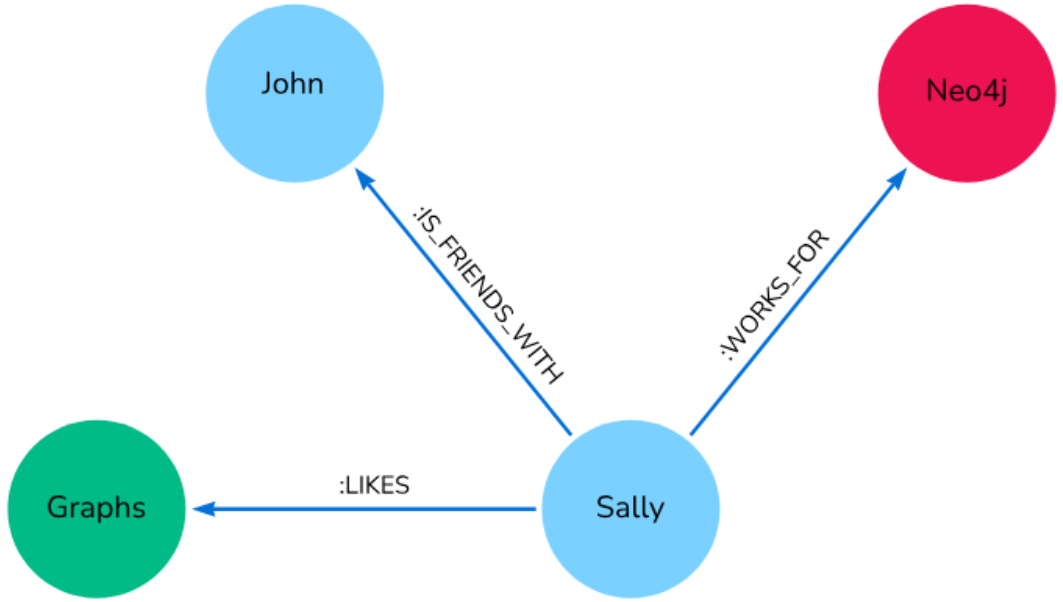
Relational databases organize data into predefined tables, requiring rigid schemas. Relationships are expressed through primary–foreign key references, with JOIN operations computed at query time by matching keys across tables, an approach that becomes computationally expensive as data complexity increases. Many-to-many relationships further amplify this cost by necessitating additional associative tables to mediate the connections. In **graph databases**, relationships are as fundamental as the data itself, eliminating the need to infer connections via foreign keys or external processing. By organizing nodes and relationships into connected structures, the database can model the problem domain directly.





Neo4J Cypher

Cypher is Neo4j's declarative query language, conforming to the GQL standard and available as open source through the openCypher project. While conceptually similar to SQL, it is specifically optimized for graph data. Its intuitive, nearly natural-language syntax uses ASCII-art–style notation to represent graph patterns and relationships, for example, (:Node)-[:RELATES_TO]->(:OtherNode), where parentheses denote nodes and square brackets with arrows indicate directed relationships. This syntax supports all standard CRUD operations on graph structures.




```
(:Sally)-[:LIKES]->(:Graphs)
(:Sally)-[:IS_FRIENDS_WITH]->(:John)
(:Sally)-[:WORKS_FOR]->(:Neo4j)
```

Neo4J Cypher

Node variables allow parts of a query to reference matched nodes in later clauses. Variables, written in lowercase, can be single letters (e.g., p:Person) or full words (person:Person). Relationships are always directional, indicated by arrows, which may point left-to-right, right-to-left, or remain unspecified for undirected links.
An undirected relationship can be traversed in either direction, even though all relationships have an inherent direction; Cypher allows querying them undirected using clauses like MATCH. This is useful when the relationship direction is unknown, ensuring all connected nodes of a given type are retrieved. Relationship types classify connections and convey meaning, typically using verbs to describe how nodes relate, making queries resemble natural language, with nodes as nouns and relationships as predicates.
Relationships can also be assigned variables, just like nodes. Once defined, these variables allow the relationship to be referenced in subsequent parts of the query.

```
MATCH (p:Person)
RETURN p
```

```
(p:Person)-[:LIKES]->(t:Technology)
```

```
(p:Person)<-[:LIKES]-(t:Technology)
```

```
MATCH (p:Person)-[:LIKES]-(t:Technology)
```

```
MATCH (p:Person)-[r:LIKES]->(t:Technology)
RETURN p,r,t
```

Neo4J Cypher

- Create nodes and relationships:

```
CREATE (p:Person {name: 'Alice’})
CREATE (p)-[:KNOWS]->
(:Person {name: 'Bob'})
```

- Matching Patterns

```
MATCH (p:Person)-[:KNOWS]-
>(f:Person)
RETURN p.name, f.name
```

- Variables

```
MATCH (p:Person {name: 'Alice'})
SET p.age = 30
```

- Querying relationship directions

```
MATCH (a)-[:KNOWS]-(b)
RETURN a, b
```

- Filtering and conditions

```
MATCH (p:Person)
WHERE p.age > 25
RETURN p
```

- CUD

```
CREATE (:Person {name: 'Alice', age:
30})
MATCH (p:Person {name: 'Alice'})
SET p.age = 31
MATCH (p:Person {name: 'Bob'})
DELETE p
```

- Return and aggregate results

```
MATCH (p:Person)-[:KNOWS]->(f)
RETURN p.name, count(f) AS num_friends
```

Cloud Graph Database Providers

Neo4j / Neo4j Aura: fully managed cloud native edition of Neo4j, available on major cloud platforms (AWS, Azure, GCP).
It preserves Neo4j's native graph storage and its expressive query language Cypher, while removing the burden of infrastructure management — which is ideal if you want scalability without operational overhead.
Amazon Neptune (on AWS): fully managed graph database supporting both property graph and RDF models. It supports multiple popular query languages (e.g. Apache TinkerPop Gremlin, SPARQL, and openCypher) enabling flexibility depending on your data model and application needs.
As a managed service it handles scaling, backups, failover, and integrates with other AWS infrastructure (security, networking, etc.), which simplifies operations for large-scale or production workloads.
Microsoft Azure Cosmos DB (Gremlin API): a multi model globally distributed database that offers graph capabilities via the Gremlin API among its supported models. That makes it a good fit if you already are invested in Azure or need a database that supports graph + other data models alongside.
Open source/self-managed but cloud deployable: JanusGraph; not strictly a "cloud service," but because it supports backends such as Cassandra, HBase or Bigtable, you can deploy it in cloud infrastructure (e.g. on virtual machines or managed clusters) and treat it as a cloud hosted graph solution.

Choices

Choose Neo4j Aura if needed native graph storage, the expressive power of Cypher, and is preferred a fully managed, cloud native graph solution without managing infrastructure.
Use Amazon Neptune when already within the AWS ecosystem, or when need support for both property graph and RDF models, or expect to scale heavily with reliability and integration with AWS services.
Go with Azure Cosmos DB (Gremlin API) when need a multimodel database in Azure and plan to mix graph data with other data paradigms (document, key-value etc.).
Opt for JanusGraph (self-managed on cloud) when need flexibility, want open-source freedom, or need to integrate graph storage with other bespoke distributed back ends or big data infrastructures.

Sample App

http://my-neo4j-movies-app.herokuapp.com/

Resources

Neo4j Getting Started
Cypher
Neo4J AuraDB

Date Ierarhice in RDBMS

Introducere

- Exista momente cand avem nevoie sa stocam date arborescente sau ierarhice pentru a modela diverse probleme:
  - Categorii, sub-categorii si sub-sub-categorii intr-un online shop
  - Meniul de navigare intr-un sitemap complex
  - Relatiile dintre membrii unui sistem multilevel (eg: MLM)
- Organizarea datelor si indexarea ar trebui sa ofere:
  - Afisarea rezultatelor (cum se construieste un arbore de navigare?)
  - Raport statistic (cate persoane sunt pe o ramura?)
  - Extragera unui subarbore specific (se doreste afisarea produselor cu categorii dintr-o anumita sub-categorie, vezi cursul SD)

Arbori

Exista o diferenta intre arbori si ierarhii.

Arborii sunt un caz special de graf; ierarhiile sunt un caz special de arbori si au legatura cu mostenirea si subordonarea.

- Arborii sunt grafuri cu urmatoarele proprietati:

un arbore

  - este un graf conex fara cicluri. Un graf conex este un graf in care exista un drum intre oricare 2 noduri. Nu exista nod deconectat de restul grafului.
  - orice nod este radacina unui subarbore.
  - oricare 2 noduri sunt conectate de un singur drum.
  - Este un graf conex ce are numarul de muchii mai mic cu 1 decat numarul de noduri.

In arbore, cand se sterge o muchie, rezultatul este o padure (forest) cu 2 arbori disjuncti.

Ierarhii

- O ierarhie este un arbore directionat cu proprietati suplimentare de subordonare si mostenire. Relatia reprezentata de muchiile grafului poate fi de la radacina sau spre radacina:
  - Intr-o organizatie autoritatea porneste de obicei de la radacina, cu CEO, si descreste spre frunze.
- Mostenirea, fie de la sau spre radacina, este proprieatea cea mai importanta a unei ierarhii. Aceasta proprietate nu este caracteristica unui arbore.
  - O ierarhie permite unui nod sa aiba mai multe roluri. Intr-o organizatie o persoana poate avea mai multe roluri.
- Ierarhiile au roluri ce sunt ocupate de entitati. Proprietatea de rol nu exista intr-un arbore; fiecare nod apare o data intr-un arbore si este unic.

Tipuri de Ierarhii

Putem identifica 4 tipuri de modelare:

1. **Noduri statice si muchii statice**: schema contabila dintr-un sistem economic nu se va schima (mult/des) in timp.

2. **Noduri statice si muchii dinamice:** message board-ul unui forum/chat; continutul mesajelor (nodurile) nu se schimba dupa ce sunt postate; raspunsurile noi sunt postate ca subordonate ale unui mesaj existent (muchie noua).

3. **Noduri dinamice si muchii statice:** schema organizationala, in care organizarea de functii ramane aceeasi, oamenii ce ocupa aceste functii se schimba/muta.

4. **Noduri dinamice si muchii dinamice:** modelul graf al unei retele de transport sau comunicatii. Traficul in retea este in continua schimbare. Un exemplu: Waze, rute diferite la momente diferite.

Consideratii de Manipulare a Datelor

Problema poate fi privita din urmatoarele (nu e lista completa de) perspective relativ la manipularea datelor:

- inserarea unui nou nod
- stergerea unui nod
- mutarea unui nod
- navigarea in structura
  - gasirea descendentilor
  - gasirea ascendentilor
  - gasirea nivelului unui nod

Modele de Proiectare

In literatura se gasesc diverse modele cu caracteristici diferite pentru organizarea datelor ierarhice:

- liste de adiacenta (adjacency list)
- enumerarea cailor (path enumeration)
- multimi imbricate (nested sets)
- tabele de legatura (bridge table)
- intervale imbricate (nested intervals)
- tabele late (flat table)
- altele ...

Liste de Adiacenta

O metoda simpla (folosita anterior) de stocare si intretinere a ierarhiei sub forma de relatii parinte-fiu este de a inlude pentru fiecare inregistrare o referinta catre inregistrarea parinte. Acest model se numeste “Adjacency List Model”.

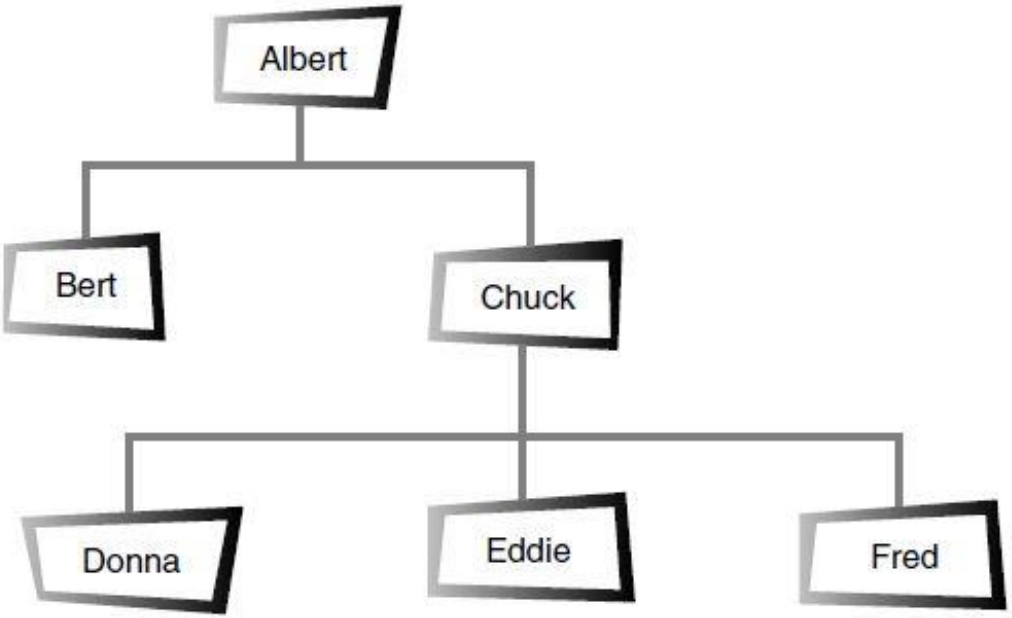
Acest model este un fel de graf de relatii si reprezinta un caz special in care graful formeaza o structura arborescenta.

Reprezentarea arborescenta permite interogarea usoara a nodurilor imediat adiacente unui nod dat (deasupra sau dedesubt in ierarhie)

Necesita recursivitate pentru interogarea datelor ce se gasesc pe mai multe nivele in arbore.

Liste de Adiacenta

```
CREATE TABLE Personnel_OrgChart
(emp VARCHAR(10) NOT NULL PRIMARY KEY,
boss VARCHAR(10), -- null means root
salary DECIMAL(6,2) NOT NULL,
... );
```



Liste de Adiacenta

- Cea mai des intalnita solutie
- Fiecare intrare isi cunoaste parintele
- Trebuiesc evitate unele anomalii:
  - anomalii de update (un angajat isi schimba numele)
  - anomalii de insert (crearea unui ciclu in graf prin inserarea unei relatii)
  - anomalii de delete (un angajat este sters si un subarbore devine deconectat)
  - anomalii de structura (referinta spre sine, null-uri multiple)
- Aceste anomalii pot aparea din cauza **DKNF**; este o forma normala folosita in normalizarea bazei de date ce necesita ca o tabela sa nu contina alte constrangeri decat de domeniu si cheie.

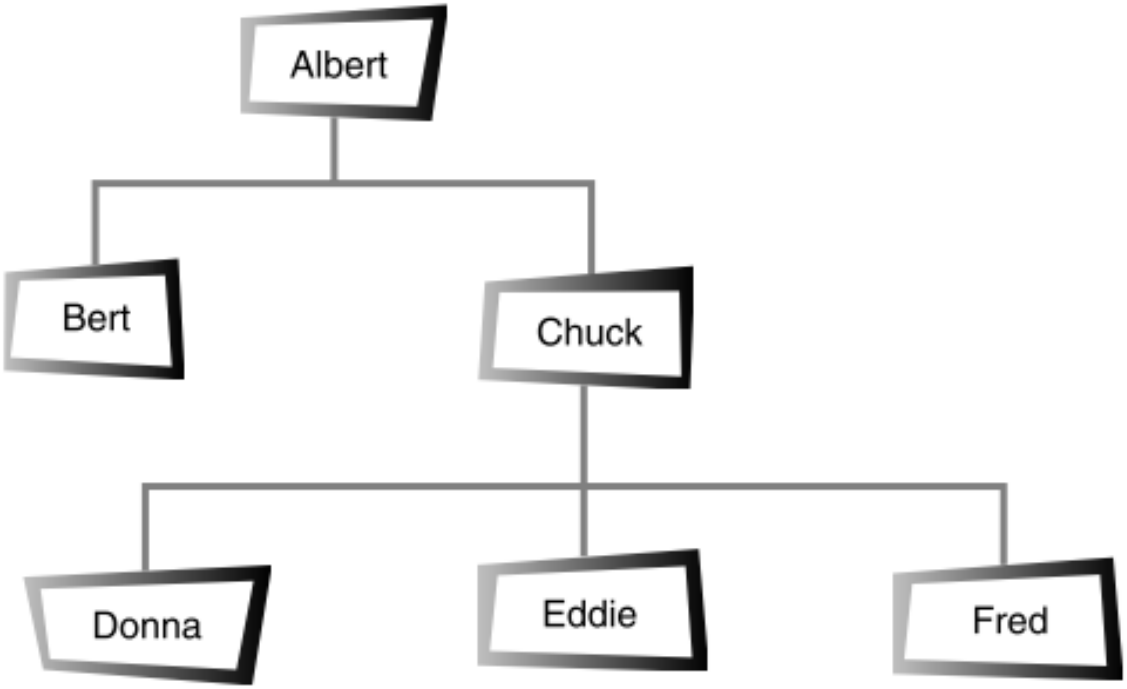
  Constangerea de domeniu specifica valorile permise pentru un atribut, iar cele de cheie specifica atributele ce identifica unic o inregistrare in tabela.

Liste de Adiacenta

```
CREATE TABLE Personnel
(emp_nbr INTEGER DEFAULT 0 NOT NULL PRIMARY
KEY,
emp_name VARCHAR(10) DEFAULT '{ {vacant} }'
NOT NULL,
address VARCHAR(35) NOT NULL,
birth_date DATE NOT NULL,
...);
```

Consideram ca exista un angajat virtual numit '{ {vacant} }' cu marca angajat (emp_nbr) egala cu valoarea 0

Liste de Adiacenta



Liste de Adiacenta

```
CREATE TABLE OrgChart
(job_title VARCHAR(30) NOT NULL PRIMARY KEY,
emp_nbr INTEGER DEFAULT 0 NOT NULL REFERENCES
Personnel(emp_nbr)
ON DELETE SET DEFAULT ON UPDATE CASCADE,
-- zero is vacant position
boss_emp_nbr INTEGER -- null means root node
REFERENCES Personnel(emp_nbr),
UNIQUE (emp_nbr, boss_emp_nbr),
salary DECIMAL (12,4) NOT NULL CHECK (salary
>= 0.00),
...);
```

Liste de Adiacenta

Trebuie sa adaugam constrangeri tabelei pentru a forta proprietatea de arbore (de asemenea sa asiguram faptul ca functia nu e detinuta de o persoana ce nu este angajata in companie)

Conditia ar putea fi:

```
CHECK (boss_emp_nbr <> emp_nbr) -- cannot be
your own boss!
```

dar, nu e suficient din cauza angajatului virtual cu marca 0, functia 'vacant'.

```
CHECK ((boss_emp_nbr <> emp_nbr) OR
(boss_emp_nbr = 0 AND emp_nbr = 0))
```

Alte cicluri sunt evitate folosind constrangerea `UNIQUE (emp, boss)` ce limiteaza la 1 singur manager (boss) intre angajati.

Liste de Adiacenta

Din teoria grafelor stim ca numarul de muchii intr-un arbore este numarul de noduri - 1:

```
CHECK ((SELECT COUNT(*) FROM OrgChart) - 1
-- edges
= (SELECT COUNT(boss_emp_nbr) FROM OrgChart))
-- nodes
```

Conditia `COUNT (boss_emp_nbr)` va elimina NULL din radacina.

Are acelasi efect cu a avea o constrangere de verificare a unui NULL:

```
CHECK((SELECT COUNT(*) FROM OrgChart WHERE
boss_emp_nbr IS NULL) = 1)
```

Este o conditie necesara, dar nu si suficienta.

Ex: emp1 este boss pentru emp2 si emp2 este boss pentru emp1

Liste de Adiacenta

O abordare posibila ar fi eliminarea tuturor nodurilor frunza si repetarea procedurii pana cand se obtine multimea vida. Daca arborele nu se reduce la multimea vida inseamna ca exista un ciclu deconectat.

```
CREATE FUNCTION TreeTest() RETURNS CHAR(6)
LANGUAGE SQL
DETERMINISTIC
BEGIN ATOMIC
-- put a copy in a temporary table
INSERT INTO Tree SELECT emp, boss FROM
Personnel_OrgChart;
-- prune the leaves
WHILE (SELECT COUNT(*) FROM Tree) - 1
= (SELECT COUNT(boss) FROM Tree)
```

Liste de Adiacenta

```
DO DELETE FROM Tree
WHERE Tree.emp
NOT IN (SELECT T2.boss
FROM Tree AS T2
WHERE T2.boss IS NOT NULL);
IF NOT EXISTS (SELECT * FROM Tree)
THEN RETURN ('Tree');
ELSE RETURN ('Cycles');
END IF;
END WHILE;
END;
```

Liste de Adiacenta

Structura originala Personnel_OrgChart se poate reconstitui usor cu urmatorul VIEW:

```
CREATE VIEW Personnel_OrgChart (emp_nbr, emp,
boss_emp_nbr, boss)
AS SELECT E1.emp_nbr, E1.emp_name,
E1.boss_emp_nbr, B1.emp_name
FROM Personnel AS E1, Personnel AS B1,
OrgChart AS O1
WHERE B1.emp_nbr = O1.boss_emp_nbr
AND E1.emp_nbr = O1.emp_nbr;
```

NULL in Liste de Adiacenta

Utilizarea NULL pentru a reprezenta radacina nu este singura modalitate;

Sa ne aducem aminte cum am rezolvat la cursul SD in semestrul anterior

Navigarea in Liste de Adiacenta

- foarte simplu de obtinut **copiii unui nod**

```
SELECT * FROM OrgChart AS O1 LEFT JOIN
OrgChart AS O2 ON O2.emp_nbr = O1.boss_emp_nbr
```

- foarte simplu de obtinut **parintele unui nod**

```
SELECT * FROM OrgChart AS O1 INNER JOIN
OrgChart AS O2 ON O1.emp_nbr = O2.boss_emp_nbr
```

- Mai dificil de afisat **subarborele complet al unui nod**

```
SELECT * FROM OrgChart AS O1 LEFT JOIN
OrgChart AS O2 ON O2.emp_nbr = O1.boss_emp_nbr
LEFT JOIN OrgChart AS O3 ON O3.emp_nbr =
O2.boss_emp_nbr
```

…

- SQL-99 introduce recursive CTE

```
WITH [RECURSIVE] ...
```

Navigarea in Liste de Adiacenta

- **Inserarea unui nod**: doar se insereaza perechea (emp_nbr, boss_emp_nbr) in tabel; nu trebuie schimbat niciun alt rand.
- S**tergerea unui nod**:
  - Stergerea unui nod frunza: doar se sterge randul din structura arborescenta.
  - Cand se sterge un nod intern din arbore problema este complexa. Este necesara o decizie asupra nodurilor subordonate; exista mai multe abordari:

1. Ancient Egyptian school of management: cand se sterge un nod, toate nodurile subordonate se sterg. (Cand moare faraonul, se ingroapa toti sclavii cu el.)

2. Send the orphans to Grandmother: nodurile subordinate nodului sters devin noduri subordonate ale parintelui nodului sters (bunic).

Navigarea in Liste de Adiacenta

3. The oldest son takes over the shop: unul din nodurile subordonate preia locul nodului sters. Aceasta promovare poate provoca o cascada de promovari in subarbore

Enumerarea Cailor

Se bazeaza pe proprietatea arborilor ca exista doar cate o singura cale de la radacina la orice nod din arbore. Modelul stocheaza calea sub forma de string prin concatenarea fie a muchiilor fie a cheilor nodurilor din cale. Cautarile se fac pe baza aplicarii de functii pentru stringuri.

Exista 2 metode pentru enumerarea cailor: **enumerarea muchiilor** si **enumerarea nodurilor**. Enumerarea nodurilor este metoda mai des folosita; de altfel exista o mica diferenta in operatiile pe stringuri necesare pentru cele 2 modele.

```
--CHECK() prevents separator in the column.
CREATE TABLE Personnel_OrgChart
(emp_name CHAR(10) NOT NULL,
emp_id CHAR(1) NOT NULL PRIMARY KEY
CHECK(REPLACE (emp_id, '/', '') = emp_id) ),
path_string VARCHAR(500) NOT NULL);
```

Enumerarea Cailor

Este necesara prevenirea ciclurilor in graf: un ciclu s-ar reprezenta ca un string de cale in care cel putin o cheie (emp_id) apare de 2 ori. Acest lucru se poate obtine cu o constrangere ce foloseste o subinterogare:

```
CHECK (NOT EXISTS
(SELECT *
FROM Personnel_OrgChart AS D1,
Personnel_OrgChart AS P1
WHERE CHAR_LENGTH (REPLACE (D1.emp_id,
P1.path_string, ‘’))
< (CHAR_LENGTH(P1.path_string)- 1) --size of one
emp_id string
) )
```

Enumerarea Cailor

Nicio cale nu poate avea lungimea mai mare decat numarul de noduri din arbore

```
CHECK ((SELECT MAX(CHAR_LENGTH(path_string) )
FROM Personnel_OrgChart AS P1)
<= (SELECT COUNT(emp_id) * CHAR_LENGTH(emp_id)
FROM Personnel_OrgChart AS P2))
```

Se presupune ca cheia (emp_id) are lungime fixa si nu se foloseste niciun separator in stringul de cale.

Enumerarea Cailor

emp_name	emp_id	path_string
'Albert'	'A'	'A'
'Bert'	'B'	'AB'
'Chuck'	'C'	'AC'
'Donna'	'D'	'ACD'
'Eddie'	'E'	'ACE'
'Fred'	'F'	'ACF'

Enumerarea Cailor

Pentru un anumit nod, **gasiti toti subarborii sai**. Cea mai simpla solutie:

```
SELECT *
FROM Personnel_OrgChart
WHERE path_string LIKE '%' || :parent_emp_id ||
'%';
```

Dezavantaj:

Cautarile folosind predicate LIKE ce incep cu wildcard '%' nu pot folosi indexuri si gereaza table scan.

Enumerarea Cailor

O alta posibila abordare este utilizarea interogarii:

```
SELECT * FROM Personnel_OrgChart
WHERE path_string
LIKE (SELECT path_string
FROM Personnel_OrgChart
WHERE emp_id = :parent_emp_id) || '%';
```

Subinterogarea va putea folosi un index pe coloana emp_id pentru a gasi “inceputul” stringului, de la radacina la parintele nodului de interes.

Enumerarea Cailor

Navigarea in adancime in arbore e usoara. In loc de wildcard-ul ‘%’, se foloseste un string ce contine wildcard-uri underscore (‘_’) de lungime potrivita. De exemplu, interogarea de mai jos va returna copiii directi ai unui nod parinte dat (emp_id):

```
SELECT * FROM Personnel_OrgChart
WHERE path_string
LIKE (SELECT path_string
FROM Personnel_OrgChart
WHERE emp_id = :parent_emp_id) ||'_';
```

Enumerarea Cailor

Navigarea in adancime in arbore e usoara. In loc de wildcard-ul ‘%’, se foloseste un string ce contine wildcard-uri underscore (‘_’) de lungime potrivita. De exemplu, interogarea de mai jos va returna copiii directi ai unui nod parinte dat (emp_id):

```
SELECT * FROM Personnel_OrgChart
WHERE path_string LIKE (SELECT path_string
FROM Personnel_OrgChart
WHERE emp_id = :parent_emp_id) || REPLICATE(‘_’,
:n);
```

Enumerarea Cailor

Pentru un nod dat, **sa se gaseasca toti superiorii** (ascendentii). Aceasta problema necesita dezasamblarea caii in identificatorii componenti. Se poate utiliza o tabela de secvente de intregi pentru a determina substringul cerut:

```
SELECT SUBSTRING (P1.path_string
FROM (seq * CHAR_LENGTH(P1.emp_id) )
FOR CHAR_LENGTH(P1.emp_id) ) AS emp_id
FROM Personnel_OrgChart AS P1, Sequence AS S1
WHERE P1.emp_id = :search_emp_id
AND S1.seq <=
CHAR_LENGTH(path_string)/CHAR_LENGTH(emp_id);
```

Enumerarea Cailor

Problema este ca nu vom obtine si relatia intre superiori, doar lista acestora.

```
SELECT P2.*
FROM Personnel_OrgChart AS P1,
Personnel_OrgChart AS P2
WHERE P1.emp_id = :search_emp_id
AND POSITION (P2.path_string IN P1.path_string) =
1;
```

Enumerarea Cailor

Stergerea subarborelui cu radacina in acel nod se poate face cu acelasi predicat utilizat la gasirea subordonatilor:

```
DELETE FROM Personnel_OrgChart
WHERE path_string
LIKE (SELECT path_string
FROM Personnel_OrgChart
WHERE emp_id = :dead_guy) || '%';
```

Enumerarea Cailor

Acest model are aceleasi proprietati de **inserare** cu modelul lista de adiacenta. Noul nod (emp_id) se concateneaza la sfarsitul caii nodului parinte.

```
INSERT INTO Personnel_OrgChart VALUES (:new_emp,
:new_emp_id,
(SELECT path_string FROM Personnel_OrgChart WHERE
emp_id= :new_emp_boss) || :new_emp_id);
```

inserarea unui subarbore:

```
INSERT INTO Personnel_OrgChart SELECT N1.emp,
N1.emp_id,
(SELECT path_string FROM Personnel_OrgChart WHERE
emp_id= :new_tree_boss) || N1.emp_id
FROM NewTree AS N1;
```

Multimi Imbricate

Foloseste o tehnica ce utilizeaza 2 valori aditionale pentru fiecare nod, conform cu traversarea arborelui, ce viziteaza fiecare nod de 2 ori, atribuind valorile (numere) in ordinea vizitarii, la ambele vizite.

Cele 2 valori numerice sunt stocate ca 2 atribute. Apartenenta la ierarhie este simplu de verificat prin compararea celor 2 valori.

Actualizarea este mult mai costisitoare si necesita renumerotare. Se pot face anumite imbunatatiri folosind numere rationale in loc de intregi, in felul acesta evitand renumerotarea, obtinand o actualizare mai rapida, dar mai complexa.

Multimi Imbricate

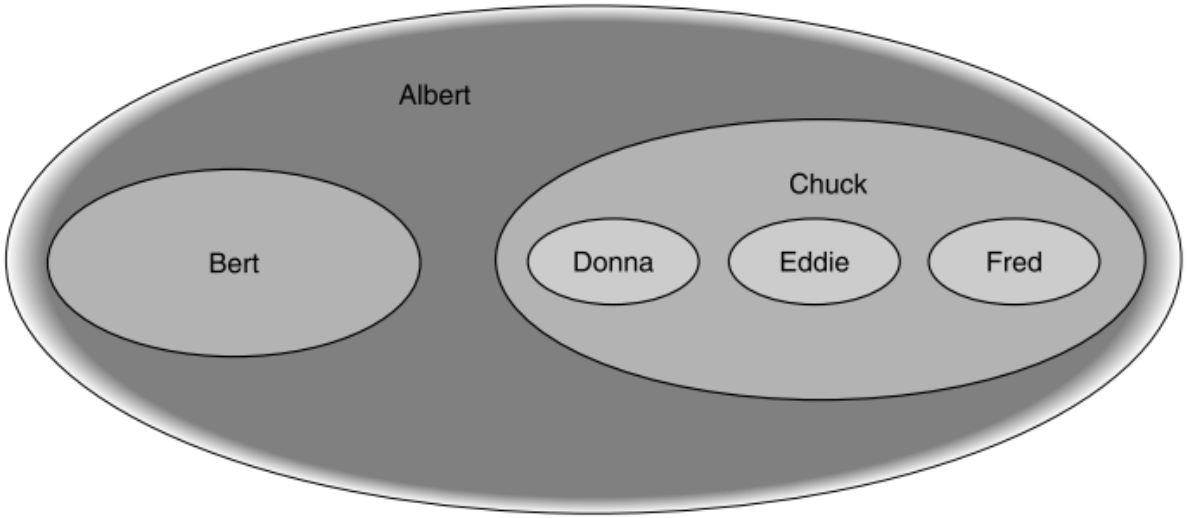
Pentru reprezentarea grafica a unui arbore sub forma de multimi imbricate se pot folosi elipse si imbricarea elipselor subordonate in interiorul parintilor. Daca o elipsa este continuta intr-o alta, va reprezenta subordonare.

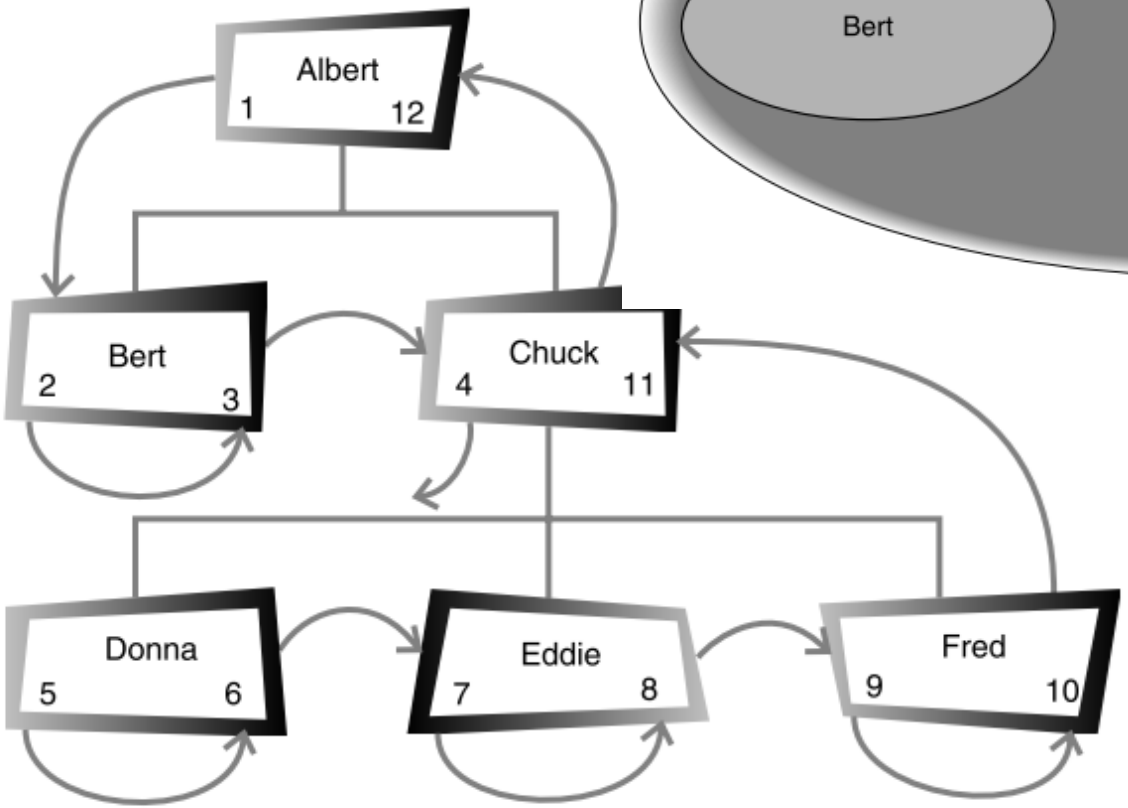
Radacina va fi reprezentata de cea mai mare (exterioara) elipsa si va contine toate celelalte noduri. Nodurile frunza vor fi reprezentate de elipsele cele mai interioare, fara a contine alte elipse, iar imbricarea elipselor va arata relatia ierarhica.

Putem modela arborele cu (lft, rgt) multimi imbricate folosind perechi de intregi.

Aceste perechi de intregi vor contine intotdeauna perechile corespunzand nodurilor subordonate, astfel ca un nod copil va fi intre limitele nodului parinte.

Multimi Imbricate





Multimi Imbricate

Nodul radacina va avea intotdeauna ca in coloana lft valoarea 1, iar in cloana rgt valoarea dublului numarului de noduri.

Radacina se poate determina cu interogarea:

```
SELECT * FROM Orgchart WHERE lft = 1;
```

Interogarea poate beneficia de existenta unui index pe valorile coloanei lft. Un nod frunza este unul ce nu are niciun copil. In modelul liste de adiacenta nu se determina foarte usor astfel de noduri:

```
SELECT * FROM OrgChart AS O1
WHERE NOT EXISTS
(SELECT * FROM OrgChart AS O2
WHERE O1.member = O2.boss);
```

Multimi Imbricate

In modelul de multimi imbricate pentru nodurile frunza avem proprietatea ca diferenta intre valoarea lui lft si cea a lui rgt este intotdeauna 1. Putem determina nodurile frunza:

```
SELECT * FROM Orgchart WHERE(rgt - lft) = 1;
```

Pentru a putea eficientiza astfel de determinari se poate utiliza un index fie pe coloana lft, fie pe perechea de coloane (lft, rgt), optimizatorul putand utiliza acel index in loc de un table scan:

```
SELECT * FROM Orgchart
WHERE lft =(rgt - 1);
```

Multimi Imbricate

Un arbore este un graf fara cicluri; de asemenea, exista intotdeauna o cale de la radacina la orice nod din arbore.

O alta proprietate este ca orice nod in arbore este radacina unui subarbore; unele proprietati ale acelui subarbore sunt utile perechii de coloane (lft, rgt). In tabela multimi imbricate toti descendentii unui nod pot fi regasiti prin cautarea nodurilor ce au valorile (lft, rgt) intre valorile (lft, rgt) ale nodului parinte.

Toti subordonatii fiecarui boss din ierarhia organizationala:

```
SELECT Mgrs.member AS boss, Workers.member AS
worker
FROM Orgchart AS Mgrs, Orgchart AS Workers
WHERE Workers.lft BETWEEN Mgrs.lft AND Mgrs.rgt
AND Workers.rgt BETWEEN Mgrs.lft AND Mgrs.rgt;
```

Multimi Imbricate

Nivelul unui nod in arbore este numarul de muchii dintre acel nod si radacina. O cale este o multime de muchii ce conecteaza 2 noduri. Modelul multimi imbricate se foloseste de faptul ca fiecare pentru fiecare parinte are “latimea” (latimea = [rgt - lft]) mai mare decat a multimilor continute. Radacina va avea “latimea’ cea mai mare.

Pentru a determina nivelul fiecarui nod (employee) :

```
SELECT O2.member, COUNT(O1.member) AS level
FROM OrgChart AS O1, OrgChart AS O2
WHERE O2.lft BETWEEN O1.lft AND O1.rgt
GROUP BY O2.member;
```

Expresia COUNT(01.member) va contoriza si nodul insusi.

Multimi Imbricate

Pentru a **sterge un nod intern** din arbore este mai dificil decat eliminarea intregului subarbore in modelul nested sets. Cand se elimina nodul intern trebuie decis cum se va “umple” nodul lipsa. Se pot aplica mai multi algoritmi. O metoda este de a conecta copii la parintele nodului de sters (Mom dies and Grandma adopts the kids). Reprezinta o promovare verticala a intregului subarbore.

O alta promovare verticala este de a muta un singur copil pe pozitia nodului sters (give the business to the oldest son). Dezavantajul este ca atunci cand nodul copil este promovat va determina o lipsa in acea pozitie.

O alta solutie este promovarea orizontala. Nodul frate din dreapta al nodului sters se muta pe pozitia vacanta (Dad dies and his oldest brother takes over the business). Se presupune ca exista un astfel de nod frate care sa preia acea pozitie.

Conversia Nested Sets la Adjacency List

Modelul adjacency list este cel mai des utilizat pentru ca:

- E. F. Codd a publicat o lucrare ce utiliza adjacency list, el fiind parintele domeniului BD.
- Reprezinta o modalitate de a construi inlantuiri de pointeri, metoda traditionala de programare pentru gestiunea arborilor in limbajele procedurale.

```
SELECT B.member AS boss, P.member
FROM OrgChart AS P LEFT OUTER JOIN
Personnel AS B
ON B.lft = (SELECT MAX(S.lft)
FROM OrgChart AS S
WHERE P.lft > S.lft AND P.lft < S.rgt);
```

Conversia Nested Sets la Adjacency List

O alta versiune a aceleiasi interogari:

```
SELECT B.member AS boss, P.member
FROM OrgChart AS B, Personnel AS P
WHERE P.lft BETWEEN B.lft AND B.rgt
AND B.member = (SELECT MAX(S.member)
FROM OrgChart AS S
WHERE S.lft < P.lft AND S.rgt > P.rgt);
```

Versiunea aceasta ar trebui sa fie mai simpla si mai rapida. Rezultatul va contine doar muchiile grafului, nu si o inregistrare (NULL, <root>).

Tabele de Legatura

Modalitatea cea mai intalnita pentru stocarea ierarhii este sa utilizam un "pointer" (foreign key) spre parinte. Una din probleme este ca in unele versiuni de SGBD nu se poate obtine simplu un intreg arbore. Problema este asemanatoare cu regasirea tuturor descendentilor sau ascendentilor unui nod.

Modelele prezentate incerca sa depaseasca aceste probleme dar au fiecare dezavantaje. Problema este ca gestiunea trebuie facuta la nivel de aplicatie: trebuie aleasa performanta fie pentru timpul de regasire a datelor, fie pentru cel de insert/delete.

Mai exista un model numit **tabel de legatura (bridge table/closure table).** Modelul permite regasirea tuturor descendentilor unui nod (**optional pana la o adancime specificata**) doar cu un join cu tabela de legatura. Avantajul principal este ca atunci cand se adauga sau sterge o relatie parinte-copil, doar o operatie este necesara (poate fi configurata sa ruleze ca trigger pe tabela principala).

Tabele de Legatura

O tabela de legatura (closure table) este o tabela ce stocheaza inchiderea tranzitiva ("transitive closure“) a relatiilor parinte-copil din tabela de baza.

In aceasta structura, avand o tabela cu campuri pentru parinte, copil si adancime, ierarhia poate fi gestionata pentru orice adancime.

Numarul total de inregistrari in tabela de legatura este egal cu numarul de inregistrari din tabela de baza inmultit cu adancimea medie a arborelui din tabela de baza. Fiecare inregistrare din tabela de baza are o inregistrare corespondenta in tabela de legatura, plus o inregistrare pentru fiecare parinte.

Tabele de Legatura

Pentru a face nodul PARENT_ITEM parinte al nodului CHILD_ITEM, codul poate fi:

```
insert into closure(parent, child, depth)
select p.parent, c.child, p.depth+c.depth+1
  from closure p, closure c
 where p.child=PARENT_ITEM and
c.parent=CHILD_ITEM
```

Costul acestei operatii este direct proportional cu marimea subarborelui cu radacina CHILD_ITEM; daca se adauga o frunza, este doar un simplu insert de inregistrari pentru parinte pentru noul nod. Daca se adauga un parinte la un arbore existent, se va insera o noua inregistrare pentru fiecare nod “adoptat” din subarbore.

Tabele de Legatura

Elementul principal pentru tabela de legatura sunt inregistrarile cu depth 0. Fara ele, 4 inserari separate ar fi necesare, sau o interogare cu UNION pentru a colecta relatiile unui nou link.

Stergerea unei legaturi (link) poate utiliza o interogare precum:

```
delete link
  from closure p, closure link, closure c
 where p.parent = link.parent and c.child =
link.child and p.child=PARENT_ITEM    and
c.parent=CHILD_ITEM
```

Tabele de Legatura

Datorita faptului ca inserarea si stergerea sunt simple si modifica doar tabela de legatura, se pot automatiza aceste operatii cu triggere pe tabela de baza.

Sunt necesare actiuni suplimentare: in momentul inserarii, se insereaza inregistrarea (self,self,0) in tabela de legatura, iar in momentul stergerii, se sterge atat self-link-ul cat si toate legatura care depind de el:

```
delete link
  from closure p, closure link, closure c,
closure to_delete
 where p.parent=link.parent and
c.child=link.child and p.child=to_delete.parent
and c.parent= to_delete.child and
(to_delete.parent=DELETE_ITEM or
to_delete.child=DELETE_ITEM) and
to_delete.depth<2
```

Intervale Imbricate

Reprezinta generalizarea Multimilor Imbricate folosind numerele rationale (de exemplu numere rationale diadice)

Multimile Imbricate sunt potrivite pentru unele aplicatii, dar au si dezavantaje:

1. Codificare este volatila. In jur de jumatate din nodurile arborelui ar trebui reetichetate cand se adauga un nod.

2. Interogarea intervalelor este asimetrica din perspectiva performantei. Este usor de raspuns daca un punct este in interval, dar este greu de indexat o multime de intervale ce contin un anumit punct. Acest fapt conduce la dificultatea de a raspunde la interogari despre ascendentii unui nod.

Intervale Imbricate

Modelul Nested Sets (Seturi Imbricate) folosește numere întregi pentru a codifica nodurile, ceea ce înseamnă că există spații finite (lacune) pentru inserarea de noi noduri. Pentru a depăși această problemă, se poate utiliza un domeniu mai dens, cum ar fi numerele raționale.

Modelul Nested Intervals (Intervale Imbricate) codifică fiecare nod al arborelui cu o pereche de numere, numită cap (head) și coadă (tail). Intervalul alocat unui nod copil este întotdeauna conținut în intervalul nodului părinte.

Cu aceasta etichetare inchiderea tranzitiva poate fi interogata astfel:

```
select e1.ename, e2.ename
from emp e1, emp e2
where e2.head >= e1.head
and e2.head < e1.tail
```

Intervale Imbricate

Subarborele tuturor descendentilor unui nod poate fi regasit prin restrictia asupra view-ului anterior cu un predicat

```
select e2.ename from emp e1, emp e2
where e2.head >= e1.head
and e2.head < e1.tail and e1.ename = 'SCOTT'
```

Calea pentru ascendeti poate fi obtinuta simetric

```
select e1.ename from emp e1, emp e2
where e2.head >= e1.head
and e2.head < e1.tail and e2.ename = 'SCOTT'
```

Tabela Flat

- O Lista de Adiacenta modificata prin adaugarea coloanelor Level si Rank (ordonare) pentru fiecare inregistrare.
- Simplu de iterat/paginat
- Sterge si mutare costisitoare
- Utile la: discutii/formuri/comentarii pe bloguri, organizate in threaduri.

Implementari

http://www.mysqltutorial.org/mysql-adjacency-list-tree/

https://docs.microsoft.com/en-us/sql/relational-databases/hierarchical-data-sql-server?view=sql-server-ver15

https://coderwall.com/p/whf3-a/hierarchical-data-in-postgres

http://www.klempert.de/php/nested_sets/demo/ (germana)

Resurse

- https://www.databasestar.com/hierarchical-data-sql/
- https://learnsql.com/blog/how-to-query-hierarchical-data/